

```
const shipping = 3000;

const menuPrice: number = 4000;

console.log(menuPrice);

console.log(total);

console.log(cafe.name);

console.log(found);

const applePrice: number = 1000;
```

자바스크립트를 배운 다음

타입스크립트

같은 파일인데 node 로는 잘 돌고 검사기로는 에러가 납니다.
그 차이를 아는 것이 이 책의 전부입니다.

9

단원

33

개념

143

직접 해보기

138

문제

차례

01	타입스크립트 시작하기	11
01	왜 타입인가	12
02	실행과 검사는 따로다 ★ 이 단원에서 제일 중요한 파일	19
03	에러 메시지 읽는 법	25
02	기본 타입과 추론	41
01	기본 타입	42
02	추론: 대부분은 안 적어도 된다	49
03	any 는 왜 금지인가 ★ 이 자료의 규칙 하나	56
03	함수 타입	73
01	매개변수와 반환값	74
02	선택 매개변수와 기본값	81
03	함수를 값으로 넘기기	88
04	객체와 타입 별칭	105
01	객체 타입 적기	106
02	type: 타입에 이름 붙이기	115
03	선택 속성 · readonly · interface	122
04	서버 데이터에 모양 적기	129
05	유니온과 타입 좁히기	145
01	유니온: 둘 중 하나	146
02	좁히기: 확인하면 쓸 수 있다 ★ 실무 체감 1위	155
03	null 과 undefined 다루기	164
04	판별 유니온: 모양이 여러 가지일 때	172
06	제네릭	193
01	제네릭이 왜 필요한가	194

02	제네릭 쓰기	201
03	이미 쓰고 있던 제네릭	208
04	keyof: 속성 이름을 타입으로 다루기	216
07	React와 타입스크립트	237
01	props 에 타입 붙이기	238
02	useState 와 타입	245
03	이벤트와 폼	248
04	화면 상태 다루기 ★ 이 단원의 결론	255
04	주문 화면	262
08	기존 프로젝트에 타입 입히기	283
01	as 를 왜 쓰지 않나	284
02	기존 JS 프로젝트 옮기기	291
03	tsconfig 읽기 · 그리고 다음 단계	298
04	타입이 없는 패키지 만나기	305
09	종합 실습	329
01	주문 관리	330
02	재고 보고서	334
03	서버 응답 다루기	338
부록 가	연습문제·종합연습 정답	343
부록 나	심화문제 정답	439

여는 글

시작하기 전에

다 배우고 나면 이런 것을 직접 만듭니다. 지금은 무슨 코드인지 하나도 몰라도 됩니다.

이 책의 표시

블록마다 왼쪽 가장자리 표시가 다릅니다. 표시만 보고 “지금 내가 무엇을 해야 하는지” 알 수 있습니다.

```
console.log("안녕하세요");
```

— 직접 쳐 볼 코드입니다. 파일을 열고 그대로 따라 치세요.

출력 **안녕하세요**

— 실행하면 컴퓨터가 내놓는 값입니다. 내 화면과 같은지 맞춰 보세요.

▢ 직접 해보기 **1**

자기 이름을 출력해 보세요.

— **여러분 차례입니다.** 이 표시가 보이면 손을 움직이세요. 정답은 그 개념 맨 끝에 있습니다.

여기에 코드를 쓰세요

— 연필로 직접 적는 자리입니다.

이런 실수를 합니다

따옴표를 빠뜨림

```
console.log(안녕하세요);
```

따옴표가 없으면 컴퓨터는 ‘안녕하세요’라는 이름의 변수를 찾습니다.

— 여기서 많이 넘어집니다. 미리 보고 피해 가세요.

RUN node 개념01_console_log로_출력하기.js

— 개념마다 맨 위에 있습니다. 이대로 실행하면 됩니다.

시작하기 전에

준비물 두 가지

① **Node 22.18 이상**. 터미널에서 확인하세요.

```
node --version
```

v22.18 미만이면 nodejs.org 에서 새로 설치하세요. (22.18 부터 **node** 가 타입 표기를 옵션 없이 지원합니다. 그 아래 버전은 실행이 안 됩니다) 이 자료는 **node** 파일 **.ts** 로 타입스크립트를 **바로 실행**합니다. **ts-node** 나 **tsx** 니 하는 도구는 필요 없습니다.

② **VS Code**. 타입 에러는 편집기의 **빨간 물결 밑줄**로 보는 것이 기본입니다. 메모장으로 열면 이 자료의 절반이 안 보입니다.

한 번만 하는 준비

이 폴더에서:

```
npm install
```

맥에서 자료를 옮겨 왔다면 한 번 더:

```
npm run fix:names
```

맥은 한글 파일명을 NFD(자모 분리)로 저장합니다. 글자로는 똑같이 보이는데 바이트가 다릅니다. 압축을 풀거나 옮기기만 해도 이렇게 바뀌고, 그러면 검증도구가 파일을 못 찾아 **npm run typecheck** 가 **안** **푼** **연습문제까지 검사해서 시끄러워집니다**. 검사 결과가 이상하면 이것부터 돌리세요. 고칠 게 없으면 조용히 끝납니다.

07단원에 들어갈 때 한 번 더:

```
cd 07_React와_타입스크립트/실습프로젝트
npm install
```

명령 네 개

JS자료에서는 `node` 파일 `.js` 하나였습니다. 여기서는 넷입니다.

명령	하는 일	언제
<code>node</code> 파일 <code>.ts</code>	실행만 한다. 타입은 안 본다	개념 파일을 돌려 볼 때
<code>npm run typecheck</code>	개념·정답 파일을 검사한다 (<code>npx tsc --noEmit</code> 과 같은 것입니다. 짧게 쓰려고 등록해 둔 이름입니다)	언제나 조용해야 정상
<code>npm run grade</code>	연습문제를 채점한다	문제를 풀면서 (전부 일치하면 다 맞은 것)
<code>npm run check</code>	연습문제를 타입 검사한다	풀면서 같이 (일부 문제만 잡습니다)
<code>npm run fix:names</code>	한글 파일명을 NFC 로 되돌린다	자료를 옮긴 뒤 검사가 이상할 때

채점은 `npm run grade` 로 합니다. 연습문제를 실제로 돌려서 문제에 적힌 '기대 출력' 과 줄 단위로 맞춰
봅니다.

`npm run check`(타입 검사)만 믿으면 안 됩니다. **138문항 중 21개(15%)만 잡습니다.** "예상한 값을
출력하세요" 같은 문제는 안 풀어도 타입 에러가 안 나기 때문입니다. 02단원과 09단원 종합01~03은 타입
검사로 잡히는 문제가 아예 없습니다. 어느 문제가 어느 쪽으로 잡히는지는 `npm run audit:exercises` 로 볼
수 있습니다.

07단원과 09단원 종합04는 브라우저라 자동 채점이 없습니다. '기대 화면' 을 눈으로 확인하세요.

이 자료에서 제일 중요한 것 — `node` 는 타입을 검사하지 않고 **지웁니다**. 그래서 타입이 틀려도 화면은 그냥
돌아갑니다. 이걸 모르면 반드시 헤맬니다. 01단원 개념02가 이 이야기만 합니다.

`tsc` 가 아무 말도 안 하면 통과입니다. "성공" 같은 글자는 안 나옵니다.

파일 읽는 법

각 단원 폴더는 이렇게 생겼습니다.

```

04_객체와_타입_별칭/
  개념01_객체_타입_적기.ts      ← 읽고 따라 치는 파일
  개념02_type_별칭.ts
  개념03_선택속성과_interface.ts
  개념04_서버_데이터에_모양_적기.ts
  연습문제.ts                  ← 직접 푸는 파일
  연습문제_정답.ts             ← 해설이 본체입니다

09_종합_실습/                  ← 개념 파일이 없습니다
  종합01_주문_관리.ts          ← 여러 단원을 엮는 실습
  종합01_주문_관리_정답.ts
  ...

```

개념 파일 안의 표시:

표시	뜻
— 섹션 N: 제목 —	한 덩어리의 시작
// 출력:	그 위 <code>console.log</code> 가 실제로 찍는 것
// 에러: TS2322 ...	아래 주석 처리된 줄의 // 를 지우면 나는 에러
// 🖊️ 직접 해보기 N	손으로 해 보는 자리. 정답은 파일 맨 아래
— 자주 하는 실수 —	실제로 많이 나는 실수
— 정리 —	그 파일의 요약

// 에러: 가 보이면 주석을 풀어서 직접 확인해 보세요. 빨간 밑줄이 어떻게 생기는지 한 번 보는 것이 설명 열 줄보다 낫습니다. 확인했으면 다시 // 를 붙이세요.

이 자료가 정한 규칙 셋

세 가지를 쓰지 않습니다. 셋 다 "검사를 끄는 도구" 입니다.

금지	왜	어디서
<code>any</code>	검사를 통째로 끈다. 그리고 옆으로 번진다	02단원 개념03
<code>!</code>	"없을 리 없다" 고 우기는 것. 있으면 터진다	05단원 개념03
<code>as</code>	값을 안 바꾸고 생각만 바꾼다	08단원 개념01

셋 다 "검사는 통과하는데 돌리면 터진다" 를 만듭니다. 이게 타입스크립트를 쓰는 이유를 정면으로 없앱니다.

이 셋 중 하나를 쓰고 싶어질 때는 대개 아직 안 배운 것이 있다는 신호입니다.

- 모양을 못 적겠다 → **04단원**
- 확인을 못 하겠다 → **05단원**
- 타입마다 같은 함수를 또 만들고 있다 → **06단원**

자주 보게 될 에러 다섯 개

번호	뜻
TS2322	답을 수 없다 (Type 'A' is not assignable to type 'B')
TS2339	그런 이름이 없다
TS2551	그런 이름이 없다 + 이거 아니냐
TS2345	넘긴 값이 안 맞는다
TS18048 / TS18047	undefined / null 일 수도 있다

Type 'A' is not assignable to type 'B' 에서 **A** 는 내가 넣은 것, **B** 는 들어가야 하는 것입니다. 거꾸로 읽으면 엉뚱한 곳을 고칩니다.

자세한 것은 01단원 개념03에 있습니다.

단 원 01

타입스크립트 시작하기

OPEN 01_타입스크립트_시작하기

```
const shipping = 3000;
const menuPrice: number = 4000;
console.log(menuPrice);
console.log(total);
```

01 왜 타입인가

02 실행과 검사는 따로다 ★ 이 단원에서 제일 중요한 파일

03 에러 메시지 읽는 법

– 연습문제

왜 타입인가

RUN node 개념01_왜_타입인가.ts

검사: npm run typecheck

JS자료를 끝내고 오셨습니다. 그때 이런 일을 겪었습니다.

"10" + 5 → 105 (더하기가 아니라 이어붙이기)

이 파일은 그 사고를 다시 꺼내서 타입스크립트가 무엇을 막아 주고, 무엇을 못 막는지를 봅니다.

처음부터 말해 두겠습니다. 타입스크립트는 만능이 아닙니다. 위의 사고 중 절반은 타입스크립트도 그냥 통과시킵니다. 어디까지 막아 주는지를 정확히 아는 것이 이 단원의 목표입니다.

문자열에 + 를 쓰는 것은 타입스크립트도 막지 않는다

섹션 1

장바구니 코드입니다. 수량은 화면 입력창에서 받아왔다고 생각하세요. 입력창 값은 무엇을 입력해도 언제나 문자열입니다.

```
const quantityText = "3"; // 입력창에서 온 값
const shipping = 3000;

console.log(quantityText + shipping);
```

출력 33000

3 + 3000 = 3003 이 아닙니다. "3" 뒤에 "3000" 을 붙여서 "33000" 입니다.

놀라운 사실 — 이 줄은 타입스크립트에서도 에러가 안 납니다. 빨간 밑줄도 안 생깁니다. npx tsc --noEmit 도 조용히 통과합니다.

왜냐하면 문자열을 + 로 잇는 것은 원래 정당한 연산이기 때문입니다.

"봄날" + "카페" → "봄날카페" ← 이건 아무 문제가 없죠

타입스크립트는 이 줄을 보고 "결과는 문자열이겠군" 하고 넘어갑니다. 여러분이 숫자를 원했다는 것을 알 방법이 없습니다.

◁ 직접 해보기

1

quantityText + shipping 의 결과에 `typeof` 를 찍어서

무슨 타입인지 확인해 보세요.

반대로 - 와 * 는 바로 걸린다

섹션 2

문자열끼리 빼는 것은 아무 의미가 없습니다. 그래서 타입스크립트가 막습니다.

이 코드는 검사에서 걸립니다

TS2362

```
console.log(quantityText - shipping);
```

The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.

“빼기의 왼쪽은 숫자여야 합니다” 라는 뜻입니다.

JS에서는 이 줄이 -2997 을 조용히 내놓던 자리입니다.
확인했으면 다시 // 를 붙이세요.

곱하기도 같습니다.

이 코드는 검사에서 걸립니다

TS2362

```
console.log(quantityText * 2);
```

The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.

여기서 규칙이 하나 보입니다.

+ 만 특별하다. - * / % 는 숫자 전용이라 바로 걸린다.

하필 사고가 제일 많이 나는 것이 + 입니다. 그래서 섹션 3이 필요합니다.

◁ 직접 해보기

2

quantityText / 3 을 써 보고 빨간 밑줄이 생기는지,

메시지가 섹션 2와 같은지 확인해 보세요. 확인 후 지우세요.

담을 자리에 타입을 적으면 + 사고도 잡힌다

섹션 3

타입을 적는 방법은 이름 뒤에 콜론(:)과 종류를 쓰는 것입니다.

```
const menuPrice: number = 4000;
```

```
_____
```

이 부분이 새로 배우는 전부입니다.

```
console.log(menuPrice);
```

```
출력      4000
```

이제 섹션 1의 그 줄을 `number` 자리에 담아 보겠습니다.

이 코드는 검사에서 걸립니다

TS2322

```
const wrongTotal: number = quantityText + shipping;
```

Type 'string' is not assignable to type 'number'.

이제야 걸립니다.

"이 자리에는 숫자가 와야 하는데 문자열이 왔습니다" 라는 뜻입니다.

섹션 1에서는 왜 안 걸렸을까요?

그때는 결과를 어디에도 안 담고 그냥 출력만 했기 때문입니다.

타입스크립트는 "내가 원하는 것이 무엇인지" 를 적어 줘야 비교할 수 있습니다.

이것이 타입을 적는 이유입니다.

타입은 '내가 기대하는 것' 을 코드에 적어 두는 일이고,
타입스크립트는 그 기대와 실재가 어긋날 때만 말해 줄 수 있다.

아무것도 안 적으면 아무것도 안 막아 줍니다.

물론 제대로 고치면 통과합니다.

```
const total: number = Number(quantityText) + shipping;
console.log(total);
```

```
출력      3003
```

▹ 직접 해보기 3

```
const stock: number = "열개"; 를 써 보고
```

빨간 밑줄에 마우스를 올려 메시지를 읽어 보세요. 확인 후 지우세요.

타입이 잡는 것과 못 잡는 것

섹션 4

잡는 것 1 · 값의 종류를 착각한 것 — 섹션 2, 3에서 봤습니다.

잡는 것 2 · 이름 오타

```
const cafe = { name: "봄날카페", price: 4500 };
console.log(cafe.name);
```

출력 봄날카페

이 코드는 검사에서 걸립니다

TS2551

```
console.log(cafe.pirce);
```

Property 'pirce' does not exist on type '{ name: string; price: number; }'. Did you mean 'price'?

JS에서는 `undefined` 가 조용히 찍히던 자리입니다.

"혹시 price 를 쓰려던 것 아닌가요?" 까지 물어봐 줍니다.

다만 제안이 항상 나오지는 않습니다. 이름이 짧으면 후보를 못 고릅니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(cafe.nmae);
```

Property 'nmae' does not exist on type '{ name: string; price: number; }'.

같은 오타인데 이번엔 'Did you mean' 이 없습니다. 에러 번호도 다릅니다(TS2339).

name 은 네 글자뿐이라 nmae 와 너무 멀다고 판단한 것입니다.

"제안이 안 뜨네?" 하고 당황할 필요 없습니다. 없는 이름인 건 똑같습니다.

잡는 것 3 · 있을 수도, 없을 수도 있는 값

```
const found = [10, 20, 30].find((n) => n > 100);
console.log(found);
```

출력 undefined

find 는 못 찾으면 `undefined` 를 줍니다. JS에서는 이것 잊고 바로 쓰다가 터집니다.

이 코드는 검사에서 걸립니다

TS18048

```
console.log(found + 1);
```

'found' is possibly 'undefined'.

“없을 수도 있는데 그냥 쓰시려고요?” 라고 막아 줍니다.

이 상황을 다루는 방법은 05단원에서 배웁니다.

반대로, 타입이 못 잡는 것도 분명합니다

못 잡는 것 1 · 계산이 틀린 것

```
const applePrice: number = 1000;
const bananaPrice: number = 2000;
const fruitTotal: number = applePrice - bananaPrice; // 더해야 하는데
뺐습니다 console.log(fruitTotal);
```

출력 -1000

둘 다 number 라서 타입은 아무 말도 안 합니다. 완벽하게 통과합니다. 총액이 음수인 것은 타입의 관심사가 아닙니다.

못 잡는 것 2 · 섹션 1의 + 사고 — 결과를 안 담으면 안 걸립니다.

못 잡는 것 3 · 화면이 이상한 것, 글자가 겹치는 것 — 타입과 무관합니다.

못 잡는 것 4 · 없는 데이터를 만들어 주지도 않습니다.

서버가 안 보내 준 값이 타입을 적는다고 생기지 않습니다.

정리하면 타입이 하는 일은 딱 한 가지입니다.

"이 자리에 들어올 수 있는 값의 종류" 를 미리 적어 두고,
어긋나면 실행하기 전에 알려 주는 것.

▹ 직접 해보기 4

fruitTotal 을 applePrice + bananaPrice 로 고치면

출력이 어떻게 바뀌는지, 타입 검사는 뭐라고 하는지 확인해 보세요.

덤으로 따라오는 것 — 자동완성

섹션 5

실제로 가장 크게 체감되는 이득은 이것입니다.

```
const order = { menu: "아메리카노", count: 2, isHot: true };
```

VS Code 에서 order. 까지만 치면 menu / count / isHot 이 목록으로 뜹니다. 외우고 있을 필요가 없어집니다. 오타가 날 자리도 없어집니다.

```
console.log(order.menu, order.count, order.isHot);
```

출력 아메리카노 2 true

남이 만든 함수를 쓸 때도 "이건 무엇을 받고 무엇을 돌려주는가" 가 마우스만 올리면 보입니다. 03단원에서 다시 다룹니다.

▸ 직접 해보기

5

order 에 마우스를 올려 보세요.

VS Code 가 어떤 설명을 보여 주는지 확인하면 됩니다.

자주 하는 실수

섹션 6

이런 실수를 합니다

"타입을 적었으니 이제 안전하다" 고 믿기

타입은 값의 '종류' 만 봅니다. 값이 옳은지는 안 봅니다. 위 fruitTotal 이 그 예입니다. 나이에 -5 를 넣어도 number 라서 통과합니다.

이런 실수를 합니다

"타입스크립트니까 알아서 다 잡아 주겠지" 라고 믿기

섹션 1이 반례입니다. 결과를 담을 자리에 타입을 안 적으면 안 잡힙니다. 막아 주는 것은 내가 기대를 적어 둔 자리뿐입니다.

이런 실수를 합니다

타입 에러가 나면 코드가 안 돌아간다고 생각하기

반대입니다. 타입이 틀려도 코드는 그냥 돌아갑니다. 이게 워낙 중요해서 다음 파일(개념02) 전체를 여기에 씁니다.

정리

- 1 타입은 "이 자리에 올 값의 종류" 를 미리 적어 두는 것이다. 문법은 : 하나다.
- 2 문자열에 + 를 쓰는 것은 타입스크립트도 안 막는다. 이어붙이기는 정당한 연산이라서다.
- 3 - * / 는 숫자 전용이라 바로 걸린다. 사고가 잦은 것은 하필 안 걸리는 + 다.
- 4 + 사고를 잡으려면 결과를 담을 자리에 : number 를 적어야 한다. 적어 둔 만큼만 막아 준다.
- 5 잡아 주는 것 — 종류 착각, 이름 오타, 없을 수도 있는 값.
- 6 못 잡는 것 — 계산 로직, 화면, 없는 데이터.
- 7 실제로 가장 크게 체감되는 이득은 자동완성이다.

직접 해보기 정답

```
1) console.log(typeof (quantityText + shipping));    // 출력: string
```

숫자가 하나 섞여 있어도 결과는 문자열입니다.

- 2 다음 메시지가 나옵니다. 섹션 2와 같은 TS2362 입니다. The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type. / 도 - * 와 같은 무리입니다. 숫자 전용입니다.
- 3 다음 메시지가 나옵니다. Type 'string' is not assignable to type 'number'. "문자열을 숫자 자리에 넣을 수 없습니다"
- 4 출력: 3000 타입 검사는 아무 말도 안 합니다. - 일 때도 + 일 때도 결과는 number 라 타입 입장에서는 둘 다 똑같이 옳은 코드입니다. "타입 검사를 통과했다" 와 "코드가 맞다" 는 다른 말입니다.
- 5 **const** order: { menu: **string**; count: **number**; isHot: **boolean** }

적지 않았는데도 타입스크립트가 알아서 알아냈습니다. 이것을 '추론' 이라고 하고 02단원에서 배웁니다.

실행과 검사는 따로다 ★ 이 단원에서 제일 중요한 파일

```
RUN node 개념02_실행과_검사는_따로다.ts
```

01

검사: npm run typecheck

이 파일을 안 읽으면 반드시 헤맵니다. 여기만은 꼭 읽으세요.

JS자료에서는 명령이 하나였습니다.

```
node 파일.js ← 실행하면 결과도 보이고 에러도 보였다
```

타입스크립트는 명령이 둘입니다. 그리고 둘이 하는 일이 완전히 다릅니다.

```
node 파일.ts ← 실행만 한다. 타입은 안 본다.
npx tsc --noEmit ← 검사만 한다. 실행은 안 한다.
```

이걸 모르면 "타입을 틀리게 썼는데 왜 아무 말도 없지?" 하고 한참 헤맵니다.

node 는 타입을 '지우고' 실행한다

섹션 1

여러분이 이렇게 쓰면

```
const price: number = 4000;
const menus: string[] = ["아메리카노", "라떼"];
```



"문자열이 들어 있는 배열" 이라는 뜻입니다. 02단원에서 배웁니다.
지금은 '타입 표기가 하나 더 있다' 정도로만 보세요.

node 는 실행하기 전에 타입 표기를 지웁니다. 이렇게 됩니다.

```
const price = 4000;
const menus = ["아메리카노", "라떼"];
```

: number 와 : string[] 은 흔적도 없이 사라집니다.

```
console.log(price, menus);
```

출력 4000 ['아메리카노', '라떼']

여기서 중요한 것은 '지운다' 지 '검사하고 지운다' 가 아니라는 점입니다. 맞는지 틀리는지 보지도 않고 그냥 지웁니다.

그래서 타입이 틀려도 화면은 그냥 돌아갑니다.
빨간 화면도, 콘솔 경고도, 아무것도 안 뜹니다.

JS자료(01~13단원)에서는 실수하면 화면이 깨지거나 콘솔에 빨간 줄이 나왔습니다. 그것에 익숙해져 있으면 여기서 반드시 걸려 넘어집니다.

▫ 직접 해보기 1

위 price 를 `const price: string = 4000;` 으로 고치고

node 로 실행해 보세요. 화면이 어떻게 되는지 확인한 뒤 되돌리세요.

왜 그렇게 만들어 냈나

섹션 2

(✎ 없음: 왜 그렇게 만들었는지를 읽는 섹션입니다. 해 볼 것이 없습니다)

타입 검사는 프로젝트 전체를 한 번 훑어야 합니다. 파일이 많으면 시간이 걸립니다. 실행할 때마다 그걸 기다리면 개발이 느려집니다.

그래서 역할을 나눴습니다.

node : 빨리 돌려서 결과를 보여 주는 일
tsc : 꼼꼼히 검사하는 일

07단원에서 만나는 Vite 도 똑같습니다. 타입을 검사하지 않고 지웁니다. 도구가 달라져도 이 구조는 그대로입니다.

직접 확인해 보기 ★

섹션 3

이 단원 폴더 안에 틀린예제/ 라는 폴더가 있습니다. 그 안의 타입이_틀린_파일.ts 는 타입이 세 군데나 틀린 채로 두었습니다.

TS자료 폴더에서 아래 두 줄을 순서대로 쳐 보세요.

1 node 01_타입스크립트_시작하기/틀린예제/타입이_틀린_파일.ts

2 npx tsc --noEmit -p 01_타입스크립트_시작하기/틀린예제

1번 결과 — 끝까지 잘 돌아갑니다.

```
가격: 4000
정말 숫자인가: string
메뉴 가격: undefined
100보다 큰 수 + 1 = NaN
---
```

이 파일은 끝까지 다 돌았습니다. 타입이 세 군데나 틀렸는데도요.

2번 결과 — 같은 파일인데 에러가 세 개 나옵니다.

```
...(23,7): error TS2322: Type 'string' is not assignable to type 'number'.
...(41,28): error TS2551: Property 'pirce' does not exist on ... Did you mean 'price'?
...(53,32): error TS18048: 'big' is possibly 'undefined'.
```

1번에서 눈여겨볼 줄은 이것입니다.

```
정말 숫자인가: string
```

: number 라고 적어 놓았는데 실제로 들어 있는 것은 문자열입니다. node 는 그 표기를 지우고 실행했기 때문에 아무 일도 일어나지 않았습니다. 화면의 “가격: 4000” 은 숫자 4000 과 눈으로 구별이 안 됩니다.

▫ 직접 해보기 2

위 두 명령을 실제로 쳐 보고,

2번에서 나온 에러 세 개가 각각 몇 번째 줄인지 파일을 열어 확인해 보세요.

그럼 타입 에러는 어디서 보나 — 두 곳뿐이다

섹션 4

곳 1 · VS Code 의 빨간 물결 밑줄

틀린 자리에 빨간 물결이 생깁니다. 저장하지 않아도 바로 나옵니다. 그 위에 마우스를 올리면 메시지가 뜹니다. 메모장이나 다른 편집기로 열면 이 밑줄이 안 보입니다. 이 자료는 VS Code 로 여는 것을 전제합니다.

곳 2 · 터미널에서 npx tsc --noEmit

TS자료 폴더에서 칩니다. 프로젝트 전체를 한 번에 검사합니다.

tsc 는 TypeScript Compiler 의 줄임말이고, --noEmit 은 “파일은 만들지 말고 검사만 해라” 라는 뜻입니다. 실행은 node 가 하니까 검사만 시키는 것입니다.

짧은 이름

이 긴 명령을 매번 치기 번거로우니 짧은 이름으로 등록해 두었습니다.

`npm run typecheck` ← `npx tsc --noEmit` 과 완전히 같습니다

앞으로 파일 머리글에는 짧은 쪽이 적혀 있습니다. 어느 쪽을 쳐도 결과는 같으니 편한 것을 쓰세요. (등록해 둔 곳은 `package.json` 의 `scripts` 입니다. 08단원에서 다시 봅니다)

▸ 직접 해보기

3

지금 TS자료 폴더에서 `npx tsc --noEmit` 을 쳐 보세요.

무엇이 나오는지(또는 안 나오는지) 확인하면 됩니다.

조용하면 통과다

섹션 5

문제가 없으면 `tsc` 는 아무것도 안 찍습니다.

```
> npx tsc --noEmit
>
```

"성공" 이나 "OK" 같은 글자는 안 나옵니다. 처음 보면 "명령이 안 먹었나?" 싶지만 정상입니다.

아무것도 안 나오는 것이 통과입니다.

이 자료에서는 이것을 '조용하다' 고 부르겠습니다.

▸ 직접 해보기

4

이 파일 아무 데나 `const x: number = "글자";` 를 한 줄 넣고

`npx tsc --noEmit` 을 쳐 보세요. 조용하지 않게 되는 것을 확인한 뒤 지우세요.

확장자가 바뀐다

섹션 6

지금까지 타입을 쓰려면

```
.js → .ts
.jsx → .tsx (07단원 React 에서)
```

이 자료의 01~06단원은 전부 `.ts` 입니다. 실행은 JS자료와 똑같이 `node 파일이름.ts` 입니다. 빌드 같은 것은 없습니다.

참고: `node` 가 `.ts` 를 바로 실행하는 것은 Node 22 이후 기능입니다.

예전 자료나 인터넷 글에서 ts-node 나 tsx 니 하는 도구를 설치하라고 하는 것을 볼 수 있는데, 지금은 필요 없습니다.
터미널에서 node --version 을 쳐서 v22 이상이면 됩니다.

▹ 직접 해보기 5

node --version 을 쳐서 버전을 확인해 보세요.

자주 하는 실수

섹션 7

이런 실수를 합니다

node 로 돌려 보고 “타입 다 맞았네” 라고 넘어가기

node 는 타입을 안 봅니다. 검사는 npx tsc --noEmit 로만 됩니다. 이 자료에서 제일 많이 나오는 실수입니다.

이런 실수를 합니다

빨간 밑줄이 있는데 화면이 잘 나오니까 무시하기

지금은 넘어가도 되지만, 07단원에서 화면이 안 나오는 진짜 원인이 됩니다. 빨간 밑줄은 “지금 당장은 안 터지지만 곧 터진다” 는 뜻입니다.

이런 실수를 합니다

tsc 가 조용한 것을 “명령이 실패했다” 고 착각하기

조용한 것이 통과입니다. 섹션 5를 다시 보세요.

이런 실수를 합니다

타입 에러가 났는데 파일을 저장 안 하고 다시 검사하기

VS Code 의 빨간 밑줄은 저장 없이도 갱신되지만, npx tsc --noEmit 은 ‘저장된 파일’ 을 읽습니다. 고쳤으면 저장부터 하세요.

정리

- 1 명령이 두 개다. node 는 실행만, npx tsc --noEmit 은 검사만 한다.
- 2 node 는 타입을 검사하지 않고 지운다. 그래서 타입이 틀려도 그냥 돌아간다.
- 3 타입 에러를 보는 곳은 둘뿐이다 — VS Code 빨간 밑줄, npx tsc --noEmit.
- 4 tsc 가 아무 말도 안 하면 통과다. '성공' 같은 글자는 안 나온다.
- 5 확장자는 .ts (React 는 .tsx). 실행법은 node 파일.ts 로 JS자료와 같다.
- 6 07단원의 Vite 도 똑같이 타입을 지운다. 도구가 바뀌어도 구조는 같다.

직접 해보기 정답

1) 화면은 아무 일 없이 그대로 돌아갑니다. 출력도 4000 그대로입니다. : string 이라고 적어 놓고 숫자 4000 을 넣었는데도 그렇습니다. VS Code 에는 빨간 밑줄이 생기고, npx tsc --noEmit 에는 이렇게 나옵니다. error TS2322: Type 'number' is not assignable to type 'string'. 재현:

```
const price: string = 4000;
```

→ 화면과 검사가 따로 논다는 것을 직접 본 것입니다.

- 2 23줄 · 41줄 · 53줄 입니다. 파일을 열어 보면 각각 `const price: number = "4000" / cafe.pirce / big + 1` 자리입니다. 괄호 안의 두 번째 숫자(7, 28, 32)는 그 줄의 몇 번째 글자인지입니다.
- 3 아무것도 안 나옵니다. 이 자료의 개념·정답 파일은 전부 검사를 통과한 상태입니다. 두 가지가 이 검사에서 빠져 있습니다.

- 틀린예제/ 폴더 – 일부러 틀린 파일이니까요
- 연습문제.ts – 안 푼 상태에서 걸리는 게 정상이라 npm run check 로 따로 봅니다

그래서 연습문제를 아직 안 풀었어도 이 명령은 조용합니다.

4) 이렇게 나옵니다. 01_타입스크립트_시작하기/개념02_실행과_검사는_따로다.ts(줄,열): error TS2322: Type 'string' is not assignable to type 'number'. 재현:

```
const x: number = "글자";
```

지우고 다시 치면 다시 조용해집니다.

5) v22 이상이면 됩니다. 낮은 버전이면 node 파일.ts 가 실행 자체를 거부합니다 (" .ts 확장자를 모른다" 는 뜻의 에러가 납니다. 문구는 버전마다 조금 다릅니다). 그때는 nodejs.org 에서 새 버전을 받아 까세요.

에러 메시지 읽는 법

RUN node 개념03_에러_메시지_읽는_법.ts

01

검사: npm run typecheck

타입스크립트 에러 메시지는 전부 영어입니다. 그리고 길어 보입니다. 그런데 실제로는 몇 가지 문장이 계속 반복됩니다. 다섯 개만 알면 앞으로 만날 것의 대부분이 해결됩니다.

이 파일에서는 그 다섯 개를 하나씩 직접 만들어 보면서 읽는 법을 익힙니다.

에러 한 줄의 구조

섹션 1

tsc 가 내는 에러는 언제나 이 모양입니다.

```
개념03_에러_메시지_읽는_법.ts(23,7): error TS2322: Type 'string' is not ...
└── 1 ───┘ └── 2 ─┘ └── 3 ─┘ └── 4 ───┘
```

1. 어느 파일인가
2. 몇 번째 줄, 몇 번째 글자인가 ← (줄, 열) 입니다
3. 에러 번호
4. 무슨 일인가

순서대로 “어디서 · 무엇이” 입니다. 2번의 줄 번호만 봐도 절반은 해결됩니다. 그 줄로 가면 되니까요.

3번 번호는 외울 필요가 없습니다. 다만 인터넷에 검색할 때 긴 영어 문장 대신 TS2322 라고만 치면 되니 편합니다.

```
const 예시 = "에러 번호는 검색어로 쓰면 편합니다";
console.log(예시);
```

출력 에러 번호는 검색어로 쓰면 편합니다

⇒ 직접 해보기 1

개념02의 틀린예제를 검사했을 때 나온 에러 세 줄에서

‘줄 번호’만 세 개 뽑아 적어 보세요.

① TS2322 : 담을 수 없다

이 코드는 검사에서 걸립니다

TS2322

```
const price: number = "4000";
```

Type 'string' is not assignable to type 'number'.

assignable = "넣을 수 있는". not assignable = "넣을 수 없다".

읽는 법: "string 을 number 자리에 넣을 수 없습니다"

가장 많이 보게 될 에러입니다. 왼쪽이 실제, 오른쪽이 기대한 것입니다.

이 에러는 배열 안에서도 그대로 나옵니다.

이 코드는 검사에서 걸립니다

TS2322

```
const scores: number[] = [90, 85, "칠십"];
```

Type 'string' is not assignable to type 'number'.

배열 전체가 아니라 "칠십" 이 있는 그 자리를 가리킵니다.

열 번호를 보면 정확히 어느 값인지 알 수 있습니다.

② TS2339 : 그런 이름이 없다

```
const count: number = 3;
console.log(count);
```

출력 3

이 코드는 검사에서 걸립니다

TS2339

```
console.log(count.toUpperCase());
```

Property 'toUpperCase' does not exist on type 'number'.

property = "속성", 점 뒤에 붙이는 이름을 말합니다.

읽는 법: "number 에는 toUpperCase 라는 게 없습니다"

toUpperCase 는 문자열에만 있습니다. 숫자에 쓰려고 한 것입니다.

JS에서는 실행하다가 TypeError 로 터지던 자리입니다.

③ TS2551 : 그런 이름이 없다 + 이거 아니냐

```
const cafe = { name: "봄날카페", price: 4500 };
console.log(cafe.name);
```

출력 봄날카페

이 코드는 검사에서 걸립니다

TS2551

```
console.log(cafe.pirce);
```

Property 'pirce' does not exist on type '{ name: string; price: number; }'. Did you mean 'price'?

② 와 같은 상황인데 비슷한 이름을 찾아 준 경우입니다.

Did you mean 'price'? = "price 를 쓰려던 것 아닌가요?"
이게 나오면 십중팔구 그게 맞습니다. 그대로 고치면 됩니다.

④ TS2345 : 넘긴 값이 안 맞는다

```
console.log("하".repeat(3));
```

출력 하하하

이 코드는 검사에서 걸립니다

TS2345

```
console.log("하".repeat("3"));
```

Argument of type 'string' is not assignable to parameter of type 'number'.

argument = "넘긴 값", parameter = "받기로 한 자리".

읽는 법: "숫자를 받기로 한 자리에 문자열을 넘겼습니다"

① 과 문장이 거의 같습니다. 다른 점은 '변수에 담을 때' 가 아니라 '괄호 안에 넘길 때' 라는 것뿐입니다.

argument 와 parameter 는 03단원에서 제대로 다룹니다.

⑤ TS18048 : 없을 수도 있는 값이다

```
const numbers = [10, 20, 30];
const big = numbers.find((n) => n > 100);
console.log(big);
```

출력 undefined

이 코드는 검사에서 걸립니다

TS18048

```
console.log(big + 1);
```

'big' is possibly 'undefined'.

possibly = “어쩌면”. 읽는 법: “big 은 `undefined` 일 수도 있습니다”

"지금은 값이 있을 수도 있지만 없을 때도 있는데, 그건 생각해 보셨나요?"
라는 뜻입니다. 이 상황을 다루는 방법은 05단원에서 배웁니다.

▹ 직접 해보기

2

`const t: string = 100;` 을 써 보고

나오는 에러가 위 다섯 개 중 무엇인지, ‘왼쪽 오른쪽’ 이 뭐가 뭔지 말해 보세요.

에러가 여러 개 나오면 위에서부터 하나씩

섹션 3

타입 에러는 한 번에 여러 개가 쏟아지는 일이 흔합니다. 스무 줄쯤 나오면 겁이 나지만, 대개 원인은 한두 개입니다.

1 맨 위 에러의 줄로 간다

2 그것만 고친다

3 다시 `npx tsc --noEmit` 을 친다

한 개를 고치면 밑에 있던 열 개가 같이 사라지는 일이 자주 있습니다. 앞의 에러 때문에 뒤가 줄줄이 틀린 것으로 보였던 것뿐이기 때문입니다.

반대로 아래쪽 에러부터 손대면 원인이 아닌 곳을 고치게 됩니다.

▹ 직접 해보기

3

아무 파일에 틀린 줄을 두 개 넣고 검사한 뒤,

위엣것 하나만 고치고 다시 검사해서 몇 개가 남는지 보세요.

메시지에 나오는 영어 단어

섹션 4

자주 나오는 단어는 이만큼입니다.

type	타입, 값의 종류
assignable	넣을 수 있는 (not assignable = 넣을 수 없다)
property	속성, 점 뒤의 이름
argument	넘긴 값
parameter	받기로 한 자리
possibly	어쩌면 ~일 수도
expected	기대한 것
required	반드시 있어야 하는
missing	빠졌다

이 아홉 개면 이 자료에서 만나는 메시지는 다 읽힙니다.

```
const 단어수 = 9;
console.log("외울 단어:", 단어수, "개");
```

출력 외울 단어: 9 개

⇒ 직접 해보기 4

위 표를 보지 말고

‘Property ... is missing’ 이 무슨 뜻인지 말해 보세요.

자주 하는 실수

섹션 5

이런 실수를 합니다

메시지를 안 읽고 코드부터 고치기

메시지에 답이 그대로 적혀 있는 경우가 대부분입니다. 특히 Did you mean 은 거의 항상 맞습니다.

이런 실수를 합니다

왼쪽과 오른쪽을 거꾸로 읽기

Type ‘A’ is not assignable to type ‘B’ 에서 A 는 ‘내가 넣은 것’, B 는 ‘들어가야 하는 것’ 입니다. 순서를 바꿔 읽으면 엉뚱한 곳을 고치게 됩니다.

이런 실수를 합니다

에러 스무 개를 보고 파일을 통째로 지우기

섹션 3을 보세요. 맨 위 하나만 고치면 대부분 같이 사라집니다.

이런 실수를 합니다

줄 번호는 봤는데 열 번호를 안 보기

한 줄에 값이 여러 개일 때(배열, 함수 인자) 열 번호가 “그중 어느 것” 인지 알려 줍니다.

정리

- 1 에러 한 줄은 '파일(줄,열): error 번호: 내용' 이다. 줄 번호부터 본다.
- 2 TS2322 담을 수 없다 / TS2339 그런 이름 없다 / TS2551 이거 아니냐 / TS2345 넘긴 값이 안 맞는다 / TS18048 없을 수도 있다.
- 3 Type 'A' is not assignable to type 'B' 는 A 가 내가 넣은 것, B 가 필요한 것.
- 4 에러가 여러 개면 맨 위부터 하나씩. 하나 고치면 여러 개가 같이 사라진다.
- 5 번호(TS2322)는 외우지 말고 검색어로 쓰면 된다.

직접 해보기 정답

- 1 23줄 · 41줄 · 53줄 각각 `const price: number = "4000" / cafe.pirce / big + 1` 자리입니다.
- 2 TS2322 입니다. error TS2322: Type 'number' is not assignable to type 'string'. 재현:

```
const t: string = 100;
```

왼쪽 'number' 가 내가 넣은 100, 오른쪽 'string' 이 t 가 받기로 한 종류입니다. 섹션 2의 ㉠ 과 왼쪽 오른쪽만 서로 바뀐 모양입니다.

- 3 상황마다 다릅니다. 서로 관계없는 두 줄이면 하나만 남고, 앞 줄이 뒤 줄의 원인이었다면 둘 다 사라집니다. 후자가 실제로 훨씬 자주 일어납니다.
- 4 "반드시 있어야 하는 속성이 빠졌습니다" 객체에 필요한 항목을 안 넣었을 때 나옵니다. 04단원에서 실제로 만납니다.

타입스크립트 시작하기

RUN node 연습문제.ts

01

채점: npm run grade ← 출력을 기대 출력과 맞춰 봅니다

검사: npm run check ← 타입 검사 (일부 문제만 잡습니다)

★ 채점은 npm run grade 로 합니다. 실제 출력을 문제의 '기대 출력' 과 줄 단위로 맞춰 봅니다. 전부 일치하면 다 맞은 것입니다.

npm run check(타입 검사)도 같이 쓰세요. 다만 이쪽은 '일부 문제만' 잡습니다.

138문항 중 21개뿐입니다. "출력하세요" 류는 안 풀어도 타입 에러가 안 나거든요. 그러니 check 가 조용하다고 다 맞은 것이 아닙니다. grade 로 확인하세요. (npm run typecheck 는 개념·정답 파일만 봅니다. 그쪽은 언제나 조용합니다)

- 1 TODO 자리에 코드를 씁니다.
- 2 npm run grade 로 채점합니다. (직접 node 연습문제.ts 로 돌려 봐도 됩니다)
- 3 npm run check 로 타입도 봅니다.
- 4 10분 고민해도 안 되면 연습문제_정답.ts 를 보세요.

순서대로 푸세요. 문제 1~8은 기본, 9~11은 응용, 12는 [도전]입니다. 문제 13은 에러를 직접 보는 문제라 맨 뒤에 있습니다.

```
console.log("=== 01단원 연습문제 ===");
```

문제 1 (개념01 섹션3)

아래 변수에 타입을 적어 주세요.

【지금까지 하던 방법】(JS)

```
const cafeName = "봄날카페";
```

【이번에 배울 방법】(TS) — 이름 뒤에 : 과 종류를 씁니다

```
const cafeName: string = "봄날카페";
```

아래 seatCount 에 숫자 타입을 붙이고 출력하세요.

기대 출력 24

기대 검사 결과: 조용함

{

아래 줄에 : number 를 붙이세요

```
const seatCount = 24;
  console.log(seatCount);
}
```

문제 2 (개념01 섹션3)

아래 두 변수에 각각 알맞은 타입을 붙이세요. 문자열은 string, 참/거짓은 boolean 입니다.

기대 출력 아메리카노 true

기대 검사 결과: 조용함

{

두 줄에 타입을 붙이세요

```
const menuName = "아메리카노";
  const isHot = true;
  console.log(menuName, isHot);
}
```

문제 3 (개념01 섹션1)

아래 코드의 결과가 무슨 '타입' 일지 먼저 예상해서 적어 보고, `typeof` 로 출력해서 맞는지 확인하세요.

기대 출력 string

기대 검사 결과: 조용함

```
{
  const quantityText = "2";
  const shipping = 3000;
```

quantityText + shipping 의 typeof 를 출력하세요

```
}
```

문제 4 (개념01 섹션3)

문제 3의 결과를 number 자리에 담으려고 하면 걸립니다. 걸리지 않게, 값도 올바르게 나오도록 고쳐주세요. (힌트: JS자료 01단원에서 쓰던 Number() 를 씁니다)

기대 출력 3002

기대 검사 결과: 조용함

```
{
  const quantityText = "2";
  const shipping = 3000;
```

total 에 올바른 합계가 담기게 하세요

```
const total: number = 0;
console.log(total);
}
```

문제 5 (개념01 섹션4)

아래 세 줄 중 '타입 검사가 잡아 주는 것'은 몇 번일까요? 번호를 출력하세요.

1번: `const age: number = -5;` (나이가 음수) 2번: `const age: number = "스물";` (숫자 자리에 한글) 3번: `const total: number = 1000 - 2000;` (더해야 하는데 뺀)

기대 출력 2

기대 검사 결과: 조용함

```
{
```

정답 번호를 출력하세요

```
}
```

문제 6 (개념02 섹션1)

아래 코드는 타입이 틀렸습니다. 그런데 node 로 돌리면 어떻게 될까요? 두 가지를 예상해서 순서대로 출력하세요. ① `console.log(price)` 가 화면에 찍는 것 ② `console.log(typeof price)` 가 찍는 것

```
const price: number = "4000";
console.log(price);
console.log(typeof price);
```

★ ②를 조심하세요. : number 라고 적혀 있다고 number 가 아닙니다. (문제 13에서 직접 돌려 확인합니다)

기대 출력 4000
string

기대 검사 결과: 조용함

```
{
```

두 줄을 출력하세요

```
}
```

문제 7 (개념02 섹션5)

npx tsc --noEmit 을 쳤더니 아무것도 안 나왔습니다. ① 이게 무슨 뜻인가요? “통과” 또는 “실패” 중 하나를 출력하세요. ② 타입 에러를 볼 수 있는 곳은 둘뿐입니다(개념02 정리 3번).

그 둘을 적힌 그대로 순서대로 출력하세요.

기대 출력 통과
VS Code 빨간 밑줄
npx tsc --noEmit

기대 검사 결과: 조용함

```
{
```

세 줄을 출력하세요

```
}
```

문제 8 (개념03 섹션1)

아래 코드는 검사에서 걸립니다.

- ① npm run check 를 쳐서 에러를 봅니다. ② 괄호 안 첫 숫자가 ‘몇 번째 줄’ 인지입니다. 그 줄로 갑니다.
③ 그 줄을 고쳐서 25 가 나오게 하세요.

힌트: 따옴표가 붙어 있으면 문자열입니다.

기대 출력 25

기대 검사 결과: 조용함

```
{
```

걸리는 줄을 찾아 고치세요

```
const seatCount: number = "24";
  console.log(seatCount + 1);
}
```

문제 9 (응용 · 개념03 섹션2)

아래 에러 메시지에서 '내가 넣은 것' 과 '들어가야 하는 것' 을 순서대로 출력하세요.

```
error TS2322: Type 'boolean' is not assignable to type 'string'.
```

기대 출력 내가 넣은 것: boolean
 들어가야 하는 것: string

기대 검사 결과: 조용함

```
{
```

두 줄을 출력하세요

```
}
```

문제 10 (응용 · 개념01 섹션2)

아래 코드는 검사에서 걸립니다. 걸리는 줄이 몇 번인지 찾아서 값이 올바르게 나오도록 고치세요.

```
const stockText = "12";
const sold = 5;
const left = stockText - sold;
```

기대 출력 7

기대 검사 결과: 조용함

```
{
  const stockText = "12";
  const sold = 5;
```

left 에 남은 수량이 담기게 하세요

```
const left: number = 0;
console.log(left);
}
```

문제 11 (응용 · 개념03 섹션2)

아래 코드에서 나올 에러 번호를 출력하세요. TS 까지 붙여서 쓰세요.

```
const score: number = 90;
console.log(score.toUpperCase());
```

힌트: 없는 이름을 부른 것입니다. 비슷한 이름 제안은 안 나옵니다.

기대 출력 TS2339

기대 검사 결과: 조용함

```
{
```

에러 번호를 출력하세요

}

문제 12 ([도전] · 개념01~03 종합)

아래는 주문 정보입니다. 세 가지를 한 번에 하세요. ① 세 변수에 알맞은 타입을 붙인다 ② 총액(단가 × 수량 + 배송비)을 number 로 구한다 ③ 결과를 "총액: 11000원" 형태로 출력한다

주의: quantityText 는 입력창에서 온 값이라 문자열입니다.

기대 출력 총액: 11000원

기대 검사 결과: 조용함

{

타입을 붙이고 총액을 구해 출력하세요

```
const unitPrice = 4000;
const quantityText = "2";
const shipping = 3000;
}
```

문제 13 (에러 확인 · 개념02)

아래 두 줄의 // 를 지우고 저장한 뒤,

① node 연습문제.ts 로 실행해 보세요 → 무슨 일이 일어나나요? ② npm run check 로 검사해 보세요 → 무슨 일이 일어나나요?

둘의 차이를 확인하는 것이 이 문제의 전부입니다. 확인했으면 다시 // 를 붙이세요.

{

```
const openHour: number = "아홉시";
console.log("여는 시간:", openHour);
```

}

개념02 부속 — 일부러 틀린 파일

★ 이 파일은 타입이 틀린 채로 두었습니다. 고치지 마세요.

이 파일로 확인할 것은 딱 하나입니다.

node 로 돌리면 잘 돌아가고,

tsc 로 검사하면 걸린다.

두 명령을 순서대로 쳐 보세요. (TS자료 폴더에서)

1) node 01_타입스크립트_시작하기/틀린예제/타입이_틀린_파일.ts

2) npx tsc --noEmit -p 01_타입스크립트_시작하기/틀린예제

1번은 결과가 나오고, 2번은 에러 3개가 나옵니다. 같은 파일인데도 그렇습니다.

틀린 곳 1: 숫자 자리에 문자열

이 코드는 검사에서 걸립니다

TS2322

Type 'string' is not assignable to type 'number'.

```
const price: number = "4000";
```

```
console.log("가격:", price);
```

출력 가격: 4000

따옴표가 붙은 “4000” 이 그대로 들어 있습니다. 숫자가 아닙니다. 그런데도 화면에는 4000 이라고 나옵니다. 눈으로는 구별이 안 됩니다.

```
console.log("정말 숫자인가:", typeof price);
```

출력 정말 숫자인가: string

typeof 로 찍어 봐야 비로소 문자열인 것이 드러납니다. : number 라고 적어 놓은 것이 무색합니다. node 는 그 표기를 지우고 실행하기 때문입니다.

틀린 곳 2: 없는 이름

```
const cafe = { name: "봄날카페", price: 4500 };
```

이 코드는 검사에서 걸립니다

TS2551

Property 'pirce' does not exist on type '{ name: string; price: number; }'. Did you mean 'price'?

```
console.log("메뉴 가격:", cafe.pirce);
```

출력 메뉴 가격: undefined

없는 이름을 읽으면 `undefined` 입니다. 에러가 아닙니다. 그냥 `undefined` 입니다. 이 값으로 계산을 하면 그때서야 `NaN` 이 되어 이상한 화면이 나옵니다.

틀린 곳 3: 없을 수도 있는 값을 그냥 쓰기

```
const numbers = [10, 20, 30];
const big = numbers.find((n) => n > 100);
```

이 코드는 검사에서 걸립니다

TS18048

'big' is possibly 'undefined'.

```
console.log("100보다 큰 수 + 1 =", big + 1);
```

출력 100보다 큰 수 + 1 = NaN

`undefined + 1` 은 `NaN` 입니다. 이것도 에러가 아닙니다. 화면에 `NaN` 이 찍히고 나서야 뭔가 잘못됐다는 걸 알게 됩니다.

여기까지 정리

```
console.log("---");
```

출력 ---

```
console.log("이 파일은 끝까지 다 돌았습니다. 타입이 세 군데나 틀렸는데도요.");
```

출력 이 파일은 끝까지 다 돌았습니다. 타입이 세 군데나 틀렸는데도요.

기본 타입과 추론

OPEN 02_기본_타입과_추론

```
const cafeName: string = "봄날카페";  
const seatCount: number = 24;  
const isOpen: boolean = true;  
const price: number = 4500;
```

01 기본 타입

02 추론: 대부분은 안 적어도 된다

03 any 는 왜 금지인가 ★ 이 자료의 규칙 하나

- 연습문제

기본 타입

RUN node 개념01_기본_타입.ts

검사: npm run typecheck

01단원에서 : number 와 : string 을 봤습니다. 이 파일에서는 실제로 쓰게 될 기본 타입을 한 번에 정리합니다.

개수가 적습니다. string · number · boolean 셋 + 배열이 전부입니다.

string · number · boolean

섹션 1

이 셋이 전체의 90% 입니다.

```
const cafeName: string = "봄날카페";
const seatCount: number = 24;
const isOpen: boolean = true;

console.log(cafeName, seatCount, isOpen);
```

출력 봄날카페 24 true

이름은 전부 소문자로 시작합니다. 이것만 지키면 됩니다.

```
string    글자
number   숫자 (정수·소수 구분이 없습니다)
boolean   true / false
```

JS자료 01단원에서 typeof 로 찍어 보던 그 이름과 똑같습니다.

```
console.log(typeof cafeName, typeof seatCount, typeof isOpen);
```

출력 string number boolean

number 는 정수와 소수를 나누지 않습니다. 둘 다 number 입니다.

```
const price: number = 4500;
const rate: number = 0.1;
console.log(price * rate);
```

출력 450

▹ 직접 해보기 1

여러분 이름(string), 나이(number),

커피를 좋아하는지(boolean) 를 타입과 함께 만들어 출력해 보세요.

대문자로 시작하는 String 은 쓰면 안 된다

섹션 2

문법상으로는 이렇게도 써집니다.

```
const wrong: String = "이것도 되기는 합니다";
console.log(wrong);
```

출력 이것도 되기는 합니다

되기는 하는데 쓰면 안 됩니다. 다른 것이기 때문입니다.

```
string    우리가 쓰는 글자값
String    글자를 감싼 '객체' ← JS자료 06단원에서 잠깐 나온 그것
```

문제가 언제 생기냐면 이럴 때입니다.

이 코드는 검사에서 걸립니다

TS2322

```
const plain: string = wrong;
```

Type 'String' is not assignable to type 'string'.

String 은 string 자리에 못 들어갑니다. 반대는 됩니다.

규칙은 하나입니다 – 타입 이름은 전부 소문자.
Number, Boolean 도 마찬가지입니다.

▹ 직접 해보기 2

const n: Number = 3; 을 만들고

```
const m: number = n; 을 써 보세요. 무슨 에러가 나는지 확인 후 지우세요.
```

배열 — 타입 뒤에 []

섹션 3

배열은 “무엇이 들어 있는 배열인가” 를 적습니다.

```
const menus: string[] = ["아메리카노", "라떼", "카페모카"];
const prices: number[] = [4000, 4500, 5000];

console.log(menus);
```

출력 ['아메리카노', '라떼', '카페모카']

```
console.log(prices.length);
```

출력 3

string[] 은 “문자열이 들어 있는 배열” 이라고 읽습니다. [] 는 “여러 개” 라는 뜻입니다.

다른 종류를 넣으면 그 자리에서 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```
const wrongMenus: string[] = ["아메리카노", 4500];
```

Type 'number' is not assignable to type 'string'.

배열 전체가 아니라 4500 이 있는 자리를 콕 집어 줍니다.

열 번호를 보면 몇 번째 값인지 알 수 있습니다.

배열에 넣을 때도 검사합니다.

```
const newMenus: string[] = ["아메리카노"];
newMenus.push("라떼");
console.log(newMenus);
```

출력 ['아메리카노', '라떼']

이 코드는 검사에서 걸립니다

TS2345

```
newMenus.push(4500);
```

Argument of type 'number' is not assignable to parameter of type 'string'.

push 는 ‘넘기는’ 것이라 TS2345 입니다. 01단원 개념03 ④ 를 보세요.

답을 때는 TS2322, 넘길 때는 TS2345 – 이 구분이 계속 나옵니다.

▸ 직접 해보기 3

boolean 이 들어가는 배열 checkList 를 만들고

`true, false, true` · 를 담아 출력해 보세요.

배열 안의 값을 꺼내 쓸 때

섹션 4

배열에서 꺼낸 값에는 그 배열의 타입이 그대로 붙습니다.

```
const firstMenu = menus[0];
console.log(firstMenu.toUpperCase());
```

출력 아메리카노

`firstMenu` 는 `string` 이라서 `toUpperCase` 를 쓸 수 있습니다. (한글은 대문자가 없어서 그대로 나옵니다)
숫자 배열에서 꺼낸 값에는 문자열 메소드를 못 씁니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(prices[0].toUpperCase());
```

Property 'toUpperCase' does not exist on type 'number'.

`prices` 는 `number[]` 니까 `prices[0]` 은 `number` 입니다.

JS에서는 실행하다 `TypeError` 로 터지던 자리입니다.

배열끼리 섞으면 이렇게 됩니다.

```
const mixed = [1, "둘", 3];
console.log(mixed);
```

출력 [1, '둘', 3]

`mixed` 의 타입은 `(string | number)[]` 가 됩니다. “문자열이거나 숫자인 것들의 배열”이라는 뜻입니다.
세로줄(`|`)이 ‘이거나’ 입니다. 05단원에서 제대로 배웁니다.

이렇게 섞인 배열은 꺼내 써먹기가 번거롭습니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(mixed[0].toUpperCase());
```

Property 'toUpperCase' does not exist on type 'string | number'.

“문자열일 수도 숫자일 수도 있는데 `toUpperCase` 를 쓰겠다고요?” 입니다.

숫자일 때는 없는 기능이니깐요.
→ 배열에는 되도록 한 종류만 넣으세요. 이게 실무 규칙입니다.

▹ 직접 해보기 4

prices 의 첫 번째 값에 100 을 더해 출력해 보세요.

자리마다 종류가 다른 배열 — 튜플

섹션 5

string[] 은 “문자열이 몇 개든” 입니다. 개수도 순서도 정해져 있지 않습니다. 그런데 “첫 번째는 이름, 두 번째는 가격” 처럼 자리가 정해진 경우가 있습니다. 그때는 대괄호 안에 종류를 순서대로 적습니다.

```
const 메뉴한줄: [string, number] = ["아메리카노", 4000];
```

```
console.log(메뉴한줄[0]);
```

출력 아메리카노

```
console.log(메뉴한줄[1] + 500);
```

출력 4500

[0] 은 string, [1] 은 number 로 나옵니다. 자리마다 종류가 다릅니다. (string | number)[] 과는 다릅니다. 그쪽은 꺼낼 때마다 어느 쪽인지 확인해야 합니다. 튜플은 자리로 이미 정해져 있어서 바로 쓸 수 있습니다.

순서를 바꾸면 두 자리 다 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```
에러: TS2322 Type 'string' is not assignable to type 'number'.  
const 뒤집힘: [string, number] = [4000, "아메리카노"];
```

Type 'number' is not assignable to type 'string'.

개수도 약속입니다. 튜플은 길이까지 정해 둔 것입니다.

이 코드는 검사에서 걸립니다

TS2322

```
const 짧음: [string, number] = ["아메리카노"];
```

Type '[string]' is not assignable to type '[string, number]'.

07단원에서 만날 React 의 useState 가 바로 이 모양입니다.

```
const [count, setCount] = useState(0);
//      ^ number      ^ 값을 바꾸는 함수
```

배열처럼 생겼는데 자리마다 종류가 다른 것 — 그게 튜플입니다. 자주 쓰지는 않습니다. 자리가 두셋으로 딱 정해진 것에만 쓰고, 그보다 늘어나면 객체로 이름을 붙이는 편이 낫습니다(04단원).

▹ 직접 해보기

5

[string, number, boolean] 로 주문한줄 을 만들고

세 값을 각각 출력해 보세요. 두 번째 자리에 문자열을 넣으면 어떻게 되나요?

자주 하는 실수

섹션 6

이런 실수를 합니다

타입 이름을 대문자로 쓰기

String / Number / Boolean 은 다른 것입니다. 섹션 2를 보세요. VS Code 가 자동완성으로 대문자를 먼저 보여 줄 때가 있어 잘 걸립니다.

이런 실수를 합니다

배열 타입을 []string 이라고 쓰기

순서가 반대입니다. string[] 입니다. "무엇이" 가 먼저입니다.

이런 실수를 합니다

배열에 여러 종류를 섞어 넣기

문법상 되지만 꺼내 쓸 때마다 걸립니다. 섹션 4 마지막을 보세요.

이런 실수를 합니다

number 에 정수·소수 구분이 있다고 생각하기

int 나 float 같은 것은 없습니다. 전부 number 입니다. (다른 언어를 해 본 분들이 자주 찾습니다)

정리

- 1 기본 타입은 string · number · boolean 셋이면 대부분 끝난다.
- 2 타입 이름은 전부 소문자로 시작한다. String 은 다른 것이니 쓰지 않는다.
- 3 배열은 종류 뒤에 [] 를 붙인다. string[] = 문자열들의 배열.
- 4 배열에서 꺼낸 값에는 그 종류가 그대로 붙는다. number[] 에서 꺼내면 number.
- 5 담을 때 안 맞으면 TS2322, 넘길 때 안 맞으면 TS2345.
- 6 number 에 정수·소수 구분은 없다.

직접 해보기 정답

- 1 `const myName: string = "홍길동"; const myAge: number = 25; const likesCoffee: boolean = true; console.log(myName, myAge, likesCoffee);` // 출력: 홍길동 25 true
- 2 error TS2322: Type 'Number' is not assignable to type 'number'. 재현:

```
const n: Number = 3;
const m: number = n;
```

대문자 Number 를 소문자 number 자리에 넣을 수 없습니다. 반대로 `const n2: Number = 3;` 은 됩니다. 그래서 더 헷갈립니다. 소문자만 쓰면 이 문제는 아예 안 생깁니다.

- 3 `const checkList: boolean[] = [true, false, true]; console.log(checkList);` // 출력: [true, false, true]
- 4 `console.log(prices[0] + 100);` // 출력: 4100

prices[0] 이 number 라서 그냥 더해집니다. menus[0] + 100 이었다면 "아메리카노100" 이 됐을 것입니다. (문자열에 + 는 이어붙이기 — 01단원 개념01 섹션1)

```
5) const 주문한줄: [string, number, boolean] = ["아메리카노", 2, true];
console.log(주문한줄[0]); // 출력: 아메리카노
console.log(주문한줄[1]); // 출력: 2
console.log(주문한줄[2]); // 출력: true
```

두 번째에 문자열을 넣으면 그 자리에서 걸립니다. error TS2322: Type 'string' is not assignable to type 'number'.

재현: `const 잘못: [string, number, boolean] = ["아메리카노", "둘", true];`

추론: 대부분은 안 적어도 된다

```
RUN node 개념02_추론.ts
```

검사: npm run typecheck

지금까지 타입을 꼬박꼬박 적었습니다. 손에 익히려고 그런 것입니다. 그런데 실무 코드를 보면 타입이 생각보다 훨씬 적게 적혀 있습니다.

타입스크립트가 값을 보고 알아서 알아내기 때문입니다. 이것을 '추론' 이라고 합니다.

값이 옆에 있으면 안 적어도 된다

섹션 1

이렇게 적어도

```
const price1: number = 4000;
```

이렇게 적어도

```
const price2 = 4000;
```

둘은 완전히 같습니다. 아래쪽도 number 입니다.

```
console.log(price1, price2);
```

```
출력      4000 4000
```

4000 을 보고 "이건 숫자군" 하고 알아냈기 때문입니다. 사람이 봐도 뻔한 것은 타입스크립트도 압니다.

안 적었어도 검사는 똑같이 합니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(price2.toUpperCase());
```

Property 'toUpperCase' does not exist on type '4000'.

타입을 안 적었다고 검사가 느슨해지는 게 아닙니다.

적어 둔 것과 똑같이 막습니다.

▹ 직접 해보기 1

`const city = "서울";` 을 만들고

`city.toFixed(2)` 를 써 보세요. 무슨 에러가 나는지 확인 후 지우세요.

추론된 타입을 눈으로 보는 두 가지 방법

섹션 2

```
const menu = "아메리카노";
```

방법 1 · VS Code 에서 이름 위에 마우스를 올린다

`const menu`: "아메리카노" 라고 뜹니다. 가장 편한 방법입니다.

방법 2 · 일부러 틀리게 담아 보고 에러 메시지를 읽는다

이 방법은 편집기 없이도 되고, 나중에 복잡한 타입을 볼 때 유용합니다.

이 코드는 검사에서 걸립니다

TS2322

```
const peek: null = menu;
```

Type '"아메리카노"' is not assignable to type 'null'.

`null` 자리에는 아무것도 못 들어가니 반드시 에러가 납니다.

그 메시지의 왼쪽에 '진짜 타입' 이 그대로 적혀 나옵니다.
궁금할 때 쓰는 요령이니 알아 두세요. 확인 후 지웁니다.

```
console.log(menu);
```

출력 아메리카노

▹ 직접 해보기 2

`const stock = [1, 2, 3];` 의 타입이 무엇인지

위 두 방법 중 하나로 확인해 보세요.

`const` 와 `let` 은 추론 결과가 다르다

섹션 3

같은 값을 넣었는데 타입이 다릅니다.

```
const fixed = 5; // 타입: 5
let changing = 5; // 타입: number console.log(fixed, changing);
```

출력 5 5

`const` 는 '5' 라는 값 자체가 타입이 됩니다. 리터럴 타입이라고 부릅니다. 왜냐하면 `const` 는 다시 대입할 수 없으니, 영원히 5 일 것이 확실하기 때문입니다.

`let` 은 나중에 바뀔 수 있으니 넉넉하게 `number` 로 잡습니다.

`let` 은 같은 종류 안에서는 자유롭게 바뀝니다.

```
changing = 10;
console.log(changing);
```

출력 10

하지만 종류를 넘어가면 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```
changing = "열";
```

Type 'string' is not assignable to type 'number'.

처음에 5 를 넣은 순간 "이 변수는 숫자를 담는 자리" 로 정해졌습니다.

JS 에서는 아무 값이나 다시 넣을 수 있던 것이 여기서는 막힙니다.

리터럴 타입이 왜 쓸모가 있는지는 05단원에서 나옵니다. 지금은 "const 는 값 자체가 타입이 된다" 정도만 기억하세요.

▹ 직접 해보기 3

```
let city = "서울"; 을 만든 뒤
```

```
city = "부산"; 과 city = 3; 을 각각 해 보세요. 어느 쪽이 걸리나요?
```

객체와 배열도 알아서 알아낸다

섹션 4

```
const order = {
  menu: "라떼",
  count: 2,
  isHot: true,
};
```

order 의 타입은 이렇게 추론됩니다.

```
{ menu: string; count: number; isHot: boolean; }
```

하나하나 적을 필요가 없습니다.

```
console.log(order.menu, order.count);
```

```
출력      라떼 2
```

없는 이름은 여전히 막힙니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(order.size);
```

Property 'size' does not exist on type '{ menu: string; count: number; isHot: boolean; }'.

적어 두지 않았는데도 “이 객체에 뭐가 있는지”를 정확히 알고 있습니다.

배열도 같습니다.

```
const prices = [4000, 4500, 5000];
console.log(prices.length);
```

```
출력      3
```

prices 의 타입은 number[] 입니다.

이 코드는 검사에서 걸립니다

TS2345

```
prices.push("육천원");
```

Argument of type 'string' is not assignable to parameter of type 'number'.

처음에 숫자만 넣었으니 숫자 배열로 정해졌습니다.

참고 — 객체 안의 객체, 배열 안의 객체도 전부 알아냅니다.

```
const shop = {
  name: "봄날카페",
  menus: [
    { name: "아메리카노", price: 4000 },
    { name: "라떼", price: 4500 },
  ],
};
console.log(shop.menus[0].price);
```

```
출력      4000
```

shop.menus[0].price 까지 자동완성이 됩니다. 아무것도 안 적었는데도요. 이게 추론의 진짜 위력입니다.

▹ 직접 해보기 4

위 shop 에서 두 번째 메뉴의 이름을 출력해 보세요.

그럼 언제 적어야 하나

섹션 5

규칙은 한 줄로 정리됩니다.

초기값이 옆에 있으면 안 적는다. 없으면 적는다.

초기값이 없으면 타입스크립트가 볼 것이 없어서 알아낼 수가 없습니다.

적어야 하는 경우 1 · 나중에 값이 정해지는 변수

```
let selectedMenu: string;
selectedMenu = "카페모카";
console.log(selectedMenu);
```

출력 카페모카

적어야 하는 경우 2 · 처음엔 비어 있고 나중에 채우는 것

05단원에서 null 을 배우면 이 경우를 제대로 다룹니다.

적어야 하는 경우 3 · 함수의 매개변수 — 03단원에서 배웁니다.

이건 예외 없이 무조건 적어야 합니다. 추론할 근거가 아예 없기 때문입니다.

적어야 하는 경우 4 · 객체의 모양을 미리 정해 둘 때 — 04단원에서 배웁니다.

그리고 이런 경우에도 '일부러' 적습니다. 값이 옆에 있어도, 내가 무엇을 기대하는지 남겨 두고 싶을 때입니다.

```
const total: number = 4000 + 4500;
console.log(total);
```

출력 8500

01단원 개념01 섹션3에서 본 그것입니다. 적어 둔 자리에서만 걸리니, 중요한 계산 결과에는 적어 두는 편이 낫습니다.

▹ 직접 해보기 5

let count; 라고만 쓰고 count = 3; 을 한 뒤

count.toUpperCase() 를 써 보세요. 걸리나요? 왜 그럴까요?

자주 하는 실수

섹션 6

이런 실수를 합니다

모든 곳에 타입을 적으려고 하기

초보자가 가장 많이 하는 것입니다. 코드만 길어지고 얻는 게 없습니다. `const price: number = 4000; 은 : number` 가 하는 일이 없습니다.

이런 실수를 합니다

타입을 안 적으면 검사를 안 한다고 생각하기

섹션 1을 보세요. 안 적어도 똑같이 막습니다.

이런 실수를 합니다

let 에 처음 넣은 값의 종류를 나중에 바꾸려 하기

```
let x = 5; 뒤에 x = "다섯"; 은 안 됩니다.
```

여러 종류를 담고 싶으면 05단원의 유니온이 필요합니다.

이런 실수를 합니다

const 의 타입이 number 라고 생각하기

`const n = 5` 의 타입은 `number` 가 아니라 `5` 입니다. 보통은 차이가 없지만 05단원에서 이 차이가 중요해집니다.

정리

- 1 값이 옆에 있으면 타입스크립트가 알아서 알아낸다. 이것이 추론이다.
- 2 안 적어도 검사는 똑같이 한다. 적은 것과 효력이 같다.
- 3 추론 결과를 보려면 마우스를 올리거나, 일부러 `null` 에 담아 에러를 읽는다.
- 4 `const` 는 값 자체가 타입(리터럴 타입), `let` 은 넉넉한 타입이 된다.
- 5 객체·배열·중첩된 것까지 전부 추론한다. 자동완성도 그대로 된다.
- 6 적어야 하는 곳은 '초기값이 없는 자리' 다. 그리고 함수 매개변수는 예외 없이 적는다(03단원).

직접 해보기 정답

1) error TS2551: Property 'toFixed' does not exist on type '"서울"'. Did you mean 'fixed'?
`toFixed` 는 숫자에만 있습니다. Did you mean 'fixed'? 는 무시하세요. 문자열에 정말로 `fixed` 라는 옛날

기능이 있어서 비슷한 이름으로 골라 준 것입니다. 제안이 항상 옳지는 않습니다. 재현:

```
const city = "서울";
console.log(city.toFixed(2));
```

메시지에 'string' 이 아니라 ""서울"" 이라고 나오는 것에 주목하세요. `const` 라서 리터럴 타입이 된 것입니다 (섹션 3).

2 number[] 입니다. 마우스를 올리면 `const stock: number[]` 라고 뜹니다. `const` 인데 왜 리터럴이 아니냐면, 배열 '안' 의 값은 나중에 바뀔 수 있기 때문입니다. `const` 가 막는 것은 `stock` 자체를 다른 배열로 바꾸는 것뿐입니다. (JS자료 06단원에서 배운 그 내용입니다)

3 city = 3; 만 걸립니다.

error TS2322: Type 'number' is not assignable to type 'string'. `let` 이라 타입이 string 으로 넉넉하게 잡혔고, 문자열끼리는 자유롭습니다. 재현:

```
let city = "서울";
city = "부산";
city = 3;
```

4 console.log(shop.menus[1].name); // 출력: 라떼

5 걸립니다. error TS2339: Property 'toUpperCase' does not exist on type 'number'. 재현:

```
let count;
count = 3;
console.log(count.toUpperCase());
```

타입을 안 적었지만 `count = 3;` 을 보고 그 뒤부터는 `number` 로 취급합니다. 타입스크립트는 코드가 흘러가는 순서를 따라가며 타입을 좁혀 나갑니다. 이 성질이 05단원의 핵심이 됩니다.

any 는 왜 금지인가 ★ 이 자료의 규칙 하나

RUN node 개념03_any는_왜_금지인가.ts

검사: npm run typecheck

타입 에러가 나면 반드시 이런 유혹이 옵니다.

"귀찮은데 any 라고 적으면 에러가 없어진대"

없어집니다. 진짜 없어집니다. 그래서 위험합니다.

이 자료에는 규칙이 하나 있습니다.

★ any 를 쓰지 않습니다.

이 파일은 그 이유를 보여 줍니다. 이유를 알아야 안 쓰게 됩니다.

any 는 타입 검사를 '끄는' 스위치다

섹션 1

```
const data: any = { name: "봄날카페" };
```

any 를 붙이면 이 값에 대해서는 타입스크립트가 아무것도 안 봅니다. 아래 줄들이 전부 검사를 통과합니다. 하나도 안 걸립니다.

코드	출력
<code>console.log(data.name);</code>	▶ 봄날카페
<code>console.log(typeof data.존재하지않는속성);</code>	▶ undefined

없는 이름인데 아무 말이 없습니다. TS2339 가 안 납니다.

```
const asNumber: number = data;
console.log(typeof asNumber);
```

출력 object

객체를 number 자리에 넣었는데 통과했습니다. : number 라고 적어 놓은 것이 완전히 무의미해졌습니다.

any 는 "이 값은 검사하지 마세요" 라는 뜻입니다. 타입을 적는 것이 아니라, 타입 검사를 끄는 것입니다.

▸ 직접 해보기

1

`const x: any = "글자";` 를 만들고

`x.toFixed(2)` 를 써 보세요. 검사는 통과하나요? 실행하면 어떻게 되나요?

검사를 꺾으니 실행할 때 터진다

섹션 2

01단원에서 "타입스크립트는 실행 전에 잡아 준다" 고 배웠습니다. `any` 를 쓰면 그 보호가 사라지고, JS 로 되돌아갑니다.

```
try {
  console.log(data.price.toFixed(2));
} catch (e) {
  console.log("터졌습니다:", String(e));
}
```

출력 터졌습니다: TypeError: Cannot read properties of undefined (reading 'toFixed')

`data` 에 `price` 가 없으니 `data.price` 는 `undefined` 이고, `undefined` 에 `.toFixed` 를 부르니 터졌습니다.

중요한 것은 이겁니다 —

이 줄은 타입 검사를 통과했습니다.
`npx tsc --noEmit` 이 조용했습니다.
 그런데 실행하니 터졌습니다.

01단원 개념02에서 본 "돌아가는데 검사는 걸린다" 의 정반대입니다. 이번엔 "검사는 통과하는데 돌리면 터진다" 입니다. 이쪽이 훨씬 나쁩니다.

▸ 직접 해보기

2

`위 try` 를 지우고 그냥 `console.log(data.price.toFixed(2))` 를

써 보세요. 프로그램이 어디까지 돌아가는지 확인한 뒤 되돌리세요.

any 는 옆으로 번진다

섹션 3

이게 `any` 의 진짜 문제입니다. 한 군데만 끄고 끝나지 않습니다.

```
const settings: any = { theme: "dark", fontSize: 14 };
```

`any` 에서 점(`.`)으로 꺼낸 값도 `any` 가 됩니다.

```
const theme = settings.theme; // 이것도 any
const upper = theme.toUpperCase(); // 이것도 any console.log(upper);
```

출력 DARK

upper 는 실제로는 문자열인데 타입은 any 입니다. 그래서 이런 것도 통과합니다.

```
console.log(upper.toFixed === undefined);
```

출력 true

문자열에 toFixed(숫자 전용)가 있는지 물어봤는데 아무도 안 막았습니다.

아무리 깊이 들어가도 마찬가지입니다.

```
console.log(typeof settings.없는것?.더없는것);
```

출력 undefined

settings.없는것.더없는것 이라고 써도 검사는 통과합니다. (실행하면 터지니까 여기서는 ?. 를 붙여 두었습니다)

그림으로 보면 이렇습니다.

```
settings(any) —.theme—> theme(any) —.toUpperCase()—> upper(any) —> ...
  ↳ 여기서 한 번 꺾더니 점을 따라 계속 꺼진 채로 흘러갑니다
```

처음 한 줄만 any 였는데, 그 값을 파고드는 코드 전체가 검사 밖으로 나갑니다. 그래서 "여기 한 군데만" 이 안 됩니다.

다만, 번짐이 끊기는 자리가 있습니다 ———

계산을 거치면 결과는 any 가 아니라 제대로 된 타입이 됩니다.

```
const doubled = settings.fontSize * 2; // any 가 아니라 number
const label = "테마: " + settings.theme; // any 가 아니라 string console.log(doubled, label);
```

출력 28 테마: dark

* 의 결과는 무조건 숫자이고, 문자열에 + 한 결과는 무조건 문자열이라 타입스크립트가 확신할 수 있기 때문입니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(doubled.toUpperCase());
```

Property 'toUpperCase' does not exist on type 'number'.

doubled 는 number 라서 여기서는 정상적으로 막힙니다.

"any 가 무조건 끝까지 번진다" 는 아닙니다.
점(.)을 따라갈 때 번지고, 계산을 거치면 끊깁니다.

▹ 직접 해보기 3

settings 의 : any 를 지우고 저장해 보세요.

upper.toFixed === undefined 줄이 걸리는지 확인한 뒤 되돌리세요.

안 적었는데 any 가 몰래 생기는 곳

섹션 4

any 라고 쓴 적이 없어도 any 가 들어오는 자리가 있습니다. 가장 흔한 것이 `JSON.parse` 입니다.

```
const parsed = JSON.parse('{ "name": "봄날카페", "seats": 24 }');
```

parsed 의 타입은 any 입니다. JSON 문자열 안에 뭐가 들었는지는 실행해 봐야 알 수 있으니,
타입스크립트가 알아낼 방법이 없어서 any 로 둔 것입니다.

코드	▶ 출력
<code>console.log(parsed.name);</code>	▶ 봄날카페
<code>console.log(parsed.아무거나 === undefined);</code>	▶ true

없는 이름인데 통과합니다. any 니까요.

서버에서 받아온 데이터가 바로 이 자리입니다. fetch 로 받은 것을 .json() 하면 그것도 마찬가지입니다
(07단원에서 만납니다).

그래서 이렇게 하는 것이 실무 방식입니다.

```
받자마자 "이런 모양일 것이다" 를 적어 준다
```

그 '모양을 적는 법' 이 04단원입니다.

▹ 직접 해보기 4

parsed.seats + 10 을 출력해 보세요.

parsed.seats 가 문자열이었어도 이 줄이 통과할까요?

“정말 뭐가 올지 모르는” 자리에는 any 말고 unknown 을 씁니다.

```
const mystery: unknown = "사실은 글자입니다";
```

unknown 은 any 와 반대입니다. 확인하기 전까지 아무것도 못 하게 막습니다.

이 코드는 검사에서 걸립니다

TS18046

```
console.log(mystery.length);
```

'mystery' is of type 'unknown'.

“뭔지 모른다면서 length 를 쓰시겠다고요?” 라고 막습니다.

any 였다면 그냥 통과했을 자리입니다.

담는 것도 막습니다.

이 코드는 검사에서 걸립니다

TS2322

```
const text: string = mystery;
```

Type 'unknown' is not assignable to type 'string'.

그럼 unknown 은 어떻게 쓰나 — 확인하고 나서 씁니다.

```
if (typeof mystery === "string") {
  console.log(mystery.length);
}
```

출력 9

typeof 로 확인한 안쪽에서는 mystery 가 string 인 것을 타입스크립트가 압니다. 이것을 ‘타입 좁히기’ 라고 하고 05단원의 주제입니다.

차이를 한 줄로 하면 이렇습니다.

any	=	검사하지 마세요	(위험을 넘김)
unknown	=	확인한 다음에 쓰겠습니다	(위험을 막음)

▫ 직접 해보기

5

mystery 에 숫자 42 를 넣고 위 if 를 그대로 두면

무엇이 출력될지 예상한 뒤 확인해 보세요.

그럼 any 는 절대 안 쓰나

섹션 6

(✎ 없음: 규칙의 예외를 설명하는 섹션입니다. 해 볼 것이 없습니다) 딱 한 자리에서 씁니다. 08단원의 '기존 JS 프로젝트를 옮길 때' 입니다.

수천 줄짜리 JS 를 하루아침에 다 고칠 수는 없으니, 일단 any 로 막아 두고 한 파일씩 제대로 고쳐 나가는 방법을 씁니다. 그때도 "빔을 지는 것" 이라는 표시를 남깁니다.

그 경우가 아니라면, any 를 쓰고 싶어질 때는 대개 이 셋 중 하나입니다.

- 1 04단원의 '모양 적기' 를 아직 안 배운 것
- 2 05단원의 '좁히기' 를 아직 안 배운 것
- 3 06단원의 '제네릭' 을 아직 안 배운 것

세 개 다 이 자료 안에 있습니다. any 없이 갈 수 있습니다.

자주 하는 실수

섹션 7

이런 실수를 합니다

에러를 없애려고 any 를 붙이기

에러가 없어진 게 아니라 검사가 꺼진 것입니다. 문제는 그대로 있습니다. 섹션 2에서 본 것처럼 실행할 때 터집니다.

이런 실수를 합니다

"한 군데만 any 니까 괜찮겠지"

섹션 3을 보세요. 그 값을 쓰는 코드 전체로 번집니다.

이런 실수를 합니다

JSON.parse 결과를 그냥 쓰기

any 라는 자각 없이 쓰는 가장 흔한 경우입니다. 서버 데이터는 04단원에서 모양을 적어 주고 씁니다.

이런 실수를 합니다

any 와 unknown 을 같은 것으로 알기

정반대입니다. any 는 다 열고, unknown 은 다 막습니다.

정리

- 1 any 는 타입을 적는 것이 아니라 타입 검사를 끄는 스위치다.
- 2 검사를 켜오니 실행할 때 터진다. "검사는 통과하는데 돌리면 터진다" 가 된다.
- 3 any 는 점(.)을 따라갈 때 번진다. any 에서 꺼낸 값도 any 다. 다만 계산을 거치면 끊긴다 — any * 2 는 number, "글자" + any 는 string.
- 4 `JSON.parse` 결과처럼 안 적었는데 any 인 자리가 있다. 서버 데이터가 대표적이다.
- 5 정말 모를 때는 any 말고 unknown. 확인한 뒤에 쓰게 강제한다.
- 6 any 를 쓰고 싶어지면 04·05·06단원 중 하나를 아직 안 배운 것이다.

직접 해보기 정답

- 1 검사는 통과합니다. `npx tsc --noEmit` 이 조용합니다. 실행하면 터집니다. `TypeError: x.toFixed is not a function` 문자열에는 `toFixed` 가 없기 때문입니다. any 를 안 썼다면 TS2339 로 실행 전에 잡혔을 자리입니다.
- 2 거기서 멈춥니다. 그 아래 코드는 하나도 안 돌아갑니다. try 로 감싸면 터진 것을 잡아서 계속 갈 수 있지만, 실제 화면에서는 그 지점부터 아무것도 안 그려집니다.
- 3 걸립니다. error TS2551: Property 'toFixed' does not exist on type 'string'. Did you mean 'fixed'? (Did you mean 'fixed'? 는 무시하세요. 문자열에 옛날 fixed 가 있어서 나온 제안입니다) 재현:

```
const settings = { theme: "dark", fontSize: 14 };
const theme = settings.theme;
const upper = theme.toUpperCase();
console.log(upper.toFixed === undefined);
```

: any 를 지우면 `{ theme: string; fontSize: number }` 로 추론되고, theme 이 string, upper 도 string 이 되어 번짐이 사라집니다. 세 글자 지웠을 뿐인데 검사가 되살아납니다.

- 4 출력: 34 통과합니다. `parsed.seats` 가 문자열 "24" 였어도 통과합니다. 그때는 "2410" 이 나왔을 것입니다(문자열 이어붙이기). any 라서 아무도 안 막아 줍니다. 이게 섹션 4의 요점입니다.
- 5 아무것도 출력되지 않습니다. 42 는 문자열이 아니니 if 안으로 안 들어갑니다. unknown 은 이렇게 "확인을 통과한 것만" 쓰게 만듭니다.

기본 타입과 추론

RUN node 연습문제.ts

채점: npm run grade ← 출력을 기대 출력과 맞춰 봅니다

검사: npm run check ← 타입 검사 (일부 문제만 잡습니다)

★ 채점은 npm run grade 로 합니다. 실제 출력을 문제의 '기대 출력' 과 줄 단위로 맞춰 봅니다. 전부 일치하면 다 맞은 것입니다.

npm run check(타입 검사)도 같이 쓰세요. 다만 이쪽은 '일부 문제만' 잡습니다.

138문항 중 21개뿐입니다. "출력하세요" 류는 안 풀어도 타입 에러가 안 나거든요. 그러니 check 가 조용하다고 다 맞은 것이 아닙니다. grade 로 확인하세요. (npm run typecheck 는 개념·정답 파일만 봅니다. 그쪽은 언제나 조용합니다)

- 1 TODO 자리에 코드를 씁니다.
- 2 npm run grade 로 채점합니다. (직접 node 연습문제.ts 로 돌려 봐도 됩니다)
- 3 npm run check 로 타입도 봅니다.
- 4 10분 고민해도 안 되면 연습문제_정답.ts 를 보세요.

문제 1~8은 기본, 9~12는 응용, 13은 [도전]입니다. 문제 14는 에러를 직접 보는 문제라 맨 뒤에 있습니다.

```
console.log("=== 02단원 연습문제 ===");
```

문제 1 (개념01 섹션1)

세 변수에 알맞은 타입을 붙이세요.

【이번에 배울 방법】 — 이름 뒤에 : 과 종류

```
const cafeName: string = "봄날카페";
```

기대 출력 라떼 4500 true

기대 검사 결과: 조용함

```
{
```

세 줄에 타입을 붙이세요

```
const menuName = "라떼";
const price = 4500;
const isIced = true;
console.log(menuName, price, isIced);
}
```

문제 2 (개념01 섹션3)

배열 두 개에 타입을 붙이세요. "무엇이" 가 먼저, [] 가 뒤입니다.

기대 출력 3 2

기대 검사 결과: 조용함

```
{
```

두 줄에 타입을 붙이세요

```
const sizes = ["S", "M", "L"];
const stocks = [12, 5];
console.log(sizes.length, stocks.length);
}
```

문제 3 (개념01 섹션4)

아래 배열에서 첫 번째 값을 꺼내 100 을 더한 값을 출력하세요.

기대 출력 4100

기대 검사 결과: 조용함


```
{
  const prices: number[] = [4000, 4500, 5000];
```

첫 번째 값에 100 을 더해 출력하세요

```
}
```

문제 4 (개념02 섹션1)

아래 세 줄에서 타입 표기를 지워도 되는 것은 지우세요. (값이 옆에 있으면 안 적어도 됩니다) 지운 뒤에도 검사가 조용해야 합니다.

기대 출력 봄날카페 24

기대 검사 결과: 조용함

```
{
```

지워도 되는 표기를 지우세요

```
const shopName: string = "봄날카페";
const seats: number = 24;
console.log(shopName, seats);
}
```

문제 5 (개념02 섹션2)

아래 변수의 추론된 타입이 무엇인지 알아내서, 그 이름을 문자열로 출력하세요. (마우스를 올리거나, `null` 에 담아 보는 방법을 쓰세요)

기대 출력 `number[]`

기대 검사 결과: 조용함

```
{
  const counts = [1, 2, 3];
```

`counts` 의 타입 이름을 문자열로 출력하세요

```
}
```

문제 6 (개념02 섹션3)

아래 두 줄의 추론된 타입을 각각 알아내서 순서대로 출력하세요.

```
const a = "커피";
let b = "커피";
```

기대 출력 "커피"
 string

기대 검사 결과: 조용함

```
{
```

두 줄을 출력하세요

```
}
```

문제 7 (개념02 섹션4)

아래 객체에서 두 번째 메뉴의 가격을 출력하세요. 타입은 적지 마세요.

기대 출력 4500

기대 검사 결과: 조용함

```
{
  const shop = {
    name: "봄날카페",
    menus: [
      { name: "아메리카노", price: 4000 },
      { name: "라떼", price: 4500 },
    ],
  };
}
```

두 번째 메뉴의 가격을 출력하세요

```
}
```

문제 8 (개념02 섹션5)

아래 두 줄 중 타입을 '반드시' 적어야 하는 것은 몇 번일까요?

```
1번: const price = 4000;
2번: let selected; ... 나중에 selected = "라떼";
```

기대 출력 2

기대 검사 결과: 조용함

```
{
```

번호를 출력하세요

```
}
```

문제 9 (응용 · 개념03 섹션1)

아래 코드는 검사를 통과합니다. 왜 통과하는지 이유를 한 단어로 출력하세요.

```
const data: any = "글자입니다";
const n: number = data;
```

기대 출력 any

기대 검사 결과: 조용함

```
{
```

이유를 출력하세요

```
}
```

문제 10 (응용 · 개념03 섹션3)

any 는 점(.)을 따라갈 때는 번지고, 계산을 거치면 끊깁니다. 아래 두 값 중 '번짐이 끊겨서 number 가 되는 것' 은 몇 번일까요?

```
1번: const x = settings.fontSize;
2번: const y = settings.fontSize * 2;
```

기대 출력 2

기대 검사 결과: 조용함

```
{
```

번호를 출력하세요

}

문제 11 (응용 · 개념03 섹션4)

`JSON.parse`의 결과는 `any`라서 아무 이름이나 다 통과합니다. 아래 코드는 지금 검사도 통과하는데 실행하면 `NaN`이 나옵니다.

① `parsed`에 `{ seats: number }` 모양을 적어 주세요. ② 그러면 숨어 있던 오타가 드러납니다. 그것도 고치세요.

기대 출력 34

기대 검사 결과: 조용함

{

`parsed`에 모양을 적고, 드러나는 오타를 고치세요

```
const parsed = JSON.parse('{ "seats":24 }');
console.log(parsed.seat + 10);
}
```

문제 12 (응용 · 개념03 섹션5)

아래 `mystery`는 `unknown`이라 그냥은 못 씁니다. `typeof`로 확인한 뒤에 글자 수를 출력하도록 고치세요.

기대 출력 4

기대 검사 결과: 조용함

{

```
const mystery: unknown = "봄날카페";
```

`typeof` 로 확인한 뒤 글자 수를 출력하세요

```
}
```

문제 13 ([도전] · 개념01~03 종합)

아래 코드에는 `any` 가 하나 있습니다. 세 가지를 하세요. ① : `any` 를 지운다 ② ③ 을 쓰다 보면 걸리는 줄이 생긴다. 그 줄을 올바르게 고친다 ③ 좌석당 매출(총매출 ÷ 좌석수)을 정수로 출력한다

힌트: `seatsText` 는 문자열입니다. 소수점 버리기는 `Math.floor` 를 씁니다.

기대 출력 좌석당 매출: 5000

기대 검사 결과: 조용함

```
{
```

`any` 를 없애고 올바르게 고치세요

```
const shop: any = { revenue: 120000, seatsText: "24" };
}
```

문제 14 (에러 확인 · 개념03 섹션1)

아래 세 줄의 `//` 를 지우고 저장한 뒤,

① `npm run check` 로 검사해 보세요 → 걸릴까요? ② `node 연습문제.ts` 로 실행해 보세요 → 어떻게 될까요?

`any` 를 쓰면 무슨 일이 생기는지 직접 보는 문제입니다. 확인했으면 다시 `//` 를 붙이세요.

```
{
```

```
const value: any = "글자입니다";  
console.log(value.toFixed(2));  
console.log("이 줄은 실행될까요?");
```

```
}
```


함수 타입

OPEN 03_함수_타입

03

```
function greet(name: string) {  
  return "안녕하세요 " + name + "님";  
  console.log(greet("홍길동"));  
  return a + b;  
}
```

01 매개변수와 반환값

02 선택 매개변수와 기본값

03 함수를 값으로 넘기기

- 연습문제

매개변수와 반환값

RUN node 개념01_매개변수와_반환값.ts

검사: npm run typecheck

02단원에서 “초기값이 옆에 있으면 안 적어도 된다” 고 했습니다. 함수의 매개변수는 그 반대입니다. 값이 옆에 없습니다.

```
function greet(name) { ... }
    └─ 여기에 무엇이 들어올지 알 방법이 없다
```

그래서 매개변수는 예외 없이 적습니다. 이 단원에서 제일 중요한 규칙입니다.

매개변수는 무조건 적는다

섹션 1

안 적으면 이렇게 됩니다.

이 코드는 검사에서 걸립니다

TS7006

```
function greetWrong(name) {
  return "안녕하세요 " + name + "님";
}
```

Parameter 'name' implicitly has an 'any' type.

implicitly = “은근슬쩍”, “적지도 않았는데 any 가 됐다” 는 뜻입니다.

02단원 개념03에서 any 를 쓰지 않기로 했는데,
매개변수를 비워 두면 자동으로 any 가 되어 버립니다.
→ 그래서 매개변수만은 반드시 적어야 합니다.

제대로 적으면 이렇습니다.

```
function greet(name: string) {
  return "안녕하세요 " + name + "님";
}
```

```
console.log(greet("홍길동"));
```

출력 안녕하세요 홍길동님

이제 잘못 넘기면 그 자리에서 걸립니다.

이 코드는 검사에서 걸립니다

TS2345

```
console.log(greet(123));
```

Argument of type 'number' is not assignable to parameter of type 'string'.

argument(넘긴 값) 가 parameter(받기로 한 자리) 에 안 맞는다는 뜻입니다.

01단원 개념03 ④ 에서 본 그 예러입니다.

개수가 틀려도 걸립니다.

이 코드는 검사에서 걸립니다

TS2554

```
console.log(greet("홍길동", "추가"));
```

Expected 1 arguments, but got 2.

"1개를 받기로 했는데 2개를 주셨습니다" 입니다.

JS에서는 남는 것을 그냥 버리고 조용히 돌아가던 자리입니다.

▹ 직접 해보기 1

숫자 두 개를 받아 곱한 값을 돌려주는 multiply 를 만들고

multiply(3, 4) 를 출력해 보세요.

반환값은 안 적어도 된다

섹션 2

돌려주는 값은 함수 몸통을 보면 알 수 있으니 추론됩니다.

```
function add(a: number, b: number) {
  return a + b;
}
```

add 의 타입은 (a: number, b: number) ⇒ number 로 추론됩니다. 화살표 오른쪽이 돌려주는 것입니다.

```
const sum = add(3, 4);
console.log(sum);
```

출력 7

그리고 돌려받은 값에도 타입이 붙어 있습니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(add(3, 4).toUpperCase());
```

Property 'toUpperCase' does not exist on type 'number'.

add 가 숫자를 준다는 것을 알기 때문에 막습니다.

JS에서는 실행하다 터지던 자리입니다.

▹ 직접 해보기 2

greet("홍길동")의 결과에 .length를 붙여 출력해 보세요.

걸리나요? 왜 그럴까요?

그래도 반환값을 적으면 좋은 경우

섹션 3

안 적어도 되지만, 적어 두면 '실수를 함수 안에서' 잡을 수 있습니다.

안 적은 경우 · — 실수가 함수 밖으로 새어 나갑니다

```
function getPriceLoose(count: number) {  
  return String(count * 4000); // 실수로 문자열을 돌려줌  
}
```

```
const looseResult = getPriceLoose(2);  
console.log(looseResult, typeof looseResult);
```

출력 8000 string

숫자를 원했는데 문자열이 나왔습니다. 함수는 아무 말도 안 했습니다. 이 값을 쓰는 쪽에 가서야 문제가 드러납니다.

적은 경우 · — 함수 안에서 바로 잡힙니다

이 코드는 검사에서 걸립니다

TS2322

```
function getPriceStrict(count: number): number {  
  return String(count * 4000);  
}
```

Type 'string' is not assignable to type 'number'.

return 하는 그 줄에서 걸립니다.

"number 를 주기로 해 놓고 string 을 주시네요" 입니다.
실수한 자리에서 바로 잡히니 원인을 찾을 필요가 없습니다.

제대로 쓰면 이렇습니다.

```
function getPrice(count: number): number {
  return count * 4000;
}
console.log(getPrice(2));
```

출력 8000

규칙으로 정하면 이렇습니다.

짧고 뻥한 함수 → 안 적어도 된다
길거나 중요한 함수 → 적어 두면 실수를 그 자리에서 잡는다

▸ 직접 해보기 3

getPrice 의 : number 는 그대로 두고

몸통을 return "공짜"; 로 바꿔 보세요. 어디서 걸리나요? 확인 후 되돌리세요.

void — 돌려주는 것이 없는 함수

섹션 4

화면에 찍기만 하고 아무것도 안 돌려주는 함수가 있습니다.

```
function printReceipt(menu: string, price: number): void {
  console.log("주문: " + menu);
  console.log("금액: " + price + "원");
}

printReceipt("라떼", 4500);
```

출력 주문: 라떼
 금액: 4500원

void 는 "돌려주는 것이 없다" 는 뜻입니다. return 이 없는 함수는 안 적어도 void 로 추론됩니다. 적는 것은 선택입니다.

void 함수의 결과를 받아서 쓰려고 하면 막아 줍니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(printReceipt("라떼", 4500).length);
```

Property 'length' does not exist on type 'void'.

아무것도 안 돌려주는 함수의 결과를 쓰려고 한 것입니다.

JS에서는 undefined가 되어 "Cannot read properties of undefined"로 실행 중에 터지던 자리입니다.

▫ 직접 해보기 4

printReceipt의 : void를 : string으로 바꿔 보세요.

무슨 에러가 나는지 확인한 뒤 되돌리세요.

화살표 함수도 똑같다

섹션 5

JS자료 05단원에서 배운 화살표 함수도 적는 자리가 같습니다.

```
const double = (n: number): number => n * 2;  
console.log(double(21));
```

출력 42

반환 타입을 생략하면 이렇게 됩니다. 이쪽이 더 흔합니다.

```
const triple = (n: number) => n * 3;  
console.log(triple(5));
```

출력 15

적는 자리를 그림으로 보면 이렇습니다.

```
const double = (n: number): number => n * 2;  
               |      |  
               매개변수 반환값  
               (필수)   (선택)
```

매개변수가 없어도 괄호는 있어야 합니다.

```
const now = (): string => "지금";  
console.log(now());
```

출력 지금

▸ 직접 해보기

5

문자열을 받아 "안녕, OO!" 를 돌려주는

화살표 함수 hello 를 만들어 hello("봄날") 을 출력해 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

매개변수 타입을 빼먹기

가장 흔합니다. TS7006 이 나오면 100% 이것입니다. 에러 메시지에 implicitly 라는 단어가 보이면 매개변수를 확인하세요.

이런 실수를 합니다

매개변수와 반환값의 콜론을 헛갈리기

```
function f(a: number): string { }
```

└─ 매개변수 └─ 반환값 반환값 콜론은 닫는 괄호 '}' 입니다.

이런 실수를 합니다

반환 타입을 적어 놓고 return 을 안 쓰기

```
function f(): number { console.log(1); } 은
```

TS2355 A `function` whose declared type is neither `'undefined'`, `'void'`, nor `'any'` must `return` a value. 로 걸립니다.

이런 실수를 합니다

void 를 '아무 타입이나' 로 알기

void 는 "돌려줄 것이 없다" 입니다. any 와는 정반대입니다.

정리

- 1 매개변수는 예외 없이 적는다. 안 적으면 TS7006 으로 은근슬쩍 any 가 된다.
- 2 반환값은 몸통을 보고 추론되므로 안 적어도 된다.
- 3 반환값을 적어 두면 실수를 함수 '안' 에서 잡는다. 중요한 함수에는 적는다.
- 4 돌려줄 것이 없으면 void 다. void 함수의 결과는 쓸 수 없다.
- 5 화살표 함수도 적는 자리가 같다. 매개변수는 괄호 안, 반환값은 괄호 뒤.
- 6 넘긴 값이 안 맞으면 TS2345, 개수가 안 맞으면 TS2554.

직접 해보기 정답

```
1) function multiply(a: number, b: number) {  
  
    return a * b;  
  
}  
console.log(multiply(3, 4));           // 출력: 12
```

2) 걸리지 않습니다.

```
console.log(greet("홍길동").length);   // 출력: 10
```

greet 는 문자열을 돌려주니 .length 를 쓸 수 있습니다. "안녕하세요 홍길동님" 이 10글자입니다(띄어쓰기 포함). 반대로 add(3,4).length 는 TS2339 로 걸립니다. 숫자에는 length 가 없으니까요.

3) return 하는 줄에서 걸립니다. error TS2322: Type 'string' is not assignable to type 'number'.
재현:

```
function getPrice(count: number): number { return "공짜"; }
```

함수를 쓰는 쪽이 아니라 함수 '안' 에서 잡히는 것이 핵심입니다. 이게 반환 타입을 적는 이유입니다.

4) error TS2355: A function whose declared type is neither 'undefined', 'void', nor 'any' must return a value. "string 을 주기로 해 놓고 아무것도 안 돌려주시네요" 입니다. 재현:

```
function printReceipt(menu: string): string { console.log(menu); }
```

```
5) const hello = (name: string) => "안녕, " + name + "!";  
console.log(hello("봄날"));           // 출력: 안녕, 봄날!
```


선택 매개변수와 기본값

RUN node 개념02_선택_매개변수와_기본값.ts

검사: npm run typecheck

개념01에서 “개수가 틀리면 TS2554 로 걸린다” 고 했습니다. 그런데 실제로는 “있어도 되고 없어도 되는” 값이 자주 있습니다.

주문할 때 메모는 남겨도 되고 안 남겨도 된다
할인율은 대부분 0 이지만 가끔 다르다

이 파일은 그 두 가지를 다루는 방법입니다.

? — 없어도 되는 매개변수

섹션 1

이름 뒤에 ? 를 붙이면 “안 남겨도 된다” 가 됩니다.

```
function order(menu: string, memo?: string) {
  console.log("주문:", menu);
  console.log("메모:", memo);
}
```

```
order("아메리카노", "얼음 적게");
```

출력 주문: 아메리카노
 메모: 얼음 적게

```
order("라떼");
```

출력 주문: 라떼
 메모: undefined

두 번째 호출에서 memo 를 안 넘겼는데 에러가 안 났습니다. ? 가 붙어 있으니 개수 검사가 통과한 것입니다.

그리고 안 넘긴 memo 는 `undefined` 가 됩니다. JS자료에서 “없는 값은 `undefined`” 라고 배운 그대로입니다.

▸ 직접 해보기 1

이름을 받고 직급은 선택으로 받는 함수 intro 를 만들어

intro("홍길동") 과 intro("홍길동", "대리") 를 각각 불러 보세요.

? 의 대가 — undefined 일 수 있다

섹션 2

? 를 붙이면 편해지는 대신, 그 값의 타입이 바뀝니다.

```
memo?: string → memo 의 타입은 string | undefined
```

“문자열이거나, 아예 없거나” 라는 뜻입니다. 세로줄 (|)은 ‘이거나’ 입니다. 05단원에서 제대로 배웁니다.
그래서 그냥 쓰려고 하면 막힙니다.

이 코드는 검사에서 걸립니다

TS18048

```
function orderWrong(menu: string, memo?: string) {  
  console.log(menu, memo.length);  
}
```

'memo' is possibly 'undefined'.

“안 넘길 수도 있다고 해 놓고 .length 를 쓰시겠다고요?” 입니다.

안 넘기면 undefined 이고, undefined 에는 length 가 없으니까요.
JS 에서는 이 자리가 "Cannot read properties of undefined" 였습니다.

그럼 어떻게 쓰나 — 있는지 확인하고 씁니다.

```
function orderSafe(menu: string, memo?: string) {  
  if (memo) {  
    console.log(menu, "/ 메모", memo.length, "글자");  
  } else {  
    console.log(menu, "/ 메모 없음");  
  }  
}
```

```
orderSafe("아메리카노", "얼음 적게");
```

출력 아메리카노 / 메모 5 글자

```
orderSafe("라떼");
```

출력 라떼 / 메모 없음

if (memo) 안쪽에서는 memo 가 확실히 문자열입니다. 타입스크립트가 그걸 알고 .length 를 허락해 줍니다. 이것이 02단원 개념03에서 잠깐 본 ‘타입 좁히기’ 이고, 05단원의 주제입니다.

▹ 직접 해보기 2

orderSafe 의 if (memo) 를 지우고 memo.length 만 남겨 보세요.

무슨 에러가 나는지 확인한 뒤 되돌리세요.

기본값을 주면 ? 가 필요 없다

섹션 3

“안 넘기면 이 값으로 해라” 를 정해 줄 수 있습니다. JS자료 05단원의 그것입니다.

```
function orderWithDefault(menu: string, count: number = 1) {
  console.log(menu, count + "잔");
}

orderWithDefault("아메리카노", 3);
```

출력 아메리카노 3잔

```
orderWithDefault("라떼");
```

출력 라떼 1잔

기본값을 주면 좋은 점이 두 가지입니다.

- ① 안 넘겨도 된다 (? 와 같음) ② 타입이 number 그대로다. `undefined` 가 섞이지 않는다
- ② 가 핵심입니다. 섹션 2처럼 매번 확인할 필요가 없습니다.

```
function totalPrice(unit: number, count: number = 1) {
  return unit * count; // 확인 없이 바로 계산해도 됩니다
}

console.log(totalPrice(4000));
```

출력 4000

```
console.log(totalPrice(4000, 3));
```

출력 12000

그리고 기본값이 있으면 타입을 안 적어도 됩니다. 값을 보고 추론하니까요.

```
function greetWithDefault(name: string, greeting = "안녕하세요") {
  return greeting + " " + name + "님";
}

console.log(greetWithDefault("홍길동"));
```

출력 안녕하세요 홍길동님

```
console.log(greetWithDefault("홍길동", "반갑습니다"));
```

출력 반갑습니다 홍길동님

규칙으로 정리하면 이렇습니다.

쓸 만한 기본값이 있다 → = 기본값 (이쪽을 먼저 고려하세요)
없는 것이 의미가 있다 → ?

▫ 직접 해보기 3

할인율을 받되 안 넘기면 0 이 되는 함수 discount 를 만들어

원가 10000 에 대해 discount(10000) 과 discount(10000, 0.2) 를 출력해 보세요.

순서 규칙 — 필수가 먼저

섹션 4

? 가 붙은 것은 뒤로 가야 합니다.

이 코드는 검사에서 걸립니다

TS1016

```
function wrongOrder(memo?: string, menu: string) {  
    console.log(menu, memo);  
}
```

A required parameter cannot follow an optional parameter.

“필수 매개변수가 선택 매개변수 뒤에 올 수 없습니다” 입니다.

생각해 보면 당연합니다. wrongOrder("라떼") 라고 부르면
"라떼" 가 memo 인지 menu 인지 알 방법이 없습니다.

기본값도 같은 규칙입니다. 다만 이유가 조금 다릅니다.

```
function tagged(prefix = "[알림]", message: string) {  
    return prefix + " " + message;  
}
```

이건 문법상 걸리지는 않습니다. 대신 앞엿것을 생략할 수가 없어서 tagged("[알림]", "문 닫습니다") 처럼
항상 둘 다 넘겨야 합니다. 쓸모가 없습니다.

```
console.log(tagged("[공지]", "문 닫습니다"));
```

출력 [공지] 문 닫습니다

그래서 실무 규칙은 하나입니다.

반드시 필요한 것부터 쓰고, 선택인 것을 뒤에 쓴다.

▫ 직접 해보기 4

위 tagged 의 순서를 (message: string, prefix = "[알림]") 으로

바꾸고 tagged("문 닫습니다") 를 출력해 보세요.

자주 하는 실수

섹션 5

이런 실수를 합니다

? 를 붙여 놓고 그냥 쓰기

TS18048 이 나오는 가장 흔한 이유입니다. ? 를 붙였으면 쓰기 전에 확인해야 합니다(섹션 2).

이런 실수를 합니다

? 와 기본값을 같이 쓰기

`function f(a?: number = 1)` 은 TS1015 로 걸립니다. 기본값이 있으면 이미 생략 가능하니 ? 가 필요 없습니다. 둘 중 하나만 쓰세요.

이런 실수를 합니다

선택 매개변수를 앞에 두기

TS1016 입니다. 섹션 4를 보세요.

이런 실수를 합니다

"안 넘기면 null 이 들어오겠지" 라고 생각하기

`undefined` 입니다. `null` 이 아닙니다. 둘은 다른 값입니다. 05단원에서 이 둘의 차이를 다룹니다.

정리

- 1 매개변수 뒤에 ? 를 붙이면 안 넘겨도 된다.
- 2 그 대가로 타입이 'string | undefined' 가 된다. 쓰기 전에 확인해야 한다.
- 3 기본값(= 값)을 주면 안 넘겨도 되면서 타입도 깨끗하게 유지된다. 쓸 만한 기본값이 있으면 ? 보다 이쪽이 낫다.
- 4 기본값이 있으면 타입 표기도 생략할 수 있다. 값을 보고 추론한다.
- 5 선택 매개변수는 반드시 뒤에. 앞에 두면 TS1016.
- 6 안 넘기면 undefined 다. null 이 아니다.

직접 해보기 정답

```
1) function intro(name: string, rank?: string) {
```

```
    console.log(name, rank);
```

```
}
```

```
intro("홍길동"); // 출력: 홍길동 undefined
```

```
intro("홍길동", "대리"); // 출력: 홍길동 대리
```

2) error TS18048: 'memo' is possibly 'undefined'. 재현:

```
function orderSafe(menu: string, memo?: string) { console.log(menu, memo.length); }
```

? 를 붙인 순간 memo 는 string | undefined 가 됩니다. if 로 확인해야만 .length 를 쓸 수 있습니다.

```
3) function discount(price: number, rate: number = 0) {
```

```
    return price - price * rate;
```

```
}
```

```
console.log(discount(10000)); // 출력: 10000
```

```
console.log(discount(10000, 0.2)); // 출력: 8000
```

기본값 0 을 주었으니 rate 의 타입은 그냥 number 입니다. 확인 없이 바로 곱셈에 쓸 수 있습니다.

```
4) function tagged2(message: string, prefix = "[알림]") {
```

```
    return prefix + " " + message;
```

```
}  
console.log(tagged2("문 답습니다"));    // 출력: [알림] 문 답습니다
```

이렇게 해야 기본값이 제 역할을 합니다.

함수를 값으로 넘기기

RUN node 개념03_함수를_값으로_넘기기.ts

검사: npm run typecheck

JS자료 08단원에서 map · filter · forEach 를 배웠습니다. 그때 “함수를 값처럼 넘긴다” 는 것을 익혔습니다.

```
numbers.map(함수)
  └ 이 자리에 함수가 들어간다
```

함수도 값이니 타입이 있습니다. 이 파일은 그 타입을 적는 법입니다.

함수 타입은 화살표로 적는다

섹션 1

지금까지 본 것 — 개념01의 add 는 이런 타입이었습니다.

```
function add(a: number, b: number) {
  return a + b;
}
console.log(add(3, 4));
```

출력 7

```
(a: number, b: number) => number
└─ 받는 것 ─┘ └─ 주는 것
```

화살표 왼쪽이 매개변수, 오른쪽이 반환값입니다. 화살표 함수를 쓸 때의 모양과 비슷해서 외우기 쉽습니다.

이 표기를 변수에 쓸 수 있습니다.

```
const calc: (a: number, b: number) => number = add;
console.log(calc(10, 20));
```

출력 30

다른 모양의 함수를 넣으면 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```
const wrong: (a: number, b: number) => number = (s: string) => s;
```

Type '(s: string) => string' is not assignable to type '(a: number, b: number) => number'.

받는 것도 주는 것도 다릅니다.

함수끼리도 "모양이 맞는가" 를 검사합니다.

▹ 직접 해보기 1

문자열 하나를 받아 숫자를 돌려주는 함수 타입을 적어 보세요.

(답의 모양: (○○: ○○) ⇒ ○○)

콜백은 대개 안 적어도 된다

섹션 2

map 안에 넣는 함수의 매개변수에는 타입을 안 적어도 됩니다.

```
const prices = [4000, 4500, 5000];

const doubled = prices.map((p) => p * 2);
console.log(doubled);
```

출력 [8000, 9000, 10000]

p 에 아무것도 안 적었는데 걸리지 않았습니다. 개념01에서 "매개변수는 무조건 적어야 한다" 고 했는데 왜 여기는 괜찮을까요?

prices 가 number[] 이므로,
map 이 하나씩 꺼내 주는 값은 number 일 수밖에 없기 때문입니다.

넘겨받을 쪽이 이미 정해져 있어서 알아낼 근거가 있는 것입니다. 이런 추론을 '문맥에서 알아낸다' 고 합니다.

그래서 p 를 잘못 쓰면 여전히 걸립니다.

이 코드는 검사에서 걸립니다

TS2339

```
const broken = prices.map((p) => p.toUpperCase());
```

Property 'toUpperCase' does not exist on type 'number'.

p 가 number 인 것을 알고 있으니 막습니다.

"안 적었으니 any 겠지" 가 아닙니다.

filter 도 같습니다.

```
const expensive = prices.filter((p) => p >= 4500);
console.log(expensive);
```

출력 [4500, 5000]

문자열 배열이면 문자열로 알아냅니다.

```
const menus = ["아메리카노", "라떼"];
const lengths = menus.map((m) => m.length);
console.log(lengths);
```

출력 [5, 2]

규칙으로 정리하면 이렇습니다.

내가 만드는 함수의 매개변수 → 반드시 적는다
남의 함수에 넘기는 콜백 → 대개 안 적어도 된다

▫ 직접 해보기 2

menus 에서 글자 수가 3 이상인 것만 골라 출력해 보세요.

콜백을 받는 함수를 직접 만들기

섹션 3

이번엔 반대 입장입니다. 함수를 '받는' 함수를 만들어 봅시다.

```
function applyTwice(value: number, fn: (n: number) => number) {
  return fn(fn(value));
}

console.log(applyTwice(3, (n) => n * 2));
```

출력 12

3 → 6 → 12 입니다.

```
console.log(applyTwice(3, (n) => n + 10));
```

출력 23

fn 의 타입을 적어 두었기 때문에, 넘기는 쪽에서 (n) => ... 의 n 을 안 적어도 됩니다. 섹션 2와 같은 이유입니다.

모양이 안 맞으면 넘기는 자리에서 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```
console.log(applyTwice(3, (n) => "결과 " + n));
```

Type 'string' is not assignable to type 'number'.

숫자를 돌려주기로 한 자리에 문자열을 돌려주는 함수를 넘겼습니다.

에러가 넘기는 줄 전체가 아니라 "결과 " + n 부분을 콧 집어 나옵니다.
fn 의 모양을 미리 적어 둔 덕분에 화살표 안쪽까지 검사한 것입니다.

아무것도 안 돌려주는 콜백은 void 로 적습니다.

```
function repeat(times: number, action: (i: number) => void) {
  for (let i = 1; i <= times; i++) {
    action(i);
  }
}
```

```
repeat(3, (i) => console.log("반복", i));
```

```
출력      반복 1
          반복 2
          반복 3
```

◀ 직접 해보기

3

숫자 배열과 함수를 받아,

각 값에 그 함수를 적용한 결과를 출력하는 함수를 만들어 보세요. (map 을 써도 되고 for 를 써도 됩니다)

같은 함수 타입을 여러 번 쓸 때

섹션 4

함수 타입 표기는 길어서 여러 번 쓰면 지저분해집니다.

```
function runA(fn: (a: number, b: number) => number) {
  return fn(1, 2);
}
function runB(fn: (a: number, b: number) => number) {
  return fn(10, 20);
}

console.log(runA(add), runB(add));
```

출력 3 30

$(a: \text{number}, b: \text{number}) \Rightarrow \text{number}$ 를 두 번 적었습니다. 세 번, 네 번 나오면 고칠 때마다 전부 찾아 고쳐야 합니다.

이럴 때 타입에 이름을 붙일 수 있습니다. 04단원에서 배웁니다. 미리 모양만 보면 이렇습니다.

```
type Calc = (a: number, b: number) => number;
function runA(fn: Calc) { ... }
```

지금은 “길어지면 이름을 붙일 수 있구나” 정도만 알아 두세요.

▹ 직접 해보기

4

runA 와 runB 에 $(n) \Rightarrow n * 100$ 을 넘겨 보세요.

걸리나요? 왜 그럴까요?

자주 하는 실수

섹션 5

이런 실수를 합니다

함수 타입에 콜론을 쓰기

$(a: \text{number}, b: \text{number}): \text{number}$ 는 함수를 ‘정의’ 할 때의 모양입니다. 함수 ‘타입’ 을 적을 때는 $\Rightarrow$ 를 씁니다. 이 둘을 자주 헷갈립니다.

이런 실수를 합니다

콜백 매개변수에 굳이 타입을 적기

`prices.map((p: number) => p * 2)` 도 되기는 합니다. 틀린 건 아니지만 길어지기만 합니다. 문맥에서 알아내니 말기세요.

이런 실수를 합니다

함수를 넘겨야 하는데 호출한 결과를 넘기기

`applyTwice(3, double(2))` 처럼 괄호를 붙이면 함수가 아니라 값이 넘어갑니다. JS자료 11단원에서 본 `onclick = handler` 와 `onclick = handler()` 의 차이입니다.

이런 실수를 합니다

반환 타입이 void 인 콜백에서 값을 돌려주기

막지는 않습니다. 그냥 무시됩니다. void 는 “돌려줘도 안 쓴다” 에 가깝습니다.

정리

- 1 함수 타입은 (매개변수) ⇒ 반환값 으로 적는다. 화살표 왼쪽이 받는 것.
- 2 함수끼리도 모양이 맞는지 검사한다. 매개변수와 반환값 둘 다 본다.
- 3 `map · filter` 같은 곳에 넘기는 콜백의 매개변수는 안 적어도 된다. 넘겨받을 쪽이 정해져 있어서 문맥에서 알아낸다.
- 4 그래도 검사는 그대로 한다. 콜백 안에서 잘못 쓰면 걸린다.
- 5 콜백을 받는 함수를 만들 때는 그 자리에 함수 타입을 적어 준다.
- 6 아무것도 안 돌려주는 콜백은 ⇒ void 로 적는다.

직접 해보기 정답

1) (s: string) ⇒ number 이름(s)은 아무거나 써도 됩니다. 중요한 것은 종류와 순서입니다.

```
예: const len: (s: string) => number = (t) => t.length;
```

```
2) console.log(menus.filter((m) => m.length >= 3)); // 출력: [ '아메리카노' ]
```

‘라떼’ 는 2글자라 빠집니다.

```
3) function applyAll(values: number[], fn: (n: number) => number) {
```

```
  console.log(values.map(fn));
```

```
}
```

```
  applyAll([1, 2, 3], (n) => n * 10); // 출력: [ 10, 20, 30 ]
```

`map` 에 `fn` 을 그대로 넘긴 것에 주목하세요. (n) ⇒ `fn(n)` 이라고 쓸 필요가 없습니다.

4) 걸리지 않습니다. 그냥 됩니다.

```
console.log(runA((n) => n * 100), runB((n) => n * 100)); // 출력: 100 1000
```

두 개짜리 함수를 받기로 한 자리에 한 개짜리를 넘겼는데도 통과합니다. 두 번째 값을 그냥 안 쓸 뿐이니 문제가 될 게 없기 때문입니다. runA 는 fn(1, 2) 를 부르고, n 은 첫 번째 값 1 만 받아서 100 이 됩니다.

JS자료 08단원에서 arr.map((x) => x) 처럼 index 를 안 받고 써도 됐던 이유가 이것입니다. map 은 값·인덱스·배열 셋을 넘기지만 안 받으면 그만입니다.

반대로 '종류' 가 다르면 걸립니다. (s: string) => number 를 넘겨 보세요. 개수가 모자란 것은 봐주고, 종류가 다른 것은 안 봐줍니다(04단원 개념02 섹션3).

함수 타입

RUN node 연습문제.ts

채점: npm run grade ← 출력을 기대 출력과 맞춰 봅니다

검사: npm run check ← 타입 검사 (일부 문제만 잡습니다)

★ 채점은 npm run grade 로 합니다. 실제 출력을 문제의 '기대 출력' 과 줄 단위로 맞춰 봅니다. 전부 일치하면 다 맞은 것입니다.

npm run check(타입 검사)도 같이 쓰세요. 다만 이쪽은 '일부 문제만' 잡습니다.

138문항 중 21개뿐입니다. "출력하세요" 류는 안 풀어도 타입 에러가 안 나거든요. 그러니 check 가 조용하다고 다 맞은 것이 아닙니다. grade 로 확인하세요. (npm run typecheck 는 개념·정답 파일만 봅니다. 그쪽은 언제나 조용합니다)

- 1 TODO 자리에 코드를 씁니다.
- 2 npm run grade 로 채점합니다. (직접 node 연습문제.ts 로 돌려 봐도 됩니다)
- 3 npm run check 로 타입도 봅니다.
- 4 10분 고민해도 안 되면 연습문제_정답.ts 를 보세요.

문제 1~8은 기본, 9~12는 응용, 13은 [도전]입니다. 문제 14는 에러를 직접 보는 문제라 맨 뒤에 있습니다.

```
console.log("=== 03단원 연습문제 ===");
```

문제 1 (개념01 섹션1)

아래 함수의 매개변수에 타입을 붙이세요.

【지금까지 하던 방법】(JS)

```
function greet(name) { ... }
```

【이번에 배울 방법】(TS)

```
function greet(name: string) { ... }
```

기대 출력 라떼 나왔습니다

기대 검사 결과: 조용함

```
{
```

매개변수에 타입을 붙이세요

```
function callMenu(menu) {  
    return menu + " 나왔습니다";  
}  
console.log(callMenu("라떼"));  
}
```

문제 2 (개념01 섹션1)

숫자 두 개를 받아 큰 쪽을 돌려주는 함수 bigger 를 만드세요. 반환 타입은 적지 마세요(추론에 맡깁니다).

기대 출력 9

기대 검사 결과: 조용함

```
{
```

bigger 를 만들고 bigger(4, 9) 를 출력하세요

```
}
```

문제 3 (개념01 섹션3)

아래 함수에 반환 타입 : number 를 붙이세요. 붙이면 몸통에서 에러가 하나 드러납니다. 그것도 고치세요.

기대 출력 8000

기대 검사 결과: 조용함

{

반환 타입을 붙이고 몸통을 고치세요

```
function getTotal(count: number) {
    return String(count * 4000);
}
console.log(getTotal(2));
}
```

문제 4 (개념01 섹션4)

아무것도 돌려주지 않고 화면에 찍기만 하는 함수 announce 를 만드세요. 반환 타입을 : void 로 명시하세요.

기대 출력

공지 · 문 답습니다

기대 검사 결과: 조용함

{

announce 를 만들고 announce(“문 답습니다”) 를 호출하세요

}

문제 5 (개념01 섹션5)

아래 함수를 화살표 함수로 바꿔 쓰세요. 타입은 그대로 유지하세요.

```
function half(n: number): number {
  return n / 2;
}
```

기대 출력 21

기대 검사 결과: 조용함

```
{
```

화살표 함수 half 를 만들고 half(42) 를 출력하세요

```
}
```

문제 6 (개념02 섹션1)

두 번째 매개변수를 '없어도 되게' 만드세요.

기대 출력 아메리카노 undefined
아메리카노 얼음 적게

기대 검사 결과: 조용함

```
{
```

memo 를 선택 매개변수로 바꾸세요

```
function order(menu: string, memo: string) {
  console.log(menu, memo);
}
order("아메리카노");
order("아메리카노", "얼음 적게");
}
```

문제 7 (개념02 섹션3)

두 번째 매개변수에 기본값 1 을 주세요.

기대 출력 4000
 12000

기대 검사 결과: 조용함

```
{
```

count 에 기본값을 주세요

```
function total(unit: number, count: number) {
  return unit * count;
}
console.log(total(4000));
console.log(total(4000, 3));
}
```

문제 8 (개념03 섹션2)

아래 배열에서 각 값에 10 을 곱한 새 배열을 만들어 출력하세요. 콜백 매개변수에는 타입을 적지 마세요.

기대 출력

— 10, 20, 30

기대 검사 결과: 조용함

```
{
  const nums: number[] = [1, 2, 3];
```

map 으로 새 배열을 만들어 출력하세요

```
}
```

문제 9 (응용 · 개념02 섹션2)

아래 함수는 검사에서 걸립니다. if 로 확인해서 걸리지 않게 고치세요. memo 가 없으면 "메모 없음" 을 찍으세요.

기대 출력 5
 메모 없음

기대 검사 결과: 조용함

```
{
```

걸리지 않게 고치고, 아래 두 줄의 주석도 푸세요

```
function showMemo(memo?: string) {  
    console.log(memo.length);  
}
```

```
showMemo("얼음 적게");  
showMemo();
```

```
}
```

문제 10 (응용 · 개념03 섹션1)

아래 변수에 '숫자 하나를 받아 문자열을 돌려주는 함수' 타입을 붙이세요.

기대 출력 4000원

기대 검사 결과: 조용함

```
{
```

타입을 붙이세요

```
const format = (n: number) => n + "원";
console.log(format(4000));
}
```

문제 11 (응용 · 개념03 섹션3)

숫자 배열과 함수를 받아, 그 함수를 통과한 값들의 합을 돌려주는 `sumWith` 를 만드세요.

기대 출력 60

기대 검사 결과: 조용함

```
{
```

`sumWith` 를 만들고 `sumWith([1,2,3], (n) => n * 10)` 을 출력하세요

```
}
```

문제 12 (응용 · 개념02 섹션4)

아래 함수는 TS1016 으로 걸립니다. 순서를 고쳐서 걸리지 않게 하세요. 고친 뒤 필수 값만 넘겨서 호출하세요.

```
function tag(prefix?: string, message: string) { ... }
```

기대 출력

알림 · 문 답습니다

기대 검사 결과: 조용함

```
{
```

순서를 고쳐 만들고 호출하세요

```
}
```

문제 13 ([도전] · 개념01~03 종합)

주문 목록에서 '뜨거운 음료' 만 골라 총액을 구하는 함수를 만드세요. ① 매개변수 두 개: 주문 배열, 그리고 '뜨거운지 판단하는 함수' ② 반환 타입은 : number 로 명시 ③ 결과를 "뜨거운 음료 합계: 8500원" 형태로 출력

힌트: filter 로 고르고, 그다음 합을 구하면 됩니다.

기대 출력 뜨거운 음료 합계: 8500원

기대 검사 결과: 조용함

```
{  
  const orders = [  
    { menu: "아메리카노", price: 4000, isHot: true },  
    { menu: "아이스티", price: 5000, isHot: false },  
    { menu: "라떼", price: 4500, isHot: true },  
  ];
```

sumHot 을 만들고 결과를 출력하세요

```
}
```

문제 14 (에러 확인 · 개념01 섹션1)

아래 두 줄의 // 를 지우고 npm run check 로 검사해 보세요.

① 무슨 에러 번호가 나오나요? ② 메시지 안의 implicitly 는 무슨 뜻일까요?

확인했으면 다시 // 를 붙이세요.

```
{  
  
  function double(n) { return n * 2; }  
  console.log(double(21));  
  
}
```


객체와 타입 별칭

OPEN 04_객체와_타입_별칭

04

```
name: "라떼",  
price: 4500,  
printMenu(menu);  
printMenu(rawMenu);
```

01 객체 타입 적기

02 type: 타입에 이름 붙이기

03 선택 속성 · readonly · interface

04 서버 데이터에 모양 적기

- 연습문제

객체 타입 적기

RUN node 개념01_객체_타입_적기.ts

검사: npm run typecheck

02단원에서 객체는 추론이 잘 된다고 했습니다. 맞습니다. 그런데 값이 없는 자리에는 추론할 근거가 없습니다.

```
function order(o) { ... }      ← o 가 무슨 모양인지 알 수 없다
const data = JSON.parse(...)  ← 안에 뭐가 들었는지 알 수 없다
```

그런 자리에 “이런 모양일 것이다” 를 적는 법이 이 단원입니다.

{ 이름: 종류 } 로 적는다

섹션 1

객체의 타입은 객체처럼 생겼습니다. 값 자리에 종류를 쓴 것뿐입니다.

```
값:    { name: "라떼", price: 4500 }
타입:  { name: string, price: number }
```

```
const menu: { name: string; price: number } = {
  name: "라떼",
  price: 4500,
};

console.log(menu.name, menu.price);
```

출력 라떼 4500

타입 안에서는 쉼표(,) 대신 세미콜론(;)을 쓰는 것이 관행입니다. 쉼표를 써도 동작은 같습니다. 이 자료는 세미콜론으로 통일합니다.

함수 매개변수에 쓰면 이렇게 됩니다. 이게 진짜 쓸 자리입니다.

```
function printMenu(item: { name: string; price: number }) {
  console.log(item.name + " " + item.price + "원");
}

printMenu({ name: "아메리카노", price: 4000 });
```

출력 아메리카노 4000원

```
printMenu(menu);
```

출력 라떼 4500원

이제 매개변수 자리가 설명서 역할을 합니다. 이 함수를 쓰는 사람은 몸통을 안 열어 봐도 무엇을 넘겨야 하는지 압니다.

▹ 직접 해보기

1

이름(string)과 좌석수(number)를 가진 객체 타입을 적어

shop 이라는 변수를 만들고 출력해 보세요.

빠뜨리면 걸린다

섹션 2

이 코드는 검사에서 걸립니다

TS2741

```
const broken: { name: string; price: number } = { name: "라떼" };
```

Property 'price' is missing in type '{ name: string; }' but required in type '{ name: string; price: number; }'.

missing = “빠졌다”, required = “반드시 있어야 하는”.

읽는 법: “price 가 빠졌는데 그건 반드시 있어야 합니다”

01단원 개념03 섹션4의 단어 표에서 본 그 둘입니다.

함수에 넘길 때도 똑같이 TS2741 입니다.

이 코드는 검사에서 걸립니다

TS2741

```
printMenu({ name: "라떼" });
```

Property 'price' is missing in type '{ name: string; }' but required in type '{ name: string; price: number; }'.

02단원에서 “답을 때는 TS2322, 넘길 때는 TS2345” 라고 했는데,

‘속성이 빠진 것’ 만은 양쪽 다 TS2741 입니다.

빠진 속성 이름을 꼭 집어 주는 것이 더 쓸모 있기 때문입니다.

객체를 넘길 때 나는 에러는 세 가지로 갈립니다 ———

갈래 1 · 종류가 통째로 다르다 → TS2345

이 코드는 검사에서 걸립니다

TS2345

```
printMenu("라떼");
```

Argument of type 'string' is not assignable to parameter of type '{ name: string; price: number; }'.

객체를 받기로 한 자리에 문자열을 넘겼습니다. 통째로 다른 것입니다.

갈래 2 · 속성이 빠졌다 → TS2741 (위에서 본 것)

갈래 3 · 속성은 다 있는데 그 속성의 종류가 다르다 → TS2322

이 코드는 검사에서 걸립니다

TS2322

```
printMenu({ name: "라떼", price: "비쌘" });
```

Type 'string' is not assignable to type 'number'.

에러가 객체 전체가 아니라 “비쌘” 자리를 콕 집어 나옵니다.

열 번호를 보면 정확히 그 값입니다.

여러 속성이 있어도 어느 것이 문제인지 바로 알 수 있습니다.

▫ 직접 해보기

2

printMenu({ price: 4000 }) 을 써 보고

어느 속성이 빠졌다고 하는지, 세 갈래 중 무엇인지 확인한 뒤 지우세요.

없는 것을 넣어도 걸린다

섹션 3

이 코드는 검사에서 걸립니다

TS2353

```
const extra: { name: string; price: number } = { name: "라떼", price: 4500, size: "L" };
```

Object literal may only specify known properties, and 'size' does not exist in type '{ name: string; price: number; }'.

“적어 둔 것만 쓸 수 있습니다. size 는 그 안에 없습니다” 입니다.

이걸 '초과 속성 검사' 라고 합니다.

왜 막을까요? 대개는 오타이기 때문입니다.

priceX 라고 쓰면 "price 를 안 넣었고 priceX 라는 이상한 걸 넣었다" 가 되는데, 초과 속성 검사가 없으면 조용히 통과해 버립니다.

▹ 직접 해보기

3

```
printMenu({ name: "라떼", price: 4500, isHot: true })
```

써 보고 무슨 에러가 나는지 확인한 뒤 지우세요.

그런데 변수를 거치면 통과한다 ★

섹션 4

이건 처음 보면 반드시 놀랍니다.

```
const rawMenu = { name: "라떼", price: 4500, size: "L" };
```

방금 섹션 3에서 막혔던 그 모양인데, 변수에 담아서 넘기면 통과합니다.

```
printMenu(rawMenu);
```

출력 라떼 4500원

```
const stored: { name: string; price: number } = rawMenu;
console.log(stored.name);
```

출력 라떼

왜 이럴까요?

초과 속성 검사는 '그 자리에 직접 쓴 객체' 에만 적용됩니다.

직접 쓴 것은 오타일 가능성이 높지만, 이미 만들어져 있는 값은 "필요한 게 다 있으니 됐다" 고 봅니다. 남은 속성이 있어도 printMenu 는 안 쓸 뿐이니 문제가 없습니다.

이런 방식을 '구조적 타이핑' 이라고 합니다. 이름표가 아니라 모양만 봅니다. 필요한 것이 다 들어 있으면 통과입니다.

그래서 이런 것도 됩니다. 이름이 달라도 모양이 맞으니까요.

```
const 손님이_고른_것 = { name: "아이스티", price: 5000 };
printMenu(손님이_고른_것);
```

출력 아이스티 5000원

다만 '없는 것' 은 여전히 못 만들어 냅니다.

```
const missing = { name: "라떼" };
```

이 코드는 검사에서 걸립니다

TS2741

```
printMenu(missing);
```

Property 'price' is missing in type '{ name: string; }' but required in type '{ name: string; price: number; }'.

남는 것은 봐주지만 빠진 것은 안 봐줍니다.

변수를 거쳤는데도 걸리는 것에 주목하세요.

초과 속성 검사만 느슨해질 뿐, 필수 속성 검사는 언제나 그대로입니다.

▹ 직접 해보기

4

rawMenu 에서 price 를 지우고 printMenu(rawMenu) 를 해 보세요.

통과할까요? 확인한 뒤 되돌리세요.

배열 안의 객체

섹션 5

객체 배열은 뒤에 [] 를 붙입니다. 02단원의 string[] 과 같은 규칙입니다.

```
const menus: { name: string; price: number }[] = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
];

console.log(menus.length);
```

출력 2

```
console.log(menus[0].name);
```

출력 아메리카노

배열 메소드도 그대로 됩니다. 콜백 매개변수는 문맥에서 알아냅니다(03단원 개념03).

```
const names = menus.map((m) => m.name);
console.log(names);
```

출력 ['아메리카노', '라떼']

```
const total = menus.reduce((sum, m) => sum + m.price, 0);
console.log(total);
```

출력 8500

그런데 이 표기가 슬슬 길어집니다.

```
{ name: string; price: number }[]
```

이게 세 번, 네 번 나오면 고칠 때마다 전부 찾아 고쳐야 합니다. 그래서 이름을 붙입니다. 개념02에서 배웁니다.

▸ 직접 해보기 5

menus 에서 가격이 4200 이상인 것만 골라 출력해 보세요.

04

키 이름을 미리 모를 때

섹션 6

지금까지는 속성 이름을 전부 알고 있었습니다. name, price 처럼요. 그런데 이름을 미리 못 적는 경우가 있습니다. 메뉴가 늘어날 때마다 키가 늘어난다면요.

```
const 재고: { [메뉴이름: string]: number } = {
  아메리카노: 12,
  카페라떼: 5,
};

console.log(재고["아메리카노"]);
```

출력 12

```
console.log(재고.카페라떼);
```

출력 5

없던 키를 새로 넣어도 됩니다.

```
재고["바닐라라떼"] = 8;
console.log(재고["바닐라라떼"]);
```

출력 8

메뉴이름: string : number 는

“키는 문자열이고, 거기 담긴 값은 전부 number 다” 라는 뜻입니다. 대괄호 안의 이름(메뉴이름)은 읽는 사람을 위한 것이라 아무렇게나 지어도 됩니다. 값의 종류는 지켜집니다.

이 코드는 검사에서 걸립니다

TS2322

```
재고["에스프레소"] = "많음";
```

Type 'string' is not assignable to type 'number'.

키는 자유지만 값은 자유가 아닙니다.

★ 여기에 함정이 하나 있습니다.

```
const 없는것 = 재고["카페모카"];  
console.log(없는것);
```

출력 undefined

없는 키를 꺼냈는데도 타입은 number 라고 나옵니다. 실제 값은 undefined 인데요. 04단원 개념04의 "타입은 약속이지 검사가 아니다" 가 여기서도 그대로 나옵니다. 08단원 개념03의 noUncheckedIndexedAccess 를 켜면 number | undefined 가 되어 확인하고 쓰게 강제됩니다.

그래서 키를 아는 것은 그냥 적으세요. 이 방법은 정말 모를 때만 씁니다.

▹ 직접 해보기 6

console.log(재고["카페모카"] + 1); 을 써 보세요.

검사는 통과하나요? 실행하면 무엇이 찍히나요?

자주 하는 실수

섹션 7

이런 실수를 합니다

타입 자리에 값을 쓰기

{ name: "라떼" } 가 아니라 { name: string } 입니다. 타입 자리에는 '종류' 를 씁니다.

이런 실수를 합니다

세미콜론과 쉼표를 헷갈리기

타입 안에서는 둘 다 됩니다. 값 안에서는 쉼표만 됩니다. 타입은 ; , 값은 , 로 통일해 두면 헷갈리지 않습니다.

이런 실수를 합니다

초과 속성 검사를 '언제나' 적용된다고 알기

섹션 4를 보세요. 직접 쓴 객체에만 적용됩니다. "아까는 걸렸는데 지금은 왜 안 걸리지?" 의 답이 여기 있습니다.

이런 실수를 합니다

객체 배열을 { ... }[] 가 아니라 [{ ... }] 로 쓰기

[] 는 뒤에 붙입니다. 02단원의 string[] 과 같습니다.

정리

- 1 객체 타입은 { 이름: 종류; ... } 로 적는다. 값 자리에 종류를 쓴 모양이다.
- 2 진짜 쓸 자리는 함수 매개변수다. 매개변수가 설명서 역할을 하게 된다.
- 3 빠뜨리면 TS2741(missing), 없는 것을 넣으면 TS2353(초과 속성 검사).
- 4 초과 속성 검사는 '그 자리에 직접 쓴 객체'에만 적용된다. 변수를 거치면 남는 속성이 있어도 통과한다 (구조적 타이핑).
- 5 남는 것은 봐주지만 빠진 것은 안 봐준다.
- 6 객체 배열은 뒤에 [] 를 붙인다. 길어지면 이름을 붙인다(개념02).

직접 해보기 정답

- 1 `const shop: { name: string; seats: number } = { name: "봄날카페", seats: 24 };
console.log(shop.name, shop.seats); // 출력: 봄날카페 24`
- 2 error TS2741: Property 'name' is missing in type '{ price: number; }' but required in type '{ name: string; price: number; }'. 재현:

```
function printMenu(item: { name: string; price: number }) { console.log(item.name); }
printMenu({ price: 4000 });
```

갈래 2 · 입니다. name 이 빠졌다고 이름을 꼭 집어 알려 줍니다.

3) error TS2353: Object literal may only specify known properties, and 'isHot' does not exist in type '{ name: string; price: number; }'. 재현:

```
function printMenu(item: { name: string; price: number }) { console.log(item.name); }
printMenu({ name: "라떼", price: 4500, isHot: true });
```

직접 쓴 객체라 초과 속성 검사에 걸립니다.

4) 통과하지 않습니다. error TS2741: Property 'price' is missing in type '{ name: string; size: string; }' but required in type '{ name: string; price: number; }'. 재현:

```
function printMenu(item: { name: string; price: number }) { console.log(item.name); }  
const rawMenu = { name: "라떼", size: "L" };  
printMenu(rawMenu);
```

남는 것(size)은 봐주지만 빠진 것(price)은 안 봐줍니다. 변수를 거쳤는데도 걸립니다. 느슨해지는 것은 초과 속성 검사뿐입니다.

5 `console.log(menus.filter((m) => m.price >= 4200));` // 출력: [{ name: '라떼', price: 4500 }]

6 검사는 통과합니다. 재고["카페모카"]의 타입이 number 라고 나오기 때문입니다. 실행하면 NaN 이 찍힙니다. 실제로는 undefined + 1 이라서요. "검사는 통과하는데 돌리면 이상하다" — 이 자료가 계속 경계하는 그 자리입니다.

type: 타입에 이름 붙이기

RUN node 개념02_type_별칭.ts

검사: npm run typecheck

개념01 마지막에서 이런 표기가 나왔습니다.

```
{ name: string; price: number }[]
```

이게 세 번, 네 번 나오면 고칠 때마다 전부 찾아 고쳐야 합니다. 가격을 문자열로 바꾸기로 했다면 다섯 군데를 다 찾아야 합니다. 하나라도 빠뜨리면 어긋납니다.

그래서 이름을 붙입니다. 변수에 이름을 붙이는 것과 똑같은 이유입니다.

type 이름 = 모양;

섹션 1

```
type Menu = { name: string; price: number };
```

이제 Menu 라고만 쓰면 됩니다.

```
const latte: Menu = { name: "라떼", price: 4500 };
console.log(latte.name, latte.price);
```

출력 라떼 4500

```
function printMenu(item: Menu) {
  console.log(item.name + " " + item.price + "원");
}
printMenu(latte);
```

출력 라떼 4500원

배열도 짧아집니다.

```
const menus: Menu[] = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
];
console.log(menus.length);
```

출력 2

개념01의 그 표기와 비교해 보세요.

```
전: const menus: { name: string; price: number }[] = [...]  
후: const menus: Menu[] = [...]
```

그리고 가격을 문자열로 바꿔야 한다면, 고칠 곳은 type Menu 한 줄뿐입니다.

검사는 똑같이 합니다. 이름만 붙였을 뿐입니다.

이 코드는 검사에서 걸립니다

TS2741

```
const broken: Menu = { name: "라떼" };
```

Property 'price' is missing in type '{ name: string; }' but required in type 'Menu'.

메시지에 'Menu' 라는 이름이 그대로 나옵니다.

긴 모양이 통째로 찍히던 것보다 훨씬 읽기 쉽습니다. 이것도 이득입니다.

◦ 직접 해보기 1

이름(string)과 좌석수(number)를 가진 Shop 타입을 만들고

변수 하나를 만들어 출력해 보세요.

이름 짓는 관행

섹션 2

타입 이름은 대문자로 시작합니다.

type Menu	○
type menu	× (문법상 되지만 아무도 안 씁니다)

변수는 소문자, 타입은 대문자로 시작하니 한눈에 구별됩니다.

```
const menu: Menu = ...  
  ↳변수   ↳타입
```

이 자료도 그렇게 씁니다. 02단원에서 “타입 이름은 전부 소문자” 라고 한 것은 string · number 같은 ‘기본 타입’ 이야기입니다. 내가 만든 타입은 대문자입니다.

```
type Shop = { name: string; seats: number };  
const shop: Shop = { name: "봄날카페", seats: 24 };  
console.log(shop.name);
```

출력 봄날카페

▣ 직접 해보기 2

위 shop 에 마우스를 올려 보세요.

타입이 Shop 이라고 나오나요, 아니면 { name: string; seats: number } 라고 나오나요?

함수 타입에도 이름을 붙인다

섹션 3

03단원 개념03 섹션4에서 예고한 것입니다.

```
type Calc = (a: number, b: number) => number;

function runA(fn: Calc) {
  return fn(1, 2);
}
function runB(fn: Calc) {
  return fn(10, 20);
}

const add: Calc = (a, b) => a + b;
console.log(runA(add), runB(add));
```

출력 3 30

add 의 매개변수 (a, b) 에 타입을 안 적은 것에 주목하세요. 왼쪽에 : Calc 라고 적어 두었으니 문맥에서 알아냅니다.

여기서 놀라운 것 하나 — 매개변수를 '적게' 받는 함수는 통과합니다.

```
const half: Calc = (a) => a / 2;
console.log(runA(half));
```

출력 0.5

Calc 는 두 개를 받기로 했는데 하나만 받는 함수를 넣었는데도 통과했습니다. 두 번째 값을 그냥 안 쓸 뿐이니 문제가 될 게 없기 때문입니다.

JS자료 08단원에서 arr.map((x) => x) 처럼 index 를 안 받고 써도 됐던 이유가 이것입니다. map 은 값·인덱스·배열 셋을 넘기지만, 안 받으면 안 받는 대로 됩니다.

반대로 '돌려주는 종류' 가 다르면 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```
const label: Calc = (a, b) => "합계 " + (a + b);
```

Type 'string' is not assignable to type 'number'.

화살표 안쪽을 꼭 집어 줍니다. 숫자를 주기로 했는데 문자열을 줬습니다.

매개변수의 '종류' 가 다른 것도 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```
const wrongParam: Calc = (a: string, b: number) => 1;
```

Type '(a: string, b: number) => number' is not assignable to type 'Calc'.

메시지가 여러 줄로 나옵니다.

Types of parameters 'a' and 'a' are incompatible.

두 번째 줄까지 읽어야 어느 매개변수가 문제인지 알 수 있습니다.

정리하면 - 개수가 모자란 것은 봐주고, 종류가 다른 것은 안 봐줍니다.

▣ 직접 해보기 3

문자열 하나를 받아 숫자를 돌려주는 함수 타입 Measure 를 만들고,

글자 수를 세는 함수를 담아 써 보세요.

타입 안에 타입

섹션 4

만든 타입을 다른 타입 안에서 쓸 수 있습니다.

```
type Order = {
  shop: Shop;
  items: Menu[];
  memo: string;
};

const todayOrder: Order = {
  shop: { name: "봄날카페", seats: 24 },
  items: [
    { name: "아메리카노", price: 4000 },
    { name: "라떼", price: 4500 },
  ],
  memo: "포장",
```

```
};

console.log(todayOrder.shop.name);
```

출력 봄날카페

```
console.log(todayOrder.items[1].price);
```

출력 4500

자동완성도 끝까지 따라옵니다. todayOrder.items[0]. 까지 치면 name / price 가 뜹니다.

안쪽이 틀리면 안쪽을 콧 집어 줍니다.

이 코드는 검사에서 걸립니다

TS2322

```
const wrongOrder: Order = {
  shop: { name: "봄날카페", seats: 24 },
  items: [{ name: "라떼", price: "4500원" }],
  memo: "",
};
```

Type 'string' is not assignable to type 'number'.

Order 전체가 아니라 "4500원" 자리를 가리킵니다.

중첩이 깊어도 정확한 자리를 알려 줍니다.

▸ 직접 해보기 4

todayOrder 의 items 에 메뉴를 하나 더 넣고

전체 개수를 출력해 보세요.

언제 이름을 붙이나

섹션 5

규칙은 단순합니다.

같은 모양을 두 번 이상 쓰게 되면 이름을 붙인다.

한 번만 쓰는 모양은 그냥 그 자리에 적어도 됩니다.

```
function logSize(box: { width: number; height: number }) {
  console.log(box.width * box.height);
}
logSize({ width: 3, height: 4 });
```

다만 이름을 붙이면 부수적인 이득이 둘 있습니다.

① 에러 메시지가 짧아진다 (섹션 1) ② 이름 자체가 설명이 된다

{ name: string; price: number } 보다 Menu 가 뜻이 분명합니다. 그래서 실무에서는 한 번만 써도 이름을 붙이는 경우가 많습니다.

▹ 직접 해보기

5

위 logSize 의 매개변수 모양에 Box 라는 이름을 붙여 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

type 뒤에 = 를 빼먹기

type Menu { ... } 는 안 됩니다. type Menu = { ... }; 입니다. (뒤에서 배울 interface 는 = 가 없어서 더 헷갈립니다. 개념03을 보세요)

이런 실수를 합니다

타입 이름을 변수처럼 쓰기

`console.log(Menu)` 는 안 됩니다. Menu 는 실행되는 값이 아닙니다. node 로 돌리면 타입은 통째로 지워집니다(01단원 개념02).

이런 실수를 합니다

세미콜론 빠뜨리기

type Menu = { ... } 뒤의 ; 는 있는 편이 안전합니다. 없어도 대개 동작하지만 다음 줄과 붙어 이상한 에러가 날 때가 있습니다.

이런 실수를 합니다

이름을 소문자로 짓기

문법상 되지만 변수와 구별이 안 됩니다. 대문자로 시작하세요.

정리

- 1 type 이름 = 모양; 으로 타입에 이름을 붙인다.
- 2 이름은 대문자로 시작한다. 변수(소문자)와 한눈에 구별된다.
- 3 고칠 곳이 한 군데로 모인다. 이게 이름을 붙이는 가장 큰 이유다.
- 4 에러 메시지에도 그 이름이 나와서 읽기 쉬워진다.
- 5 함수 타입에도 붙일 수 있다. **type Calc = (a: number, b: number) ⇒ number;**
- 6 만든 타입을 다른 타입 안에서 쓸 수 있다. 중첩이 깊어도 자리를 정확히 짚어 준다.
- 7 같은 모양을 두 번 쓰게 되면 이름을 붙인다.

직접 해보기 정답

```
1) type Cafe = { name: string; seats: number };
const myCafe: Cafe = { name: "봄날카페", seats: 24 };
console.log(myCafe.name, myCafe.seats); // 출력: 봄날카페 24
```

(이 파일에는 이미 Shop 이 있으니 다른 이름을 쓰세요.

같은 이름을 두 번 만들면 TS2300 Duplicate identifier 가 납니다)

- 2 **const** shop: Shop 이라고 나옵니다. 붙여 둔 이름을 그대로 보여 줍니다. 모양이 궁금하면 Shop 위에 마우스를 올리면 펼쳐집니다.
- 3 **type Measure = (s: string) ⇒ number; const** count: **Measure** = (s) ⇒ s.length;

console.log(count("봄날카페")); // 출력: 4

(s) 에 타입을 안 적어도 되는 것에 주목하세요. : Measure 가 알려 줍니다.

```
4) todayOrder.items.push({ name: "카페모카", price: 5000 });
console.log(todayOrder.items.length); // 출력: 3
```

push 에 넘기는 객체도 Menu 모양이어야 합니다. 아니면 TS2345 로 걸립니다.

```
5) type Box = { width: number; height: number };
function logSize2(box: Box) { console.log(box.width * box.height); }
logSize2({ width: 3, height: 4 }); // 출력: 12
```

선택 속성 · readonly · interface

RUN node 개념03_선택속성과_interface.ts

검사: npm run typecheck

개념02까지로 객체 타입은 거의 다 됩니다. 이 파일은 실무에서 꼭 만나는 세 가지를 더합니다.

? 없어도 되는 속성 · **readonly** 못 바꾸는 속성 · **interface** 또 하나의 표기

? — 없어도 되는 속성

섹션 1

03단원 개념02에서 매개변수에 ? 를 붙였습니다. 속성에도 똑같이 붙입니다.

```
type Order = {
  menu: string;
  count: number;
  memo?: string; // 메모는 없어도 된다
};

const a: Order = { menu: "라떼", count: 1 };
const b: Order = { menu: "라떼", count: 1, memo: "얼음 적게" };

console.log(a.memo);
```

출력 undefined

```
console.log(b.memo);
```

출력 얼음 적게

? 가 없으면 빠뜨렸을 때 TS2741 이 났습니다. ? 를 붙였으니 통과합니다.

대가도 같습니다. memo 의 타입이 string | undefined 가 됩니다.

이 코드는 검사에서 걸립니다

TS18048

```
console.log(a.memo.length);
```

'a.memo' is possibly 'undefined'.

“없어도 된다면 .length 를 쓰시겠다고요?” 입니다.

03단원 개념02 섹션2와 완전히 같은 이야기입니다.

쓰려면 확인하고 씁니다.

```
if (b.memo) {
  console.log("메모 길이:", b.memo.length);
}
```

출력 메모 길이: 5

▹ 직접 해보기

1

Order 에 선택 속성 isHot(boolean)을 추가하고

그것 없이 객체를 하나 만들어 보세요.

? 와 '값이 undefined 인 것' 은 다르다

섹션 2

이건 헛갈리는 자리라 따로 봅니다.

```
type Strict = { memo: string | undefined }; // ? 가 없다
type Loose = { memo?: string }; // ? 가 있다
```

Loose 는 아예 빼도 됩니다.

```
const loose: Loose = {};
console.log(loose.memo);
```

출력 undefined

Strict 는 '반드시 있어야 하되, 값이 undefined 여도 된다' 입니다.

```
const strict: Strict = { memo: undefined };
console.log(strict.memo);
```

출력 undefined

그래서 빼면 걸립니다.

이 코드는 검사에서 걸립니다

TS2741

```
const strictBroken: Strict = {};
```

Property 'memo' is missing in type '{}' but required in type 'Strict'.

“값이 undefined 인 것” 과 “속성이 아예 없는 것” 은 다릅니다.

거의 항상 ? 쪽(Loose)을 쓰게 됩니다.

Strict 같은 모양은 "빠뜨린 게 아니라 일부러 비운 것" 을 구분해야 할 때만 씁니다.

▸ 직접 해보기

2

```
const x: Loose = { memo: undefined }; 는 될까요?
```

써 보고 확인하세요.

readonly — 만든 뒤에는 못 바꾼다

섹션 3

```
type Receipt = {  
  readonly id: number;  
  menu: string;  
};  
  
const receipt: Receipt = { id: 1001, menu: "라떼" };  
  
receipt.menu = "아메리카노"; // 이걸 됩니다  
console.log(receipt.menu);
```

출력 아메리카노

이 코드는 검사에서 걸립니다

TS2540

```
receipt.id = 2002;
```

Cannot assign to 'id' because it is a read-only property.

"id 는 읽기 전용이라 바꿀 수 없습니다" 입니다.

영수증 번호처럼 한 번 정해지면 바뀌면 안 되는 값에 씁니다.

const 와 헷갈리지 마세요. 막는 대상이 다릅니다.

const receipt = ...	→ receipt 자체를 다른 객체로 바꾸는 것을 막는다
readonly id	→ 객체 '안' 의 id 를 바꾸는 것을 막는다

JS자료 06단원에서 "const 객체는 속성은 바꿀 수 있다" 고 배웠습니다. 그 구멍을 막는 것이 readonly 입니다.

배열에도 쓸 수 있습니다.

```
type Menu = { name: string; price: number };
const fixedMenus: readonly Menu[] = [
  { name: "아메리카노", price: 4000 },
];
console.log(fixedMenus.length);
```

출력 1

이 코드는 검사에서 걸립니다

TS2339

```
fixedMenus.push({ name: "라떼", price: 4500 });
```

Property 'push' does not exist on type 'readonly Menu[]'.

readonly 배열에는 push 자체가 없습니다.

"바꾸지 마세요" 가 아니라 "바꾸는 기능이 아예 없다" 로 막습니다.

▹ 직접 해보기 3

Receipt 의 menu 에도 readonly 를 붙이고

receipt.menu = "..." 를 해 보세요. 확인한 뒤 되돌리세요.

interface — 또 하나의 표기

섹션 4

같은 것을 이렇게도 쓸 수 있습니다.

```
interface Shop {
  name: string;
  seats: number;
}

const shop: Shop = { name: "봄날카페", seats: 24 };
console.log(shop.name, shop.seats);
```

출력 봄날카페 24

type 으로 쓴 것과 비교해 보세요.

```
type Shop = { name: string; seats: number };    ← = 가 있고 ; 로 끝난다
interface Shop { name: string; seats: number } ← = 가 없고 ; 가 없다
```

쓰는 쪽에서는 완전히 똑같습니다. 검사도 똑같이 합니다.

이 코드는 검사에서 걸립니다

TS2741

```
const brokenShop: Shop = { name: "봄날카페" };
```

Property 'seats' is missing in type '{ name: string; }' but required in type 'Shop'.

구조적 타이핑도 그대로입니다. type 으로 만든 것과 서로 오갑니다.

```
type ShopType = { name: string; seats: number };
const fromInterface: ShopType = shop;
console.log(fromInterface.seats);
```

출력 24

이름표가 아니라 모양만 보기 때문입니다(개념01 섹션4).

▸ 직접 해보기

4

interface 로 Customer(name: string, age: number)를 만들고

변수를 하나 만들어 출력해 보세요.

그럼 type 과 interface 중 뭘 쓰나

섹션 5

인터넷에 이 주제로 글이 아주 많습니다. 대부분은 지금 알 필요가 없습니다. 이 자료는 기준을 하나로 정합니다.

★ type 을 씁니다. 하나만 쓰세요.

이유는 이렇습니다.

① type 은 객체가 아닌 것에도 쓸 수 있습니다.

interface 는 객체 모양에만 씁니다.

```
type Calc = (a: number, b: number) => number;  ← 함수
type Id = string;                             ← 그냥 별명
type Status = "대기" | "완료";                 ← 05단원에서 배웁니다
```

interface 로는 위 셋을 못 쓰거나 어색합니다.
그러니 type 하나로 통일하는 편이 배울 것이 적습니다.

② 섞어 쓰면 "왜 여긴 이걸 썼지?" 를 매번 생각하게 됩니다.

그럼 interface 는 왜 배웠냐면, 남의 코드에 나오기 때문입니다. React 라이브러리 설명서도 interface 로 쓰인 것이 많습니다. 읽을 줄만 알면 됩니다. 쓸 때는 type 을 쓰세요.

참고로 딱 하나 실제로 다른 점이 있습니다. interface 는 같은 이름을 또 쓰면 합쳐집니다.

```
interface Box {
  width: number;
}
interface Box {
  height: number;
}
const box: Box = { width: 3, height: 4 };
console.log(box.width * box.height);
```

출력 12

type 으로 같은 짓을 하면 TS2300 Duplicate identifier 로 걸립니다. 이 '합쳐지는' 성질은 남의 라이브러리를 확장할 때 쓰는 것이고, 우리 코드에서는 오히려 실수를 놓치게 만듭니다. type 이 안전합니다.

▹ 직접 해보기

5

type Box2 = { width: number }; 를 두 번 써 보세요.

무슨 에러가 나나요?

자주 하는 실수

섹션 6

이런 실수를 합니다

? 를 붙여 놓고 그냥 쓰기

TS18048 입니다. 확인하고 쓰세요. 이 자료에서 두 번째로 자주 보게 됩니다.

이런 실수를 합니다

interface 뒤에 = 를 쓰기

interface Shop = { ... } 는 안 됩니다. type 만 = 를 씁니다. 반대로 type 에 = 를 빼먹는 실수도 흔합니다. 짝을 지어 외우세요.

이런 실수를 합니다

readonly 를 const 와 같은 것으로 알기

const 는 변수를, readonly 는 속성을 막습니다. 섹션 3을 보세요.

이런 실수를 합니다

type 과 interface 를 섞어 쓰기

동작에는 문제가 없지만 코드가 지저분해집니다. 하나로 정하세요.

정리

- 1 속성 뒤 ? 는 "없어도 된다". 대가로 타입에 `undefined` 가 섞인다.
- 2 `memo?: string` 과 `memo: string | undefined` 는 다르다. 앞은 빼도 되고, 뒤는 반드시 있어야 하되 값이 `undefined` 여도 된다.
- 3 `readonly` 는 객체 '안' 의 속성을 못 바꾸게 한다. `const` 와 막는 대상이 다르다.
- 4 `readonly` 배열에는 `push` 가 아예 없다.
- 5 `interface` 도 같은 일을 한다. 표기만 다르다(= 없음, ; 없음).
- 6 이 자료의 기준 — 쓸 때는 `type` 하나만. `interface` 는 읽을 줄만 알면 된다.

직접 해보기 정답

- 1 `type Order2 = { menu: string; count: number; memo?: string; isHot?: boolean }; const c: Order2 = { menu: "라떼", count: 1 }; console.log(c.isHot); // 출력: undefined`
- 2 됩니다. 에러가 나지 않습니다. ? 는 "빼도 되고, `undefined` 를 넣어도 된다" 입니다. 둘 다 허용합니다. 반대로 `Strict` 는 `undefined` 를 넣는 것만 되고 빼는 것은 안 됩니다.
- 3 error TS2540: Cannot assign to 'menu' because it is a read-only property. id 때와 똑같은 에러가 menu 에 대해 납니다. 재현:

```
type Receipt = { readonly id: number; readonly menu: string };
const receipt: Receipt = { id: 1001, menu: "라떼" };
receipt.menu = "아메리카노";
```

- 4 `interface Customer { name: string; age: number } const cust: Customer = { name: "홍길동", age: 25 }; console.log(cust.name, cust.age); // 출력: 홍길동 25`
- 5 error TS2300: Duplicate identifier 'Box2'. `type` 은 같은 이름을 두 번 만들 수 없습니다. `interface` 는 조용히 합쳐지므로, 오타로 같은 이름을 써도 안 걸립니다. 재현:

```
type Box2 = { width: number };
type Box2 = { width: number };
```

그래서 이 자료는 `type` 을 권합니다.

서버 데이터에 모양 적기

RUN node 개념04_서버_데이터에_모양_적기.ts

검사: npm run typecheck

02단원 개념03 섹션4에서 이런 이야기를 했습니다.

JSON.parse 의 결과는 any 다.
대응 방법은 "받자마자 이런 모양일 것이다 를 적어 주는 것" 이고,
그 적는 법이 04단원이다.

이제 적을 수 있게 됐으니 그 약속을 지킬 차례입니다. 그리고 이 파일에는 이 자료에서 가장 중요한 경고가 하나 들어 있습니다(섹션 3).

받자마자 모양을 적는다

섹션 1

```
type Menu = { name: string; price: number };
```

서버가 보내 준 문자열이라고 생각하세요.

```
const responseText = '{"name":"라떼","price":4500}';
```

전 · 아무것도 안 적으면 any 입니다.

```
const loose = JSON.parse(responseText);
console.log(loose.아무거나 === undefined);
```

출력 true

없는 이름인데 통과합니다. 검사가 꺼져 있습니다.

후 · 모양을 적어 주면 그때부터 검사가 살아납니다.

```
const menu: Menu = JSON.parse(responseText);
console.log(menu.name, menu.price);
```

출력 라떼 4500

이 코드는 검사에서 걸립니다

TS2339

```
console.log(menu.size);
```

Property 'size' does not exist on type 'Menu'.

이제 없는 이름을 막아 줍니다. menu. 까지 치면 자동완성도 뜹니다.

: Menu 한 번 적었을 뿐인데 이만큼 달라집니다.

▹ 직접 해보기 1

loose 와 menu 에 각각 마우스를 올려 보세요.

타입이 어떻게 다른가요?

계산도 안전해진다

섹션 2

```
const total: number = menu.price * 2;  
console.log(total);
```

출력 9000

menu.price 가 number 라고 적어 두었으니 곱셈이 통과합니다. loose.price 였다면 any 라 무엇을 해도 통과했을 것입니다.

만약 서버가 가격을 문자열로 준다면 모양도 그렇게 적어야 합니다.

```
type MenuFromLegacy = { name: string; price: string };  
const legacy: MenuFromLegacy = JSON.parse('{"name":"라떼","price":"4500"}');
```

그러면 곱셈을 막아 줍니다. 이게 정확한 동작입니다.

이 코드는 검사에서 걸립니다

TS2362

```
console.log(legacy.price * 2);
```

The left-hand side of an arithmetic operation must be of type 'any', 'number', 'bigint' or an enum type.

01단원 개념01 섹션2에서 본 그 에러입니다.

모양을 사실대로 적어 두면 이런 사고가 실행 전에 걸립니다.

```
console.log(Number(legacy.price) * 2);
```

출력 9000

◁ 직접 해보기 2

MenuFromLegacy 의 price 를 number 로 바꿔 보세요.

검사는 통과할까요? 실행 결과는 어떻게 될까요?

★ 타입은 '약속' 이지 '검사' 가 아니다

섹션 3

이 절이 이 파일의 핵심입니다. 안 읽으면 반드시 당합니다.

아래 JSON 에는 price 가 없습니다. 그런데도 : Menu 라고 적었습니다.

```
const broken: Menu = JSON.parse('{ "name": "라떼" }');
```

npx tsc --noEmit 은 조용합니다. 아무 말도 안 합니다.

```
console.log(broken.name);
```

출력 라떼

그런데 price 를 쓰는 순간 실행 중에 터집니다.

```
try {
  console.log(broken.price.toFixed(0));
} catch (e) {
  console.log("터졌습니다:", String(e));
}
```

출력 터졌습니다: TypeError: Cannot read properties of undefined (reading 'toFixed')

왜 이럴까요?

: Menu 라고 적는 것은 "이런 모양일 것이다" 라는 내 주장입니다.
타입스크립트는 그 주장을 검사하지 않고 믿습니다.

01단원 개념02에서 "node 는 타입을 지운다" 고 했습니다. 지워진 것은 실행할 때 아무 일도 하지 않습니다. 서버가 보낸 값이 실제로 그 모양인지 확인해 주는 코드는 어디에도 없습니다.

즉 이렇게 됩니다.

내가 쓴 코드끼리 안 맞는 것 → 타입스크립트가 잡아 준다
밖에서 들어온 값이 다른 것 → 타입스크립트가 못 잡는다

01단원 개념01 섹션4의 “타입이 못 잡는 것” 목록에 “서버가 안 보내 준 데이터가 생기지 않는다” 를 넣었던 이유가 이것입니다.

▹ 직접 해보기

3

`{"name":"라떼","price":"비쌈"}` 을 Menu 로 받아

`price * 2` 를 출력해 보세요. 검사는 통과하나요? 실행 결과는요?

그래서 확인이 필요하다

섹션 4

밖에서 온 값은 쓰기 전에 확인하는 것이 원칙입니다. 제대로 된 방법은 05단원(좁히기)에서 배우고, 여기서는 가장 단순한 형태만 봅니다.

```
function readMenu(text: string): Menu | null {
  const data = JSON.parse(text);
  if (typeof data.name === "string" && typeof data.price === "number") {
    return data;
  }
  return null;
}
```

```
const ok = readMenu('{"name":"라떼","price":4500}');
console.log(ok);
```

출력 `{ name: '라떼', price: 4500 }`

```
const bad = readMenu('{"name":"라떼"}');
console.log(bad);
```

출력 `null`

이제 부르는 쪽에서 확인을 강제당합니다.

이 코드는 검사에서 걸립니다

TS18047

```
console.log(bad.name);
```

`'bad' is possibly 'null'.`

“null 일 수도 있는데 그냥 쓰시겠다고요?” 입니다.

`Menu | null` 이라고 돌려주기로 했으니 막아 줍니다.
이 세로줄(`|`)과 확인하는 법이 05단원의 전부입니다.

```
if (bad === null) {
  console.log("서버 응답이 예상과 다릅니다");
}
```

출력 서버 응답이 예상과 다릅니다

실무에서는 이 확인을 손으로 안 쓰고 zod 같은 도구에 맡깁니다. 08단원에서 이름만 소개합니다. 지금은 “확인이 필요하다” 만 기억하세요.

◀ 직접 해보기 4

readMenu 에 '{"name":123,"price":4500}' 을 넘겨 보세요.

결과가 무엇일지 예상한 뒤 확인하세요.

목록으로 올 때

섹션 5

서버는 대개 하나가 아니라 목록을 보냅니다. 뒤에 [] 를 붙이면 됩니다.

```
const listText = '["name":"아메리카노","price":4000],{"name":"라떼","price":4500}';
const menus: Menu[] = JSON.parse(listText);

console.log(menus.length);
```

출력 2

```
console.log(menus.map((m) => m.name));
```

출력 ['아메리카노', '라떼']

```
const sum = menus.reduce((acc, m) => acc + m.price, 0);
console.log(sum);
```

출력 8500

콜백 매개변수 m 에 타입을 안 적어도 되는 것에 주목하세요. menus 가 Menu[] 라고 적혀 있으니 문맥에서 알아냅니다(03단원 개념03).

그리고 섹션 3의 경고는 여기서도 그대로입니다. 목록 안의 어느 하나가 price 를 안 갖고 있어도 검사는 조용합니다.

◀ 직접 해보기 5

menus 에서 4200원 이상인 메뉴의 이름만 뽑아 출력해 보세요.

이런 실수를 합니다

: Menu 를 적었으니 안전하다고 믿기

섹션 3입니다. 적는 것은 주장이지 확인이 아닙니다. 내가 쓴 코드끼리는 맞춰 주지만, 서버가 다른 걸 보내면 못 잡습니다.

이런 실수를 합니다

서버 응답을 그대로 any 로 쓰기

가장 흔한 any 유입 경로입니다. 적는 데 한 줄이면 됩니다.

이런 실수를 합니다

실제 응답과 다른 모양을 적기

서버가 문자열로 주는데 number 라고 적으면, 실행 전까지 아무도 모릅니다. 응답을 한 번은 눈으로 찍어 보고 적으세요(`console.log(loose)` 로 충분합니다).

이런 실수를 합니다

목록인데 [] 를 안 붙이기

Menu 라고 적고 배열을 받으면, `menus.length` 가 TS2339 로 걸립니다. 반대로 안 걸리고 넘어가는 경우도 있어 더 위험합니다.

정리

- 1 `JSON.parse` 결과에 : 타입 을 적어 주면 그때부터 검사와 자동완성이 살아난다.
- 2 응답이 문자열로 오면 모양도 string 으로 적어야 한다. 사실대로 적는 것이 원칙.
- 3 ★ 타입은 약속이지 검사가 아니다. 실제 값이 달라도 조용히 통과하고 쓰는 순간 실행 중에 터진다.
- 4 내 코드끼리 안 맞는 것은 잡아 주고, 밖에서 들어온 값은 못 잡는다.
- 5 그래서 밖에서 온 값은 쓰기 전에 확인한다. 제대로 된 방법은 05단원.
- 6 목록은 뒤에 [] 를 붙인다. 콜백 매개변수는 문맥에서 알아낸다.

직접 해보기 정답

1 loose 는 any, menu 는 Menu 로 나옵니다. loose 는 무엇을 해도 통과하고, menu 는 적어 둔 것만 됩니다. 한 줄 차이로 검사가 켜지고 꺼집니다.

2 검사는 통과합니다. 그리고 출력도 9000 으로 '맞게' 나옵니다. price 는 실제로 문자열 "4500" 인데 number 라고 적었는데도 그렇습니다. 곱하기가 문자열을 숫자로 바꿔 주기 때문입니다("4500" * 2 = 9000).

★ 이게 가장 나쁜 경우입니다. 틀렸는데 맞게 나오니까요. 같은 값으로 더하기를 하면 바로 드러납니다.

```
console.log(legacy.price + 2); → 45002
```

01단원 개념01의 "+ 만 특별하다" 가 여기서 다시 나옵니다. 검사도 통과하고 곱셈도 맞으니, 덧셈을 쓰는 날까지 아무도 모릅니다. → 모양은 실제 응답 그대로 적어야 합니다.

3 검사는 통과합니다. 실행하면 NaN 이 나옵니다. "비쌘" * 2 는 NaN 입니다. 예러도 안 납니다. 섹션 3의 경고 그대로입니다. 화면에 NaN 이 찍히고 나서야 알게 됩니다.

4 null 이 나옵니다. name 이 문자열이 아니라 숫자 123 이라 첫 번째 조건에서 걸러집니다. price 는 맞지만 하나라도 어긋나면 null 입니다.

5 console.log(menus.filter((m) ⇒ m.price >= 4200).map((m) ⇒ m.name)); // 출력: ['라떼']

객체와 타입 별칭

RUN node 연습문제.ts

채점: npm run grade ← 출력을 기대 출력과 맞춰 봅니다

검사: npm run check ← 타입 검사 (일부 문제만 잡습니다)

★ 채점은 npm run grade 로 합니다. 실제 출력을 문제의 '기대 출력' 과 줄 단위로 맞춰 봅니다. 전부 일치하면 다 맞은 것입니다.

npm run check(타입 검사)도 같이 쓰세요. 다만 이쪽은 '일부 문제만' 잡습니다.

138문항 중 21개뿐입니다. "출력하세요" 류는 안 풀어도 타입 에러가 안 나거든요. 그러니 check 가 조용하다고 다 맞은 것이 아닙니다. grade 로 확인하세요. (npm run typecheck 는 개념·정답 파일만 봅니다. 그쪽은 언제나 조용합니다)

- 1 TODO 자리에 코드를 씁니다.
- 2 npm run grade 로 채점합니다. (직접 node 연습문제.ts 로 돌려 봐도 됩니다)
- 3 npm run check 로 타입도 봅니다.
- 4 10분 고민해도 안 되면 연습문제_정답.ts 를 보세요.

문제 1~8은 기본, 9~13은 응용, 14는 [도전]입니다. 문제 15는 에러를 직접 보는 문제라 맨 뒤에 있습니다.

```
console.log("=== 04단원 연습문제 ===");
```

문제 1 (개념01 섹션1)

함수 매개변수에 객체 타입을 적으세요.

【이번에 배울 방법】 — 값 자리에 종류를 씁니다

```
{ name: string; price: number }
```

기대 출력 봄날카페 24석

기대 검사 결과: 조용함

```
{
```


매개변수에 { name: string; seats: number } 를 적으세요

```
function printShop(shop) {
  console.log(shop.name + " " + shop.seats + "석");
}
printShop({ name: "봄날카페", seats: 24 });
}
```

04

문제 2 (개념02 섹션1)

위 모양에 Shop 이라는 이름을 붙이고, 그 이름을 써서 변수를 만드세요.

기대 출력 별빛카페 12

기대 검사 결과: 조용함

```
{
```

type Shop 을 만들고 변수 하나를 만들어 출력하세요

```
}
```

문제 3 (개념01 섹션5 · 개념02 섹션1)

Menu 타입을 만들고, Menu 배열에서 이름만 뽑아 출력하세요.

기대 출력

— ‘아메리카노’, ‘라떼’

기대 검사 결과: 조용함

```
{
```

type Menu 를 만들고 menus 에 타입을 붙인 뒤 이름만 뽑으세요

```
const menus = [  
  { name: "아메리카노", price: 4000 },  
  { name: "라떼", price: 4500 },  
];  
}
```

문제 4 (개념03 섹션1)

memo 를 '없어도 되는' 속성으로 만드세요.

기대 출력 라떼 undefined

기대 검사 결과: 조용함

```
{
```

memo 를 선택 속성으로 바꾸세요

```
type Order = { menu: string; memo: string };  
const o: Order = { menu: "라떼" };  
console.log(o.menu, o.memo);  
}
```

문제 5 (개념03 섹션3)

id 를 만든 뒤에는 못 바꾸도록 만드세요. (아래 id 를 바꾸는 줄이 검사에서 걸리게 되면 성공입니다. 그 줄은 지우세요)

기대 출력 1001 아메리카노

기대 검사 결과: 조용함

```
{
```

`id` 에 `readonly` 를 붙이고, `id` 를 바꾸는 줄을 지우세요

```
type Receipt = { id: number; menu: string };
const r: Receipt = { id: 1001, menu: "라떼" };
r.id = 2002;
r.menu = "아메리카노";
console.log(r.id, r.menu);
}
```

04

문제 6 (개념02 섹션3)

‘숫자 두 개를 받아 숫자를 돌려주는 함수’ 타입에 `Calc` 라는 이름을 붙이고, 두 수를 더하는 함수를 담아 호출하세요.

기대 출력 30

기대 검사 결과: 조용함

```
{
```

`type Calc` 를 만들고 `add` 를 담아 `add(10, 20)` 을 출력하세요

```
}
```

문제 7 (개념04 섹션1)

`JSON.parse` 결과에 모양을 적어 주세요.

기대 출력 라떼 4500

기대 검사 결과: 조용함

```
{  
  type Menu = { name: string; price: number };
```

parsed 에 Menu 를 적으세요

```
const parsed = JSON.parse('{ "name": "라떼", "price": 4500 }');  
console.log(parsed.name, parsed.price);  
}
```

문제 8 (개념04 섹션5)

목록으로 온 응답에 모양을 적고, 가격의 합을 출력하세요.

기대 출력 8500

기대 검사 결과: 조용함

```
{  
  type Menu = { name: string; price: number };  
  const text = ' [{ "name": "아메리카노", "price": 4000 }, { "name": "라떼", "price": 4500 } ]';
```

list 에 모양을 적고 합을 출력하세요

```
}
```

문제 9 (응용 · 개념01 섹션2)

아래 세 줄은 각각 다른 에러가 납니다. 에러 번호를 순서대로 출력하세요.

1번: printMenu("라떼") (객체 자리에 문자열) 2번: printMenu({ name: "라떼" }) (price 가 빠짐) 3번: printMenu({ name: "라떼", price: "비쌘" }) (price 종류가 다름)

기대 출력 TS2345
 TS2741
 TS2322

기대 검사 결과: 조용함

```
{
```

세 줄을 출력하세요

```
}
```

문제 10 (응용 · 개념01 섹션3·4)

아래 두 줄 중 '초과 속성 검사'에 걸리는 것은 몇 번일까요?

1번: printMenu({ name: "라떼", price: 4500, size: "L" }) 2번: `const m = { name: "라떼", price: 4500, size: "L" }; printMenu(m)`

기대 출력 1

기대 검사 결과: 조용함

```
{
```

번호를 출력하세요

```
}
```

문제 11 (응용 · 개념03 섹션1)

아래 함수는 검사에서 걸립니다. 걸리지 않게 고치세요. memo 가 없으면 "(메모 없음)" 을 찍으세요.

기대 출력 얼음 적게 (5글자)
 (메모 없음)

기대 검사 결과: 조용함

```
{  
  type Order = { menu: string; memo?: string };
```

걸리지 않게 고치고, 아래 두 줄의 주석도 푸세요

```
function showMemo(o: Order) {  
  console.log(o.memo + " (" + o.memo.length + "글자)");  
}
```

```
showMemo({ menu: "라떼", memo: "얼음 적게" });  
showMemo({ menu: "라떼" });
```

```
}
```

문제 12 (응용 · 개념02 섹션4)

Shop 과 Menu 를 조합해서, 가게 하나와 메뉴 목록을 함께 담는 Store 타입을 만들고 두 번째 메뉴의 가격을 출력하세요.

기대 출력 4500

기대 검사 결과: 조용함

```
{  
  type Shop = { name: string; seats: number };  
  type Menu = { name: string; price: number };
```

type Store 를 만들고 값을 하나 만들어 두 번째 메뉴 가격을 출력하세요

```
}
```

문제 13 (응용 · 개념04 섹션3)

아래 코드는 검사를 통과합니다. 실행하면 무엇이 출력될까요? 예상한 값을 그대로 출력하세요.

```
type Menu = { name: string; price: number };
const m: Menu = JSON.parse('{"name":"라떼"}');
console.log(m.price);
```

기대 출력 undefined

기대 검사 결과: 조용함

```
{
```

예상한 값을 출력하세요

```
}
```

문제 14 ([도전] · 개념01~04 종합)

서버 응답에서 '재고가 있는 메뉴' 만 골라 총액을 구하세요. ① Menu 타입을 만든다 (name: string, price: number, stock?: number) ② 응답에 모양을 적는다 ③ stock 이 있고 0보다 큰 것만 고른다 ④ 결과를 "재고 있는 메뉴 합계: 8500원" 형태로 출력한다

힌트: stock 은 선택 속성이라 그냥 비교하면 걸립니다.

기대 출력 재고 있는 메뉴 합계: 8500원

기대 검사 결과: 조용함

```
{
  const text =
    '["name":"아메리카노","price":4000,"stock":3],' +
    '{"name":"아이스티","price":5000,"stock":0},' +
    '{"name":"라떼","price":4500,"stock":1},' +
    '{"name":"폼질메뉴","price":9000}]';
```

위 네 가지를 하세요

```
}
```

문제 15 (에러 확인 · 개념04 섹션3)

아래 세 줄의 // 를 지우고,

① npm run check 로 검사해 보세요 → 걸릴까요? ② node 연습문제.ts 로 실행해 보세요 → 어떻게 될까요?

“타입은 약속이지 검사가 아니다” 를 직접 보는 문제입니다. 확인했으면 다시 // 를 붙이세요.

```
{

  type Menu = { name: string; price: number };
  const m: Menu = JSON.parse('{"name":"라떼"}');
  console.log(m.price.toFixed(0));

}
```


유니온과 타입 좁히기

OPEN 05_유니온과_타입_좁히기

05

```
type Id = string | number;
function printId(id: Id) {
  console.log("번호:", id);
}
printId(3);
```

01 유니온: 둘 중 하나

02 좁히기: 확인하면 쓸 수 있다 ★ 실무 체감 1위

03 null 과 undefined 다루기

04 판별 유니온: 모양이 여러 가지일 때

- 연습문제

유니온: 둘 중 하나

RUN node 개념01_유니온.ts

검사: npm run typecheck

세로줄(|)은 앞 단원들에서 계속 스쳐 지나갔습니다.

```
string | undefined    ? 를 붙였을 때 (03단원 개념02)
Menu | null           없을 수도 있을 때 (04단원 개념04)
(string | number)[]   배열에 섞여 있을 때 (02단원 개념01)
```

그때마다 “05단원에서 배웁니다” 라고 미뤘습니다. 그 단원입니다.

이 단원은 실무에서 가장 자주 쓰게 되는 단원이기도 합니다.

A | B는 “둘 중 하나”

섹션 1

게시글 번호가 숫자로 올 때도 있고 문자열로 올 때도 있다고 해 봅시다.

```
type Id = string | number;

function printId(id: Id) {
  console.log("번호:", id);
}

printId(3);
```

출력 번호: 3

```
printId("A-3");
```

출력 번호: A-3

셋 이상도 됩니다. 개수 제한은 없습니다.

```
type Value = string | number | boolean;

function printValue(v: Value) {
  console.log("값:", v);
}

printValue(true);
```

출력 값: true

둘 다 아닌 것은 막습니다.

이 코드는 검사에서 걸립니다

TS2345

```
printId(null);
```

Argument of type 'null' is not assignable to parameter of type 'Id'.

string 도 number 도 아니니 걸립니다.

null 을 허용하려면 `string | number | null` 이라고 적어야 합니다.

▸ 직접 해보기

1

숫자이거나 불리언인 타입 Flag 를 만들고

두 종류를 각각 넘겨 출력해 보세요.

유니온에서는 '공통으로 있는 것' 만 쓸 수 있다 ★

섹션 2

이게 유니온의 핵심 규칙입니다.

```
function shout(value: string | number) {
```

toString 은 문자열에도 숫자에도 있으니 됩니다.

```
console.log(value.toString());
}
shout("라떼");
```

출력 라떼

```
shout(4500);
```

출력 4500

이 코드는 검사에서 걸립니다

TS2339

```
function shoutWrong(value: string | number) {
  console.log(value.toUpperCase());
}
```

Property 'toUpperCase' does not exist on type 'string | number'.

toUpperCase 는 문자열에만 있습니다.

숫자가 들어왔을 때는 없는 기능이니, 타입스크립트가 미리 막습니다.
JS에서는 숫자가 들어오는 날 실행 중에 터지던 자리입니다.

메시지를 두 줄까지 읽으면 어느 쪽이 문제인지 알려 줍니다.

이 코드는 검사에서 걸립니다

TS2339

```
function roundIt(value: string | number) {  
  return value.toFixed(1);  
}
```

Property 'toFixed' does not exist on type 'string | number'.

아래에 Property 'toFixed' does not exist on type 'string'. 이 따라옵니다.

"문자열 쪽에 없다" 를 짚어 줍니다.

그럼 유니온은 못 써먹는 것 아닌가? 아닙니다. "지금 어느 쪽인지" 를 확인하면 그때부터 그쪽 기능을 다 쓸 수 있습니다. 그 확인하는 방법이 개념02(좁히기)입니다.

▹ 직접 해보기

2

string | number 를 받아 length 를 출력하려고 해 보세요.

어느 쪽에 없다고 하나요? 확인한 뒤 지우세요.

리터럴 유니온 — 정해진 값만 허용하기 ★

섹션 3

여기가 실무에서 가장 많이 쓰는 자리입니다. 02단원 개념02에서 "const 는 값 자체가 타입이 된다" 고 했습니다. 그 리터럴 타입을 | 로 이으면 "이 값들 중 하나만" 이 됩니다.

```
type Status = "대기" | "조리중" | "완료";
```

```
function printStatus(status: Status) {  
  console.log("상태:", status);  
}
```

```
printStatus("대기");
```

출력 상태: 대기

```
printStatus("완료");
```

출력 상태: 완료

목록에 없는 값은 막습니다.

이 코드는 검사에서 걸립니다

TS2345

```
printStatus("취소");
```

Argument of type '"취소"' is not assignable to parameter of type 'Status'.

오타도 이걸로 잡힙니다. "완료" 라고 쓰면 그 자리에서 걸립니다.

JS에서는 오타가 조용히 통과해서, `if (status === "완료")`가 영원히 `false`가 되는 버그로 나타났습니다.

자동완성까지 됩니다. `printStatus()` 까지 치면 세 개가 목록으로 뜹니다. 외우고 있을 필요가 없습니다.

숫자도 됩니다.

```
type Rating = 1 | 2 | 3 | 4 | 5;
function printRating(r: Rating) {
  console.log("별점:", r);
}
printRating(5);
```

출력 별점: 5

이 코드는 검사에서 걸립니다

TS2345

```
printRating(6);
```

Argument of type '6' is not assignable to parameter of type 'Rating'.

별점이 6점이 되는 사고를 타입이 막아 줍니다.

01단원에서 "타입은 값이 옳은지는 안 본다" 고 했는데, 리터럴 유니온은 값의 범위까지 좁혀 주는 예외적인 도구입니다.

➤ 직접 해보기

3

"S" | "M" | "L" 인 Size 타입을 만들고

함수 하나를 만들어 "XL" 을 넘겨 보세요. 확인한 뒤 지우세요.

그런데 변수에서는 그냥 되던데요?

섹션 4

섹션 2를 읽고 나서 이걸 해 보면 헛갈립니다.

```
const fixedId: Id = "A-3";
console.log(fixedId.toUpperCase());
```

출력 A-3

id 라고 적었는데 문자열 기능이 그냥 됩니다. let 도 마찬가지입니다.

```
let movingId: Id = "A-3";
console.log(movingId.toUpperCase());
```

출력 A-3

왜냐하면 타입스크립트가 '지금 그 자리에 무엇이 들어 있는지' 를 보고 있기 때문입니다. 눈에 보이는 값을 대입했으니, 그 순간부터 그 종류로 취급합니다.

그래서 다른 종류를 넣으면 그때부터 바뀝니다.

```
movingId = 3;
console.log(movingId.toFixed(1));
```

출력 3.0

이 코드는 검사에서 걸립니다

TS2339

```
console.log(movingId.toUpperCase());
```

Property 'toUpperCase' does not exist on type 'number'.

방금 위에서는 됐는데 여기서는 안 됩니다.

같은 변수인데 줄에 따라 다릅니다. 3 을 넣은 아래부터는 숫자니까요.
메시지도 'string | number' 가 아니라 'number' 라고 나옵니다.

그럼 섹션 2는 언제 적용되는가 —

무엇이 들어올지 '모르는' 자리에서 적용됩니다.

함수 매개변수가 대표적입니다. 누가 무엇을 넘길지 알 수 없습니다.

```
function idLength(id: Id) {
```

이 코드는 검사에서 걸립니다

TS2339

```
return id.toUpperCase();
```

Property 'toUpperCase' does not exist on type 'Id'.

```
return String(id).length;
}
console.log(idLength("A-3"), idLength(3));
```

출력 3 1

함수가 돌려준 값도 마찬가지입니다. 실행해 봐야 아는 값이니깐요.

정리하면 이렇습니다.

눈에 보이는 값을 대입했다 → 그 종류로 좁혀진다. 바로 쓸 수 있다
 밖에서 들어온 값이다 → 유니온 그대로다. 확인해야 쓸 수 있다

그리고 실무 코드에서 유니온을 만나는 자리는 거의 다 아래쪽입니다.

▸ 직접 해보기

4

movingId = 3; 아래에 movingId = "B-7"; 을 한 줄 넣고

그 아래에서 .toUpperCase() 를 써 보세요. 이번에는 될까요?

05

배열에서 헛갈리는 자리

섹션 5

두 표기는 완전히 다릅니다. 괄호 하나 차이입니다.

```
type MixedArray = (string | number)[]; // 문자열이거나 숫자인 것들의 배열
type TwoArrays = string[] | number[]; // 문자열 배열이거나 숫자 배열
const mixed: MixedArray = ["라떼", 4500, "아메리카노"];
console.log(mixed.length);
```

출력 3

```
const onlyStrings: TwoArrays = ["라떼", "아메리카노"];
console.log(onlyStrings.length);
```

출력 2

이 코드는 검사에서 걸립니다

TS2322

```
const wrong: TwoArrays = ["라떼", 4500];
```

Type '(string | number)[]' is not assignable to type 'TwoArrays'.

TwoArrays 는 '섞인 배열' 을 허용하지 않습니다.

전부 문자열이거나 전부 숫자여야 합니다.

메시지를 보면 내가 넣은 것이 (string | number)[] 로 추론됐고,
 그것이 TwoArrays 에 안 들어간다고 말하고 있습니다.

왼쪽이 내가 넣은 것, 오른쪽이 필요한 것 - 01단원 개념03의 그 규칙입니다.

어느 쪽이 필요한지 헷갈리면 이렇게 생각하세요.

(A | B)[] → 한 배열 안에 A 와 B 가 섞여 있어도 된다
A[] | B[] → 배열 자체가 A 전용이거나 B 전용이다

▫ 직접 해보기

5

mixed 의 첫 번째 값에 .toUpperCase() 를 써 보세요.

걸리나요? 섹션 2와 무슨 관계일까요?

자주 하는 실수

섹션 6

이런 실수를 합니다

유니온이면 양쪽 기능을 다 쓸 수 있다고 생각하기

반대입니다. 공통으로 있는 것만 됩니다. 섹션 2가 이 단원의 핵심입니다.

이런 실수를 합니다

리터럴 유니온에 큰따옴표를 빼먹기

type Status = 대기 | 완료 는 안 됩니다. "대기" | "완료" 입니다. 따옴표가 없으면 그 이름의 '타입' 을 찾다가 TS2304 로 걸립니다.

이런 실수를 합니다

(A | B)[] 와 A[] | B[] 를 같은 것으로 알기

섹션 5입니다. 괄호 위치가 뜻을 바꿉니다.

이런 실수를 합니다

유니온을 너무 길게 만들기

string | number | boolean | null | undefined 처럼 다 넣으면 공통으로 있는 것이 거의 없어져서 아무것도 못 하게 됩니다. 그럴 때는 정말 그 모두가 필요한지 다시 생각해 보세요.

정리

- 1 $A \mid B$ 는 "둘 중 하나" 다. 개수 제한은 없다.
- 2 ★ 유니온 값으로는 '양쪽에 공통으로 있는 것' 만 쓸 수 있다. 한쪽에만 있는 기능은 TS2339 로 막힌다.
- 3 리터럴 값을 $|$ 로 이으면 "정해진 값만" 이 된다. 오타와 잘못된 값을 막아 준다. 자동완성까지 되어서 실무에서 가장 많이 쓴다.
- 4 `const` 든 `let` 이든 눈에 보이는 값을 대입했으면 그 종류로 좁혀져 바로 쓸 수 있다. 밖에서 들어오는 값 (매개변수)만 유니온 그대로다. 확인해야 쓸 수 있다.
- 5 $(A \mid B)[]$ 는 섞인 배열, $A[] \mid B[]$ 는 한 종류로만 채운 배열이다.
- 6 "지금 어느 쪽인지" 를 확인하면 그쪽 기능을 다 쓸 수 있다 → 개념02.

직접 해보기 정답

- 1

```
type Flag = number | boolean; function printFlag(f: Flag) { console.log(f); }
printFlag(1); // 출력: 1
printFlag(true); // 출력: true
```

- 2 error TS2339: Property 'length' does not exist on type 'string | number'.

Property 'length' does not exist on type 'number'.

재현:

```
function f(v: string | number) { return v.length; }
```

숫자 쪽에 없다고 알려 줍니다. 두 번째 줄까지 읽어야 어느 쪽인지 나옵니다.

```
3) type Size = "S" | "M" | "L";
```

error TS2345: Argument of type '"XL"' is not assignable to parameter of type 'Size'. 재현:

```
type Size = "S" | "M" | "L";
function printSize(s: Size) { console.log(s); }
printSize("XL");
```

목록에 없는 값이라 걸립니다.

4 됩니다. "B-7" 을 넣은 아래부터는 다시 문자열로 좁혀지기 때문입니다. 한 변수의 타입이 줄에 따라 string → number → string 으로 바뀐 것입니다. 타입스크립트는 코드가 흘러가는 순서를 따라가며 판단합니다. 이 성질이 개념02(좁히기)의 바탕입니다.

5 걸립니다. error TS2339: Property 'toUpperCase' does not exist on type 'string | number'. mixed[0] 을 꺼내면 그 값의 타입이 string | number 입니다. 재현:

```
const mixed: (string | number)[] = ["라떼", 4500];  
console.log(mixed[0].toUpperCase());
```

섹션 2의 규칙이 그대로 적용됩니다. 배열이라고 예외가 아닙니다.

좁히기: 확인하면 쓸 수 있다 ★ 실무 체감 1위

RUN node 개념02_좁히기.ts

검사: npm run typecheck

개념01에서 이렇게 끝났습니다.

유니온 값으로는 공통으로 있는 것만 쓸 수 있다.
"지금 어느 쪽인지" 를 확인하면 그쪽 기능을 다 쓸 수 있다.

그 '확인' 을 타입 좁히기(narrowing)라고 합니다.

문법을 새로 배우는 게 아닙니다. JS자료에서 쓰던 if · typeof 그대로입니다. 달라진 것은 타입스크립트가 그 if 를 읽고 안쪽에서 타입을 바꿔 준다는 것뿐입니다.

typeof 로 좁히기

섹션 1

```
function describe(value: string | number) {
  if (typeof value === "string") {
```

이 안에서 value 는 string 입니다

```
    console.log("글자 " + value.length + "자");
  } else {
```

여기서는 number 입니다

```
    console.log("숫자 " + value.toFixed(1));
  }
}

describe("아메리카노");
```

출력 글자 5자

```
describe(4500);
```

출력 숫자 4500.0

if 안에서 .length 를, else 에서 .toFixed 를 썼습니다. 개념01 섹션2에서는 둘 다 막혔던 것입니다.

재미있는 것은 else 쪽입니다. “문자열이 아니다” 만 알려 줬는데, string | number 에서 string 을 빼면 number 밖에 안 남으니 알아서 number 로 봅니다.

▹ 직접 해보기

1

boolean 을 유니온에 추가하고(string | number | boolean)

describe 를 돌려 보세요. else 쪽이 여전히 통과하나요?

=== 로 좁히기 (리터럴 유니온)

섹션 2

```
type Status = "대기" | "조리중" | "완료";

function statusMessage(status: Status) {
  if (status === "대기") {
    return "곧 시작합니다";
  } else if (status === "조리중") {
    return "만들고 있어요";
  } else {
```

여기서 status 는 “완료” 하나만 남았습니다

```
    return "나왔습니다";
  }
}

console.log(statusMessage("대기"));
```

출력 곧 시작합니다

```
console.log(statusMessage("완료"));
```

출력 나왔습니다

switch 로 써도 똑같이 좁혀집니다. 이쪽이 더 읽기 좋을 때가 많습니다.

```
function statusEmoji(status: Status): string {
  switch (status) {
    case "대기":
      return "[ ]";
    case "조리중":
      return "[.]";
    case "완료":
      return "[OK]";
  }
}
console.log(statusEmoji("조리중"));
```

출력 [.]

위 함수에 `return` 이 없는 길이 없다는 점에 주목하세요. 세 경우를 다 적었으니 타입스크립트가 “다 덮었다”고 인정해 줍니다. 하나라도 빠뜨리면 이렇게 됩니다.

이 코드는 검사에서 걸립니다

TS2366

```
function statusEmojiBroken(status: Status): string {
  switch (status) {
    case "대기":
      return "[ ]";
    case "조리중":
      return "[.]";
  }
}
```

Function lacks ending return statement and return type does not include 'undefined'.

“완료” 일 때 돌려줄 것이 없다는 뜻입니다.

나중에 `Status` 에 “취소” 를 추가하면, 이 함수가 그 자리에서 걸립니다.

→ 상태를 하나 늘렸을 때 고쳐야 할 곳을 타입이 전부 찾아 줍니다.

이게 리터럴 유니온을 쓰는 가장 큰 이득입니다.

▫ 직접 해보기 2

`Status` 에 “취소” 를 추가해 보세요.

어느 함수가 걸리나요? 확인한 뒤 되돌리세요.

```
function printNames(value: string | string[]) {
  if (Array.isArray(value)) {
    console.log(value.join(", "));
  } else {
    console.log(value);
  }
}

printNames("아메리카노");
```

출력 아메리카노

```
printNames(["아메리카노", "라떼"]);
```

출력 아메리카노, 라떼

typeof 로는 배열을 못 가립니다. 배열의 typeof 는 "object" 이기 때문입니다.

```
console.log(typeof ["a"]);
```

출력 object

JS자료 06단원에서 배운 그 함정입니다. 그래서 Array.isArray 를 씁니다.

▹ 직접 해보기 3

number | number[] 를 받아

하나면 그대로, 배열이면 합계를 출력하는 함수를 만들어 보세요.

그냥 if (값) 로 좁히기 — 그리고 함정

03단원·04단원에서 계속 쓰던 방법입니다.

```
function showMemo(memo?: string) {
  if (memo) {
    console.log("메모:", memo.length, "자");
  } else {
    console.log("메모 없음");
  }
}

showMemo("얼음 적게");
```

출력 메모: 5 자

```
showMemo();
```

출력 메모 없음

편하지만 함정이 있습니다. JS자료 01단원의 falsy 목록을 떠올리세요.

```
0 · "" · null · undefined · NaN · false
```

이 값들은 '있는데도' else 로 갑니다.

```
function showCount(count?: number) {
  if (count) {
    console.log("수량:", count);
  } else {
    console.log("수량 없음");
  }
}
showCount(3);
```

출력 수량: 3

```
showCount(0);
```

출력 수량 없음

← 0잔이라고 분명히 넘겼는데 "없음" 이 났습니다. 이게 함정입니다.

그래서 숫자·문자열에는 `undefined` 인지를 직접 물어봅니다.

```
function showCountSafe(count?: number) {
  if (count !== undefined) {
    console.log("수량:", count);
  } else {
    console.log("수량 없음");
  }
}
showCountSafe(0);
```

출력 수량: 0

규칙으로 정리하면 이렇습니다.

객체·배열	→ if (값) 로 충분하다
숫자·문자열	→ if (값 !== undefined) 로 물어봐야 한다 (0 과 "" 이 있는 값인데도 falsy 라서)

showMemo 에 "" (빈 문자열)을 넘겨 보세요.

무엇이 출력되나요? 그게 맞는 동작일까요?

좁힌 것은 어디까지 유지되나

섹션 5

```
function process(value: string | number) {
  if (typeof value !== "string") {
```

문자열이 아니면 여기서 끝냅니다

```
    console.log("숫자라서 그냥 씁니다:", value.toFixed(0));
    return;
  }
```

return 으로 걸러 났으니, 이 아래는 전부 string 입니다

```
    console.log(value.toUpperCase());
    console.log(value.length);
  }
```

```
process(4500);
```

출력 숫자라서 그냥 씁니다: 4500

```
process("latte");
```

출력 LATTE
5

if 블록 안이 아니라 '그 아래 전체' 가 좁혀졌습니다. 아니면 먼저 돌려보내는 이 방식을 '이른 반환' 이라고 하고, 중첩이 깊어지는 것을 막아 주어 실무에서 많이 씁니다.

다만 좁힌 뒤에 다른 값을 넣으면 그때부터 풀립니다.

```
function reassign(value: string | number) {
  if (typeof value === "string") {
    console.log(value.length);
    value = 100; // 여기서 숫자를 넣었습니다
```


이 코드는 검사에서 걸립니다

TS2339

```
console.log(value.length);
```

Property 'length' does not exist on type 'number'.

같은 if 안인데도 아래에서는 안 됩니다.

타입스크립트는 블록이 아니라 '줄 순서' 를 따라갑니다.

```
console.log(value.toFixed(0));
}
}
reassign("latte");
```

출력 5
 100

▹ 직접 해보기 5

process 에서 `return`; 을 지워 보세요.

아래 두 줄이 걸리나요? 왜 그럴까요?

자주 하는 실수

섹션 6

이런 실수를 합니다

typeof 결과를 오타로 쓰기

`typeof v === "sting"` 은 영원히 `false` 입니다. 다행히 이걸 타입스크립트가 잡아 줍니다(TS2367). JS 에서는 조용히 안 걸리는 버그였습니다.

이런 실수를 합니다

배열을 `typeof` 로 가리려 하기

`typeof []` 는 "object" 입니다. `Array.isArray` 를 쓰세요.

이런 실수를 합니다

if (count) 로 숫자를 확인하기

0 이 없는 것 취급됩니다. 섹션 4의 함정입니다. 실무 버그로 가장 자주 나오는 자리입니다.

이런 실수를 합니다

좁혀 놓고, 그 변수에 나중에 다시 값을 넣기

```
if (memo) { setTimeout(() => memo.length, 100) } 자체는 통과합니다.
```

그런데 같은 함수 어딘가에서 `memo = "z"`; 처럼 다시 대입하면 콜백 안의 좁힘이 풀려 TS18048 이 납니다. 나중에 실행되니 확신할 수 없어서입니다. 그럴 때는 `const m = memo;` 처럼 상수에 담아 두면 풀립니다.

정리

- 1 유니온은 확인하면 그쪽 기능을 다 쓸 수 있다. 이것이 좁히기다.
- 2 문법은 JS 그대로다. `typeof` · `===` · `switch` · `Array.isArray` · `if (값)`.
- 3 `else` 쪽은 "나머지" 로 알아서 좁혀진다. 두 개짜리 유니온이면 하나만 남는다.
- 4 `switch` 로 리터럴 유니온을 다 덮으면 `return` 이 빠졌다는 에러가 안 난다. 나중에 값을 하나 추가하면 고쳐야 할 곳을 타입이 전부 찾아 준다.
- 5 `if (값)` 은 0 과 "" 도 없는 것으로 친다. 숫자·문자열에는 `!== undefined` 를 쓴다.
- 6 좁힌 상태는 줄 순서를 따라간다. 이른 반환을 쓰면 아래 전체가 좁혀진다.

직접 해보기 정답

1) 걸립니다. error TS2339: Property 'toFixed' does not exist on type 'number | boolean'. string 을 빼고도 number | boolean 이 남기 때문입니다. `else` 가 "나머지 전부" 라는 것이 여기서 분명해집니다. 재현:

```
function describe(value: string | number | boolean) {  
  
  if (typeof value === "string") { console.log(value.length); }  
  else { console.log(value.toFixed(1)); }  
  
}
```

셋 이상이면 `else if` 로 하나씩 더 걸려야 합니다.

2) `statusEmoji` 가 걸립니다. error TS2366: Function lacks ending `return` statement and `return` type does not include 'undefined'. "취소" 일 때 돌려줄 것이 없다는 뜻입니다. `statusMessage` 는 `else` 로 나머지를 다 받고 있어서 안 걸립니다. 재현:

```
type Status = "대기" | "조리중" | "완료" | "취소";  
function statusEmoji(status: Status): string {
```

```
switch (status) {
  case "대기": return "[ ]";
  case "조리중": return "[..]";
  case "완료": return "[OK]";
}
```

```
}
```

→ else 로 뭉뚱그리면 편하지만, 이런 알림을 못 받습니다. switch 쪽이 안전합니다.

```
3) function sumAll(v: number | number[]) {
```

```
  if (Array.isArray(v)) {
    console.log(v.reduce((a, b) => a + b, 0));
  } else {
    console.log(v);
  }
}
```

```

}
sumAll(5); // 출력: 5
sumAll([1, 2, 3]); // 출력: 6
```

4 “메모 없음” 이 출력됩니다. 빈 문자열은 falsy 라서 else 로 갑니다. “메모를 빈칸으로 남긴 것” 과 “메모를 아예 안 쓴 것” 이 구별되지 않습니다. 구별해야 한다면 if (memo !== undefined) 를 써야 합니다.

5 걸립니다. error TS2339: Property 'toUpperCase' does not exist on type 'string | number'.
return 이 없으면 숫자인 경우도 아래로 흘러 내려오기 때문입니다. 재현:

```
function process(value: string | number) {
```

```
  if (typeof value !== "string") { console.log(value.toFixed(0)); }
  console.log(value.toUpperCase());
```

```
}
```

이런 반환이 좁히기와 잘 맞는 이유가 이것입니다. 걸러 낸 뒤 돌려보내야 그 아래가 깨끗해집니다.

null 과 undefined 다루기

RUN node 개념03_null과_undefined.ts

검사: npm run typecheck

JS 에서 가장 많이 본 에러가 아마 이것일 겁니다.

```
TypeError: Cannot read properties of undefined (reading 'name')
```

타입스크립트가 실제로 가장 크게 막아 주는 것이 바로 이 에러입니다. 그 방법이 이 파일입니다.

둘은 다른 값이다

섹션 1

JS자료 01단원에서 배운 그대로입니다.

```
undefined → 아직 값이 없다 (자동으로 그렇게 됨)
null       → 없다고 내가 정했다 (일부러 넣음)
```

타입스크립트에서도 이 둘은 서로 다른 타입입니다.

```
type MaybeName = string | null;

let name1: MaybeName = null;
console.log(name1);
```

출력 null

이 코드는 검사에서 걸립니다

TS2322

```
name1 = undefined;
```

Type 'undefined' is not assignable to type 'MaybeName'.

null 은 허용했지만 undefined 는 허용하지 않았습니다.

둘은 다른 값이라 따로 적어야 합니다.

```
name1 = "홍길동";
console.log(name1.length);
```

출력 3

문자열을 넣은 아래부터는 좁혀져서 바로 쓸 수 있습니다(개념01 섹션4).
실무 관행은 이렇습니다.

? 를 붙인 속성·매개변수	→ undefined
"값이 없음" 을 담는 변수	→ null
서버가 보내 주는 빈 값	→ 대개 null (JSON 에 undefined 가 없어서)

헛갈리면 둘 다 받는 방법도 있습니다. 섹션 4에서 봅니다.

직접 해보기 1

string | undefined 타입 변수를 만들고

null 을 넣어 보세요. 걸리나요?

05

확인하지 않으면 못 쓴다

섹션 2

```
type User = { name: string; nickname: string | null };

function printUser(user: User) {
  console.log("이름:", user.name);
}
```

이 코드는 검사에서 걸립니다 TS18047

console.log("별명 길이:", user.nickname.length);

'user.nickname' is possibly 'null'.

"null 일 수도 있는데 .length 를 쓰시겠다고요?" 입니다.

JS 에서는 별명 없는 사용자가 들어오는 날 터지던 자리입니다.

```
if (user.nickname !== null) {
  console.log("별명 길이:", user.nickname.length);
} else {
  console.log("별명 없음");
}

printUser({ name: "홍길동", nickname: "길동이" });
```

출력 이름: 홍길동
 별명 길이: 3

```
printUser({ name: "김철수", nickname: null });
```

출력 이름: 김철수
 별명 없음

에러 번호를 구분해 두면 편합니다.

```
TS18047 → null 일 수도 있다  
TS18048 → undefined 일 수도 있다
```

둘 다 possibly 라는 단어가 들어갑니다. 그것만 봐도 “확인해야겠구나” 입니다.

▹ 직접 해보기 2

위 if 를 if (user.nickname) 으로 바꿔 보세요.

통과하나요? 별명이 "" 인 사용자에게는 어떻게 될까요?

?. — 있으면 꺼내고 없으면 undefined

섹션 3

확인을 매번 if 로 쓰면 코드가 길어집니다. 짧게 쓰는 문법이 있습니다.

```
type Shop = { name: string; owner: { name: string; phone: string | null } | null };  
  
const shop1: Shop = { name: "봄날카페", owner: { name: "홍길동", phone: "010-0000" }  
};  
const shop2: Shop = { name: "무인카페", owner: null };
```

?. 는 “앞앤티가 없으면 거기서 멈추고 undefined” 라는 뜻입니다.

코드	▶ 출력
console.log(shop1.owner?.name);	▶ 홍길동
console.log(shop2.owner?.name);	▶ undefined

if 로 쓰면 이만큼이 됩니다. 같은 일입니다.

```
if (shop2.owner !== null) {  
  console.log(shop2.owner.name);  
} else {  
  console.log(undefined);  
}
```

출력 undefined

여러 번 이어 쓸 수 있습니다. 중간에 하나만 없어도 전체가 `undefined` 입니다.

코드	출력
<code>console.log(shop1.owner?.phone?.length);</code>	8
<code>console.log(shop2.owner?.phone?.length);</code>	undefined

중요한 것 — `?.` 의 결과에는 `undefined` 가 붙습니다.

이 코드는 검사에서 걸립니다

TS2322

```
const len: number = shop1.owner?.name.length;
```

Type 'number | undefined' is not assignable to type 'number'.

`?.` 를 썼다고 문제가 사라지는 게 아닙니다.

"없을 수도 있음" 이 결과로 옮겨 갈 뿐입니다.
메시지의 '`number | undefined`' 가 그것을 그대로 보여 줍니다.
그래서 대개 `??` 와 짝을 지어 씁니다(섹션 4).

함수와 배열에도 씁니다.

```
type Handler = { onClick?: () => void };
const h: Handler = {};
h.onClick?();
console.log("터지지 않았습니다");
```

출력 터지지 않았습니다

`onClick` 이 없어도 그냥 넘어갑니다. JS 에서는 여기서 터졌습니다.

▹ 직접 해보기 3

`shop2.owner?.phone?.length` 에서 두 번째 `?.` 를 지워 보세요.

무슨 에러가 나나요?

?? — 없으면 이 값으로

섹션 4

`??` 는 "왼쪽이 `null` 이나 `undefined` 면 오른쪽" 이라는 뜻입니다.

```
const nickname1: string | null = null;
console.log(nickname1 ?? "이름없음");
```

출력 이름없음

```
const nickname2: string | null = "길동이";
console.log(nickname2 ?? "이름없음");
```

출력 길동이

?. 와 짝을 지으면 깔끔해집니다.

```
console.log(shop2.owner?.name ?? "주인 없음");
```

출력 주인 없음

그리고 이제 결과에 `undefined` 가 안 붙으니 그냥 쓸 수 있습니다.

```
const ownerName: string = shop1.owner?.name ?? "주인 없음";
console.log(ownerName.length);
```

출력 3

★ || 와 헷갈리지 마세요. 이게 실무에서 진짜 버그를 만듭니다.

```
?? → null 과 undefined 일 때만 오른쪽
|| → falsy 전부일 때 오른쪽 (0 · "" · false 포함)
```

```
const count: number | null = 0;
console.log(count ?? 99);
```

출력 0

```
console.log(count || 99);
```

출력 99

← 0잔이라고 분명히 넣었는데 99 가 됐습니다. 개념02 섹션4의 그 함정입니다.

```
const memo: string | null = "";
console.log(JSON.stringify(memo ?? "메모 없음"));
```

출력 ""

```
console.log(JSON.stringify(memo || "메모 없음"));
```

출력 "메모 없음"

규칙 — 기본값을 줄 때는 ?? 를 쓰세요. || 는 “비어 있으면” 을 뜻할 때만.

▣ 직접 해보기

4

`false` 를 담은 `boolean` | `null` 변수에

?? `true` 와 || `true` 를 각각 써 보세요. 결과가 어떻게 다른가요?

! 는 쓰지 않습니다

섹션 5

느낌표를 붙이면 “없을 리 없다, 내가 책임진다” 는 뜻이 됩니다. 에러가 사라집니다. 그래서 위험합니다.

```
const len = shop2.owner!.name.length;
      ↳ 이것
```

이 줄은 검사를 통과하고, 실행하면 터집니다. 02단원의 `any` 와 똑같은 구조입니다. “검사는 통과하는데 돌리면 터진다”.

```
const risky: Shop = shop2;
try {
  console.log(risky.owner!.name);
} catch (e) {
  console.log("터졌습니다:", String(e));
}
```

출력 터졌습니다: TypeError: Cannot read properties of null (reading 'name')

★ 이 자료의 규칙 — ! 를 쓰지 않습니다. `any` 와 같은 급으로 취급하세요.

! 를 쓰고 싶어질 때는 대개 이 셋 중 하나입니다.

① ?. 와 ?? 로 해결된다 (섹션 3·4) ② if 로 확인하면 된다 (섹션 2) ③ 애초에 `null` 이 들어올 수 없는 구조로 바꿀 수 있다

07단원에서 `useRef` 를 쓸 때 ! 를 붙이고 싶은 순간이 반드시 옵니다. 그때도 ?. 로 해결됩니다.

▣ 직접 해보기

5

`risky.owner!.name` 을 `risky.owner?.name` ?? “없음” 으로

바꿔 보세요. 터지나요?

이런 실수를 합니다

?.를 썼으니 안전하다고 믿고 그 결과를 바로 쓰기

?. 의 결과에는 `undefined` 가 붙습니다. ?? 로 마무리해야 깨끗해집니다.

이런 실수를 합니다

기본값에 || 를 쓰기

0 과 "" 이 기본값으로 바뀝니다. ?? 를 쓰세요. 섹션 4입니다.

이런 실수를 합니다

null 과 **undefined** 를 같은 것으로 알기

타입에서는 다른 값입니다. 둘 다 받으려면 `string | null | undefined` 입니다. 확인은 `== null` 한 번으로 둘 다 잡힙니다(`=== null || === undefined` 와 같음). 반대로 둘 다 걸러내고 값만 남기려면 `!= null` 입니다.

이런 실수를 합니다

!로 에러를 없애기

섹션 5입니다. 에러가 없어진 게 아니라 검사가 꺼진 것입니다.

정리

- 1 `null` 과 `undefined` 는 다른 타입이다. 둘 다 받으려면 둘 다 적어야 한다.
- 2 `TS18047(null) · TS18048(undefined)` 가 나오면 확인하고 쓰라는 뜻이다.
- 3 `?.` 는 “없으면 거기서 멈추고 `undefined`”. 여러 번 이어 쓸 수 있다.
- 4 `?.` 의 결과에는 `undefined` 가 붙으므로 대개 `??` 와 짝지어 쓴다.
- 5 `??` 는 `null·undefined` 일 때만 오른쪽. `||` 는 0 과 "" 도 바꿔 버린다. 기본값에는 `??` 를 쓴다.
- 6 ★! 는 쓰지 않는다. `any` 와 같은 급이다. `?. · ?? · if` 로 해결된다.

직접 해보기 정답

1) 걸립니다. error TS2322: Type 'null' is not assignable to type 'string | undefined'. `undefined` 를 허용했다고 `null` 까지 되는 것이 아닙니다. 재현:

```
let name1: string | undefined = "홍길동";
name1 = null;
```

둘은 다른 값이고, 타입에서도 따로 적어야 합니다.

2 통과합니다. 다만 별명이 "" 인 사용자는 "별명 없음" 으로 갑니다. 빈 문자열이 falsy 라서입니다 (개념02 섹션4). "별명을 빈칸으로 저장한 사람" 과 "별명이 아예 없는 사람" 을 구별해야 한다면 !== null 을 써야 합니다.

3 error TS18047: 'shop2.owner.phone' is possibly 'null'. phone 자체가 string | null 이라 그렇습니다. 재현:

```
type Shop = { name: string; owner: { name: string; phone: string | null } | null };
const shop2: Shop = { name: "무인카페", owner: null };
console.log(shop2.owner?.phone.length);
```

첫 번째 ?. 는 owner 가 없는 경우만 막아 줍니다. phone 이 null 인 경우는 두 번째 ?. 가 막고 있었던 것입니다. ?. 는 '없을 수 있는 자리마다' 필요합니다. 하나 붙었다고 뒤가 안전해지지 않습니다.

```
4) const flag: boolean | null = false;
console.log(flag ?? true);           // 출력: false
console.log(flag || true);           // 출력: true
```

false 는 '있는 값' 인데 || 는 없는 것으로 칩니다. 설정 스위치를 껐는데 다시 켜지는 버그가 이렇게 생깁니다.

5) 터지지 않습니다. "없음" 이 출력됩니다. ! 는 문제를 숨기고, ?. 와 ?? 는 문제를 처리합니다. 코드 길이는 비슷한데 결과가 완전히 다릅니다.

판별 유니온: 모양이 여러 가지일 때

RUN node 개념04_판별_유니온.ts

검사: npm run typecheck

개념01~03은 string | number 처럼 '기본 타입'의 유니온이었습니다. 이번엔 '객체'의 유니온입니다.

이름이 어려워 보이지만 하는 일은 단순합니다.

여러 모양 중 하나가 오는데, 어느 모양인지 알려 주는 표찍지를 하나 붙인다.

07단원 React 에서 화면 상태를 다룰 때 그대로 씁니다.

표찍지가 없으면 못 쓴다

섹션 1

```
type Dog = { name: string; bark: string };
type Cat = { name: string; meow: string };
type Animal = Dog | Cat;

function printName(a: Animal) {
  console.log(a.name);
}

printName({ name: "바둑이", bark: "멍멍" });
```

출력 바둑이

name 은 양쪽에 다 있으니 됩니다(개념01 섹션2의 그 규칙).

이 코드는 검사에서 걸립니다

TS2339

```
function speakAnyway(a: Animal) {
  console.log(a.bark);
}
```

Property 'bark' does not exist on type 'Animal'.

아래에 Property 'bark' does not exist on type 'Cat'. 이 따라옵니다.

고양이한테는 bark 가 없으니까요.

그럼 어떻게 구분할까요? typeof 로는 안 됩니다. 둘 다 object 니까요. in 연산자로 "이 속성이 있느냐"를 물어볼 수는 있습니다.

```
function speakByIn(a: Animal) {
  if ("bark" in a) {
    console.log(a.bark);
  } else {
    console.log(a.meow);
  }
}
speakByIn({ name: "바둑이", bark: "멍멍" });
```

출력 멍멍

```
speakByIn({ name: "나비", meow: "야옹" });
```

출력 야옹

되기는 하는데 불편합니다. 모양이 다섯 개로 늘면 “어느 속성이 어느 모양에만 있는지”를 다 외워야 합니다. 그래서 표박지를 붙입니다.

▫ 직접 해보기

1

Animal에 Bird({ name, tweet })를 추가하고

speakByIn을 고쳐 보세요. 몇 줄이 늘어나나요?

표박지 하나면 끝난다

섹션 2

모든 모양에 같은 이름의 속성을 두고, 값만 다르게 합니다.

```
type DogT = { kind: "dog"; name: string; bark: string };
type CatT = { kind: "cat"; name: string; meow: string };
type BirdT = { kind: "bird"; name: string; tweet: string };
type AnimalT = DogT | CatT | BirdT;
```

kind의 타입이 “dog” 처럼 리터럴인 것이 핵심입니다(개념01 섹션3). string이라고 적으면 아무 소용이 없습니다.

```
function speak(a: AnimalT) {
  switch (a.kind) {
    case "dog":
      return a.bark;
    case "cat":
      return a.meow;
    case "bird":
      return a.tweet;
  }
}

console.log(speak({ kind: "dog", name: "바둑이", bark: "멍멍" }));
```

출력 멍멍

```
console.log(speak({ kind: "bird", name: " 짹짹이", tweet: " 짹짹 " }));
```

출력 짹짹

case "dog" 안에서 a 는 DogT 하나로 좁혀집니다. 그래서 a.bark 가 통과하고, a.meow 는 걸립니다.

이 코드는 검사에서 걸립니다

TS2339

```
function speakWrong(a: AnimalT) {
  switch (a.kind) {
    case "dog":
      return a.meow;
    default:
      return "";
  }
}
```

Property 'meow' does not exist on type 'DogT'.

개 자리에서 고양이 소리를 꺼내려 했습니다.

메시지에 'DogT' 라고 정확히 나옵니다. 좁혀졌다는 증거입니다.

▹ 직접 해보기 2

speak 에 case "cat" 을 지워 보세요.

무슨 에러가 나나요? (개념02 섹션2와 같은 것입니다)

실전 — 화면 상태 다루기 ★

섹션 3

이 패턴을 실제로 가장 많이 쓰는 곳입니다. 서버에서 데이터를 받아 오는 화면은 언제나 세 상태 중 하나입니다.

```
type Menu = { name: string; price: number };

type LoadState =
  | { status: "로딩중" }
  | { status: "성공"; data: Menu[] }
  | { status: "실패"; message: string };

function render(state: LoadState): string {
  switch (state.status) {
    case "로딩중":
      return "불러오는 중...";
    case "성공":
      return state.data.length + "개 메뉴";
    case "실패":
      return "오류: " + state.message;
  }
}

console.log(render({ status: "로딩중" }));
```

출력 불러오는 중...

```
console.log(render({ status: "성공", data: [{ name: "라떼", price: 4500 }] }));
```

출력 1개 메뉴

```
console.log(render({ status: "실패", message: "서버가 응답하지 않습니다" }));
```

출력 오류: 서버가 응답하지 않습니다

이 방식이 왜 좋은지 비교해 보면 분명해집니다.

옛날 방식 · 속성을 다 늘어놓고 선택으로 만든다

```
type BadState = {
  loading: boolean;
  data?: Menu[];
  error?: string;
};
```

이러면 이런 것들이 전부 만들어집니다.

```
{ loading: true, data: [...], error: "실패" } ← 로딩 중인데 성공이고 실패?  
{ loading: false } ← 다 끝났는데 결과가 없다?
```

말이 안 되는 조합인데 타입은 다 통과시킵니다. 그리고 data 와 error 가 선택 속성이라 쓸 때마다 확인해야 합니다.

판별 유니온 · 있을 수 없는 조합은 아예 못 만든다

"성공" 이면 data 가 반드시 있고, message 는 아예 없습니다.
"로딩중" 이면 data 도 message 도 없습니다.
확인 없이 state.data 를 바로 쓸 수 있습니다.

이 코드는 검사에서 걸립니다

TS2353

```
const impossible: LoadState = { status: "성공", data: [], message: "실패" };
```

Object literal may only specify known properties, and 'message' does not exist in type '{ status: "성공"; data: Menu[]; }'.

성공인데 오류 메시지가 있는 상태를 아예 못 만듭니다.

"잘못된 상태를 표현할 수 없게 만든다" - 이게 이 패턴의 핵심 이득입니다.

▫ 직접 해보기

3

{ status: "성공" } 만 써 보세요(data 없이).

무슨 에러가 나나요?

빠뜨린 경우를 잡아 주는 장치

섹션 4

개념02 섹션2에서 봤듯이, 반환 타입을 적어 두면 case 를 빠뜨렸을 때 TS2366 으로 걸립니다.

그런데 함수가 아무것도 안 돌려주는 경우(void)에는 그 장치가 안 먹습니다. 그럴 때 쓰는 방법이 있습니다. 참고로 알아 두세요.


```
function log(state: LoadState): void {
  switch (state.status) {
    case "로딩중":
      console.log("...");
      break;
    case "성공":
      console.log("완료:", state.data.length);
      break;
    case "실패":
      console.log("오류:", state.message);
      break;
    default: {
```

여기까지 오면 안 됩니다. 다 덮었다면 남는 것이 없으니까요.

```
const 남은것: never = state;
  console.log("모르는 상태:", 남은것);
}
}
}

log({ status: "성공", data: [] });
```

출력 완료: 0

never 는 “값이 있을 수 없는 타입” 입니다. case 를 다 적었다면 default 에 도달했을 때 남는 타입이 없어서 never 가 되고, 통과합니다.

하나라도 빠뜨리면 그 남은 타입이 never 에 안 들어가서 걸립니다. 상태를 하나 추가했을 때 “여기도 고쳐야 한다” 를 자동으로 알려 주는 장치입니다.

지금 당장 외울 필요는 없습니다. “다 덮었는지 확인하는 관용구가 있다” 정도만 기억하세요.

▹ 직접 해보기 4

LoadState 에 { status: "취소" } 를 추가해 보세요.

render 와 log 중 어느 것이 걸리나요? 둘 다 걸리나요?

표택지 이름 정하기

섹션 5

이름은 아무거나 됩니다. 관행은 이렇습니다.

```
kind    · type    · status    · tag
```

이 자료는 상황에 맞춰 씁니다. 상태를 나타내면 status, 종류면 kind 입니다.

중요한 것은 이름이 아니라 두 가지입니다.

① 모든 모양에 '같은 이름' 으로 있어야 한다 ② 값이 리터럴 타입이어야 한다 ("dog" 이지 string 이 아님)

② 를 어기면 조용히 망가집니다.

```
type LooseDog = { kind: string; bark: string };
type LooseCat = { kind: string; meow: string };
type LooseAnimal = LooseDog | LooseCat;
```

이 코드는 검사에서 걸립니다

TS2339

```
function looseSpeak(a: LooseAnimal) {
  if (a.kind === "dog") {
    return a.bark;
  }
  return "";
}
```

Property 'bark' does not exist on type 'LooseAnimal'.

kind 가 string 이라 "dog" 와 비교해도 좁혀지지 않습니다.

string 은 "dog" 일 수도 있고 "cat" 일 수도 있으니 구분이 안 되는 것입니다.
type 을 적을 때 kind: "dog" 처럼 값을 그대로 적어야 합니다.

▸ 직접 해보기

5

LooseDog 의 kind 를 "dog", LooseCat 을 "cat" 으로 고쳐 보세요.

위 함수가 통과하나요?

확인을 함수로 빼기 — a is DogT

섹션 6

지금까지 좁히는 조건은 쓸 때마다 그 자리에 적었습니다. 같은 확인을 여러 군데서 하게 되면 함수로 빼고 싶어집니다. 그런데 그냥 빼면 좁히기가 사라집니다.

```
function looksLikeDog(a: AnimalT): boolean {
  return a.kind === "dog";
}

console.log(looksLikeDog({ kind: "dog", name: "바둑이", bark: "멍멍" }));
```

출력 true

이 코드는 검사에서 걸립니다

TS2339

```
function barkOf1(a: AnimalT) {
  if (looksLikeDog(a)) return a.bark;
  return "";
}
```

Property 'bark' does not exist on type 'AnimalT'.

looksLikeDog 는 true / false 만 돌려줍니다.

"그 true 가 무슨 뜻인지" 는 어디에도 안 적혀 있습니다.
그래서 if 안으로 들어가도 a 는 여전히 AnimalT 입니다.

반환 타입을 boolean 대신 a is DogT 라고 적으면 뜻이 붙습니다.

```
function isDog(a: AnimalT): a is DogT {
  return a.kind === "dog";
}

function barkOf(a: AnimalT): string {
  if (isDog(a)) {
    return a.bark; // 여기서는 DogT 입니다
  }
  return "(개가 아님)";
}

console.log(barkOf({ kind: "dog", name: "바둑이", bark: "멍멍" }));
```

출력 멍멍

```
console.log(barkOf({ kind: "cat", name: "나비", meow: "야옹" }));
```

출력 (개가 아님)

a is DogT 는 “이 함수가 true 를 주면 a 는 DogT 다” 라는 뜻입니다. 이름이 붙은 좁히기라고 보면 됩니다. 05단원 내내 손으로 하던 일을 함수로 포장한 것입니다. 확인이 길어지거나 여러 곳에서 같은 확인을 할 때 씁니다.

★ 주의 — 몸통이 정말 맞는지는 타입스크립트가 확인하지 않습니다.

```
return a.kind === "cat";
```

이라고 적어도 조용히 통과합니다.

08단원에서 배울 as 와 같은 종류의 ‘약속’ 입니다. 그래서 몸통은 짧고 뾰족하게 적으세요. 표딱지 하나만 보는 정도가 안전합니다.

isDog 의 몸통을 `return a.kind === "cat";` 으로 바꾸고

barkOf 에 고양이를 넘겨 보세요. 검사는 조용한가요? 실행하면 무엇이 찍히나요?

자주 하는 실수

섹션 7

이런 실수를 합니다

표짜지 값을 string 으로 적기

섹션 5입니다. 가장 흔하고, 가장 찾기 어려운 실수입니다. "분명히 if 로 확인했는데 왜 안 되지?" 싶으면 여기를 보세요.

이런 실수를 합니다

표짜지 이름을 모양마다 다르게 쓰기

하나는 kind, 하나는 type 이면 공통 속성이 아니라서 못 씁니다.

이런 실수를 합니다

선택 속성으로 상태를 표현하기

섹션 3의 BadState 입니다. 있을 수 없는 조합이 만들어지고 쓸 때마다 확인해야 합니다.

이런 실수를 합니다

switch 에 break 를 빠뜨리기

`return` 을 쓰면 상관없지만, void 함수에서는 아래 case 로 흘러갑니다. JS자료 03단원에서 배운 그대로입니다. 타입스크립트도 이걸 안 막아 줍니다.

정리

- 1 객체 여러 모양 중 하나가 올 때, 모든 모양에 같은 이름의 표짜지를 둔다.
- 2 표짜지 값은 반드시 리터럴이어야 한다. "dog" 이지 string 이 아니다.
- 3 `switch (a.kind)` 하면 각 case 안에서 그 모양 하나로 좁혀진다.
- 4 가장 큰 이득은 '있을 수 없는 상태를 아예 못 만드는 것' 이다. 로딩중인데 데이터가 있는 상태 같은 것이 만들어지지 않는다.
- 5 선택 속성을 늘어놓는 방식보다 확인할 일이 적다.
- 6 default 에 never 를 두면 빠뜨린 경우를 알려 준다. 관용구로 알아 두면 된다.
- 7 07단원 React 에서 화면 상태를 이 패턴으로 다룬다.

직접 해보기 정답

```
1) type Bird = { name: string; tweet: string };
type Animal2 = Dog | Cat | Bird;
function speakByIn2(a: Animal2) {
```

```
  if ("bark" in a) return a.bark;
  if ("meow" in a) return a.meow;
  return a.tweet;
```

```
}
```

→ 한 줄 늘었습니다. 지금은 괜찮아 보이지만,

모양이 다섯 개가 되면 "어느 속성이 어느 모양에만 있는지" 를 다 알아야 합니다.
표딱지 방식은 kind 하나만 보면 됩니다.

2) error TS2366: Function lacks ending `return` statement and `return` type does not include 'undefined'. "cat" 일 때 돌려줄 것이 없다는 뜻입니다. 재현:

```
type DogT = { kind: "dog"; bark: string };
type CatT = { kind: "cat"; meow: string };
function speak(a: DogT | CatT): string {
```

```
  switch (a.kind) { case "dog": return a.bark; }
```

```
}
```

반환 타입이 있으면 이렇게 빠뜨린 case 를 잡아 줍니다.

3) error TS2322: Type '{ status: "성공"; }' is not assignable to type 'LoadState'.

```
Property 'data' is missing in type '{ status: "성공"; }'
but required in type '{ status: "성공"; data: Menu[]; }'.
```

재현:

```
type Menu = { name: string; price: number };
```

type LoadState =

```
| { status: "로딩중" }
| { status: "성공"; data: Menu[] }
| { status: "실패"; message: string };
```

```
const impossible: LoadState = { status: "성공" };
```

두 줄로 나옵니다. 첫 줄은 "LoadState 에 안 들어간다", 둘째 줄이 "왜냐하면 data 가 빠져서" 입니다. 둘째 줄까지 읽어야 답이 나옵니다. (목표가 유니온일 때는 TS2741 이 아니라 TS2322 로 나옵니다.

04단원에서 본 단일 타입일 때와 번호가 다릅니다)

"성공" 이라고 했으면 data 가 반드시 있어야 합니다. 선택 속성이었다면 이 실수가 안 걸렸을 것입니다.

4) 둘 다 걸립니다. render 는 TS2366(돌려줄 것이 없다), log 는 default 의 never 자리에서 TS2322 Type '{ status: "취소"; }' is not assignable to type 'never'. → 상태를 하나 추가하면 고쳐야 할 곳을 타입이 전부 찾아 줍니다.

이게 판별 유니온을 쓰는 실질적인 이유입니다.

5 통과합니다. kind 가 리터럴이 되는 순간 === "dog" 비교가 좁히기로 동작합니다. 딱 두 군데 고쳤을 뿐인데 함수가 살아납니다.

6 검사는 조용합니다. 그리고 실행하면 undefined 가 찍힙니다. isDog 가 고양이한테 true 를 주니 if 안으로 들어가고, 고양이에게는 bark 가 없어서 undefined 가 나오는 것입니다. 타입 서술어(a is DogT)는 몸통을 검사받지 않습니다. "검사는 통과하는데 돌리면 이상하다" 가 여기서도 나옵니다.

유니온과 타입 좁히기

```
RUN node 연습문제.ts
```

채점: `npm run grade` ← 출력을 기대 출력과 맞춰 봅니다

검사: `npm run check` ← 타입 검사 (일부 문제만 잡습니다)

★ 채점은 `npm run grade` 로 합니다. 실제 출력을 문제의 '기대 출력' 과 줄 단위로 맞춰 봅니다. 전부 일치하면 다 맞은 것입니다.

`npm run check`(타입 검사)도 같이 쓰세요. 다만 이쪽은 '일부 문제만' 잡습니다.

138문항 중 21개뿐입니다. "출력하세요" 류는 안 풀어도 타입 에러가 안 나거든요. 그러니 `check` 가 조용하다고 다 맞은 것이 아닙니다. `grade` 로 확인하세요. (`npm run typecheck` 는 개념·정답 파일만 봅니다. 그쪽은 언제나 조용합니다)

- 1 TODO 자리에 코드를 씁니다.
- 2 `npm run grade` 로 채점합니다. (직접 `node 연습문제.ts` 로 돌려 봐도 됩니다)
- 3 `npm run check` 로 타입도 봅니다.
- 4 10분 고민해도 안 되면 `연습문제_정답.ts` 를 보세요.

문제 1~8은 기본, 9~14는 응용, 15는 [도전]입니다. 문제 16은 에러를 직접 보는 문제라 맨 뒤에 있습니다.

```
console.log("=== 05단원 연습문제 ===");
```

문제 1 (개념01 섹션1)

문자열이거나 숫자인 타입 `Id` 를 만들고, 두 종류를 각각 넘겨 출력하세요.

기대 출력 3
 A-3

기대 검사 결과: 조용함

```
{
```

type Id 를 만들고 함수 하나를 만들어 두 번 부르세요

```
}
```

문제 2 (개념01 섹션3)

"S" | "M" | "L" 만 받는 Size 타입을 만들고 함수에 넘기세요.

기대 출력 사이즈: M

기대 검사 결과: 조용함

```
{
```

type Size 를 만들고 printSize("M") 을 부르세요

```
}
```

문제 3 (개념02 섹션1)

아래 함수는 걸립니다. `typeof` 로 확인해서 걸리지 않게 고치세요. 문자열이면 글자 수, 숫자면 소수 첫째 자리까지 출력하세요.

기대 출력 5
 4500.0

기대 검사 결과: 조용함

```
{
```


`typeof` 로 확인해 고치세요

```
function describe(value: string | number) {  
  console.log(value.length);  
}
```

```
describe("아메리카노");  
describe(4500);
```

```
}
```

05

문제 4 (개념02 섹션2)

Status 에 따라 다른 문구를 돌려주는 함수를 switch 로 만드세요. 반환 타입 : string 을 꼭 적으세요.

기대 출력 만들고 있어요

기대 검사 결과: 조용함

```
{  
  type Status = "대기" | "조리중" | "완료";
```

switch 로 만들고 “조리중” 을 넘겨 출력하세요

("대기"→"곧 시작합니다" / "조리중"→"만들고 있어요" / "완료"→"나왔습니다")

```
}
```

문제 5 (개념02 섹션3)

하나면 그대로, 배열이면 합계를 출력하는 함수를 만드세요.

기대 출력 5
 6

기대 검사 결과: 조용함

```
{
```

number | number[] 를 받는 sumAll 을 만들고 5 와 [1,2,3] 을 넘기세요

```
}
```

문제 6 (개념03 섹션3)

?. 를 써서 owner 가 없어도 터지지 않게 출력하세요.

기대 출력 홍길동
 undefined

기대 검사 결과: 조용함

```
{  
  type Shop = { name: string; owner: { name: string } | null };  
  const shop1: Shop = { name: "봄날카페", owner: { name: "홍길동" } };  
  const shop2: Shop = { name: "무인카페", owner: null };  
}
```

두 가게의 주인 이름을 ?. 로 출력하세요

```
}
```

문제 7 (개념03 섹션4)

?? 를 써서 주인이 없으면 "주인 없음" 이 나오게 하세요.

기대 출력	홍길동 주인 없음
-------	--------------

기대 검사 결과: 조용함

```
{
  type Shop = { name: string; owner: { name: string } | null };
  const shop1: Shop = { name: "봄날카페", owner: { name: "홍길동" } };
  const shop2: Shop = { name: "무인카페", owner: null };
```

?. 와 ?? 를 함께 써서 출력하세요

```
}
```

문제 8 (개념04 섹션2)

kind 를 표떡지로 삼는 판별 유니온을 만들고, switch 로 소리를 출력하세요.

기대 출력	멍멍 야옹
-------	----------

기대 검사 결과: 조용함

```
{
```

Dog({kind:"dog", bark}) 와 Cat({kind:"cat", meow}) 을 만들고

```
  speak 함수를 switch 로 만들어 둘 다 출력하세요
```

```
}
```

문제 9 (응용 · 개념01 섹션2)

아래 코드가 왜 걸리는지 이유를 한 줄로 출력하세요.

```
function f(v: string | number) { return v.toUpperCase(); }
```

기대 출력 숫자에는 toUpperCase 가 없어서

기대 검사 결과: 조용함

```
{
```

이유를 그대로 출력하세요

```
}
```

문제 10 (응용 · 개념02 섹션4)

아래 함수는 수량 0 을 “없음” 으로 잘못 처리합니다. 고치세요.

기대 출력 수량: 3
 수량: 0
 수량 없음

기대 검사 결과: 조용함

```
{
```

0 도 있는 값으로 처리되게 고치세요

```
function showCount(count?: number) {
  if (count) {
    console.log("수량:", count);
  } else {
    console.log("수량 없음");
  }
}
showCount(3);
showCount(0);
showCount();
}
```

문제 11 (응용 · 개념03 섹션4)

아래 두 줄의 결과를 순서대로 출력하세요.

```
const n: number | null = 0;
console.log(n ?? 99);
console.log(n || 99);
```

기대 출력 0
 99

기대 검사 결과: 조용함

```
{
```

두 결과를 출력하세요

```
}
```

문제 12 (응용 · 개념03 섹션5)

아래 코드는 검사를 통과하지만 실행하면 터집니다. !를 쓰지 않고 안전하게 고치세요. 없으면 "없음"을 출력하세요.

기대 출력 없음

기대 검사 결과: 조용함

```
{
  type Shop = { owner: { name: string } | null };
  const shop: Shop = { owner: null };
}
```

! 를 쓰지 않고 고치세요

```
console.log(shop.owner!.name);
```

```
}
```

문제 13 (응용 · 개념04 섹션5)

아래 코드는 걸립니다. 표떡지(kind) 값의 타입을 고쳐서 통과하게 만드세요.

기대 출력 멍멍

기대 검사 결과: 조용함

```
{
```

kind 의 타입을 고치세요 (힌트: string 말고 그 값 자체를 적습니다)

```
type Dog = { kind: string; bark: string };
type Cat = { kind: string; meow: string };
type Animal = Dog | Cat;

function speak(a: Animal) {
  if (a.kind === "dog") {
    return a.bark;
  }
  return "";
}
```

```
console.log(speak({ kind: "dog", bark: "멍멍" }));

}
```

문제 14 (응용 · 개념02 섹션5)

아래 함수를 '이른 반환' 으로 바꿔서, 아래쪽 두 줄이 전부 문자열로 취급되게 하세요.

기대 출력	LATTE
	5

기대 검사 결과: 조용함

```
{

숫자면 먼저 return 하고, 아래에서 문자열 기능을 쓰세요

function process(value: string | number) {
  console.log(value);
}

process("latte");

}
```

05

문제 15 ([도전] · 개념01~04 종합)

화면 상태를 판별 유니온으로 만들고, 세 상태를 각각 출력하세요. ① LoadState 를 만든다

"로딩중"	→ 추가 정보 없음
"성공"	→ data: string[]
"실패"	→ message: string

② render(state): string 를 switch 로 만든다 ③ 세 상태를 각각 넘겨 출력한다

기대 출력 불러오는 중...
2개: 아메리카노, 라떼
오류: 서버 없음

기대 검사 결과: 조용함

```
{
```

위 세 가지를 하세요

```
}
```

문제 16 (에러 확인 · 개념03 섹션5)

아래 세 줄의 // 를 지우고,

① npm run check 로 검사해 보세요 → 걸릴까요? ② node 연습문제.ts 로 실행해 보세요 → 어떻게 될까요?

! 를 쓰면 무슨 일이 생기는지 직접 보는 문제입니다. 확인했으면 다시 // 를 붙이세요.

```
{
```

```
type Shop = { owner: { name: string } | null };  
const shop: Shop = { owner: null };  
console.log(shop.owner!.name);
```

```
}
```


제네릭

OPEN 06_제네릭

```
return arr[0];  
const names = ["가", "나"];  
console.log(names.length);  
function firstAny(arr: any[]) {
```

01 제네릭이 왜 필요한가

02 제네릭 쓰기

03 이미 쓰고 있던 제네릭

04 keyof: 속성 이름을 타입으로 다루기

- 연습문제

제네릭이 왜 필요한가

RUN node 개념01_제네릭이_왜_필요한가.ts

검사: npm run typecheck

제네릭은 이름 때문에 어려워 보입니다. 하는 일은 한 줄입니다.

"무슨 타입이 올지는 쓰는 쪽이 정한다" 를 적는 방법.

왜 그런 게 필요한지부터 봅시다. 이유를 알면 문법은 5분이면 끝납니다.

같은 함수를 타입마다 만들게 된다

섹션 1

배열의 첫 번째 값을 꺼내는 함수를 만들어 봅시다.

```
function firstNumber(arr: number[]) {
  return arr[0];
}
console.log(firstNumber([10, 20, 30]));
```

출력 10

문자열 배열에도 쓰고 싶습니다. 그런데 못 씁니다.

```
const names = ["가", "나"];
```

이 코드는 검사에서 걸립니다

TS2345

```
console.log(firstNumber(names));
```

Argument of type 'string[]' is not assignable to parameter of type 'number[]'.

number[] 를 받기로 했으니 당연합니다.

아래에 Type 'string' is not assignable to type 'number'. 가 따릅니다.

배열 안의 무엇이 안 맞는지까지 알려 줍니다.

```
console.log(names.length);
```

출력 2

그래서 하나 더 만듭니다.

```
function firstString(arr: string[]) {
  return arr[0];
}
console.log(firstString(["가", "나"]));
```

출력 가

그리고 boolean 배열이 필요해집니다. 객체 배열도 필요해집니다. 몸통은 전부 `return arr[0];` 한 줄로 똑같은데 함수만 늘어납니다.

```
firstNumber · firstString · firstBoolean · firstMenu · ...
```

이게 문제입니다.

▹ 직접 해보기

1

firstBoolean 을 만들어 `[true, false]` 의 첫 값을 출력해 보세요.

몸통이 위 둘과 무엇이 다른가요?

any 로 하면 되지 않나 — 안 됩니다

섹션 2

```
function firstAny(arr: any[]) {
  return arr[0];
}
```

하나로 다 되기는 합니다.

코드

```
console.log(firstAny([10, 20]));
```

```
console.log(firstAny(["가", "나"]));
```

▶ 출력

▶ 10

▶ 가

그런데 돌려받은 값이 any 입니다. 02단원 개념03에서 본 그 문제입니다.

```
const value = firstAny([10, 20]);
console.log(value.toUpperCase === undefined);
```

출력 true

숫자인데 toUpperCase 를 물어봐도 아무도 안 막습니다.

검사가 꺼졌으니 실행할 때 터집니다.

```
try {
    console.log(firstAny([10, 20]).toUpperCase());
} catch (e) {
    console.log("터졌습니다:", String(e));
}
```

출력 터졌습니다: TypeError: firstAny(...).toUpperCase is not a function

정리하면 이런 상황입니다.

타입마다 함수를 만든다	→	검사는 되는데 함수가 계속 늘어난다
any 로 하나만 만든다	→	함수는 하나인데 검사가 꺼진다

둘 다 싫습니다. 제네릭이 이 둘 사이의 답입니다.

▸ 직접 해보기 2

`firstAny(["가", "나"]).toFixed(2)` 를 써 보세요.

거사는 통과하나요? 식해하며요?

음식단 응답이 대표: 반응이 대표:

<T> — 타입을 나중에 정하기 섹션 3

섹션 3

값을 나중에 받으려고 매개변수를 두는 것처럼, 타입을 나중에 받으려고 '타입 매개변수'를 둡니다.

```
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}
```

그림으로 보면 이렇습니다.

```
function first<T>(arr: T[]): T | undefined
    └─┬──┬──┬──
    │   │   └─ 돌려주는 것도 T
    │   └─ 받는 것은 T 들의 배열
    └─ "T" 라는 이름의 타입을 하나 받겠다"
```

T 가 무엇인지는 안 적혀 있습니다. 쓰는 쪽에서 정해집니다.

```
const n = first([10, 20, 30]);  
console.log(n);
```

출력 10

```
const s = first(["가", "나"]);
console.log(s);
```

출력 가

넘긴 것을 보고 T 를 알아냅니다. 적을 필요가 없습니다.

```
first([10, 20, 30]) → T 는 number → 돌려주는 것은 number | undefined
first(["가", "나"]) → T 는 string → 돌려주는 것은 string | undefined
```

이제 함수는 하나인데 검사는 살아 있습니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(first([10, 20])?.toUpperCase());
```

Property 'toUpperCase' does not exist on type 'number'.

any 였다면 통과했을 자리입니다. 실행 중에 터졌겠죠.

제네릭은 "무엇이든 받는다" 지 "검사를 끈다" 가 아닙니다.

▹ 직접 해보기

3

first([{ name: "라떼" }]) 의 결과에 마우스를 올려 보세요.

T 가 무엇으로 정해졌나요?

왜 | undefined 가 붙었나

섹션 4

빈 타입을 T 가 아니라 T | undefined 라고 적었습니다. 이유가 있습니다.

```
const empty = first<number>([]);
console.log(empty);
```

출력 undefined

빈 배열의 [0] 은 undefined 입니다. JS자료 06단원에서 배운 그대로입니다. 그러니 사실대로 적어야 합니다.

만약 : T 라고만 적었다면, 쓰는 쪽에서 확인 없이 바로 쓰다가 터졌을 것입니다. 04단원 개념04의 "사실대로 적는 것이 원칙" 이 여기서도 그대로입니다.

그래서 쓰는 쪽은 확인해야 합니다. 05단원에서 배운 그대로입니다.

```
const maybe = first([10, 20]);
if (maybe !== undefined) {
  console.log(maybe + 1);
}
```

출력 11

코드

```
console.log(first([10, 20]) ?? 0);
```

```
console.log(first<number>([]) ?? 0);
```

▶ 출력

▶ 10

▶ 0

▹ 직접 해보기 4

first(["가"])의 결과를 확인 없이 .length로 써 보세요.

무슨 에러가 나나요?

T라는 이름

섹션 5

T는 그냥 이름입니다. 아무거나 써도 됩니다.

```
function last<Item>(arr: Item[]): Item | undefined {
  return arr[arr.length - 1];
}
console.log(last(["가", "나", "다"]));
```

출력 다

관행은 이렇습니다.

T	Type의 첫 글자. 하나뿐일 때 가장 많이 씁니다
K, V	Key, Value. 짝으로 쓸 때
T, U	두 개일 때
Item	뜻이 분명할 때는 이렇게 풀어 써도 좋습니다

이 자료는 하나면 T, 뜻이 중요하면 풀어 씁니다.

타입 매개변수는 대문자로 시작합니다. 타입 이름이니깐요(04단원 개념02 섹션2).

▹ 직접 해보기 5

last를 써서 숫자 배열의 마지막 값을 출력해 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

<T> 를 안 쓰고 T 만 쓰기

`function first(arr: T[])` 는 TS2304 Cannot find name 'T'. 입니다. T 를 쓰려면 먼저 <T> 로 “받겠다”고 선언해야 합니다. 값 매개변수를 (arr) 이라고 적어야 arr 을 쓸 수 있는 것과 같습니다.

이런 실수를 합니다

제네릭을 any 처럼 생각하기

섹션 3입니다. 검사는 그대로 살아 있습니다.

이런 실수를 합니다

빈 배열일 때를 안 적기

섹션 4입니다. 사실대로 적어야 쓰는 쪽에서 확인하게 됩니다.

이런 실수를 합니다

필요 없는데 제네릭으로 만들기

한 종류만 쓸 함수를 <T> 로 만들면 읽기만 어려워집니다. “타입마다 같은 함수를 또 만들게 되는가?” 일때만 쓰세요.

정리

- 1 같은 몸통인데 타입만 다른 함수를 여러 개 만들게 되면 제네릭이 필요한 것이다.
- 2 any 로 하나 만들면 함수는 줄지만 검사가 꺼진다. 제네릭은 둘 다 얻는다.
- 3 <T> 는 “T 라는 이름의 타입을 하나 받겠다” 는 뜻이다. T 가 무엇인지는 쓰는 쪽에서 정해진다.
- 4 넘긴 값을 보고 알아내므로 부르는 쪽에서 적을 필요가 없다.
- 5 사실대로 적는다. 빈 배열이면 `undefined` 가 나오니 `T | undefined` 다.
- 6 T 는 이름일 뿐이다. 뜻이 중요하다면 Item 처럼 풀어 써도 된다.

직접 해보기 정답

```
1) function firstBoolean(arr: boolean[]) { return arr[0]; }
   console.log(firstBoolean([true, false])); // 출력: true
```

몸통은 완전히 같습니다. 매개변수 타입만 다릅니다. → 몸통이 같은데 타입 때문에 함수를 또 만들고 있다면 제네릭 자리입니다.

2 검사는 통과합니다. any 라서 아무도 안 막습니다. 실행하면 터집니다. `TypeError: firstAny(...).toFixed is not a function` 문자열에는 toFixed 가 없기 때문입니다.

3 T 는 { name: string } 으로 정해집니다. 결과 타입은 { name: string } | undefined 입니다. 객체든 배열이든 무엇이든 T 가 될 수 있습니다.

4 error TS2532: Object is possibly 'undefined'. 이름이 붙은 변수가 아니라 'first(...)' 라는 결과' 라서 TS18048 이 아니라 TS2532 로 나옵니다. Object 라고만 부르는 것도 그 때문입니다. 둘 다 possibly 'undefined' 이니 하는 말은 같습니다. 재현:

```
function first<T>(arr: T[]): T | undefined { return arr[0]; }
console.log(first(["가"]).length);
```

빈 배열일 수 있으니 확인하고 쓰라는 뜻입니다. `first(["가"])?length` 나 `(first(["가"]) ?? "").length` 로 쓰면 됩니다.

```
5) console.log(last([10, 20, 30]));           // 출력: 30
```


제네릭 쓰기

RUN node 개념02_제네릭_쓰기.ts

검사: npm run typecheck

개념01에서 <T> 하나를 봤습니다. 이 파일은 실제로 쓸 때 필요한 것 네 가지를 더합니다.

여러 개 받기 · 직접 정해 주기 · 조건 걸기 · 타입에 붙이기

타입 매개변수를 여러 개

섹션 1

심표로 나열하면 됩니다.

```
function pair<A, B>(a: A, b: B): [A, B] {
  return [a, b];
}
```

```
const p1 = pair("라떼", 4500);
console.log(p1);
```

출력 ['라떼', 4500]

p1의 타입은 [string, number]입니다. 02단원 개념01 섹션5의 그 튜플입니다. 거기서는 [string, number]처럼 종류를 직접 적었는데, 여기서는 [A, B]라고 적어 두고 부를 때 정해집니다. 그게 다른 점입니다.

`string[]` 길이 제한 없이 문자열만

string, number · 딱 두 개, 앞은 문자열 뒤는 숫자

A, B · 딱 두 개, 종류는 부를 때 정해짐

07단원에서 React의 useState가 정확히 이 모양을 돌려줍니다.

코드	출력
<code>console.log(p1[0].toUpperCase());</code>	▶ 라떼
<code>console.log(p1[1].toFixed(0));</code>	▶ 4500

자리마다 타입이 다른 것에 주목하세요. [0]은 문자열, [1]은 숫자입니다.

이 코드는 검사에서 걸립니다

TS2551

```
console.log(p1[0].toFixed(0));
```

Property 'toFixed' does not exist on type 'string'. Did you mean 'fixed'?

0번 자리는 문자열입니다. 순서가 타입에 적혀 있습니다.

Did you mean 'fixed'? 는 무시하세요. 문자열에 정말로 fixed 라는
옛날 기능이 있어서 비슷한 이름으로 골라 준 것입니다.
제안이 항상 옳지는 않습니다(01단원 개념03 ③).

구조분해로 꺼내면 더 편합니다. JS자료 09단원에서 배운 그대로입니다.

```
const [menuName, menuPrice] = pair("아메리카노", 4000);  
console.log(menuName, menuPrice);
```

출력 아메리카노 4000

▹ 직접 해보기 1

pair(true, "확인") 을 만들어 두 값을 각각 출력해 보세요.

T 를 직접 정해 주기

섹션 2

대개는 넘긴 값을 보고 알아냅니다. 하지만 알아낼 근거가 없을 때가 있습니다.

```
function makeEmpty<T>(): T[] {  
  return [];  
}
```

넘기는 값이 없으니 T 를 알아낼 방법이 없습니다. 그럴 때는 직접 적습니다.

```
const names = makeEmpty<string>();  
names.push("라떼");  
console.log(names);
```

출력 ['라떼']

이 코드는 검사에서 걸립니다

TS2345

```
names.push(4500);
```

Argument of type 'number' is not assignable to parameter of type 'string'.

<string> 이라고 정해 줬으니 문자열만 들어갑니다.

적어 준 것과 넘긴 것이 다르면 걸립니다.

```
function wrap<T>(value: T): T[] {
  return [value];
}
console.log(wrap<string>("라떼"));
```

출력 ['라떼']

이 코드는 검사에서 걸립니다

TS2345

```
console.log(wrap<string>(4500));
```

Argument of type 'number' is not assignable to parameter of type 'string'.
string 이라고 해 놓고 숫자를 넘겼습니다.

규칙 — 알아낼 수 있으면 말기고, 없으면 적습니다. 굳이 적으면 코드만 길어집니다.

```
console.log(wrap("라떼"));
```

출력 ['라떼']

⇒ 직접 해보기 2

makeEmpty 를 써서 숫자 배열을 만들고 3 을 넣어 출력해 보세요.

extends — 아무거나는 곤란할 때

섹션 3

T 는 정말 아무 타입이나 될 수 있습니다. 그래서 T 로는 할 수 있는 게 거의 없습니다.

이 코드는 검사에서 걸립니다

TS2339

```
function badLength<T>(value: T) {
  return value.length;
}
```

Property 'length' does not exist on type 'T'.
T 가 숫자일 수도 있는데 length 를 쓰려고 했습니다.

05단원의 "공통으로 있는 것만" 규칙과 같은 이유입니다.
T 는 '모든 타입' 이니 공통인 것이 거의 없습니다.

그래서 조건을 겁니다. "length 가 있는 것만 받겠다" 는 뜻입니다.

```
function getLength<T extends { length: number }>(value: T): number {
    return value.length;
}
```

```
console.log(getLength("아메리카노"));
```

출력 5

```
console.log(getLength([1, 2, 3]));
```

출력 3

```
console.log(getLength({ length: 99, name: "가짜" }));
```

출력 99

문자열에도 배열에도 length 가 있으니 둘 다 됩니다.

이 코드는 검사에서 걸립니다

TS2345

```
console.log(getLength(123));
```

Argument of type 'number' is not assignable to parameter of type '{ length: number; }'.

숫자에는 length 가 없어서 조건에 안 맞습니다.

부르는 쪽에서 막아 주는 것이 핵심입니다. 함수 안이 아니라고요.

extends 는 “상속” 이 아니라 “적어도 이걸 갖고 있어야 한다” 로 읽으세요.

T extends { length: number }	→ length 가 있는 것이면 무엇이든
T extends string	→ 문자열이거나 그 하위인 것
T extends { id: number }	→ id 가 있는 객체면 무엇이든

실전 예 — id 가 있는 것들 중에서 찾기

```
type Menu = { id: number; name: string };
type User = { id: number; email: string };

function findById<T extends { id: number }>(items: T[], id: number): T | undefined {
    return items.find((item) => item.id === id);
}

const menus: Menu[] = [
    { id: 1, name: "아메리카노" },
```

```
{ id: 2, name: "라떼" },
];
const users: User[] = [{ id: 7, email: "a@b.c" }];

console.log(findById(menus, 2)?.name);
```

출력 라떼

```
console.log(findById(users, 7)?.email);
```

출력 a@b.c

하나의 함수로 Menu 도 User 도 찾습니다. 그런데 돌려받은 값에는 각각 제대로 된 타입이 붙어 있습니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(findById(menus, 2)?.email);
```

Property 'email' does not exist on type 'Menu'.

menus 를 넘겼으니 T 는 Menu 입니다. Menu 에는 email 이 없습니다.

any 로 만들었다면 이게 조용히 통과했을 것입니다.

▸ 직접 해보기

3

findById(menus, 99) 를 출력해 보세요. 무엇이 나오나요?

타입에도 붙일 수 있다

섹션 4

함수뿐 아니라 type 에도 <T> 를 붙입니다.

```
type ApiResult<T> = {
  ok: boolean;
  data: T;
};
```

쓸 때 <> 안에 무엇을 담을지 적습니다.

```
const menuResult: ApiResult<Menu> = {
  ok: true,
  data: { id: 1, name: "아메리카노" },
};
console.log(menuResult.data.name);
```

출력 아메리카노

```
const countResult: ApiResult<number> = { ok: true, data: 42 };
console.log(countResult.data.toFixed(0));
```

출력 42

```
const listResult: ApiResult<Menu[]> = { ok: true, data: menus };
console.log(listResult.data.length);
```

출력 2

data 자리만 바꾸고 ok 는 그대로입니다. 서버 응답이 언제나 { ok, data } 모양이라면 이렇게 한 번만 만들어 두고 씁니다.

이 코드는 검사에서 걸립니다

TS2741

```
const wrong: ApiResult<Menu> = { ok: true, data: { id: 1 } };
```

Property 'name' is missing in type '{ id: number; }' but required in type 'Menu'.

<Menu> 라고 했으니 data 는 Menu 모양이어야 합니다.

⇒ 직접 해보기

4

ApiResponse<string> 값을 하나 만들어 data 의 글자 수를 출력해 보세요.

자주 하는 실수

섹션 5

이런 실수를 합니다

T 로 아무 일이나 하려 하기

섹션 3입니다. T 는 모든 타입이라 공통인 것이 거의 없습니다. 무언가를 하려면 extends 로 조건을 거세요.

이런 실수를 합니다

extends 를 상속으로 읽기

“적어도 이걸 갖고 있어야 한다” 입니다. 클래스 상속과는 다른 이야기입니다.

이런 실수를 합니다

부를 때마다 <타입> 을 적기

대개는 안 적어도 알아냅니다. 적어야 하는 건 알아낼 근거가 없을 때뿐입니다(섹션 2).

이런 실수를 합니다

튜플과 배열을 같은 것으로 알기

string, number · 는 딱 두 개, 순서가 정해진 것입니다(02단원 개념01 섹션5).

(string | number)[] 와 다릅니다. 05단원 개념01 섹션5의 그 구분과 비슷합니다.

정리

- 1 타입 매개변수는 <A, B> 처럼 여러 개 받을 수 있다.
- 2 [A, B] 는 튜플이다(02단원 개념01 섹션5). 종류를 부를 때 정하는 것만 다르다. 07단원의 useState 가 이 모양을 돌려준다.
- 3 알아낼 근거가 없으면 부를 때 <타입> 을 직접 적는다. 있으면 말긴다.
- 4 T 로는 할 수 있는 것이 거의 없다. extends 로 "적어도 이걸 있다" 를 걸어 준다.
- 5 type 에도 <T> 를 붙일 수 있다. ApiResponse<Menu> 처럼 담을 것만 바꿔 쓴다.
- 6 제네릭을 쓰면 함수는 하나인데 돌려받는 값에는 제대로 된 타입이 붙는다.

직접 해보기 정답

```
1) const p2 = pair(true, "확인");
   console.log(p2[0], p2[1]);           // 출력: true 확인
```

p2 의 타입은 [boolean, string] 입니다. p2[0].toUpperCase() 를 쓰면 TS2339 로 걸립니다. 0번 자리는 불리언이니깐요.

```
2) const nums = makeEmpty<number>();
   nums.push(3);
   console.log(nums);                  // 출력: [ 3 ]
```

<number> 를 안 적으면 T 를 알아낼 근거가 없어 unknown[] 이 됩니다. 그러면 push(3) 은 되지만 꺼내 쓸 때 막힙니다.

```
3) console.log(findById(menus, 99));    // 출력: undefined
```

find 는 못 찾으면 undefined 를 줍니다. 그래서 반환 타입이 T | undefined 입니다. ?. 를 붙여 둔 이유가 이것입니다(05단원 개념03).

```
4) const textResult: ApiResponse<string> = { ok: true, data: "봄날카페" };
   console.log(textResult.data.length); // 출력: 4
```

data 가 string 이라고 정해 줬으니 .length 가 통과합니다.

이미 쓰고 있던 제네릭

RUN node 개념03_이미_쓰고_있던_제네릭.ts

검사: npm run typecheck

제네릭은 새로 배우는 것처럼 느껴지지만, 사실 01단원부터 계속 쓰고 있었습니다. 표기만 다른 모습으로요.

이 파일은 그것들을 알아보는 눈을 만듭니다. 남의 코드와 라이브러리 설명서를 읽을 때 필요합니다.

string[] 은 Array<string> 이다

섹션 1

두 표기는 완전히 같습니다.

```
const a: string[] = ["라떼", "아메리카노"];
const b: Array<string> = ["라떼", "아메리카노"];

console.log(a.length, b.length);
```

출력 2 2

서로 오갈 수 있습니다. 같은 타입이니까요.

```
const c: string[] = b;
console.log(c[0]);
```

출력 라떼

즉 02단원에서 string[] 을 배웠을 때 이미 Array 라는 제네릭 타입에 string 을 넣어 쓰고 있었던 것입니다.

Array<T> 에 T = string 을 넣은 것 → Array<string> → 줄여서 string[]

검사도 당연히 같습니다.

이 코드는 검사에서 걸립니다

TS2345

```
b.push(4500);
```

Argument of type 'number' is not assignable to parameter of type 'string'.

<string> 이라고 정해 줬으니 문자열만 들어갑니다.

이 자료는 짧은 `string[]` 쪽을 씁니다. 실무 관행도 그렇습니다. `Array<...>` 는 남의 코드에서 볼 때 알아보기만 하면 됩니다.

▹ 직접 해보기

1

`Array<number>` 로 배열을 만들고 값을 하나 넣어 출력해 보세요.

Promise<T> — 나중에 올 값

섹션 2

JS자료 12단원에서 배운 Promise 도 제네릭입니다.

```
async function loadCount(): Promise<number> {
  return 42;
}
```

`Promise<number>` 는 “지금은 없지만 나중에 숫자가 나올 것” 이라는 뜻입니다. `<>` 안에 있는 것이 ‘나중에 나올 값’ 의 타입입니다.

```
const result = await loadCount();
console.log(result);
```

출력 42

`await` 하고 나면 알맹이만 남습니다. Promise 가 벗겨집니다.

```
loadCount()      → Promise<number>
await loadCount() → number
```

그래서 `await` 없이 쓰면 이렇게 됩니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(loadCount().toFixed(0));
```

Property 'toFixed' does not exist on type 'Promise<number>'.

아직 상자에 든 상태라 숫자 기능을 못 씁니다.

JS 에서는 `undefined` 나 `[object Promise]` 가 조용히 나오던 자리입니다.
`await` 를 빠뜨린 실수를 타입이 잡아 줍니다.

`async` 함수의 반환 타입은 안 적어도 추론됩니다.

```

async function loadName() {
  return "봄날카페";
}
const name1 = await loadName();
console.log(name1.length);

```

출력 4

적을 때는 반드시 Promise<> 로 감싸야 합니다.

이 코드는 검사에서 걸립니다

TS1064

```

async function loadNameWrong(): string {
  return "봄날카페";
}

```

The return type of an async function or method must be the global Promise<T> type. Did you mean to write 'Promise<string>'?

async 함수는 언제나 Promise 를 돌려줍니다.

메시지가 답까지 알려 줍니다. 그대로 고치면 됩니다.

▹ 직접 해보기 2

문자열 배열을 돌려주는 async 함수를 만들고

await 로 받아 개수를 출력해 보세요.

Map 과 Set

섹션 3

JS자료에서 잠깐 본 Map 도 제네릭입니다. 두 개를 받습니다.

```

const stock = new Map<string, number>();
stock.set("아메리카노", 12);
stock.set("라떼", 5);

console.log(stock.get("라떼"));

```

출력 5

```

console.log(stock.size);

```

출력 2

Map<K, V> 에서 K 는 열쇠(Key), V 는 값(Value)입니다. 06단원 개념01 섹션5에서 말한 이름 관행이 여기서 나옵니다.

이 코드는 검사에서 걸립니다

TS2345

```
stock.set("카페모카", "많음");
```

Argument of type 'string' is not assignable to parameter of type 'number'.

값은 숫자로 정해 뒀습니다.

없는 열쇠를 물어보면 `undefined` 입니다. 그래서 확인이 필요합니다.

```
console.log(stock.get("없는메뉴"));
```

출력 undefined

이 코드는 검사에서 걸립니다

TS2532

```
console.log(stock.get("라떼") + 1);
```

Object is possibly 'undefined'.

get 의 반환 타입이 V | `undefined` 입니다.

이번엔 TS18048 이 아니라 TS2532 입니다. 이름이 붙은 변수가 아니라 'stock.get(...)' 이라는 결과' 라서 Object 라고만 부르는 것입니다. 둘 다 possibly 'undefined' 이니 하는 말은 같습니다. 개념01 섹션4의 first 와 같은 이유입니다. 없을 수 있으니 사실대로 적힌 것입니다.

```
console.log((stock.get("라떼") ?? 0) + 1);
```

출력 6

Set 은 하나만 받습니다.

```
const tags = new Set<string>();
tags.add("따뜻한");
tags.add("따뜻한");
console.log(tags.size);
```

출력 1

▸ 직접 해보기 3

Map<string, string> 을 만들어 두 쌍을 넣고

없는 열쇠를 ?? 와 함께 꺼내 보세요.

배열 메소드들도 제네릭입니다. 그래서 타입이 정확하게 따라온 것입니다.

```
const prices = [4000, 4500, 5000];

const labels = prices.map((p) => p + "원");
console.log(labels);
```

출력 ['4000원', '4500원', '5000원']

map의 반환 타입이 string[]이 된 것에 주목하세요. 콜백이 무엇을 돌려주는지 보고 정해집니다. map<U>의 U가 그것입니다.

```
console.log(labels[0].toUpperCase());
```

출력 4000원

이 코드는 검사에서 걸립니다

TS2551

```
console.log(labels[0].toFixed(0));
```

Property 'toFixed' does not exist on type 'string'. Did you mean 'fixed'?

콜백이 문자열을 돌려줬으니 결과는 string[]입니다.

숫자 기능을 쓸 수 없습니다.

find는 개념01의 first와 완전히 같은 모양입니다.

```
const found = prices.find((p) => p > 4200);
console.log(found);
```

출력 4500

found의 타입은 number | undefined입니다. 01단원 개념01 섹션4에서 이것 처음 만났을 때는 이유를 몰랐지만, 지금 보면 “제네릭 함수가 사실대로 적어 둔 것”이라는 게 보입니다.

```
console.log((found ?? 0) + 1);
```

출력 4501

▹ 직접 해보기

4

prices.filter(...)의 결과 타입이 무엇일지 예상하고

마우스를 올려 확인해 보세요.

다음 단원 예고

섹션 5

07단원에서 React 의 useState 를 만납니다. 그것도 제네릭입니다.

```
const [count, setCount] = useState<number>(0);
```

└──┘
이 상자에 담을 것의 타입

그리고 돌려주는 것이 [값, 바꾸는 함수] 라는 튜플입니다. 개념02 섹션1에서 본 그 모양입니다.

지금 흉내만 내 보면 이렇습니다.

```
function fakeUseState<T>(initial: T): [T, (next: T) => void] {
  let value = initial;
  const set = (next: T) => {
    value = next;
    console.log("바뀜:", value);
  };
  return [value, set];
}
```

```
const [count, setCount] = fakeUseState(0);
console.log(count);
```

출력 0

```
setCount(5);
```

출력 바뀜: 5

이 코드는 검사에서 걸립니다

TS2345

```
setCount("다섯");
```

Argument of type 'string' is not assignable to parameter of type 'number'.

처음에 0 을 넣었으니 T 가 number 로 정해졌습니다.

그래서 setCount 도 숫자만 받습니다.

React 에서 겪게 될 것이 정확히 이것입니다.

▸ 직접 해보기 5

fakeUseState("대기") 를 만들고 setState 에 숫자를 넘겨 보세요.

자주 하는 실수

섹션 6

이런 실수를 합니다

await 를 빠뜨리기

`Promise<number>` 에 숫자 기능을 쓰려다 걸립니다. 에러 메시지에 `Promise` 가 보이면 `await` 를 빠뜨린 것입니다.

이런 실수를 합니다

async 함수 반환 타입을 **Promise** 없이 적기

TS1064 입니다. 메시지가 답을 알려 주니 그대로 고치면 됩니다.

이런 실수를 합니다

Map.get 결과를 그냥 쓰기

`V | undefined` 입니다. `??` 로 기본값을 주거나 확인하세요.

이런 실수를 합니다

Array<string> 과 **string[]** 이 다르다고 생각하기

같습니다. 표기만 다릅니다.

정리

- 1 `string[]` 은 `Array<string>` 의 줄임말이다. 배열도 처음부터 제네릭이었다.
- 2 `Promise<T>` 의 `T` 는 '나중에 나올 값'의 타입이다. `await` 하면 벗겨진다. `await` 를 빠뜨리면 타입이 잡아 준다.
- 3 `async` 함수의 반환 타입은 반드시 `Promise<>` 로 감싼다.
- 4 `Map<K, V>` 는 열쇠와 값 두 개를 받는다. `get` 의 결과는 `V | undefined` 다.
- 5 `map` · `find` 같은 배열 메소드도 제네릭이다. `find` 가 `number | undefined` 를 주던 이유가 이제 보인다.
- 6 `React` 의 `useState` 도 제네릭이고, `[값, 바꾸는 함수]` 튜플을 돌려준다(07단원).

직접 해보기 정답

```
1) const nums: Array<number> = [];
   nums.push(10);
   console.log(nums);           // 출력: [ 10 ]
```

number[] 라고 써도 완전히 같습니다.

```
2) async function loadMenus(): Promise<string[]> {
```

```
   return ["아메리카노", "라떼"];
```

```
   }
   console.log((await loadMenus()).length); // 출력: 2
```

반환 타입을 안 적어도 Promise<string[]> 로 추론됩니다.

```
3) const memo = new Map<string, string>(); memo.set("a", "가"); memo.set("b", "나");
   console.log(memo.get("z") ?? "없음"); // 출력: 없음
```

4) number[] 입니다. map 은 콜백이 돌려주는 것에 따라 타입이 바뀌지만, filter 는 p > 4200 처럼 '참/거짓만 보는' 조건이면 원래 타입 그대로입니다. (조건이 x !== null 처럼 종류를 걸러내는 것이면 그만큼 좁혀집니다.

```
(string | null)[] 를 그렇게 거르면 string[] 이 됩니다)
```

5) error TS2345: Argument of type 'number' is not assignable to parameter of type 'string'.
"대기" 를 넣었으니 T 가 string 으로 정해졌습니다. 재현:

```
function fakeUseState<T>(initial: T): [T, (next: T) => void] {
```

```
   let value = initial;
   return [value, (next: T) => { value = next; }];
```

```
   }
   const [s, setS] = fakeUseState("대기");
   void s;
   setS(5);
```

처음 값 하나로 그 뒤가 전부 정해지는 것이 useState 의 성질입니다.

keyof: 속성 이름을 타입으로 다루기

RUN node 개념04_keyof로_속성_다루기.ts

검사: npm run typecheck

개념02 섹션3에서 extends 로 “적어도 이걸 갖고 있어야 한다” 를 걸었습니다. 이번엔 한 걸음 더 갑니다.

“이 객체가 가진 속성 이름” 자체를 타입으로 쓴다.

속성 이름을 값으로 넘기는 함수를 만들 때 필요합니다. 목록에서 이름만 뽑기, 표를 원하는 열로 정렬하기 같은 것들입니다.

```
type Menu = { name: string; price: number };

const menu: Menu = { name: "아메리카노", price: 4000 };
```

속성 이름을 그냥 string 으로 받으면

섹션 1

“객체와 속성 이름을 받아서 그 값을 돌려주는 함수” 를 만들어 봅시다. 속성 이름은 글자니까 string 으로 받으면 될 것 같습니다.

이 코드는 검사에서 걸립니다

TS7053

```
function getLoose(item: Menu, key: string) {
  return item[key];
}
```

Element implicitly has an 'any' type because expression of type 'string' can't be used to index type 'Menu'.

key 가 string 이면 “name” 일 수도 있지만 “냐옹” 일 수도 있습니다.

Menu 에 “냐옹” 은 없으니 무엇을 돌려줄지 타입스크립트가 모릅니다.
모르면 any 가 되는데, 이 자료는 any 를 막아 두었으니 여기서 걸립니다.

역지로 통과시키면 어떻게 되는지 보겠습니다. (실제로 쓰지 마세요. 무엇이 문제인지 보려고 적은 것입니다)


```
function getByHand(item: Menu, key: "name" | "price"): string | number {
  return item[key];
}
```

```
console.log(getByHand(menu, "name"));
```

출력 아메리카노

```
console.log(getByHand(menu, "price"));
```

출력 4000

이건 되기는 합니다. 오타도 막아 줍니다.

이 코드는 검사에서 걸립니다

TS2345

```
console.log(getByHand(menu, "nmae"));
```

Argument of type '"nmae"' is not assignable to parameter of type '"name" | "price"'.

오타는 잡혔습니다. 여기까지는 좋습니다.

그런데 두 가지가 불편합니다.

① 돌려받은 값이 string | number 라 바로 못 씁니다.

"name" 을 넘겼는데도 숫자일 수 있다고 나옵니다.

② Menu 에 속성을 하나 추가하면 "name" | "price" 도 손으로 고쳐야 합니다.

① 을 확인해 봅시다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(getByHand(menu, "name").length);
```

Property 'length' does not exist on type 'string | number'.

우리는 "name" 을 넘겼으니 문자열이 온다는 것을 압니다.

그런데 타입에는 그 연결이 안 적혀 있습니다.

05단원 개념01의 "공통으로 있는 것만" 규칙에 걸린 것입니다.

▹ 직접 해보기 1

Menu 에 size: string 을 추가해 보세요.

getByHand 를 고치지 않으면 getByHand(menu, "size") 가 되나요?

손으로 적던 “name” | “price” 를 타입스크립트가 만들어 줍니다.

```
type MenuKey = keyof Menu;
```

MenuKey 는 “name” | “price” 입니다. 05단원에서 배운 유니온 그대로입니다.

```
const key1: MenuKey = "name";
const key2: MenuKey = "price";
console.log(key1, key2);
```

```
출력      name price
```

이 코드는 검사에서 걸립니다

TS2322

```
const key3: MenuKey = "nmae";
```

Type '"nmae"' is not assignable to type 'keyof Menu'.

Menu 에 없는 이름이라 걸립니다.

메시지에 keyof Menu 라고 나오는 것에 주목하세요.

“Menu 의 속성 이름이어야 한다” 는 뜻입니다.

keyof 의 좋은 점은 Menu 를 고치면 따라온다는 것입니다.

```
type Cafe = { name: string; open: boolean; seats: number };
type CafeKey = keyof Cafe;

const ck: CafeKey = "seats";
console.log(ck);
```

```
출력      seats
```

Cafe 에 속성을 더하면 CafeKey 도 저절로 늘어납니다. 손으로 적은 “name” | “open” | “seats” 였다면 고치는 것을 잊어버립니다.

▹ 직접 해보기

2

keyof Cafe 에 “price” 를 넣어 보세요. 무슨 에러가 나나요?

K extends keyof T — 값의 타입까지 따라온다 ★

섹션 1의 불편함 ① 을 해결합니다. 넘긴 이름에 맞는 타입이 그대로 돌아오게 만듭니다.

```
function getField<T, K extends keyof T>(item: T, key: K): T[K] {
  return item[key];
}
```

읽는 법은 이렇습니다.

T 객체의 타입 (Menu)
 K extends keyof T 그 객체의 속성 이름 중 하나 ("name" 또는 "price")
 T[K] T 에서 K 자리에 적힌 타입

T[K] 를 '인덱스 접근 타입' 이라고 합니다. 객체에서 값을 꺼낼 때 item["name"] 이라고 쓰는 것과 모양이 같습니다. 값이 아니라 타입에 대고 같은 일을 하는 것입니다.

```
Menu["name"]    → string
Menu["price"]   → number
```

코드	▶ 출력
console.log(getField(menu, "name").length);	▶ 5
console.log(getField(menu, "price").toFixed(0));	▶ 4000

섹션 1과 비교해 보세요. 같은 .length 가 이번에는 통과합니다. "name" 을 넘겼으니 string 이 온다는 것이 타입에 적혀 있기 때문입니다.

이 코드는 검사에서 걸립니다

TS2339

```
console.log(getField(menu, "price").toUpperCase());
```

Property 'toUpperCase' does not exist on type 'number'.

"price" 를 넘겼으니 number 가 옵니다. 숫자에는 toUpperCase 가 없습니다.

넘긴 이름에 따라 돌아오는 타입이 달라진다는 증거입니다.

오타도 그대로 걸립니다.

이 코드는 검사에서 걸립니다

TS2345

```
console.log(getField(menu, "nmae"));
```

Argument of type '"nmae"' is not assignable to parameter of type 'keyof Menu'.

keyof 가 만들어 준 목록에 없는 이름입니다.

Menu 말고 다른 타입에도 그대로 씁니다. 함수는 하나뿐입니다.

```
const cafe: Cafe = { name: "봄날카페", open: true, seats: 24 };
console.log(getField(cafe, "seats").toFixed(0));
```

출력 24

```
console.log(getField(cafe, "open"));
```

출력 true

▸ 직접 해보기 3

getField(cafe, "name")의 결과에 .toFixed(0)을 써 보세요.

무슨 에러가 나나요?

실전 — 목록에서 한 속성만 뽑기

섹션 4

이 패턴을 실제로 가장 많이 쓰는 곳입니다.

```
function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {
  return items.map((item) => item[key]);
}
```

```
const menus: Menu[] = [
  { name: "아메리카노", price: 4000 },
  { name: "라떼", price: 4500 },
  { name: "카푸치노", price: 5000 },
];
```

```
console.log(pluck(menus, "name"));
```

출력 ['아메리카노', '라떼', '카푸치노']

```
console.log(pluck(menus, "price"));
```

출력 [4000, 4500, 5000]

돌려받은 것이 제대로 된 배열이라 이어서 계산이 됩니다.

```
const total = pluck(menus, "price").reduce((sum, p) => sum + p, 0);
console.log(total);
```

출력 13500

pluck(menus, "price") 가 number[] 이니 sum + p 가 숫자 덧셈입니다. JS자료 08단원의 reduce 그대로인데, 타입이 붙어 있어서 실수가 안 납니다.

이 코드는 검사에서 걸립니다

TS2345

```
console.log(pluck(menus, "prcie"));
```

Argument of type '"prcie"' is not assignable to parameter of type 'keyof Menu'.

오타입니다. 자바스크립트였다면 [undefined, undefined, undefined] 가

조용히 나오고, 합계는 NaN 이 됐을 것입니다.

▸ 직접 해보기

4

pluck(menus, "name") 에 reduce 로 합계를 내 보세요.

무슨 일이 생기나요?

Object.keys 는 왜 keyof 를 안 주나

섹션 5

06

속성 이름을 전부 훑고 싶을 때 Object.keys 를 씁니다.

```
const loose = Object.keys(menu);
console.log(loose);
```

출력 ['name', 'price']

값은 맞는데 타입이 string[] 입니다. (keyof Menu)[] 가 아닙니다.

이 코드는 검사에서 걸립니다

TS7053

```
for (const k of loose) {
  console.log(menu[k]);
}
```

Element implicitly has an 'any' type because expression of type 'string' can't be used to index type 'Menu'.

섹션 1과 똑같은 에러입니다. loose 의 원소가 string 이기 때문입니다.

왜 이렇게 만들어 두었을까요? 04단원 개념04 섹션3의 그 이야기입니다.

타입은 약속이지 검사가 아니다.

menu 에 실제로는 타입에 안 적합한 속성이 더 붙어 있을 수 있습니다. 그래서 Object.keys 가 "keyof Menu 만 나온다" 고 약속할 수 없습니다. 거짓말을 하느니 string[] 이라고 하는 쪽을 고른 것입니다.

훔어야 한다면 키 목록을 직접 적습니다.

```
const keys: (keyof Menu)[] = ["name", "price"];

for (const k of keys) {
  console.log(k, getField(menu, k));
}
```

출력 name 아메리카노
 price 4000

이렇게 적어 두면 Menu 에서 속성을 지웠을 때 이 줄이 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```
const keysWrong: (keyof Menu)[] = ["name", "price", "size"];
```

Type '"size"' is not assignable to type 'keyof Menu'.

Menu 에 size 가 없습니다. 손으로 적은 목록도 타입이 지켜 줍니다.

⇒ 직접 해보기 5

keys 에서 "price" 를 빼고 위 for 문을 돌려 보세요.

에러가 나나요? 출력은 어떻게 되나요?

언제 쓰고 언제 안 쓰나

섹션 6

keyof 는 "속성 이름을 값으로 넘길 때" 만 필요합니다. 그 상황이 아니면 쓰지 마세요. 읽기만 어려워집니다.

— 쓸 만한 자리

· 목록에서 원하는 열만 뽑기 `pluck(menus, "name")` · 표를 원하는 열로 정렬하기 `sortBy(menus, "price")` · 설정 객체에서 이름으로 값 읽기 `getField(config, "포트")`

— 안 써도 되는 자리

· 속성이 하나로 정해져 있으면 그냥 적으면 됩니다.

이렇게 쓰지 말고

```
console.log(getField(menu, "name"));
```

출력 아메리카노

이렇게 쓰세요. 짧고 읽기 쉽습니다.

```
console.log(menu.name);
```

출력 아메리카노

둘은 같은 값을 냅니다. 아래가 낫습니다. 제네릭은 “여러 경우를 하나로 묶을 때” 쓰는 것이지, 한 가지 경우를 어렵게 쓰려고 쓰는 것이 아닙니다.

▸ 직접 해보기

6

메뉴 이름만 모아 글자 수 합계를 내 보세요.

pluck 을 쓰는 것과 `menus.map((m) => m.name)` 을 쓰는 것 중 어느 쪽이 읽기 좋은가요?

자주 하는 실수

섹션 7

이런 실수를 합니다

속성 이름을 string 으로 받기

섹션 1입니다. TS7053 이 나오면 거의 이것입니다. `key: string` 을 `key: K extends keyof T` 로 바꾸세요.

이런 실수를 합니다

keyof 를 붙일 자리를 헛갈리기

`keyof T` 는 ‘이름들’ 이고, `T[K]` 는 ‘그 이름 자리의 타입’ 입니다. 돌려주는 타입에 `keyof T` 를 적으면 이름이 나옵니다. 값이 아니라요.

이런 실수를 합니다

Object.keys 결과를 keyof 로 알기

섹션 5입니다. `string[]` 입니다. 훑어야 하면 목록을 직접 적으세요.

이런 실수를 합니다

안 써도 되는 곳에 쓰기

섹션 6입니다. `menu.name` 으로 끝나는 일에 제네릭을 넣지 마세요.

정리

- 1 `keyof T` 는 `T` 의 속성 이름을 모은 유니온이다. `keyof Menu` 는 `"name" | "price"`.
- 2 `T` 를 고치면 `keyof T` 도 따라온다. 손으로 적은 유니온은 안 따라온다.
- 3 `T[K]` 는 '인덱스 접근 타입' 이다. `Menu["price"]` 는 `number`.
- 4 `<T, K extends keyof T>(item: T, key: K): T[K]` 가 기본 꼴이다. 넘긴 이름에 맞는 타입이 그대로 돌아온다.
- 5 목록에서 한 속성만 뽑는 `pluck` 이 대표적인 쓰임이다.
- 6 `Object.keys` 는 `string[]` 을 준다. 타입은 약속이지 검사가 아니기 때문이다. 훑어야 하면 `(keyof T)[]` 목록을 직접 적는다.
- 7 속성 이름을 값으로 넘길 때만 쓴다. `menu.name` 으로 끝나는 일에는 쓰지 않는다.

직접 해보기 정답

1) `size` 를 추가해도 `getByHand(menu, "size")` 는 안 됩니다. error TS2345: Argument of type `"size"` is not assignable to parameter of type `"name" | "price"`. 재현:

```
type Menu = { name: string; price: number; size: string };
function getByHand(item: Menu, key: "name" | "price"): string | number {

    return item[key];

}

const menu: Menu = { name: "아메리카노", price: 4000, size: "L" };
void getByHand(menu, "size");
```

매개변수에 `"name" | "price"` 라고 손으로 적어 두었기 때문입니다. `Menu` 를 고쳐도 저 줄은 안 따라옵니다. 이것이 섹션 2에서 `keyof` 를 쓰는 이유입니다. 섹션 3의 `getField` 는 고칠 것 없이 바로 됩니다. `keyof Menu` 가 따라오니까요.

2) `Cafe` 에는 `price` 가 없습니다. `keyof Cafe` 는 `"name" | "open" | "seats"` 입니다. error TS2322: Type `"price"` is not assignable to type `'keyof Cafe'`. 재현:

```
type Cafe = { name: string; open: boolean; seats: number };
const ck: keyof Cafe = "price";
void ck;
```

손으로 적은 유니온이었다면 `Cafe` 를 고쳤을 때 이 줄이 안 따라옵니다.

3) "name" 을 넘겼으니 Cafe["name"] 즉 string 이 돌아옵니다. error TS2551: Property 'toFixed' does not exist on type 'string'. Did you mean 'fixed'? 재현:

```
type Cafe = { name: string; open: boolean; seats: number };
function getField<T, K extends keyof T>(item: T, key: K): T[K] {

return item[key];

}
const cafe: Cafe = { name: "봄날카페", open: true, seats: 24 };
void getField(cafe, "name").toFixed(0);
```

문자열에는 toFixed 가 없습니다. Did you mean 'fixed'? 는 무시하세요. 개념02 섹션1에서 본 그 헛다리입니다. 같은 함수인데 "seats" 를 넘기면 toFixed 가 통과합니다(섹션 3 마지막 줄).

4) 타입 에러가 납니다. 번호가 2345 가 아니라 2769 인 것에 주의하세요. error TS2769: No overload matches this call. 재현:

```
type Menu = { name: string; price: number };
function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] {

return items.map((item) => item[key]);

}
const menus: Menu[] = [{ name: "라떼", price: 4500 }];
void pluck(menus, "name").reduce((sum, p) => sum + p, 0);
```

쓰는 방법이 여러 가지인 함수라 "맞는 사용법이 하나도 없다" 고 나옵니다. 이어지는 줄에 Type 'string' is not assignable to type 'number'. 가 붙습니다. 그 줄까지 읽어야 이유가 나옵니다. pluck(menus, "name") 은 string[] 이라 sum + p 가 글자 붙이기가 되는데, 시작값 0 은 숫자라 어긋난 것입니다. 자바스크립트였다면 "0아메리카노라떼카푸치노" 가 조용히 나왔을 것입니다.

5) 에러는 안 납니다. 출력만 한 줄로 줄어듭니다.

```
출력      name 아메리카노
```

(keyof Menu)[] 는 "전부 다 적어야 한다" 는 뜻이 아니라 "Menu 의 속성 이름만 넣어라" 는 뜻이기 때문입니다. 빠뜨린 것까지 잡고 싶으면 07단원 이후에 만나는 Record 를 씁니다.

```
6) const sum = pluck(menus, "name").reduce((n, s) => n + s.length, 0);
console.log(sum); // 출력: 11
```

아메리카노 5 + 라떼 2 + 카푸치노 4 = 11 입니다. 문제 4와 달리 s.length 로 숫자를 꺼내 더했으니 타입이 맞습니다. 읽기는 menus.map((m) => m.name) 쪽이 낫습니다. 여기서는 뱀을 속성이 "name" 하나로 정해져 있기 때문입니다. pluck 은 뱀을 속성이 그때그때 달라질 때 값어치가 있습니다.

제네릭

RUN node 연습문제.ts

채점: npm run grade ← 출력을 기대 출력과 맞춰 봅니다

검사: npm run check ← 타입 검사 (일부 문제만 잡습니다)

★ 채점은 npm run grade 로 합니다. 실제 출력을 문제의 '기대 출력' 과 줄 단위로 맞춰 봅니다. 전부 일치하면 다 맞은 것입니다.

npm run check(타입 검사)도 같이 쓰세요. 다만 이쪽은 '일부 문제만' 잡습니다.

138문항 중 21개뿐입니다. "출력하세요" 류는 안 풀어도 타입 에러가 안 나거든요. 그러니 check 가 조용하다고 다 맞은 것이 아닙니다. grade 로 확인하세요. (npm run typecheck 는 개념·정답 파일만 봅니다. 그쪽은 언제나 조용합니다)

- 1 TODO 자리에 코드를 씁니다.
- 2 npm run grade 로 채점합니다. (직접 node 연습문제.ts 로 돌려 봐도 됩니다)
- 3 npm run check 로 타입도 봅니다.
- 4 10분 고민해도 안 되면 연습문제_정답.ts 를 보세요.

문제 1~8과 14는 기본, 9~12와 15는 응용, 13은 [도전]입니다. 문제 14~15는 개념04(keyof)에서 나옵니다. 문제 16은 에러를 직접 보는 문제라 맨 뒤에 있습니다.

```
console.log("=== 06단원 연습문제 ===");
```

문제 1 (개념01 섹션3)

배열의 마지막 값을 돌려주는 제네릭 함수 last 를 만드세요.

【지금까지 하던 방법】 타입마다 하나씩

```
function lastNumber(arr: number[]) { return arr[arr.length - 1]; }
```

【이번에 배울 방법】 하나로

```
function last<T>(arr: T[]): T | undefined { ... }
```

기대 출력 다
 30

기대 검사 결과: 조용함

```
{
```

`last` 를 만들고 ["가", "나", "다"] 와 [10, 20, 30] 에 각각 쓰세요

```
}
```

문제 2 (개념01 섹션4)

위 `last` 를 빈 배열에 쓰면 `undefined` 가 나옵니다. ?? 를 써서 없으면 "없음" 이 나오게 출력하세요.

기대 출력 없음

기대 검사 결과: 조용함

```
{
  function last<T>(arr: T[]): T | undefined {
    return arr[arr.length - 1];
  }
}
```

빈 문자열 배열에 `last` 를 쓰고 ?? 로 "없음" 을 내보내세요

```
}
```

문제 3 (개념02 섹션1)

값 두 개를 받아 튜플로 돌려주는 `pair` 를 만들고, 구조분해로 꺼내 출력하세요.

기대 출력 라떼 4500

기대 검사 결과: 조용함

```
{
```

`pair<A, B>` 를 만들고 `pair("라떼", 4500)` 을 구조분해해서 출력하세요

```
}
```

문제 4 (개념02 섹션2)

아래 함수는 넘기는 값이 없어 `T` 를 알아낼 수 없습니다. 부를 때 타입을 직접 정해 주세요.

기대 출력 4

기대 검사 결과: 조용함

```
{  
  function makeEmpty<T>(): T[] {  
    return [];  
  }  
}
```

`makeEmpty` 에 타입을 정해 주세요 (지금은 마지막 줄이 걸립니다)

```
const nums = makeEmpty();  
nums.push(3);  
console.log(nums[0] + 1);  
}
```

문제 5 (개념02 섹션3)

아래 함수는 걸립니다. `extends` 로 조건을 걸어 통과하게 만드세요.

기대 출력	5
	3

기대 검사 결과: 조용함

```
{

T 에 조건을 거세요

function getLength<T>(value: T): number {
    return value.length;
}

console.log(getLength("아메리카노"));
console.log(getLength([1, 2, 3]));

}
```

06

문제 6 (개념02 섹션4)

{ ok: boolean; data: T } 모양의 ApiResponse 타입을 만들고, data 가 문자열인 값을 하나 만들어 글자 수를 출력하세요.

기대 출력	4
-------	---

기대 검사 결과: 조용함

```
{

type ApiResponse<T> 를 만들고 써 보세요

}
```

문제 7 (개념03 섹션1)

아래 배열을 Array<...> 표기로 바꿔 쓰세요. 동작은 같아야 합니다.

기대 출력 2

기대 검사 결과: 조용함

```
{
```

Array<string> 표기로 바꾸세요

```
const menus: string[] = ["아메리카노", "라떼"];  
console.log(menus.length);  
}
```

문제 8 (개념03 섹션2)

숫자를 돌려주는 async 함수를 만들고, await 로 받아 출력하세요. 반환 타입을 Promise<number> 로 명시하세요.

기대 출력 42

기대 검사 결과: 조용함

```
{
```

async 함수를 만들고 await 로 받아 출력하세요

```
}
```

문제 9 (응용 · 개념03 섹션3)

Map<string, number> 로 재고를 만들고, 없는 메뉴를 ?? 0 과 함께 꺼내 출력하세요.

기대 출력 12
 0

기대 검사 결과: 조용함

{

Map 을 만들고 있는 것과 없는 것을 각각 출력하세요

}

문제 10 (응용 · 개념01 섹션2)

any[] 로 만든 함수 대신 제네릭을 쓰는 이유를 한 줄로 출력하세요.

기대 출력 돌려받은 값의 타입이 살아 있어서

기대 검사 결과: 조용함

{

이유를 그대로 출력하세요

}

문제 11 (응용 · 개념03 섹션4)

아래 두 결과의 타입 이름을 순서대로 출력하세요.

```
const a = [1, 2, 3].map((n) => n + "원");
const b = [1, 2, 3].filter((n) => n > 1);
```

기대 출력 string[]
 number[]

기대 검사 결과: 조용함

```
{
```

두 타입 이름을 출력하세요

```
}
```

문제 12 (응용 · 개념02 섹션3)

id 를 가진 것이면 무엇이든 찾을 수 있는 findById 를 만드세요. Menu 와 User 양쪽에 쓰고, 못 찾으면 "없음" 을 출력하세요.

기대 출력 라떼
 a@b.c
 없음

기대 검사 결과: 조용함

```
{
  type Menu = { id: number; name: string };
  type User = { id: number; email: string };
  const menus: Menu[] = [
    { id: 1, name: "아메리카노" },
    { id: 2, name: "라떼" },
  ];
  const users: User[] = [{ id: 7, email: "a@b.c" }];
```


findById 를 만들고 세 번 부르세요 (menus 2 / users 7 / menus 99)

```
}
```

문제 13 ([도전] · 개념01~03 종합)

서버 응답을 흉내 내는 함수와, 그 결과를 다루는 코드를 만드세요. ① type ApiResponse<T> = { ok: boolean; data: T } 를 만든다 ② async function load<T>(data: T): Promise<ApiResponse<T>> 를 만든다 ③ 메뉴 배열을 넘겨 await 로 받는다 ④ 첫 번째 메뉴 이름을, 없으면 "비어 있음" 을 출력한다

기대 출력 아메리카노

기대 검사 결과: 조용함

```
{
  type Menu = { id: number; name: string };
  const menus: Menu[] = [
    { id: 1, name: "아메리카노" },
    { id: 2, name: "라떼" },
  ];
```

위 네 가지를 하세요

```
}
```

문제 14 (개념04 섹션3)

객체와 속성 이름을 받아 그 값을 돌려주는 getField 를 만드세요. 넘긴 이름에 맞는 타입이 그대로 돌아와야 합니다.

【이렇게 하면 안 됩니다】속성 이름을 string 으로 받기

```
function getField(item: Cafe, key: string) { return item[key]; }
→ TS7053 으로 걸립니다. key 가 "냐옹" 일 수도 있으니까요.
```

【이번에 배울 방법】

```
function getField<T, K extends keyof T>(item: T, key: K): T[K] { ... }
```

```
기대 출력    봄날카페
              24
              4
```

기대 검사 결과: 조용함

```
{
  type Cafe = { name: string; open: boolean; seats: number };
  const cafe: Cafe = { name: "봄날카페", open: true, seats: 24 };
}
```

getField 를 만들어 name 과 seats 를 출력하고,

이어서 name 의 글자 수(.length)도 출력하세요

```
}
```

문제 15 (응용 · 개념04 섹션4)

목록에서 한 속성만 뽑는 pluck 을 만들어 두 가지를 하세요. ① 제목만 뽑아서 배열째 출력한다 ② 재고만 뽑아서 합계를 출력한다

【이번에 배울 방법】

```
function pluck<T, K extends keyof T>(items: T[], key: K): T[K][] { ... }
```

```
기대 출력
```

— ‘타입 입문’, ‘제네릭 연습’, ‘실전 TS’

10

기대 검사 결과: 조용함

```
{
  type Book = { title: string; stock: number };
  const books: Book[] = [
    { title: "타입 입문", stock: 3 },
    { title: "제네릭 연습", stock: 0 },
    { title: "실전 TS", stock: 7 },
  ];
}

pluck 을 만들고 ①②를 하세요 (합계는 reduce 를 쓰세요)
```

문제 16 (에러 확인 · 개념03 섹션2)

아래 세 줄의 // 를 지우고 npm run check 로 검사해 보세요.

① 무슨 에러가 나나요? ② 무엇을 빠뜨린 것일까요?

확인했으면 다시 // 를 붙이세요.

```
{

  async function loadCount(): Promise<number> { return 42; }
  const n = loadCount();
  console.log(n.toFixed(0));

}
```


React와 타입스크립트

OPEN 07_React와_타입스크립트

```
type MenuItemProps = {  
  name: string;  
  price: number;  
  return (  

```

01 props 에 타입 붙이기

02 useState 와 타입

03 이벤트와 폼

04 화면 상태 다루기 ★ 이 단원의 결론

04 주문 화면

- 연습문제

props 에 타입 붙이기

보기: npm run dev → 왼쪽 목록에서 고르기

검사: npm run typecheck

React자료 03단원에서 props 를 배웠습니다. 그때 이렇게 썼습니다.

```
function MenuItem({ name, price }) { ... }
```

여기서 name 과 price 가 무엇인지는 아무 데도 안 적혀 있었습니다. 쓰는 사람이 컴포넌트 안을 열어 봐야 알 수 있었습니다.

이 파일은 그 자리에 설명서를 붙입니다. 04단원에서 배운 객체 타입 그대로입니다.

```
import { useState } from "react";
import type { ReactNode } from "react";
import Summary from "../_ui/Summary.tsx";
```

props 는 그냥 객체다

섹션 1

props 는 객체 하나입니다. 그러니 04단원의 객체 타입을 그대로 쓰면 됩니다.

```
type MenuItemProps = {
  name: string;
  price: number;
};

function MenuItem({ name, price }: MenuItemProps) {
  return (
    <li>
      {name} - {price}원
    </li>
  );
}
```

적는 자리를 그림으로 보면 이렇습니다.

```
function MenuItem({ name, price }: MenuItemProps) {
  └── 구조분해 ─┘ └── 타입 ─┘
```

구조분해는 React자료 03단원 개념03에서 배운 그대로이고, 그 뒤에 : 타입 만 붙은 것입니다.

이제 쓰는 쪽에서 검사가 됩니다.

이 코드는 검사에서 걸립니다

TS2741

```
const 빠뜨림 = <MenuItem name="라떼" />;
```

Property 'price' is missing in type '{ name: string; }' but required in type 'MenuItemProps'.

price 를 안 넘겼다고 그 자리에서 알려 줍니다.

JS 였다면 화면에 "undefined원" 이 찍히고 나서야 알았습니다.

이 코드는 검사에서 걸립니다

TS2322

```
const 종류틀림 = <MenuItem name="라떼" price="4500" />;
```

Type 'string' is not assignable to type 'number'.

따옴표를 붙이면 문자열입니다. 숫자는 {4500} 로 넘겨야 합니다.

React자료 03단원에서 "숫자는 중괄호로" 라고 배운 그 규칙을
이제 타입이 강제해 줍니다.

▹ 직접 해보기

1

MenuItem 에 isHot(boolean) props 를 추가하고

아래 화면에서 넘겨 보세요. 안 넘기면 어떻게 되나요?

안 붙이면 어떻게 되나

섹션 2

매개변수 타입을 안 적으면 03단원에서 본 그 에러가 납니다.

이 코드는 검사에서 걸립니다

TS7031

```
function MenuItemNoType({ name }) {  
  return <li>{name}</li>;  
}
```

Binding element 'name' implicitly has an 'any' type.

03단원의 TS7006 과 형제입니다.

구조분해한 자리라서 Binding element 라고 부를 뿐,
"은근슬쩍 any 가 됐다" 는 말은 같습니다.
props 에도 예외 없이 타입을 적습니다.

▣ 직접 해보기 2

MenuItem 의 MenuItemProps 를 지우고 저장해 보세요.

무슨 에러가 나나요? 확인한 뒤 되돌리세요.

없어도 되는 props

섹션 3

04단원 개념03의 ? 를 그대로 씁니다.

```
type BadgeProps = {
  text: string;
  color?: string; // 없으면 기본값
};

function Badge({ text, color = "#555" }: BadgeProps) {
  return (
    <span style={{ background: color, color: "#fff", padding: "2px 8px",
borderRadius: 4 }}>
      {text}
    </span>
  );
}
```

기본값을 주면 color 는 함수 안에서 string 입니다. undefined 가 안 섞입니다. 03단원 개념02에서 "쓸
만한 기본값이 있으면 ? 보다 기본값" 이라고 한 그대로입니다.

▣ 직접 해보기 3

Badge 의 기본값 = "#555" 를 지워 보세요.

검사는 걸릴까요? 화면은 어떻게 되나요?

children 의 타입

섹션 4

태그 사이에 끼워 넣는 것을 받으려면 children 을 적습니다.

```
type CardProps = {
  title: string;
  children: ReactNode;
```



```
};

function Card({ title, children }: CardProps) {
  return (
    <div style={{ border: "1px solid #ddd", borderRadius: 8, padding: 12,
marginBottom: 12 }}> <h3 style={{ margin: "0 0 8px" }}>{title}</h3>
      {children}
    </div>
  );
}
```

ReactNode 는 "화면에 그릴 수 있는 것 전부" 입니다. 글자 · 숫자 · 태그 · 배열 · `null` 까지 다 들어갑니다.

```
import type { ReactNode } from "react";
```

처럼 `import` 앞에 `type` 을 붙인 것에 주목하세요.

"이건 타입만 가져오는 것" 이라는 표시입니다. 안 붙여도 대개 동작하지만, 붙이면 번들에 안 들어간다는 것이 분명해집니다.

▹ 직접 해보기 4

Card 를 태그 사이 내용 없이 `<Card title="빈 상자" />` 로 써 보세요.

걸리나요? `children` 을 `children?`: `ReactNode` 로 바꾸면 어떻게 되나요?

함수를 props 로 넘기기

섹션 5

03단원 개념03에서 배운 함수 타입을 그대로 씁니다.

```
type CounterProps = {
  label: string;
  onCount: (next: number) => void;
};

function Counter({ label, onCount }: CounterProps) {
  const [count, setCount] = useState(0);

  function handleClick() {
    const next = count + 1;
    setCount(next);
    onCount(next);
  }

  return (
    <button type="button" onClick={handleClick}>
```

```

    {label}: {count}
  </button>
);
}

```

넘기는 쪽에서 모양이 안 맞으면 걸립니다.

이 코드는 검사에서 걸립니다

TS2322

```

const 잘못넘김 = <Counter label="주문" onCount={({next: string}) =>
  console.log(next)} />;

```

Type '(next: string) => void' is not assignable to type '(next: number) => void'.

숫자를 받기로 한 자리에 문자열을 받는 함수를 넘겼습니다.

▹ 직접 해보기 5

Counter 의 onCount 를 (next: number) ⇒ string 으로 바꿔 보세요.

넘기는 쪽과 받는 쪽 중 어디가 걸리나요?

자주 하는 실수

섹션 6

이런 실수를 합니다

props 타입을 안 붙이기

TS7031 입니다. 03단원의 TS7006 과 형제입니다.

이런 실수를 합니다

children 이 자동으로 있다고 생각하기

직접 적어야 합니다. 예전 React.FC 를 쓰던 시절 이야기입니다.

이런 실수를 합니다

숫자를 따옴표로 넘기기

```

price="4500" 은 문자열입니다. price={4500} 로 써야 합니다.

```

React자료에서 배운 규칙을 이제 타입이 강제해 줍니다.

이런 실수를 합니다

props 타입 이름을 컴포넌트와 다르게 짓기

관행은 컴포넌트 이름 + Props 입니다. MenuItem → MenuItemProps. 찾기 쉬워집니다.

화면

정리

- 1 props 는 객체 하나다. 04단원의 객체 타입을 그대로 쓴다.
- 2 적는 자리는 구조분해 뒤 — `function C({ a, b }: CProps) 다.`
- 3 안 적으면 TS7031(Binding element ... implicitly has an 'any' type)이 난다.
- 4 없어도 되는 props 는 ? 를 붙이고, 쓸 만한 기본값이 있으면 = 로 준다.
- 5 태그 사이의 내용을 받으려면 children: ReactNode 를 적는다.
- 6 함수 props 는 (인자) ⇒ void 처럼 03단원의 함수 타입으로 적는다.
- 7 타입만 가져올 때는 import type 으로 쓴다.

```
export default function Page() {
  const [last, setLast] = useState(0);

  return (
    <div> <h2>개념 01 – props 에 타입 붙이기</h2> <Card title="섹션 1 – 기본
props"> <ul> <MenuItem name="아메리카노" price={4000} /> <MenuItem name="라떼" price=
{4500} /> </ul> </Card> <Card title="섹션 3 – 없어도 되는
props"> <p> <Badge text="기본색" /> <Badge text="지정색" color="#2d6cdf"
/> </p> </Card> <Card title="섹션 5 – 함수 props"> <Counter label="주문" onCount=
{setLast} /> <p>마지막으로 받은 값: {last}</p> </Card> </div>
  );
}
```

직접 해보기 정답

```
1) type MenuItemProps = { name: string; price: number; isHot: boolean };
```

안 넘기면 TS2741 Property 'isHot' is missing ... 가 납니다. 선택으로 만들려면 isHot?: boolean 입니다.

2) error TS7031: Binding element 'name' implicitly has an 'any' type. (price 에 대해서도 한 번 더 납니다) 재현:

```
function MenuItem({ name, price }) {
```

```
  return <li>{name} {price}</li>;
```

```
}  
void MenuItem;
```

구조분해한 자리라 Binding element 라고 부를 뿐, 뜻은 TS7006 과 같습니다.

3) 검사는 걸리지 않습니다. 조용합니다. 기본값을 지우면 color 가 string | undefined 가 되는데, style 의 background 는 원래 "없어도 되는" 자리라 undefined 를 그냥 받습니다.

대신 화면이 바뀝니다. <Badge text="기본색" /> 의 배경색이 사라져서 흰 바탕에 흰 글씨가 되어 글자가 안 보입니다.

→ 01단원 개념01의 "타입이 못 잡는 것 — 화면이 이상한 것" 이 이것입니다.

? 를 붙였으면 기본값을 주는 편이 안전합니다.
타입이 안 막아 주는 자리일수록 기본값이 중요합니다.

4 걸립니다. TS2741 Property 'children' is missing ... 입니다. children?: ReactNode 로 바꾸면 통과하고, 안쪽이 비어 있게 그려집니다. "내용이 없어도 되는 상자" 라면 ? 를 붙이는 것이 맞습니다.

5 넘기는 쪽이 걸립니다.

onCount={setLast} 에서 setLast 는 문자열을 안 돌려주기 때문입니다.

받는 쪽(Counter 안)은 onCount(next) 를 부르지만 하므로 문제가 없습니다.

useState 와 타입

보기: npm run dev → 왼쪽 목록에서 고르기

검사: npm run typecheck

06단원 개념03 섹션5에서 useState 를 미리 흉내 내 봤습니다.

```
function fakeUseState<T>(initial: T): [T, (next: T) => void]
```

진짜 useState 도 이 모양입니다. 제네릭 함수이고 튜플을 돌려줍니다. 그래서 대부분은 아무것도 안 적어도 됩니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.tsx";
```

대개는 안 적어도 된다

섹션 1

```
function Counter() {
```

0 을 보고 number 로 정해집니다. useState<number> 라고 적을 필요가 없습니다.

```
const [count, setCount] = useState(0);

return (
  <button type="button" onClick={() => setCount(count + 1)}>
    눌린 횟수: {count}
  </button>
);
}
```

정해지고 나면 검사가 살아 있습니다.

이 코드는 검사에서 걸립니다

TS2345

```
function CounterWrong() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount("다섯")}>{count}</button>;
}
```

Argument of type 'string' is not assignable to parameter of type 'SetStateAction<number>'.

처음에 0 을 넣었으니 숫자 상자입니다.

`SetStateAction<number>` 는 "number 이거나, number 를 돌려주는 함수" 라는 뜻입니다.
`setCount(count + 1)` 도 되고 `setCount((c) => c + 1)` 도 되기 때문에 이런 이름입니다.

▹ 직접 해보기 1

Counter 를 `useState("0")` 으로 바꾸고

`setCount(count + 1)` 이 어떻게 되는지 확인해 보세요.

적어야 하는 경우 ① null 로 시작할 때

섹션 2

```
type Menu = { name: string; price: number };

function Selected() {
```

`null` 만 보고는 "나중에 Menu 가 들어올 것" 을 알 수가 없습니다. 02단원 개념02 섹션5의 "초기값이 없으면 적는다" 와 같은 상황입니다.

```
const [selected, setSelected] = useState<Menu | null>(null);

return (
  <div> <button type="button" onClick={() => setSelected({ name: "라떼", price:
4500 })}>
    라떼 고르기
  </button>{" "}
  <button type="button" onClick={() => setSelected(null)}>
    취소
  </button>
  { /* selected 는 Menu | null 이라 그냥 못 씁니다. 05단원 개념03 그대로입니다
*/}
  <p>고른 것: {selected?.name ?? "없음"}</p> </div>
);
}
```

안 적으면 이렇게 됩니다.

이 코드는 검사에서 걸립니다

TS2353

```
function SelectedWrong() {
  const [selected, setSelected] = useState(null);
  return <button onClick={() => setSelected({ name: "라떼", price: 4500 })}>
    고르기</button>;
}
```

Object literal may only specify known properties, and 'name' does not exist in type '(prevState: null) => null'.

null 만 보고 "이 상자에는 null 만 들어간다" 고 정해 버린 것입니다.

null 로 시작하는 useState 는 거의 항상 <T | null> 을 적어야 합니다.

메시지에 (prevState: null) => null 이라는 낯선 것이 나오는데 놀라지 마세요.
 setState 는 '값' 도 받고 '이전 값을 받아 새 값을 돌려주는 함수' 도 받습니다.
 값 쪽이 안 맞으니 함수 쪽으로도 맞춰 보다가 낸 말입니다.
 핵심은 하나입니다 - 상자가 null 전용으로 잡혔다.

▹ 직접 해보기 2

Selected 의 selected?.name 에서 ?. 를 . 로 바꿔 보세요.

무슨 에러가 나나요? 확인한 뒤 되돌리세요.

적어야 하는 경우 ② 빈 배열로 시작할 때

섹션 3

```
function Cart() {
```

[] 만 보고는 무엇이 들어올 배열인지 알 수 없습니다.

```
const [items, setItems] = useState<Menu[]>([]);
```

```
function add() {
```

React자료 07단원에서 배운 불변 갱신 그대로입니다.

```
setItems([...items, { name: "아메리카노", price: 4000 }]);
}

const total = items.reduce((sum, m) => sum + m.price, 0);

return (
  <div> <button type="button" onClick={add}>
    담기
```

이벤트와 폼

보기: npm run dev → 왼쪽 목록에서 고르기

검사: npm run typecheck

React자료 06단원에서 폼과 입력을 배웠습니다. 그때 이렇게 썼습니다.

```
function handleChange(e) { setText(e.target.value); }
```

여기서 e 가 무엇인지는 아무도 안 알려 줬습니다. 타입스크립트에서는 이 자리가 바로 TS7006 이 나는 자리입니다.

다행히 대부분은 안 적어도 됩니다. 적어야 할 때만 구분하면 됩니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.tsx";
```

태그 안에 바로 쓰면 안 적어도 된다

섹션 1

```
function InlineHandler() {
  const [count, setCount] = useState(0);

  return (
    <div>
      { /* onClick 안에 바로 쓴 함수의 e 에는 타입이 자동으로 붙습니다 */ }
      <button type="button" onClick={(e) => setCount(count + e.detail)}>
        누른 횟수만큼 더하기: {count}
      </button> <p>더블클릭하면 2씩 오릅니다(e.detail 이 클릭 횟수입니다).</p> </div>
    );
}
```

e 에 타입을 안 적었는데 e.detail 이 통과했습니다. onClick 이 무엇을 넘겨줄지 React 쪽에 이미 적혀 있어서, 03단원 개념03의 '문맥에서 알아내기' 가 그대로 작동한 것입니다.

그래서 없는 것을 쓰면 걸립니다.

이 코드는 검사에서 걸립니다

TS2339

```
const 없는것 = <button onClick={(e) => console.log(e.value)} />;
```

Property 'value' does not exist on type 'MouseEvent<HTMLButtonElement, MouseEvent>'.

클릭 이벤트에는 value 가 없습니다.

메시지에 `MouseEvent<HTMLButtonElement, ...>` 라고 나오는 것에 주목하세요.
"버튼에서 난 마우스 이벤트" 라는 뜻입니다. 이름이 길 뿐 무섭지 않습니다.

▸ 직접 해보기 1

InlineHandler 의 (e) 에 마우스를 올려 보세요.

무슨 타입이라고 나오나요?

함수를 따로 빼면 적어야 한다

섹션 2

이때는 문맥이 없습니다. 그냥 함수 하나일 뿐이니 알아낼 근거가 없습니다.

이 코드는 검사에서 걸립니다

TS7006

```
function handleClickNoType(e) {
  console.log(e.detail);
}
```

Parameter 'e' implicitly has an 'any' type.

03단원의 그 에러 그대로입니다. 매개변수는 적어야 합니다.

그럼 무슨 타입을 적어야 하나 — 알아내는 요령이 있습니다.

① 일단 태그 안에 인라인으로 써 본다 ② e 에 마우스를 올려 뜨는 이름을 그대로 베끼다 ③ 그 이름을 import 해서 함수 밖으로 옮긴다

외울 필요가 없습니다. 편집기가 알려 줍니다.

```
function OutsideHandler() {
  const [count, setCount] = useState(0);
```

위 요령으로 알아낸 이름을 적었습니다.

```
function handleClick(e: React.MouseEvent<HTMLButtonElement>) {
  setCount(count + e.detail);
}

return (
  <button type="button" onClick={handleClick}>
    따로 뺀 핸들러: {count}
  </button>
);
}
```

React.MouseEvent 처럼 React. 을 붙여 쓰면 import 를 따로 안 해도 됩니다.

```
import type { MouseEvent } from "react"; 로 가져와 MouseEvent 라고만 써도 됩니다.
```

이 자료는 짧게 React. 을 붙이는 쪽을 씁니다.

⇒ 직접 해보기 2

OutsideHandler 의 handleClick 에서 타입 표기를 지워 보세요.

무슨 에러가 나나요? 확인한 뒤 되돌리세요.

입력창 — onChange

섹션 3

```
function TextInput() {
  const [text, setText] = useState("");
```

입력창의 이벤트는 ChangeEvent 이고, 안에 어떤 태그인지까지 적습니다.

```
function handleChange(e: React.ChangeEvent<HTMLInputElement>) {
  setText(e.target.value);
}

return (
  <div> <input value={text} onChange={handleChange} placeholder="메뉴 이름" /> <p>
    {text.length}자 입력됨</p> </div>
);
}
```

<HTMLInputElement> 를 적어야 e.target.value 가 통과합니다. 그 자리가 무엇인지 알려 줘야 target 에 무엇이 있는지도 알 수 있기 때문입니다.

태그마다 이름이 다릅니다. 셋만 알면 됩니다.

```

<input>    → HTMLInputElement
<textarea> → HTMLTextAreaElement
<select>   → HTMLSelectElement

```

▫ 직접 해보기 3

TextInput의 `<HTMLInputElement>`를 지워 보세요.

무슨 에러가 나나요?

폼 — onSubmit

섹션 4

```

function OrderForm() {
  const [menu, setMenu] = useState("");
  const [orders, setOrders] = useState<string[]>([]);

  function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault(); // React자료 06단원에서 배운 그것입니다 if (menu.trim() ===
    "") return;
    setOrders([...orders, menu]);
    setMenu("");
  }

  return (
    <form onSubmit={handleSubmit}> <input value={menu} onChange={(e) =>
setMenu(e.target.value)} placeholder="주문할 메뉴" />{" "}
    <button type="submit">주문</button> <ul>
      {orders.map((o, i) => (
        <li key={i}>{o}</li>
      ))}
    </ul> </form>
  );
}

```

폼은 `FormEvent`입니다. 오타를 내면 `preventDefault`에서 걸립니다.

이 코드는 검사에서 걸립니다

TS2339

```

function handleSubmitWrong(e: {}) {
  e.preventDefault();
}

```

Property 'preventDefault' does not exist on type '{}'.

타입을 아무렇게나 적으면 그 안에 없는 기능은 못 씁니다.

모르겠으면 섹션 2의 요령을 쓰세요. 인라인으로 써 보고 이름을 베깁니다.

▹ 직접 해보기 4

OrderForm 의 `e.preventDefault()` 를 지우고

화면에서 주문을 눌러 보세요. 무슨 일이 일어나나요? (타입 검사는 뭐라고 하나요?)

자주 하는 실수

섹션 5

이런 실수를 합니다

이벤트 타입을 외우려 하기

외울 필요가 없습니다. 인라인으로 써 보고 마우스를 올려 베끼세요. 실무 개발자도 그렇게 합니다.

이런 실수를 합니다

<HTMLInputElement> 를 빼먹기

`React.ChangeEvent` 라고만 쓰면 TS2314 Generic type requires 1 type argument(s). 입니다. 무엇에서 난 이벤트인지까지 적어야 합니다.

이런 실수를 합니다

`e.target` 과 `e.currentTarget` 을 헷갈리기

`currentTarget` 은 핸들러를 붙인 그 태그이고, `target` 은 실제로 눌린 것입니다. 버튼 안에 `<span>` 이 있으면 `target` 은 `span` 일 수 있습니다. 입력창에서는 둘이 같아서 문제가 안 생깁니다.

이런 실수를 합니다

`any` 로 넘어가기

`function handleChange(e: any)` 는 검사를 통째로 끕니다. 02단원의 규칙 그대로입니다. 쓰지 마세요.

화면

정리

- 1 태그 안에 인라인으로 쓴 핸들러의 e 는 안 적어도 된다(문맥 추론).
- 2 따로 뺀 함수는 적어야 한다. 안 적으면 TS7006 이다.
- 3 타입 이름은 외우지 말고, 인라인으로 써 본 뒤 마우스를 올려 베킨다.
- 4 클릭은 React.MouseEvent
- 5 입력은 React.ChangeEvent
- 6 폼은 React.FormEvent
- 7 를 빼먹으면 TS2314 로 걸린다. 무엇에서 난 이벤트인지까지 적는다.

```
export default function Page() {
  return (
    <div> <h2>개념 03 - 이벤트와 폼</h2> <p>섹션 1 - 인라인은 안 적어도 됨
  </p> <InlineHandler /> <p>섹션 2 - 따로 뺀 핸들러</p> <OutsideHandler /> <p>섹션 3 -
  입력창</p> <TextInput /> <p>섹션 4 - 폼</p> <OrderForm /> </div>
  );
}
```

직접 해보기 정답

- 1 React.MouseEvent<HTMLButtonElement, MouseEvent> 라고 나옵니다. 이 이름을 그대로 베껴서 함수 밖으로 옮기면 됩니다. (뒤쪽, MouseEvent 는 생략해도 됩니다)
- 2 error TS7006: Parameter 'e' implicitly has an 'any' type. 재현:

```
function handleClick(e) { console.log(e.detail); }
void handleClick;
```

태그 안에 있을 때는 문맥이 있어서 안 적어도 됐지만, 밖으로 빼면 그냥 함수 하나일 뿐이라 알아낼 근거가 없습니다.

- 3) error TS2339: Property 'value' does not exist on type 'EventTarget & Element'. 재현:

```
function handleChange(e: React.ChangeEvent) { console.log(e.target.value); }
void handleChange;
```

적는 자리에서는 안 걸립니다. <> 를 빼면 기본값이 들어가기 때문입니다. 걸리는 곳은 e.target.value 를 '쓰는' 줄입니다. 기본값이 그냥 Element 라서 그 안에 value 가 없습니다. 무엇에서 난 이벤트인지까지 적어야 target 에 무엇이 있는지 알 수 있습니다.

4) 화면이 새로고침되면서 지금까지 쌓은 주문 목록이 전부 사라집니다. 타입 검사는 아무 말도 안 합니다.
01단원 개념01의 "타입이 못 잡는 것" 이 이런 것입니다. 폼의 기본 동작은 타입의 관심사가 아닙니다.

화면 상태 다루기 ★ 이 단원의 결론

보기: npm run dev → 왼쪽 목록에서 고르기

검사: npm run typecheck

05단원 개념04 섹션3에서 이런 이야기를 했습니다.

서버에서 데이터를 받아 오는 화면은 언제나 세 상태 중 하나다.
07단원에서 React 화면 상태를 이 패턴 그대로 다룬다.

그 단원입니다. 05단원에서 만든 LoadState 를 진짜 화면에 붙입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.tsx";

type Menu = { id: number; name: string; price: number };
```

흔히 하는 방식 — 그리고 그 문제

섹션 1

React자료에서 이렇게 배웠고, 실제로 이렇게 쓰는 코드가 많습니다.

```
function BadVersion() {
  const [loading, setLoading] = useState(false);
  const [menus, setMenus] = useState<Menu[]>([]);
  const [error, setError] = useState<string | null>(null);

  function load(shouldFail: boolean) {
    setLoading(true);
    setError(null);
    setTimeout(() => {
      setLoading(false);
      if (shouldFail) setError("서버가 응답하지 않습니다");
      else setMenus([
        { id: 1, name: "라떼", price: 4500 }
      ], 400);
    });
  }

  return (
    <div> <button type="button" onClick={() => load(false)}>
      성공하는 요청
    </button>{" "}
  )
}
```

```

실패하는 요청
    </button>
    {loading && <p>불러오는 중...</p>}
    {error !== null && <p style={{ color: "#c00" }}>오류: {error}</p>}
    {!loading && error === null && <p>{menus.length}개 메뉴</p>}
  </div>
);
}

```

돌아가기는 합니다. 그런데 문제가 셋 있습니다.

① 상태가 셋으로 흩어져 있어서 세 개를 매번 같이 맞춰야 합니다.

```
setError(null) 을 한 줄 빠뜨리면 실패했던 오류 메시지가 남습니다.
```

② 있을 수 없는 조합이 만들어집니다.

```
loading: true 이면서 error 가 있는 상태를 타입이 막지 않습니다.
```

③ 화면 조건이 지저분해집니다.

```
!loading && error === null 처럼 "아닌 것" 을 나열하게 됩니다.
상태가 하나 늘면 이 조건을 전부 다시 봐야 합니다.
```

▸ 직접 해보기

1

BadVersion 에서 setError(null) 한 줄을 지우고

실패 → 성공 순서로 눌러 보세요. 무엇이 이상한가요?

판별 유니온으로 바꾸기

섹션 2

05단원 개념04 그대로입니다. 상태를 '하나' 로 만듭니다.

```

type LoadState =
  | { status: "시작전" }
  | { status: "로딩중" }
  | { status: "성공"; data: Menu[] }
  | { status: "실패"; message: string };

function GoodVersion() {
  const [state, setState] = useState<LoadState>({ status: "시작전" });

  function load(shouldFail: boolean) {
    setState({ status: "로딩중" });
    setTimeout(() => {
      if (shouldFail) setState({ status: "실패", message: "서버가 응답하지 않습니다"

```



```

});
    else setState({ status: "성공", data: [{ id: 1, name: "라떼", price: 4500 }]
});
    }, 400);
  }

  return (
    <div> <button type="button" onClick={() => load(false)}>
      성공하는 요청
    </button>{" "}
    <button type="button" onClick={() => load(true)}>
      실패하는 요청
    </button> <View state={state} /> </div>
  );
}

```

화면 그리는 부분을 따로 뺐습니다. switch 하나로 끝냅니다.

```

function View({ state }: { state: LoadState }) {
  switch (state.status) {
    case "시작전":
      return <p>버튼을 눌러 보세요.</p>;
    case "로딩중":
      return <p>불러오는 중...</p>;
    case "실패":
      return <p style={{ color: "#c00" }}>오류: {state.message}</p>;
    case "성공":
      return (
        <ul>
          {state.data.map((m) => (
            <li key={m.id}>
              {m.name} - {m.price}원
            </li>
          ))}
        </ul>
      );
  }
}

```

case "성공" 안에서 state.data 를 확인 없이 바로 씁니다. "성공이면 data 가 반드시 있다" 가 타입에 적혀 있기 때문입니다.

반대로 없는 곳에서 쓰면 걸립니다.

이 코드는 검사에서 걸립니다

TS2339

```
function ViewWrong({ state }: { state: LoadState }) {  
  if (state.status === "로딩중") {  
    return <p>{state.data.length}</p>;  
  }  
  return null;  
}
```

Property 'data' does not exist on type '{ status: "로딩중"; }'.

로딩 중에는 데이터가 없습니다. 그걸 타입이 알고 막아 줍니다.

섹션 1의 방식이었다면 menus 가 그냥 빈 배열이라 조용히 0 이 나왔을 것입니다.

그리고 있을 수 없는 상태는 아예 못 만듭니다.

이 코드는 검사에서 걸립니다

TS2353

```
const 불가능: LoadState = { status: "성공", data: [], message: "실패" };
```

Object literal may only specify known properties, and 'message' does not exist in type '{ status: "성공"; data: Menu[]; }'.

성공이면서 오류 메시지가 있는 상태를 만들 수 없습니다.

"잘못된 상태를 표현할 수 없게 만든다" - 이 패턴의 핵심 이득입니다.

▸ 직접 해보기

2

GoodVersion 에서 같은 실수를 낼 수 있나요?

 1 처럼 '지우면 상태가 어긋나는 줄' 을 찾아보세요.

상태가 하나 늘면

섹션 3

LoadState 에 { status: "빈결과" } 를 추가한다고 해 봅시다. 그러면 View 의 switch 가 그 자리에서 걸립니다. "빈결과일 때 무엇을 그릴지 안 적었습니다" 라고요.

섹션 1의 방식이었다면 아무 일도 안 일어납니다. 조건 세 개를 손으로 다시 훑어야 하고, 빠뜨려도 아무도 안 알려 줍니다.

이게 판별 유니온을 쓰는 실질적인 이유입니다.

고쳐야 할 곳을 타입이 전부 찾아 준다.

▹ 직접 해보기 3

LoadState 에 { status: "빈결과" } 를 추가해 보세요.

어디가 걸리나요? 확인한 뒤 되돌리세요.

props 로 상태를 넘길 때

섹션 4

View 의 props 를 { state: LoadState } 라고 그 자리에 적었습니다. 짧으면 이렇게 써도 되고, 길어지면 04단원처럼 이름을 붙입니다.

```
type ViewProps = { state: LoadState };
function View({ state }: ViewProps) { ... }
```

어느 쪽이든 됩니다. 이 자료는 한 줄이면 그 자리에, 길면 이름을 붙입니다.

▹ 직접 해보기 4

View 의 매개변수를 { state: LoadState } 대신

type ViewProps 로 빼 보세요. 동작이 달라지나요?

자주 하는 실수

섹션 5

이런 실수를 합니다

status 를 string 으로 적기

05단원 개념04 섹션5입니다. 리터럴이 아니면 좁히기가 안 됩니다. useState<LoadState> 를 적어 두면 이 실수가 안 생깁니다.

이런 실수를 합니다

useState 에 <LoadState> 를 안 적기

{ status: "시작전" } 만 보고 그 모양으로만 잡습니다. 그러면 setState({ status: "로딩중" }) 이 걸립니다. 개념02 섹션4의 리터럴 유니온 문제와 같습니다.

이런 실수를 합니다

상태를 여러 개로 흩뿌리기

섹션 1입니다. 돌아가지만 조합이 어긋나기 시작하면 원인을 못 찾습니다.

이런 실수를 합니다

switch 에 default 를 넣어 뭉뚱그리기

편하지만, 상태를 추가했을 때 알려 주는 기능을 잃습니다. 05단원 개념02  2 에서 확인한 그대로입니다.

화면

정리

- 1 서버를 부르는 화면은 언제나 몇 가지 상태 중 하나다.
- 2 상태를 여러 useState 로 흩뿌리면 있을 수 없는 조합이 만들어진다.
- 3 판별 유니온으로 하나로 묶으면 그런 조합을 아예 못 만든다.
- 4 `useState({ status: \"시작전\" })` 처럼 타입을 꼭 적는다.
- 5 화면은 switch (state.status) 하나로 끝난다. '아닌 것' 을 나열하지 않는다.
- 6 case 안에서는 그 상태로 좁혀져서 data 를 확인 없이 쓸 수 있다.
- 7 상태를 하나 추가하면 고쳐야 할 곳을 타입이 전부 찾아 준다.

```
export default function Page() {
  return (
    <div> <h2>개념 04 - 화면 상태 다루기</h2> <p>섹션 1 - 흩어진 상태(권하지 않음)
    </p> <BadVersion /> <p>섹션 2 - 판별 유니온</p> <GoodVersion /> </div>
  );
}
```

직접 해보기 정답

- 1 실패한 뒤 성공을 누르면, 성공했는데도 빨간 오류 문구가 남아 있습니다. 상태 세 개를 매번 같이 맞춰줘야 하는데 한 줄을 빠뜨렸기 때문입니다. 타입은 아무 말도 안 합니다. 세 상태가 서로 모순이어도 각각은 올바르니까요.
- 2 낼 수 없습니다. setState 를 한 번 부르면 상태가 통째로 바뀌기 때문에 '옛 오류가 남는' 일이 구조적으로 안 생깁니다. → 실수를 조심해서 막는 것이 아니라, 실수할 수 없게 만든 것입니다.
- 3 View 의 switch 가 걸립니다. error TS2366: Function lacks ending `return` statement and `return` type does not include 'undefined'. "빈결과일 때 무엇을 그릴지 안 적었습니다" 라는 뜻입니다. BadVersion 쪽이었다면 아무 일도 안 일어났을 것입니다. 재현: type LoadState =

```
| { status: "로딩중" }
| { status: "성공" }
| { status: "빈결과" };
```

```
function View({ state }: { state: LoadState }): string {
```

```
  switch (state.status) {
    case "로딩중": return "a";
    case "성공": return "b";
  }
```

```
}
void View;
```

4) 달라지지 않습니다. 완전히 같습니다. 한 줄이면 그 자리에 적고, 길어지거나 여러 번 쓰면 이름을 붙입니다. 04단원 개념02 섹션5의 기준이 그대로입니다.

주문 화면

보기: npm run dev → 왼쪽 목록에서 고르기

검사: npm run check

사용 단원: 07단원 전체 (props · useState · 이벤트 · 판별 유니온)

앞의 종합01~03에서 만든 것을 화면에 올립니다.

★ 규칙 — any · ! · as 를 쓰지 않습니다.

이 단원은 자동 채점이 없습니다. 두 가지로 확인하세요. ① npm run check 가 조용해지는가 ② '기대 화면' 대로 나오는가 (npm run dev 로 직접 눌러 보기)

★ check 가 잡아 주는 것은 문제 1·2·6 뿐입니다. 문제 3·4·5 는 안 풀어도 조용합니다. 반드시 화면으로 판정하세요. (문제 4 는 문제 5 에서 setCart 를 부르기 시작하면 그때 걸립니다)

막히면 종합04_주문_화면_정답.tsx 를 보세요.

```
import { useState } from "react";
import Summary from "../_ui/Summary.tsx";

type Menu = { id: number; name: string; price: number };

const MENUS: Menu[] = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "카페라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
];
```

서버라고 치는 함수입니다. 400ms 뒤에 JSON 문자열을 돌려줍니다. 총액이 20000원 이상이면 서버가 이상한 것을 보내는 상황을 흉내 냅니다. (new Promise 는 안 배웠습니다. 그냥 쓰세요 — await 하면 문자열이 나옵니다)

```
function 주문전송(총액: number): Promise<string> {
  const 응답 =
    총액 >= 20000
      ? "<html>502 Bad Gateway</html>"
      : `{"ok":true,"orderId":${1000 + (총액 % 1000)}}`;
  return new Promise((resolve) => setTimeout(() => resolve(응답), 400));
}
```

```

type Status = "대기" | "조리중" | "완료";
const STATUSES: Status[] = ["대기", "조리중", "완료"];

function statusText(s: Status): string {
  switch (s) {
    case "대기":
      return "곧 시작합니다";
    case "조리중":
      return "만들고 있어요";
    case "완료":
      return "나왔습니다";
  }
}

```

문제 1 props 에 타입 붙이기 (개념01)

MenuItem 은 메뉴 하나와 '담기 버튼을 눌렀을 때 부를 함수' 를 받습니다. type MenuItemProps 를 만들어 붙이세요.

기대 출력 메뉴 세 줄에 [담기] 버튼

type MenuItemProps 를 만들고 붙이세요

```

function MenuItem({ menu, onAdd }) {
  return (
    <li>
      {menu.name} - {menu.price}원{" "}
      <button type="button" onClick={() => onAdd(menu)}>
        담기
      </button> </li>
    );
}

```

문제 2 children 받기 (개념01 섹션4)

Panel 은 제목과 '태그 사이의 내용' 을 받습니다. children 을 받도록 고치세요.

기대 출력 제목이 붙은 상자 안에 내용이 들어감

children 을 받도록 고치세요

```
type PanelProps = { title: string };
function Panel({ title }: PanelProps) {
  return (
    <div style={{ border: "1px solid #ddd", borderRadius: 8, padding: 12,
marginBottom: 12 }}> <h3 style={{ margin: "0 0 8px" }}>{title}</h3> </div>
  );
}
```

문제 3 화면 상태 (개념04)

주문 상태를 판별 유니온으로 만들고, switch 로 그리는 OrderView 를 만드세요.

"시작전" → 담고 주문 버튼을 눌러 보세요. "보내는중" → 보내는 중...

"성공"	→ 주문 완료 - N원	(total 을 가집니다)
"실패"	→ 오류: 메시지	(message 를 가집니다)

기대 출력 주문 버튼을 누르면 "보내는 중..." 이 잠깐 뜨고 "주문 완료 - N원"

type OrderState 와 OrderView 를 만드세요

정리

- 1 props 에 타입을 붙이는 것이 이 단원의 절반이다.
- 2 useState 는 null · 빈 배열 · 리터럴 유니온 이 셋만 적으면 된다.
- 3 이벤트 타입은 인라인으로 써 본 뒤 마우스를 올려 베킨다.
- 4 화면 상태는 판별 유니온 하나로 묶는다.
- 5 any · ! · as 를 쓰지 않는다.


```
export default function Page() {
```

문제 4 useState 세 가지 (개념02)

아래 셋에 타입을 정해 주세요. 07단원에서 배운 '적어야 하는 세 경우'가 전부 나옵니다. cart : 빈 배열로 시작 → 지금은 never[] 라 담기가 안 됩니다 selected : null 로 시작 → 지금은 null 전용 상자입니다 status : "대기" 로 시작 → 지금은 string 이라 오타가 안 걸립니다

기대 출력 담기를 누르면 목록이 늘고, "마지막으로 고른 것" 이 바뀜

세 줄에 타입을 정해 주세요

```
const [cart, setCart] = useState([]);
const [selected, setSelected] = useState(null);
const [status, setStatus] = useState("대기");

const [memo, setMemo] = useState("");

const total = 0; // ← 문제 5에서 고칩니다
```

문제 5 장바구니 담기 (개념02 · React자료 07단원 불변 갱신)

① addToCart 를 완성하세요.

- 이미 담긴 메뉴면 count 를 1 늘립니다
- 아니면 { menu, count: 1 } 을 새로 담습니다
- 담을 때 selected 도 갱신합니다

② 위 total 을 '장바구니 합계' 로 고치세요 (reduce 를 쓰면 짧습니다)

★ cart.push(...) 는 화면을 안 바꿉니다. 새 배열을 만들어 setCart 하세요.

기대 출력 아메리카노를 두 번 담으면 '담긴 종류' 는 1가지 그대로인데
합계만 8000원으로 늘니다. (2가지가 되면 새로 담은 것이라 틀린 것입니다)

addToCart 를 완성하고 total 을 고치세요

```
function addToCart(menu: Menu) {  
    console.log(menu.name);  
}
```

문제 6 진짜 서버처럼 (개념03 · 06단원 async · 종합03 확인)

이 실습의 마지막이자, 09단원 전체가 만나는 자리입니다.

① handleSubmit 의 매개변수에 타입을 붙이고, 새로고침을 막으세요. ② 장바구니가 비었으면 실패 상태로 두고 끝냅니다. ③ 아니면 "보내는중" 으로 바꾼 뒤 await 주문전송(total) 로 응답을 받습니다. (handleSubmit 앞에 async 를 붙여야 await 를 쓸 수 있습니다 — 06단원 개념03) ④ ★ 받은 것은 그냥 문자열입니다. 종합03에서 한 대로 확인하고 쓰세요.

- JSON.parse 를 try / catch 로 감쌉니다. 실패하면 실패 상태로.
- unknown 으로 받아 ok 가 true 인지 확인합니다. 아니면 실패 상태로.
- 통과하면 { status: "성공", total } 입니다.

⑤ handleMemo 의 매개변수에도 타입을 붙이세요.

★ 총액 20000원이 넘으면 서버가 JSON 이 아닌 것을 보냅니다(위 주문전송 참고). try 를 빠뜨리면 그 순간 화면이 통째로 멈춥니다. 직접 빼 보고 확인하세요.

기대 출력 주문하면 "보내는 중..." 이 잠깐 뜨고 "주문 완료 - N원".
빈 장바구니면 오류 문구. 20000원 넘게 담고 주문하면
화면이 안 터지고 "오류: 서버 응답을 알아볼 수 없습니다" 가 뜸.

두 핸들러에 타입을 붙이고 내용을 채우세요

```

function handleSubmit(e) {
  console.log(e);
}

function handleMemo(e) {
  setMemo(e.target.value);
}

return (
  <div>
    <h2>종합 04 – 주문 화면</h2> <Panel title="메뉴"> <ul>
      {MENUS.map((m) => (
        <MenuItem key={m.id} menu={m} onAdd={addToCart} />
      ))}
    </ul> <p>마지막으로 고른 것: {selected?.name ?? "없음"}
  </p> </Panel> <Panel title="장바구니"> <p>담긴 종류: {cart.length}가지</p> <p>합계:
  {total}원</p> <button type="button" onClick={() => setCart([])}>
    비우기
  </button> </Panel> <Panel title="주문"> <form onSubmit=
  {handleSubmit}> <input value={memo} onChange={handleMemo} placeholder="메모 (선택)"
  />{" "}
    <button type="submit">주문하기</button> </form> <p>메모: {memo.trim() ===
    "" ? "(없음)" : memo}</p>
    { /* TODO: 문제 3에서 만든 OrderView 를 여기에 넣으세요 */ }
  </Panel>

  <Panel title="상태">
    {STATUSES.map((s) => (
      <button key={s} type="button" onClick={() => setStatus(s)} style={{
marginRight: 6 }}>
        {s}
      </button>
    ))}
    <p>{statusText(status)}</p> </Panel>

  </div>
);
}

```

React 와 타입스크립트

보기: npm run dev → 왼쪽 목록에서 “연습문제” 고르기

검사: npm run check ← 연습문제 전용 검사입니다

★ 이 단원은 브라우저라 자동 채점이 없습니다. 화면으로 판정하세요. (01~06·08단원은 npm run grade 로 출력을 대조합니다)

npm run check(타입 검사)가 10문항 중 8개를 잡아 줍니다.

나머지 문제 5 와 10 은 검사가 못 잡습니다. ‘기대 화면’ 을 눈으로 확인하세요. ★ 문제 10 은 [도전] 인데도 검사에 안 걸립니다. check 가 조용해도 다 푼 게 아닙니다. 문제 2·3·9 는 만드는 쪽이 아니라 ‘쓰는 쪽’(아래 Page 화면)에서 걸립니다. 걸리는 것과 별개로 동작(기본값·children·onStep)은 화면으로 봐야 합니다. (npm run typecheck 는 개념·정답 파일만 봅니다. 그쪽은 언제나 조용합니다)

- 1 TODO 자리에 코드를 씁니다.
- 2 화면이 ‘기대 화면’ 처럼 나오는지 봅니다.
- 3 npm run check 로 타입도 봅니다.
- 4 10분 고민해도 안 되면 연습문제_정답.tsx 를 보세요.

문제 1~6은 기본, 7~9는 응용, 10은 [도전]입니다.

```
import { useState } from "react";
import Summary from "../_ui/Summary.tsx";
```

문제 1 (개념01 섹션1)

아래 컴포넌트의 props 에 타입을 붙이세요.

기대 출력 아메리카노 – 4000원

type MenuItemProps 를 만들고 붙이세요

```
function MenuItem({ name, price }) {
  return (
    <li>
      {name} - {price}원
    </li>
  );
}
```

문제 2 (개념01 섹션3)

color 를 '없어도 되는' props 로 만들고, 없으면 "#555" 가 되게 하세요.

기대 출력 회색 배지 하나, 파란 배지 하나

color 를 선택 props 로 바꾸고 기본값을 주세요

```
type BadgeProps = { text: string; color: string };
function Badge({ text, color }: BadgeProps) {
  return (
    <span style={{ background: color, color: "#fff", padding: "2px 8px",
borderRadius: 4 }}>
      {text}
    </span>
  );
}
```

문제 3 (개념01 섹션4)

태그 사이의 내용을 받을 수 있게 children 을 추가하세요.

기대 출력 제목이 붙은 상자 안에 글이 들어감

children 을 받도록 고치세요

```

type BoxProps = { title: string };
function Box({ title }: BoxProps) {
  return (
    <div style={{ border: "1px solid #ddd", borderRadius: 8, padding: 12,
marginBottom: 12 }}> <h3 style={{ margin: "0 0 8px" }}>{title}</h3> </div>
  );
}

```

문제 4 (개념02 섹션3)

아래 useState 는 never[] 로 잡혀서 담기가 안 됩니다. 고치세요.

기대 출력 담기를 누르면 목록이 늘어남

```

type Menu = { id: number; name: string };

function Cart() {

```

타입을 정해 주세요

```

const [items, setItems] = useState([]);

return (
  <div> <button type="button" onClick={() => setItems([...items, { id: items.length
+ 1, name: "라떼" }])}>
    담기 ({items.length})
  </button> <ul>
    {items.map((m) => (
      <li key={m.id}>{m.name}</li>
    ))}
  </ul> </div>
);
}

```

문제 5 (개념02 섹션2)

고른 메뉴를 담은 상태를 만드세요. 처음에는 아무것도 안 골랐습니다. 안 골랐으면 "없음" 이 나와야 합니다.

기대 출력 없음 → (버튼) → 라떼

```
function Selected() {
```

`null` 로 시작하는 상태를 만드세요

```
    return (
      <div> <button type="button">라떼 고르기</button> <p>고른 것: 없음</p> </div>
    );
  }
}
```

문제 6 (개념03 섹션3)

입력창 핸들러를 함수 밖으로 뺐습니다. 매개변수에 타입을 붙이세요.

기대 출력 입력할수록 글자 수가 늘어남

```
function TextInput() {
  const [text, setText] = useState("");
```

`e` 에 타입을 붙이세요

```
function handleChange(e) {
  setText(e.target.value);
}

return (
  <div> <input value={text} onChange={handleChange} placeholder="메뉴 이름" /> <p>
    {text.length}자</p> </div>
);
}
```

문제 7 (응용 · 개념02 섹션4)

status 가 정해진 세 값만 되도록 고치세요. (지금은 string 이라 "취소" 같은 오타가 안 걸립니다)

기대 출력 버튼 세 개, 누르면 아래 글자가 바뀜

```
type Status = "대기" | "조리중" | "완료";
const STATUSES: Status[] = ["대기", "조리중", "완료"];

function statusLabel(s: Status): string {
  return s === "완료" ? "다 됐습니다" : s;
}

function StatusPicker() {
```

타입을 정해 주세요 (지금은 아래 statusLabel(status) 가 걸립니다)

```
const [status, setStatus] = useState("대기");

return (
  <div>
    {STATUSES.map((s) => (
      <button key={s} type="button" onClick={() => setStatus(s)} style={{
marginRight: 6 }}>
        {s}
      </button>
    ))}
    <p>현재: {statusLabel(status)}</p> </div>
  );
}
```

문제 8 (응용 · 개념03 섹션4)

폼 핸들러에 타입을 붙이고, 새로고침이 안 되게 막으세요.

기대 출력 입력 후 주문을 누르면 목록에 쌓임 (화면이 새로고침되면 안 됨)


```
function OrderForm() {
  const [menu, setMenu] = useState("");
  const [orders, setOrders] = useState<string[]>([]);
```

e 에 타입을 붙이고 기본 동작을 막으세요

```
function handleSubmit(e) {
  if (menu.trim() === "") return;
  setOrders([...orders, menu]);
  setMenu("");
}

return (
  <form onSubmit={handleSubmit}> <input value={menu} onChange={(e) =>
setMenu(e.target.value)} placeholder="주문할 메뉴" />{" "}
    <button type="submit">주문</button> <ul>
      {orders.map((o, i) => (
        <li key={i}>{o}</li>
      ))}
    </ul> </form>
  );
}
```

문제 9 (응용 · 개념01 섹션5)

버튼을 눌렀을 때 부모에게 값을 넘겨 주는 props 를 추가하세요.

기대 출력 누르면 아래에 "마지막 값: N" 이 갱신됨

```
type StepperProps = { label: string };

function Stepper({ label }: StepperProps) {
  const [n, setN] = useState(0);

  function handleClick() {
    const next = n + 1;
    setN(next);
```

부모에게 next 를 넘기세요

```
}

return (
  <button type="button" onClick={handleClick}>
    {label}: {n}
  </button>
);
}
```

문제 10 ([도전] · 개념04)

아래 흩어진 상태를 판별 유니온 하나로 바꾸세요. ① type LoadState 를 만든다 ("시작전" / "로딩중" / "성공"(data) / "실패"(message)) ② useState<LoadState> 하나만 쓴다 ③ 화면은 switch 하나로 그린다

기대 출력 버튼을 누르면 로딩중 → 결과

```
function Loader() {
```

아래 세 상태를 하나로 합치세요

```
const [loading, setLoading] = useState(false);
const [data, setData] = useState<string[]>([]);
const [error, setError] = useState<string | null>(null);

function load(shouldFail: boolean) {
  setLoading(true);
  setError(null);
  setTimeout(() => {
    setLoading(false);
    if (shouldFail) setError("서버가 응답하지 않습니다");
  });
}
```

```

    }, 400);
  }

  return (
    <div> <button type="button" onClick={() => load(false)}>
      성공
    </button>{" "}
    <button type="button" onClick={() => load(true)}>
      실패
    </button>
    {loading && <p>불러오는 중...</p>}
    {error !== null && <p style={{ color: "#c00" }}>오류: {error}</p>}
    {!loading && error === null && <p>{data.length}개</p>}
    </div>
  );
}

```

화면 —————

정리 —————

- 1 props 에 타입을 붙이는 것이 이 단원의 절반이다.
- 2 useState 는 null · 빈 배열 · 리터럴 유니온 이 셋만 적으면 된다.
- 3 이벤트 타입은 외우지 말고 인라인으로 써 본 뒤 베킨다.
- 4 화면 상태는 판별 유니온 하나로 묶는다.

```

export default function Page() {
  const [last, setLast] = useState(0);

  return (
    <div>
      <h2>07단원 연습문제</h2> <p>문제 1</p> <ul> <MenuItem name="아메리카노" price=
{4000} /> </ul> <p>문제 2</p> <Badge text="기본색"
/> <Badge text="지정색" color="#2d6cdf" /> <p>문제 3</p> <Box title="상자 제목">
여기가 children 자리입니다.</Box> <p>문제 4</p> <Cart /> <p>문제 5</p> <Selected
/> <p>문제 6</p> <TextInput /> <p>문제 7</p> <StatusPicker /> <p>문제
8</p> <OrderForm /> <p>문제 9</p> <Stepper label="더하기" onStep={setLast} /> <p>
마지막 값: {last}</p>
    </div>
  );
}

```

```
<p>문제 10</p>  
<Loader />
```

```
</div>  
);  
}
```

App.tsx

예제 고르기 화면 — 07단원의 예제를 왼쪽 목록에서 골라 봅니다

이 파일도 자료가 미리 만들어 둔 '틀'입니다. 고치지 않아도 됩니다.

단원 폴더에 .tsx 파일을 새로 만들면 왼쪽 목록에 저절로 나타납니다.

(React자료 실습프로젝트와 같은 구조입니다. 타입만 붙었습니다)

```
import { Suspense, lazy, useState, useMemo } from "react";
import type { ComponentType } from "react";
import ErrorBox from "../_ui/ErrorBox.tsx";
```

단원 폴더 안의 모든 예제 파일을 자동으로 찾아 옵니다.

```
const modules = import.meta.glob<{ default: ComponentType }>(
  "./[0-9][0-9]_*/*.tsx",
);

type Item = {
  path: string;
  unit: string;
  unitLabel: string;
  title: string;
};
```

"./07_React와_타입스크립트/개념01_props에_타입_붙이기.tsx"

```
→ { unit: "07_React와_타입스크립트", title: "개념01 props에 타입 붙이기" }
```

```
function parse(path: string): Item {
  const parts = path.split("/");
  const unit = parts[1] ?? "";
  const file = parts[2] ?? "";
  return {
    path,
    unit,
    unitLabel: unit.replace(/_/g, " "),
    title: file.replace(/\.tsx$/, "").replace(/_/g, " "),
  };
}
```

```

const items: Item[] = Object.keys(modules)
  .map(parse)
  .sort((a, b) => a.path.localeCompare(b.path, "ko"));

type Unit = { unit: string; label: string; items: Item[] };

const units: Unit[] = [];
for (const item of items) {
  const found = units.find((u) => u.unit === item.unit);
  if (found) found.items.push(item);
  else units.push({ unit: item.unit, label: item.unitLabel, items: [item] });
}

export default function App() {
  const [current, setCurrent] = useState<string | null>(items[0]?.path ?? null);

```

고른 예제만 그때그때 불러옵니다. 한 파일이 망가져도 나머지는 볼 수 있습니다.

```

const Current = useMemo(() => {
  if (current === null) return null;
  const loader = modules[current];
  if (loader === undefined) return null;
  return lazy(loader);
}, [current]);

return (
  <div className="layout">
    <nav className="sidebar">
      <h1>TS자료 07단원</h1>
      {units.length === 0 && (
        <p className="empty">
          아직 예제가 없습니다.
          <br />
          src 안에 단원 폴더를 만들어 보세요.
        </p>
      )}
      {units.map((unit) => (
        <section key={unit.unit}>
          <h2>{unit.label}</h2>
          <ul>
            {unit.items.map((item) => (
              <li key={item.path}>
                <button
                  type="button"
                  className={item.path === current ? "navBtn on" : "navBtn"}
                  onClick={() => setCurrent(item.path)}
                >
                  {item.title}
                </button>
              </li>
            )}}
          </ul>
        </section>
      )}}
    </nav>
  </div>
)

```

```

    </nav>

    <main className="stage">
      {Current !== null && (
        <ErrorBox key={current}> <Suspense fallback={<p>불러오는 중...</p>}>
          <Current /> </Suspense> </ErrorBox>
        )}
    </main>
  </div>
);
}

```


main.tsx

실습 프로젝트 시작점

이 파일은 자료가 미리 만들어 둔 '틀'입니다. 고치지 않아도 됩니다.

React자료의 main.jsx 와 딱 두 가지가 다릅니다.

① 확장자가 .tsx

② `querySelector` 결과가 `null` 일 수 있어서 확인이 필요함 (아래 참고)

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.tsx";
import "./index.css";
```

`document.querySelector` 의 반환 타입은 `Element | null` 입니다. `index.html` 에 `#root` 가 분명히 있지만, 타입스크립트는 HTML 을 안 읽습니다. 그래서 "없을 수도 있다" 고 보고 확인을 요구합니다.

React자료에서는 이 줄을 그냥 썼습니다. JS 였으니까요.

```
createRoot(document.querySelector("#root")).render(...)
```

여기서는 ! 를 쓰지 않고 이렇게 처리합니다(05단원 개념03 섹션5).

```
const root = document.querySelector("#root");

if (root === null) {
  throw new Error("index.html 에 <div id=\"root\"> 가 없습니다.");
}

createRoot(root).render(
  <StrictMode> <App /> </StrictMode>,
);
```


기존 프로젝트에 타입 입히기

OPEN 08_기존_프로젝트에_타입_입히기

```
const menu = raw as Menu;  
console.log(menu.name);  
console.log(menu.price);  
} catch (e) {
```

08

01 as 를 왜 쓰지 않나

02 기존 JS 프로젝트 옮기기

03 tsconfig 읽기 · 그리고 다음 단계

04 타입이 없는 패키지 만나기

– 연습문제

as 를 왜 쓰지 않나

RUN node 개념01_as를_왜_쓰지_않나.ts

검사: npm run typecheck

이 자료가 쓰지 말라고 정한 것이 셋입니다.

```
any    (02단원 개념03)
!      (05단원 개념03 섹션5)
as     ← 마지막 하나입니다
```

셋 다 “검사를 끄는 도구” 라는 공통점이 있습니다. as 는 남의 코드에서 가장 자주 보게 되므로, 읽을 줄은 알아야 합니다.

as 는 “내가 안다고 우기는 것” 이다

섹션 1

```
type Menu = { name: string; price: number };
```

04단원 개념04에서 본 그 상황입니다. 밖에서 온 값은 타입을 모릅니다.

```
const raw: unknown = JSON.parse('{ "name": "라떼" }');
```

unknown 이라 그냥은 못 씁니다(02단원 개념03 섹션5).

이 코드는 검사에서 걸립니다

TS18046

```
console.log(raw.name);
```

'raw' is of type 'unknown'.

as 를 붙이면 “이건 Menu 다” 라고 우기는 것이 됩니다. 그러면 통과합니다.

```
const menu = raw as Menu;
console.log(menu.name);
```

출력 라떼

문제는 여기부터입니다. 저 JSON 에는 price 가 없습니다.

```
console.log(menu.price);
```

출력 undefined

타입에는 price: number 라고 적혀 있는데 실제로는 없습니다.

```
try {
  console.log(menu.price.toFixed(0));
} catch (e) {
  console.log("터졌습니다:", String(e));
}
```

출력 터졌습니다: TypeError: Cannot read properties of undefined (reading 'toFixed')

검사는 조용했고, 실행하니 터졌습니다. 02단원의 any, 05단원의 ! 와 완전히 같은 구조입니다.

as 는 값을 바꾸지 않습니다. 타입스크립트의 생각만 바꿉니다.

▹ 직접 해보기 1

menu 를 만들 때 as Menu 를 지워 보세요.

무슨 에러가 나나요?

그래도 as 가 막아 주는 것이 조금은 있다

섹션 2

아무 데나 아무거나 우길 수는 없습니다.

이 코드는 검사에서 걸립니다

TS2352

```
const wrong = "4500" as number;
```

Conversion of type 'string' to type 'number' may be a mistake because neither type sufficiently overlaps with the other. If this was intentional, convert the expression to 'unknown' first.

“문자열과 숫자는 너무 달라서 실수 같습니다” 라고 말합니다.

그런데 메시지 뒷부분을 보세요.

“정말 그럴 생각이면 unknown 을 거쳐 가세요” 라고 방법까지 알려 줍니다.

그 방법을 쓰면 정말로 통과합니다.

```
const forced = "4500" as unknown as number;
console.log(typeof forced);
```

출력 string

number 라고 우겼는데 실제로는 문자열입니다. 아무도 안 막았습니다.

as unknown as ... 를 보면 “여기서 검사를 완전히 꺾다” 는 뜻입니다. 남의 코드에서 이걸 만나면 그 근처를 의심하세요.

▫ 직접 해보기

2

forced * 2 와 forced + 2 를 각각 출력해 보세요.

왜 다르게 나오나요?

as 대신 무엇을 쓰나

섹션 3

대신 ① · 확인하고 쓰기 — 05단원에서 배운 그대로

```
function readMenu(text: string): Menu | null {  
  const data: unknown = JSON.parse(text);
```

조건이 여섯 줄인 이유 — 앞에서부터 하나씩 좁혀 나가기 때문입니다. ① 객체인가 ② null 이 아닌가 (typeof null 은 “object” 라서 따로 봅니다) ③④ 그 이름이 있는가 ⑤⑥ 그 이름의 값이 제 종류인가 ③ 과 ⑤ 를 둘 다 보는 이유는, 이름이 있어도 값이 숫자일 수 있기 때문입니다.

```
if (  
  typeof data === "object" &&  
  data !== null &&  
  "name" in data &&  
  "price" in data &&  
  typeof data.name === "string" &&  
  typeof data.price === "number"  
) {  
  return { name: data.name, price: data.price };  
}  
return null;  
}  
  
console.log(readMenu('{ "name": "라떼", "price": 4500 }'));
```

출력 { name: '라떼', price: 4500 }

```
console.log(readMenu('{ "name": "라떼" }'));
```

출력 null

길긴 합니다. 그래서 실무에서는 이 일을 도구에 맡깁니다(개념03에서 소개). 중요한 것은 “as 로 우기는 것과 달리, 이건 진짜로 확인한다” 는 점입니다.

대신 ② · satisfies — “이 모양이 맞는지 검사만 해 줘”

```
const 확인된메뉴 = { name: "아메리카노", price: 4000 } satisfies Menu;

console.log(확인된메뉴.name);
```

출력 아메리카노

as 와 satisfies 의 차이가 핵심입니다.

as Menu	→ "Menu 라고 쳐 줘" (검사 안 함, 타입이 Menu 로 바뀜)
satisfies Menu	→ "Menu 가 맞는지 봐 줘" (검사 함, 타입은 그대로)

그래서 satisfies 는 틀리면 걸립니다.

이 코드는 검사에서 걸립니다

TS2741

```
const 모자란메뉴 = { name: "아메리카노" } satisfies Menu;
```

Property 'price' is missing in type '{ name: string; }' but required in type 'Menu'.

as 였다면 조용히 통과했을 자리입니다.

그리고 ‘확인’은 받되 원래 타입은 그대로 남는다’는 것도 이득입니다. : Menu 라고 적으면 그 변수는 그때부터 Menu 로만 보입니다. satisfies 는 Menu 에 맞는지 확인만 하고, 변수 자신은 적은 그대로 남습니다.

여기 Menu 는 name 이 string 이라 둘의 결과가 같습니다. 차이가 드러나는 것은 목표 타입이 더 험령할 때입니다.

```
type 설정 = { 모드: string; 값: string | number[] };

const a: 설정 = { 모드: "빠름", 값: [1, 2] };
const b      = { 모드: "빠름", 값: [1, 2] } satisfies 설정;

a.값.push(3); // TS2339 - a.값 은 string | number[] 라 push 가 없다
b.값.push(3); // 통과 - b.값 은 number[] 그대로
```

확인’은 둘 다 받았는데, satisfies 쪽만 원래 타입이 남아 있습니다.

▹ 직접 해보기 3

```
{ name: "라떼", price: 4500, size: "L" } 를
```

satisfies Menu 로 써 보세요. 걸리나요?

아주 드물게, 사람이 알고 타입스크립트가 모르는 경우가 있습니다. 예를 들어 값 목록을 그 자리에서 바로 늘어놓을 때입니다.

```
(["대기", "조리중", "완료"] as Status[]).map(...)
```

배열 리터럴만 보면 string[] 로 추론되는데, 우리는 Status[] 인 것을 압니다. 이럴 때는 as 가 실용적으로 보입니다. 값이 바로 옆에 눈으로 보이니까요.

판단 기준은 이것입니다.

값이 눈앞에 있어서 내가 확인할 수 있다	→ as 를 써도 위험이 작다
값이 밖에서 온다(서버·파일·입력)	→ as 를 쓰면 안 된다

사고는 전부 아래쪽에서 납니다.

그런데 위 경우도 as 없이 됩니다. 그리고 이쪽이 더 좋습니다.

```
type Status = "대기" | "조리중" | "완료";
const statuses: Status[] = ["대기", "조리중", "완료"];
console.log(statuses.length);
```

출력 3

이러면 목록에 오타가 있을 때 그 자리에서 걸립니다. as 로는 안 걸립니다. 07단원 실습프로젝트가 이 형태 (STATUSES)를 쓰고 있습니다. 열어서 확인해 보세요.

그래서 이 자료는 as 를 해법으로 쓴 적이 한 번도 없습니다. (이 단원에 나오는 as 는 전부 “이렇게 하지 말라” 고 보여 주는 자리입니다)

▫ 직접 해보기 4

```
const wrong = ["대기", "취소"] as Status[]; 와
```

```
const wrong2: Status[] = ["대기", "취소"]; 를 각각 써 보세요. 어느 쪽이 걸리나요?
```

자주 하는 실수

이런 실수를 합니다

에러가 나면 as 를 붙여 없애기

가장 흔합니다. 에러가 사라진 게 아니라 검사가 꺼진 것입니다. any · ! · as 셋 다 같습니다.

이런 실수를 합니다**as 가 값을 바꾼다고 생각하기**

안 바꿉니다. "4500" as number 를 해도 실행할 때는 여전히 문자열입니다. 값을 바꾸려면 Number("4500") 처럼 진짜 함수를 써야 합니다.

이런 실수를 합니다**as 와 satisfies 를 같은 것으로 알기**

반대입니다. as 는 검사를 끄고, satisfies 는 검사만 합니다.

이런 실수를 합니다**as unknown as ... 를 아무렇게 쓰지 않게 쓰기**

타입스크립트가 "이건 실수 같다" 고 말린 것을 억지로 뚫은 것입니다. 그 자리는 거의 항상 설계가 잘못된 것입니다.

이런 실수를 합니다**as const 를 '금지된 as' 로 알기**

이름만 같고 다른 것입니다. 남의 코드에서 자주 보게 되니 구별해 두세요.

```
const 상태들 = ["대기", "조리중", "완료"] as const;
```

as 는 "이 타입이라고 우겨라" 인데, as const 는 "이 값들을 그대로 굳혀라" 입니다. 검사를 끄는 게 아니라 오히려 더 조입니다 — 위 배열은 읽기 전용이 되어 상태들.push("취소") 가 TS2339 로 막힙니다. 이 자료는 as const 를 쓰지 않지만, 봤을 때 "아 저건 다른 거구나" 만 알면 됩니다.

정리

- 1 as 는 값을 바꾸지 않는다. 타입스크립트의 생각만 바꾼다.
- 2 우긴 것이 사실이 아니면 검사는 통과하고 실행할 때 터진다. any · ! 와 완전히 같은 구조다.
- 3 너무 동떨어진 타입으로 우기면 TS2352 로 말린다. as unknown as ... 는 그것마저 뚫은 것이니 남의 코드에서 보면 의심하라.
- 4 대신 쓸 것 — ① 확인하고 쓰기(05단원) ② satisfies
- 5 satisfies 는 "맞는지 봐 줘" 다. 틀리면 걸리고, 타입은 그대로 남는다.
- 6 값이 눈앞에 있으면 as 도 위험이 작다. 밖에서 온 값에는 절대 쓰지 않는다.

직접 해보기 정답

1) error TS18046: 'raw' is of type 'unknown'. unknown 은 확인하기 전에는 아무것도 못 하게 막습니다(02단원 개념03 섹션5). as 는 그 확인을 건너뛰겠다는 뜻입니다. 재현:

```
const raw: unknown = JSON.parse('{ "name": "라떼" }');
console.log(raw.name);
```

```
2) console.log(forced * 2);           // 출력: 9000
   console.log(forced + 2);           // 출력: 45002
```

실제 값이 문자열 "4500" 이라 곱하기는 우연히 맞고 더하기는 이어붙입니다. 01단원 개념01의 "+ 만 특별하다" 가 마지막 단원에서 또 나옵니다. number 라고 우겼으니 타입스크립트는 둘 다 숫자라고 믿고 있습니다.

3) 걸립니다. error TS2353: Object literal may only specify known properties, and 'size' does not exist in type 'Menu'. satisfies 는 초과 속성 검사까지 그대로 합니다(04단원 개념01 섹션3). as Menu 였다면 조용히 통과했을 것입니다. 재현:

```
type Menu = { name: string; price: number };
const m = { name: "라떼", price: 4500, size: "L" } satisfies Menu;
void m;
```

4) 두 번째만 걸립니다.

```
const wrong2: Status[] = ["대기", "취소"];
```

→ error TS2322: Type '"취소"' is not assignable to type 'Status'. as 는 우기는 것이라 오타를 못 잡고, : 타입 은 검사라서 잡습니다. 재현:

```
type Status = "대기" | "조리중" | "완료";
const wrong2: Status[] = ["대기", "취소"];
```

같은 일을 하는 두 방법 중 하나는 검사를 하고 하나는 안 합니다. 되도록 : 타입 쪽을 쓰세요.

기존 JS 프로젝트 옮기기

```
RUN node 개념02_기존_JS_옮기기.ts
```

검사: npm run typecheck

회사에 이미 자바스크립트로 만들어 둔 것이 수천 줄 있습니다. 하루아침에 다 고칠 수는 없습니다.

다행히 안 고쳐도 됩니다. 한 파일씩 옮기면 됩니다. 이 파일은 그 순서입니다.

세 단계로 나눈다

섹션 1

이 단원 폴더 안에 옮기기_예제/ 가 있습니다. 직접 따라 해 보세요.

이전.js	옮기기 전의 코드 (그냥 자바스크립트)
이후.ts	같은 코드를 옮긴 것

1단계 · 확장자는 그대로 두고 검사만 켜 본다

```
npx tsc --noEmit -p 08_기존_프로젝트에_타입_입히기/옮기기_예제
```

이렇게 나옵니다.

```
이전.js(14,19): error TS7006: Parameter 'items' implicitly has an 'any' type.
이전.js(22,23): error TS7006: Parameter 'total' implicitly has an 'any' type.
이전.js(22,30): error TS7006: Parameter 'shipping' implicitly has an 'any' type.
이전.js(26,19): error TS7006: Parameter 'item' implicitly has an 'any' type.
```

파일 이름을 하나도 안 바꿨는데 “여기 타입이 없다” 는 목록이 나왔습니다. 이게 앞으로 할 일의 목록입니다. 분량을 미리 가늠할 수 있습니다.

컨 것은 tsconfig 두 줄뿐입니다.

```
"allowJs": true,   .js 도 봐 준다
"checkJs": true,  .js 도 검사한다
```

2단계 · 파일 하나를 .ts 로 바꾸고 타입을 적는다

이전.js → 이후.ts 가 그것입니다. 달라진 것은 딱 둘입니다. 확장자와, 매개변수 타입.

그랬더니 1단계에서 안 보이던 진짜 버그가 드러났습니다.

```
withShipping(getTotal(cart), shippingText)
→ TS2345 Argument of type 'string' is not assignable to parameter of type 'number'.
```

이전.js 를 실행하면 이렇게 나옵니다.

총액: 125003000원

15,500원이어야 하는데 배송비가 이어붙었습니다. 에러는 안 났습니다. 1단계(checkJs)에서는 이게 안 잡혔습니다. 매개변수가 전부 any 였으니까요.

→ 그래서 1단계만으로는 부족합니다. 타입을 적어야 잡힙니다.

3단계 · 나머지 파일도 하나씩 반복한다

전부 옮기고 나면 checkJs 를 끄고 allowJs 도 끕니다. 그때 끝납니다.

▹ 직접 해보기 1

위 두 명령을 실제로 쳐 보세요.

node 로 이전.js 를 돌린 결과와, tsc 로 검사한 결과를 비교해 보세요.

어느 파일부터 옮기나

섹션 2

순서에는 요령이 있습니다.

- ① 아무것도 **import** 하지 않는 파일부터 (계산 함수, 상수 모음, 유틸)
- ② 그다음 그것을 쓰는 파일
- ③ 화면(컴포넌트)은 마지막

아래에서 위로 올라가는 순서입니다. 왜냐하면 — A 가 B 를 쓰는데 B 에 타입이 없으면, A 를 옮겨도 B 쪽이 any 라서 소득이 적습니다. 반대로 B 를 먼저 옮기면 A 는 아직 .js 여도 그 타입의 덕을 봅니다.

```
const 옮기는순서 = ["상수·유틸", "계산 함수", "데이터 다루는 곳", "화면"];
console.log(옮기는순서.join(" → "));
```

출력 상수·유틸 → 계산 함수 → 데이터 다루는 곳 → 화면

▹ 직접 해보기 2

옮기는순서 배열의 두 번째 값을 출력해 보세요.

중간에 막히면 — any 를 '빔' 으로 쓴다

섹션 3

02단원에서 any 를 쓰지 않기로 했습니다. 그 규칙에 딱 하나 예외가 여기입니다.

옮기다 보면 이런 자리가 나옵니다.

· 남이 만든 오래된 라이브러리라 타입이 아예 없다 · 지금 제대로 적으려면 다른 파일 열 개를 같이 고쳐야 한다

이럴 때는 일단 넘어가고 표시를 남깁니다.

```
type Legacy = {
```

TODO(타입): 결제 모듈 옮길 때 제대로 적을 것 — 2026-09 목표

```
payment: unknown;
  name: string;
};

const legacy: Legacy = { payment: {}, name: "옛날 주문" };
console.log(legacy.name);
```

```
출력      옛날 주문
```

any 대신 unknown 을 쓴 것에 주목하세요.

any	→	아무도 안 막아 준다. 그 값을 쓰는 코드 전체가 검사 밖으로 나간다
unknown	→	확인하기 전에는 못 쓰게 막는다. 빔이 번지지 않는다

그래서 “일단 넘어갈 때” 도 unknown 이 낫습니다. 정말 any 를 써야 한다면 반드시 주석으로 이유와 기한을 남기세요.

```
// eslint-disable-next-line 같은 표시가 없으면
6개월 뒤에 아무도 그게 임시였다는 것을 모릅니다.
```

◁ 직접 해보기 3

legacy.payment 를 그냥 출력해 보세요. 되나요?

console.log(legacy.payment.amount) 는 어떨까요?

옮기는 중에 자주 만나는 것

섹션 4

만나는 것 1 · 배열에서 꺼낸 값이 `undefined` 일 수 있다

```
const cart = [{ name: "라떼", price: 4500 }];
const first = cart[0];
```

기본 설정에서는 `cart[0]` 이 그냥 `{ name, price }` 로 나옵니다. 빈 배열이면 `undefined` 인데요. 이걸 타입스크립트가 봐주는 부분입니다.

```
console.log(first.name);
```

출력 라떼

실무에서는 이걸 안 봐주게 커는 설정이 있습니다(개념03에서 봅니다). 커면 아래처럼 확인해야 합니다.

```
const safe = cart[0];
console.log(safe === undefined ? "비었음" : safe.name);
```

출력 라떼

만나는 것 2 · 객체에 나중에 속성을 붙이던 코드

JS 에서는 이렇게 쓰던 코드가 흔합니다.

```
const o = {};
o.name = "라떼";
```

이 코드는 검사에서 걸립니다

TS2339

```
const empty = {};
empty.name = "라떼";
```

Property 'name' does not exist on type '{}'.

처음 만들 때 없던 속성은 나중에 못 붙입니다.

한 번에 다 적거나, 타입을 미리 정해 두어야 합니다.

```
const o: { name?: string } = {};
o.name = "라떼";
console.log(o.name);
```

출력 라떼

만나는 것 3 · 함수가 여러 모양으로 쓰이던 것

JS에서는 같은 함수에 문자열도 넣고 배열도 넣던 코드가 있습니다. 05단원의 유니온과 좁히기로 정리하면 됩니다.

```
function toList(value: string | string[]): string[] {
  return Array.isArray(value) ? value : [value];
}
console.log(toList("라떼"), toList(["라떼", "아메리카노"]));
```

출력 ['라떼'] ['라떼', '아메리카노']

▸ 직접 해보기 4

toList에 숫자를 넘겨 보세요. 무슨 에러가 나나요?

자주 하는 실수

섹션 5

이런 실수를 합니다

한꺼번에 다 바꾸려 하기

수천 줄을 하루에 옮기면 에러가 수백 개 나오고 어디서부터 볼지 알 수 없게 됩니다. 한 파일씩, 아래에서 위로.

이런 실수를 합니다

1단계(checkJs)만 하고 끝났다고 생각하기

섹션 1에서 봤듯이 진짜 버그는 타입을 적어야 드러납니다.

이런 실수를 합니다

막힐 때마다 any를 쓰고 표시를 안 남기기

6개월 뒤에는 그게 임시였다는 것을 아무도 모릅니다. unknown을 쓰고, 정말 any를 썼다면 이유와 기한을 적으세요.

이런 실수를 합니다

화면(컴포넌트)부터 옮기기

가장 많이 얽혀 있는 곳이라 제일 어렵습니다. 마지막에 하세요.

정리

- 1 한꺼번에 안 바뀌도 된다. allowJs · checkJs 로 검사만 먼저 켜다.
- 2 1단계는 '할 일 목록' 을 만들어 준다(대부분 TS7006).
- 3 진짜 버그는 2단계(.ts 로 바꾸고 타입 적기)에서 드러난다.
- 4 순서는 아래에서 위로 — 유틸 → 계산 → 데이터 → 화면.
- 5 막히면 any 말고 unknown 으로 넘어가고, 표시를 남긴다.
- 6 다 옮기면 checkJs · allowJs 를 끈다. 그때가 끝이다.

직접 해보기 정답

1) node 로 돌리면 "총액: 125003000원" 이 나옵니다. 에러는 없습니다. tsc 로 검사하면 TS7006 이 네 개 나옵니다. 총액 문제는 안 잡힙니다. → "돌아간다" 와 "맞다" 가 다르다는 것, 그리고

1단계만으로는 부족하다는 것을 한 번에 보여 주는 예입니다.

```
2 console.log(옮기는순서[1]);           // 출력: 계산 함수

3 console.log(legacy.payment); 는 됩니다. // 출력: {}
  console.log(legacy.payment.amount); 는 걸립니다.
```

error TS18046: 'legacy.payment' is of type 'unknown'. 재현:

```
type Legacy = { payment: unknown; name: string };
const legacy: Legacy = { payment: {}, name: "옛날 주문" };
console.log(legacy.payment.amount);
```

→ unknown 은 '담고 옮기는 것' 은 되고 '파고드는 것' 만 막습니다.

그래서 빛이 번지지 않습니다. any 였다면 둘 다 통과했을 것입니다.

4) error TS2345: Argument of type 'number' is not assignable to parameter of type 'string | string[]'. 재현:

```
function toList(value: string | string[]): string[] {

return Array.isArray(value) ? value : [value];

}
console.log(toList(3));
```


유니온에 없는 종류라 걸립니다.

tsconfig 읽기 · 그리고 다음 단계

RUN node 개념03_tsconfig와_다음_단계.ts

검사: npm run typecheck

01단원에서 “tsconfig.json 은 고치지 마세요” 라고 했습니다. 이제 마지막 단원이니 그 안에 뭐가 있는지 봅니다.

그리고 이 자료가 끝난 뒤 무엇을 더 보면 되는지로 마무리합니다.

이 자료의 tsconfig

섹션 1

TS자료 폴더의 tsconfig.json 을 열어 보세요. 이렇게 되어 있습니다.

"target": "ES2022"	어느 시대의 자바스크립트로 볼 것인가
"lib": ["ES2022"]	쓸 수 있는 내장 기능 목록
"module": "NodeNext"	import/export 를 어떤 방식으로 볼 것인가
"types": ["node"]	console 같은 Node 기능을 알게 해 줌
"strict": true	★ 이것이 핵심입니다
"noEmit": true	파일은 만들지 말고 검사만
"erasableSyntaxOnly": true	node 가 지우지 못하는 문법을 미리 막음

07단원의 실습프로젝트는 이 둘이 핵심적으로 다릅니다.

"jsx": "react-jsx"	.tsx 안의 태그를 이해하게 함
"lib": [..., "DOM"]	브라우저라서 document.window 를 앞

나머지 차이는 ‘무엇으로 돌리느냐’ 때문에 따라오는 것들입니다. node 로 돌리는 쪽은 module/types 가 node 쪽이고, Vite(번들러)로 돌리는 쪽은 bundler 쪽입니다. 세어 보면 열 곳쯤 됩니다.

node 쪽	Vite 쪽
module: NodeNext	module: ESNext
moduleResolution: nodenext	moduleResolution: bundler
types: ["node"]	types: ["vite/client"]
erasableSyntaxOnly: true	(없음 – 번들러가 처리하므로 필요 없음)

중요한 것은 strict · noEmit · target 이 양쪽 다 같다는 점입니다. 그 셋이 ‘타입을 어떻게 볼 것인가’를 정하고, 나머지는 ‘어디서 돌리는가’ 일 뿐입니다.

```
const 핵심설정 = ["strict", "noEmit", "target", "lib"];
console.log(핵심설정.length + "개만 알면 됩니다");
```

출력 4개만 알면 됩니다

⇒ 직접 해보기 1

TS자료의 tsconfig.json 을 열어

07단원 실습프로젝트의 것과 나란히 놓고 비교해 보세요.

strict 가 켜는 것

섹션 2

strict: true 는 스위치 여러 개를 한 번에 켜는 묶음입니다. 그중 이 자료에서 실제로 켜는 것이 둘입니다.

noImplicitAny · 매개변수에 타입을 안 적으면 걸린다

```
function f(x) { } → TS7006 Parameter 'x' implicitly has an 'any' type.
```

03단원부터 계속 만난 그 에러입니다. 이걸 끄면 x 가 조용히 any 가 됩니다. 02단원에서 본 그 위험 그대로입니다.

strictNullChecks · null 과 undefined 를 다른 값으로 본다

```
const s: string = null; → TS2322
v.length (v 가 없을 수 있음) → TS18047 / TS18048
```

05단원 전체가 이 설정 덕분에 의미가 있습니다. 이걸 끄면 모든 타입에 null 이 몰래 들어갈 수 있게 되어, "Cannot read properties of null" 을 타입스크립트가 못 막습니다.

그래서 규칙은 하나입니다.

★ strict 는 끄지 않는다.

인터넷에 “에러가 많이 나면 strict 를 끄세요” 라는 글이 있습니다. 그건 타입스크립트를 쓰는 이유를 없애는 것입니다. 에러가 많으면 08단원 개념02처럼 한 파일씩 옮기세요.

⇒ 직접 해보기 2

```
const s: string = null;
```

 을 써 보세요.

무슨 에러가 나나요? 이 에러가 안 난다면 strict 가 꺼진 것입니다.

strict 에 안 들어 있지만 실무에서 자주 켜는 것이 있습니다.

```
"noUncheckedIndexedAccess": true
```

배열에서 꺼낸 값에 `undefined` 를 붙여 줍니다.

```
const menus = ["아메리카노", "라떼"];
const tenth = menus[10];
```

지금 설정에서는 tenth 가 string 입니다. 실제로는 `undefined` 인데요.

```
console.log(tenth === undefined ? "없음" : tenth);
```

```
출력      없음
```

이 설정을 켜면 tenth 가 string | `undefined` 가 되어, 확인 없이 `tenth.length` 를 쓰면 TS18048 로 막힙니다.

직접 확인해 보려면 이렇게 쳐 보세요.

```
npx tsc --noEmit --strict --noUncheckedIndexedAccess ^
  --target ES2022 --lib ES2022 --module nodenext --moduleResolution nodenext ^
  --types node --ignoreConfig 08_기존_프로젝트에_타입_입히기/개념03_tsconfig와_다음_
  단계.ts
```

이 자료는 이 설정을 끈 채로 두었습니다. 초보자에게는 배열을 쓸 때마다 확인이 붙어서 부담이 크기 때문입니다. 다만 06단원 개념03  와 08단원 개념02에서 ? 를 붙여 둔 이유가 이것입니다. 습관을 들여 두면 나중에 이 설정을 켤 때 고칠 것이 적습니다.

▫ 직접 해보기

3

`menus[10].length` 를 지금 설정에서 써 보세요. 걸리나요?

실행하면 어떻게 되나요?

실무에서 같이 쓰는 것

zod · 밖에서 온 값을 진짜로 확인해 주는 도구

04단원 개념04에서 “타입은 약속이지 검사가 아니다” 라고 했습니다. 그래서 손으로 확인하는 코드를 썼는데, 길었습니다. zod 는 그 일을 대신해 줍니다. 모양을 한 번 적으면 ‘검사’ 와 ‘타입’ 을 둘 다 만들어 줍니다.

```
const Menu = z.object({ name: z.string(), price: z.number() });
const menu = Menu.parse(await res.json()); // 틀리면 여기서 에러를 던짐
```

서버 데이터를 다루기 시작하면 반드시 만나게 됩니다.

ESLint · 규칙을 자동으로 지켜 주는 도구

“any 를 쓰지 마세요” 같은 이 자료의 규칙을 기계가 검사하게 할 수 있습니다. 사람이 리뷰에서 매번 지적하지 않아도 됩니다.

유틸리티 타입 · Partial · Pick · Omit · Record

이미 있는 타입을 조금 바꿔서 새 타입을 만드는 도구입니다.

```
Partial<Menu>      모든 속성을 선택으로
Pick<Menu, "name"> name 만 뽑아서
Omit<Menu, "price"> price 만 빼고
```

이 자료에서 일부러 썼습니다. 지금 외워도 쓸 자리가 없어서 잊어버립니다. “같은 타입을 조금씩 바꿔 여러 개 만들고 있다” 는 생각이 들 때 찾아보세요.

```
const 다음에볼것 = ["zod", "ESLint", "유틸리티 타입"];
console.log(다음에볼것.join(", "));
```

```
출력      zod, ESLint, 유틸리티 타입
```

▸ 직접 해보기

4

04단원 개념04 섹션4의 readMenu 를 다시 보세요.

zod 가 대신해 주는 일이 무엇인지 말해 보세요.

이 자료에서 일부러 안 다룬 것

섹션 5

다음 것들은 이 자료에 없습니다. 없다는 것을 아는 것도 중요합니다.

- 조건부 타입 (`T extends U ? A : B`)
- 매핑된 타입 (`{ [K in keyof T]: ... }`)
- 데코레이터
- enum (node 가 못 지우는 문법이라 이 자료에서는 아예 막아 두었습니다)
- 네임스페이스 (옛날 방식입니다. 쓸 일 없습니다)

앞의 둘은 ‘타입 체조’ 라고 부르는 영역입니다. 라이브러리를 만드는 사람에게서는 필요하지만, 화면과 기능을 만드는 일에는 거의 안 씁니다. 필요해지는 날이 오면 그때 배워도 늦지 않습니다.

enum 대신 무엇을 쓰냐면, 이미 배운 것을 씁니다.

```
type Status = "대기" | "조리중" | "완료";
const statuses: Status[] = ["대기", "조리중", "완료"];
console.log(statuses[0]);
```

출력 대기

05단원의 리터럴 유니온이 enum 이 하려던 일을 거의 다 합니다. 게다가 실행할 때 아무것도 안 남기고 사라집니다. enum 은 코드를 남깁니다.

👉 직접 해보기 5

enum Color { Red } 를 써 보세요.

무슨 에러가 나나요? (힌트: erasableSyntaxOnly)

자주 하는 실수

섹션 6

이런 실수를 합니다

에러가 많다고 strict 를 끄기

타입스크립트를 쓰는 이유를 없애는 것입니다. 개념02처럼 한 파일씩 옮기세요.

이런 실수를 합니다

tsconfig 를 인터넷에서 통째로 복사해 오기

무슨 설정인지 모르는 줄이 스무 개쯤 붙어 옵니다. 섹션 1의 일곱 줄에서 시작해서 필요할 때 하나씩 더하세요.

이런 실수를 합니다

배열에서 꺼낸 값을 확인 없이 쓰기

기본 설정에서는 안 걸립니다. 그래서 더 위험합니다. 빈 배열일 수 있는 자리에는 ?. 를 붙이는 습관을 들이세요.

이런 실수를 합니다

유틸리티 타입이나 타입 체조를 먼저 공부하기

쓸 자리가 생기기 전에 외우면 그냥 잊어버립니다. 필요해지는 순간이 오면 그때 찾아보세요.

정리하며

섹션 7

(🖋️ 없음: 자료 전체를 마무리하는 섹션입니다) 이 자료가 정한 규칙 셋을 다시 확인하고 끝냅니다.

any 를 쓰지 않는다 (02단원 개념03)
! 를 쓰지 않는다 (05단원 개념03)
as 를 쓰지 않는다 (08단원 개념01)

셋 다 “검사를 끄는 도구”입니다. 셋 다 “검사는 통과하는데 돌리면 터진다” 를 만듭니다.

타입 에러가 났을 때 이 셋 중 하나가 떠오르면, 그건 대개 아직 안 배운 것이 있다는 신호였습니다. 이제는 다 배웠습니다.

모양을 못 적겠다 → 04단원
 확인을 못 하겠다 → 05단원
 타입마다 함수를 또 만들고 있다 → 06단원

```
const 규칙 = ["any 금지", "! 금지", "as 금지"];
console.log("규칙 " + 규칙.length + "개: " + 규칙.join(" / "));
```

출력 규칙 3개: any 금지 / ! 금지 / as 금지

정리

- 1 tsconfig 에서 실제로 중요한 것은 strict 하나다.
- 2 strict 는 noImplicitAny 와 strictNullChecks 를 켜다. 이 둘이 이 자료의 03단원과 05단원을 의미 있게 만든다.
- 3 에러가 많다고 strict 를 끄지 않는다. 한 파일씩 옮긴다.
- 4 noUncheckedIndexedAccess 는 배열 꺼내기까지 확인하게 한다. 이 자료는 켜지만, ? 를 붙이는 습관을 들여 두면 나중에 편하다.
- 5 다음에 볼 것 — zod(밖에서 온 값 확인) · ESLint(규칙 자동 검사) · 유틸리티 타입.
- 6 타입 체조와 enum 은 안 배워도 된다. 리터럴 유니온이 enum 을 대신한다.

직접 해보기 정답

1) 핵심적으로 다른 것은 jsx 와 lib 둘입니다. 그 밖에 module·moduleResolution·types·erasableSyntaxOnly 등이 다르고, 다 세면 열 곳쯤 됩니다. 다만 그 열 곳은 전부 ‘무엇으로 돌리는가’(node vs Vite) 때문에 따라오는 것입니다.

★ 중요한 것은 strict · noEmit · target 이 양쪽 다 같다는 점입니다. ‘타입을 어떻게 볼 것인가’ 는 안 바뀝니다. “React 를 하면 타입 설정이 복잡해진다” 는 오해를 깨는 것이 이 실습의 목적입니다.

2) error TS2322: Type ‘null’ is not assignable to type ‘string’. strictNullChecks 가 켜져 있어서 납니다. 재현:

```
const s: string = null;
```

이게 안 난다면 strict 가 꺼진 것이고, 그러면 05단원 내용이 통째로 무의미해집니다.

3) 걸리지 않습니다. 검사는 조용합니다. 실행하면 터집니다. `TypeError: Cannot read properties of undefined (reading 'length')` → 이 자료가 못 잡는 몇 안 되는 자리입니다.

`noUncheckedIndexedAccess` 를 켜면 잡힙니다.

4 readMenu 가 손으로 하던 일 — `typeof` 로 하나씩 확인하고, 맞으면 그 모양의 값을 돌려주고, 아니면 `null` 을 돌려주는 것. zod 는 모양을 한 번만 적으면 그 확인 코드를 대신 만들어 줍니다. 게다가 그 모양에서 타입까지 뽑아 줘서 `type` 을 따로 안 적어도 됩니다.

5 error TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled. node 가 .ts 를 실행할 때 enum 은 '지우기' 만으로 처리가 안 됩니다. 재현:

```
enum Color { Red }  
void Color;
```

그대로 두면 실행할 때 `ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX` 로 터지므로, 검사 단계에서 미리 막아 두었습니다.

타입이 없는 패키지 만나기

RUN node 개념04_타입이_없는_패키지.ts

검사: npm run typecheck

개념02에서 '우리 파일' 을 옮겼습니다. 그런데 실제 프로젝트에는 우리가 안 만든 것이 훨씬 많습니다. npm 으로 받은 패키지들입니다.

우리 파일은 고치면 됩니다. 남의 패키지는 못 고칩니다.

그 패키지에 타입이 없으면 어떻게 하는지가 이 파일의 내용입니다. 옮기기를 시작하면 거의 반드시 만나는 상황입니다.

```
import { getDiscounted, getShipping, DEFAULT_PERCENT } from "./타입없는_모듈/
할인계산기.js";
```

타입이 없는 모듈을 부르면

섹션 1

이 단원 폴더 안에 타입없는_모듈/ 이 있습니다. 두 파일이 들어 있습니다.

할인계산기.js 그냥 자바스크립트입니다. '남의 패키지' 라고 생각하세요.
할인계산기.d.ts 거기에 손으로 붙인 타입입니다.

지금은 .d.ts 가 있어서 잘 돌아갑니다.

코드	출력
console.log(getDiscounted(4500, DEFAULT_PERCENT));	4050
console.log(getShipping(12500));	3000
console.log(getShipping(30000));	0

타입도 제대로 붙어 있습니다.

이 코드는 검사에서 걸립니다

TS2345

```
console.log(getDiscounted("4500", DEFAULT_PERCENT));
```

Argument of type 'string' is not assignable to parameter of type 'number'.

화면 입력창에서 온 값을 그대로 넘긴 상황입니다.

개념02 섹션4에서 본 그 함정인데, 남의 패키지에서도 똑같이 잡힙니다.

그런데 .d.ts 가 없으면 어떻게 될까요? 할인계산기.d.ts 의 이름을 잠깐 바꾸고 npm run typecheck 를 해 보세요.

```
TS7016 Could not find a declaration file for module './타입없는_모듈/할인계산기.js'.  
... implicitly has an 'any' type.
```

“이 모듈의 모양을 적어 둔 파일을 못 찾겠다. 그래서 any 가 된다” 는 뜻입니다. any 를 막아 둔 자료라 여기서 걸립니다.

이 에러를 만나면 순서대로 세 가지를 해 봅니다. 섹션 2부터 하나씩 봅니다.

▣ 직접 해보기 1

할인계산기.d.ts 의 이름을 할인계산기.txt 로 바꾸고

npm run typecheck 를 해 보세요. 확인했으면 이름을 되돌리세요.

① 패키지가 타입을 갖고 있는지 본다

섹션 2

요즘 패키지는 대개 타입을 같이 넣어 보냅니다. 그런 패키지는 설치만 하면 아무것도 안 해도 됩니다.

구별하는 법은 간단합니다. npm 페이지에서 패키지 이름 옆을 보세요.

TS 라고 적힌 파란 딱지가 있으면	→ 타입이 같이 옵니다. 끝.
DT 라고 적힌 딱지가 있으면	→ @types 를 따로 받아야 합니다(섹션 3).
아무것도 없으면	→ 직접 적어야 합니다(섹션 4).

zod(개념03 섹션4에서 본 것)는 앞쪽입니다. 설치하면 타입이 딸려 옵니다.

▣ 직접 해보기 2

<<https://www.npmjs.com/package/zod>> 를 열어

이름 옆에 어떤 딱지가 있는지 보세요.

② @types 를 받는다

섹션 3

옛날에 만들어진 패키지에는 타입이 없습니다. 그런 것들은 뜻있는 사람들이 타입만 따로 만들어 올려 두었습니다. 그게 @types 입니다.

```
npm i -D @types/패키지이름
```

사실 이 자료도 이미 쓰고 있습니다. package.json 을 열어 보세요.

```
"devDependencies": { "@types/node": "...", "typescript": "..." }
```

node 는 자바스크립트 실행기라 그 자체에는 타입이 없습니다. @types/node 가 있어서 console.log 나 process 에 타입이 붙는 것입니다.

-D 는 --save-dev 의 줄임입니다. '만들 때만 쓰고 배포에는 안 들어간다' 는 뜻입니다. 타입은 실행할 때 사라지니 당연히 개발용입니다.

```
console.log(typeof console.log);
```

출력 function

▸ 직접 해보기

3

package.json 을 열어 @types/node 가 어디에 적혀 있는지

찾아보세요. dependencies 인가요, devDependencies 인가요?

③ 그래도 없으면 내가 적는다 ★

섹션 4

@types 에도 없는 패키지가 있습니다. 회사 안에서만 쓰는 옛날 모듈이 특히 그렇습니다. 그럴 때 .d.ts 파일을 직접 만듭니다.

타입없는_모듈/할인계산기.d.ts 를 열어 보세요. 전부입니다.

```
export declare function getDiscounted(price: number, percent: number): number;
export declare function getShipping(total: number): number;
export declare const DEFAULT_PERCENT: number;
```

declare 는 "코드는 저쪽에 있고, 여기엔 모양만 적는다" 는 뜻입니다. 그래서 함수 몸통 { } 이 없습니다. 있으면 안 됩니다.

.d.ts 파일에는 실행되는 코드가 한 줄도 없습니다. 검사가 끝나면 통째로 사라집니다.

이름 규칙은 하나뿐입니다.

할인계산기.js → 같은 자리에 할인계산기.d.ts

확장자 앞 이름이 같으면 타입스크립트가 알아서 짝지어 줍니다.

```
const 상품합계 = 4000 * 2 + 4500;
console.log(상품합계);
```

출력 12500

```
const 할인된값 = getDiscounted(상품합계, DEFAULT_PERCENT);
console.log(할인된값);
```

출력 11250

```
console.log(할인된값 + getShipping(할인된값));
```

출력 14250

남의 .js 인데도 계산이 전부 숫자로 이어집니다. .d.ts 하나 덕분입니다.

▸ 직접 해보기

4

할인계산기.d.ts 에 export declare function 세금(total: number): number;

을 한 줄 더하고, 개념04 에서 세금(1000) 을 불러 보세요.

npm run typecheck 는 통과하나요? node 로 실행하면 어떻게 되나요?

선언 파일은 '약속' 이라 틀려도 조용하다

섹션 5

04단원 개념04 섹션3의 그 이야기가 여기서 제일 무섭게 나옵니다.

타입은 약속이지 검사가 아니다.

.d.ts 는 내가 손으로 적는 약속입니다. 아무도 맞는지 확인해 주지 않습니다. 실제 .js 와 다르게 적어도 타입스크립트는 조용합니다.

예를 들어 할인계산기.d.ts 에 이렇게 적었다고 해 봅시다.

```
export declare function getShipping(total: number): string;
                    ^^^^^^ 실제로는 number
```

그러면 이런 코드가 검사를 통과합니다.

```
getShipping(12500).toUpperCase();
```

검사는 통과하는데 실행하면 터집니다. 숫자 3000 에는 toUpperCase 가 없으니까요.

```
TypeError: getShipping(...).toUpperCase is not a function
```

그래서 .d.ts 를 적을 때는 실제 .js 를 열어 보고 적어야 합니다. 짐작으로 적으면 안 됩니다. 짐작한 타입은 as 와 다를 바가 없습니다(개념01).

▣ 직접 해보기 5

실제로 해 보세요. 할인계산기.d.ts 의 getShipping 반환을

string 으로 바꾸고 npm run typecheck 를 해 보세요. 조용한가요? 확인했으면 number 로 되돌리세요.

as any 로 덮지 않기

섹션 6

TS7016 을 없애는 가장 빠른 방법은 이것입니다.

```
const 계산기 = require("할인계산기") as any;
```

한 줄로 조용해집니다. 그리고 그 순간 그 모듈에서 오는 모든 값이 any 가 됩니다. 개념01에서 as 를 안 쓰는 이유로 든 것이 정확히 이 상황입니다.

급하면 어떻게 하나면, 개념02 섹션3의 '빋' 방식을 씁니다. 덮지 말고, 좁게 적고, 자국을 남깁니다.

```
// TODO(타입빋): 할인계산기 타입을 제대로 적을 것. 지금은 쓰는 것만 적어 둬.
export declare function getShipping(total: number): number;
```

as any 와 뭐가 다르냐면, 범위가 다릅니다. as any 는 모듈 전체를 포기하는 것이고, 위처럼 적으면 '적어 둔 함수 하나' 만 약속하고 나머지는 아예 못 씁니다. 못 쓰는 편이 낫습니다. 쓰려고 하면 에러가 나서 그때 적으면 되니까요.

▣ 직접 해보기 6

할인계산기.d.ts 에서 DEFAULT_PERCENT 줄만 지우고

```
npm run typecheck 를 해 보세요. 무슨 에러가 나나요?
```

"안 적으면 못 쓴다" 가 왜 안전한지 생각해 보세요.

어디까지 적어야 하나

섹션 7

남의 패키지 전체를 적을 필요는 없습니다. 그건 며칠짜리 일이 됩니다.

지금 쓰는 것만 적는다. 나중에 더 쓰게 되면 그때 한 줄 더한다.

할인계산기.js 에는 함수가 둘, 상수가 하나 있습니다. 셋 다 쓰니까 셋 다 적었습니다. 만약 getShipping 을 안 쓴다면 안 적어도 됩니다.

판단 기준은 이렇습니다.

· 우리 코드가 부르는 것 → 적는다 · 그 함수가 돌려주는 값의 모양 → 적는다 (여기를 대충 적으면 섹션 5 사고가 납니다) · 패키지 안에서만 쓰는 것 → 안 적는다

그리고 옮기기가 끝난 뒤에 @types 가 생겼는지 가끔 확인해 보세요. 생겼으면 손으로 적은 .d.ts 를 지우고 그쪽으로 갈아타는 편이 낫습니다. 남이 관리해 주는 쪽이 언제나 더 정확합니다.

▫ 직접 해보기

7

할인계산기.js 를 열어 보세요.

.d.ts 에 안 적힌 것이 있나요? 없다면 왜 셋 다 적었을까요?

node_modules 안의 패키지는 옆에 못 놓는다

섹션 8

섹션 4의 방법에는 조건이 하나 붙어 있었습니다. 할인계산기.js 가 '내 폴더 안' 에 있어서 옆에 .d.ts 를 놓을 수 있었던 것입니다.

그런데 진짜 npm 패키지는 node_modules 안에 있습니다. 거기에 파일을 놓으면 · npm install 한 번에 지워지고 · git 에도 안 올라갑니다(node_modules 는 보통 제외합니다)

그래서 남의 패키지에는 섹션 4의 방법을 못 씁니다. declare module 을 씁니다. "이 이름으로 import 되는 것의 모양은 이렇다" 를 내 폴더에서 선언하는 것입니다.

```
// types/이상한계산기.d.ts ← 파일 이름·위치는 자유. 내 프로젝트 안이면 됩니다
declare module "이상한계산기" {
  export function 더하기(a: number, b: number): number;
  export const 버전: string;
}
```

이 파일 하나만 있으면 아래가 통과합니다.

```
import { 더하기 } from "이상한계산기";
console.log(더하기(1, 2).toFixed(0));
```

섹션 1에서 본 TS7016 메시지가 사실 이 방법까지 같이 알려 주고 있었습니다. 뒷부분을 다시 읽어 보세요 — "or add a new declaration (.d.ts) file containing declare module '...';" 가 바로 이것입니다.

정리하면 갈림길은 하나입니다.

내 폴더 안의 .js	→	옆에 같은 이름의 .d.ts	(섹션 4)
node_modules 의 패키지	→	declare module "이름"	(여기)

★ 섹션 5의 위험이 여기에도 그대로 있습니다. 오히려 더 큼니다. 내 .js 는 열어서 확인이라도 하지만, 남의 패키지는 그러지 않게 되기 때문입니다. declare module 은 '임시 처방' 입니다. @types 가 생기면 갈아타세요.

☞ 직접 해보기 8

위 예제에서 `export const` 버전: `string`; 을

`export const` 버전: `number`; 로 바꿨다고 생각해 보세요.

버전.toFixed(1) 은 검사를 통과할까요? 실제 패키지가 "1.0" 을 준다면요?

자주 하는 실수

섹션 9

이런 실수를 합니다

.d.ts 에 실행되는 코드를 적기

선언 파일에는 모양만 적습니다. 함수 몸통 `{ }` 을 적으면 안 됩니다. `declare` 를 붙였는데 몸통을 적으면 바로 걸립니다.

이런 실수를 합니다

파일 이름을 다르게 짓기

할인계산기.js 의 짝은 할인계산기.d.ts 입니다. 계산기.d.ts 라고 지으면 아무 일도 안 일어납니다. 조용히 TS7016 이 계속 납니다.

이런 실수를 합니다

실제 .js 를 안 보고 짐작으로 적기

섹션 5입니다. 틀려도 조용하기 때문에 제일 위험합니다. 반드시 .js 를 열어서 무엇을 돌려주는지 확인하고 적으세요.

이런 실수를 합니다

as any 로 덮고 넘어가기

섹션 6입니다. 한 줄로 조용해지지만 그 모듈 전체를 포기하는 것입니다.

이런 실수를 합니다

@types 를 dependencies 에 넣기

타입은 실행할 때 사라집니다. -D 를 붙여 devDependencies 에 넣으세요.

정리

- 1 타입이 없는 모듈을 부르면 TS7016 이 난다. "모양을 적어 둔 파일이 없다" 는 뜻이다.
- 2 순서는 셋이다. ① 패키지에 타입이 있나 ② @types 가 있나 ③ 없으면 내가 적는다.
- 3 @types 는 npm i -D @types/이름 으로 받는다. 이 자료의 @types/node 가 그것이다.
- 4 직접 적을 때는 .d.ts 파일을 만든다. 짝은 파일 이름으로 맞춘다. 할인계산기.js 의 짝은 할인계산기.d.ts 다.
- 5 declare 는 "코드는 저쪽, 모양만 여기" 다. 함수 몸통을 적지 않는다.
- 6 .d.ts 는 내가 적는 약속이라 틀려도 조용하다. 실제 .js 를 보고 적어야 한다. 짐작으로 적은 타입은 as 와 다를 바가 없다.
- 7 전부 적지 않는다. 지금 쓰는 것만 적고, 나중에 필요하면 한 줄 더한다.
- 8 나중에 @types 가 생기면 손으로 적은 것을 지우고 갈아탄다.

직접 해보기 정답

- 1 이름을 바꾸면 짝을 못 찾아서 이렇게 납니다. error TS7016: Could not find a declaration file for module './타입없는_모듈/할인계산기.js'. 재현: 없음 — 파일을 지워야 재현되는 상황이라 조각 코드로 못 만듭니다. 되돌리면 다시 조용해집니다. .d.ts 는 '있으면 붙고 없으면 안 붙는' 파일이라는 것이 눈으로 확인됩니다.
- 2 TS 딱지가 붙어 있습니다. zod 는 타입을 같이 넣어 보내는 패키지입니다. 그래서 npm i zod 만 하면 끝이고 @types/zod 같은 것은 없습니다. (있는지 찾아보면 아예 존재하지 않습니다. 필요가 없으니까요)
- 3 devDependencies 에 있습니다. 타입은 검사할 때만 쓰고 실행하면 사라지기 때문입니다. dependencies 에 넣어도 동작은 하지만, 배포할 때 쓸데없이 따라갑니다.
- 4 npm run typecheck 는 통과합니다. 그리고 실행하면 터집니다. SyntaxError: The requested module './타입없는_모듈/할인계산기.js'

does not provide an **export** named '세금'

할인계산기.js 에는 세금 이라는 함수가 없기 때문입니다. ★ 파일 맨 위 `console.log` 조차 안 찍히는 것을 확인하세요.

import 는 실행 전에 연결되므로, 없는 이름을 가져오면 그 자리에서 끝납니다.

.d.ts 는 약속일 뿐이라 없는 것도 있다고 적을 수 있습니다. 섹션 5가 말하는 위험이 이것입니다. 확인했으면 그 줄을 지우세요.

5 조용합니다. 아무 에러도 안 납니다. 그 상태에서 `getShipping(12500).toUpperCase()` 를 적으면 그것도 통과합니다. 실행해야 `TypeError` 가 납니다. 검사를 통과했다고 맞는 것이 아니라는 가장 분명한 예입니다.

6 `error TS2305: Module '"/타입없는_모듈/할인계산기.js"' has no exported member 'DEFAULT_PERCENT'. 재현: 없음` — 짝이 되는 .js 와 .d.ts 두 파일이 있어야 재현되는 상황입니다. 안 적으면 못 씁니다. 이게 안전한 이유는 '모르는 것을 아는 척하지 않기' 때문입니다. `as any` 로 덮으면 이 에러가 안 나고, 대신 오타가 나도 안 납니다.

7 안 적힌 것은 없습니다. .js 에 있는 셋(`getDiscounted·getShipping·DEFAULT_PERCENT`)을 개념04 에서 전부 쓰고 있기 때문입니다. 쓰지 않는 것이 있었다면 안 적어도 됩니다. 섹션 7의 기준입니다.

8 통과합니다. `declare module` 은 내가 적어 둔 것을 그대로 믿기 때문입니다. 그리고 실제 패키지가 "1.0" 이라는 문자열을 준다면 실행할 때 터집니다. `TypeError: 버전.toFixed is not a function` 섹션 5에서 본 것과 똑같은 일인데, 남의 패키지라 더 알아채기 어렵습니다. 그래서 `declare module` 로 적을 때는 그 패키지의 문서나 소스를 꼭 열어 보세요.

기존 프로젝트에 타입 입히기

RUN node 연습문제.ts

채점: npm run grade ← 출력을 기대 출력과 맞춰 봅니다

검사: npm run check ← 타입 검사 (일부 문제만 잡습니다)

★ 채점은 npm run grade 로 합니다. 실제 출력을 문제의 '기대 출력' 과 줄 단위로 맞춰 봅니다. 전부 일치하면 다 맞은 것입니다.

npm run check(타입 검사)도 같이 쓰세요. 다만 이쪽은 '일부 문제만' 잡습니다.

138문항 중 21개뿐입니다. "출력하세요" 류는 안 풀어도 타입 에러가 안 나거든요. 그러니 check 가 조용하다고 다 맞은 것이 아닙니다. grade 로 확인하세요. (npm run typecheck 는 개념·정답 파일만 봅니다. 그쪽은 언제나 조용합니다)

- 1 TODO 자리에 코드를 씁니다.
- 2 npm run grade 로 채점합니다. (직접 node 연습문제.ts 로 돌려 봐도 됩니다)
- 3 npm run check 로 타입도 봅니다.
- 4 10분 고민해도 안 되면 연습문제_정답.ts 를 보세요.

문제 1~6과 11~12는 기본, 7~9와 13~14는 응용, 10은 [도전]입니다. 문제 11~14는 개념04(타입이 없는 패키지)에서 나옵니다. 문제 15는 에러를 직접 보는 문제라 맨 뒤에 있습니다.

문제 11~14에서 쓸 모듈입니다. 타입이 없는 .js 인데 짝이 되는 할인계산기.d.ts 가 있어서 타입이 붙습니다 (개념04 섹션4).

```
import { getDiscounted, getShipping, DEFAULT_PERCENT } from "./타입없는_모듈/
할인계산기.js";

console.log("=== 08단원 연습문제 ===");
```

문제 1 (개념01 섹션1)

아래는 as 로 우긴 코드입니다. 실행하면 무엇이 출력될까요?

```
type Menu = { name: string; price: number };
const raw: unknown = JSON.parse('{ "name": "라떼" }');
const m = raw as Menu;
console.log(m.price);
```

기대 출력 undefined

기대 검사 결과: 조용함

```
{
```

예상한 값을 출력하세요

```
}
```

문제 2 (개념01 섹션3)

as 대신 satisfies 를 써서, 모양이 맞는지 검사받게 고치세요.

기대 출력 아메리카노

기대 검사 결과: 조용함

```
{
  type Menu = { name: string; price: number };

```

as 를 satisfies 로 바꾸세요

```
const menu = { name: "아메리카노", price: 4000 } as Menu;
console.log(menu.name);
}
```

문제 3 (개념01 섹션3)

아래 객체에는 Menu 에 없는 속성이 하나 있습니다. satisfies 로 바꿔서 그것이 걸리게 만든 뒤, 그 속성을 지우세요.

기대 출력 4500

기대 검사 결과: 조용함

```
{  
  type Menu = { name: string; price: number };  
}
```

satisfies 로 바꾸고 걸리는 속성을 지우세요

```
const menu = { name: "라떼", price: 4500, size: "L" } as Menu;  
console.log(menu.price);  
}
```

문제 4 (개념02 섹션3)

아래 any 를 unknown 으로 바꾸세요. 바꾸면 아래 줄이 걸립니다. 확인하고 쓰도록 고치세요.

기대 출력 글자입니다

기대 검사 결과: 조용함

```
{  

```

any 를 unknown 으로 바꾸고, 확인하고 쓰도록 고치세요

```
const legacy: any = " 글자입니다 ";
console.log(legacy.trim());
}
```

문제 5 (개념02 섹션4)

아래 코드는 걸립니다. 처음 만들 때 없던 속성은 나중에 못 붙입니다. 걸리지 않게 고치세요.

기대 출력

라떼

기대 검사 결과: 조용함

```
{
```

걸리지 않게 고치세요

```
const o = {};
o.name = "라떼";
console.log(o.name);
}
```

문제 6 (개념03 섹션5)

enum 대신 이 자료에서 배운 것으로 같은 일을 하세요. "대기" · "조리중" · "완료" 중 하나만 담기는 타입을 만들고 값을 하나 출력하세요.

기대 출력

조리중

기대 검사 결과: 조용함

```
{
```

리터럴 유니온으로 만드세요

```
}
```

문제 7 (응용 · 개념01 섹션2)

아래 값의 실제 타입(`typeof` 결과)을 출력하세요.

```
const forced = "4500" as unknown as number;
```

기대 출력 `string`

기대 검사 결과: 조용함

```
{
```

예상한 값을 출력하세요

```
}
```

문제 8 (응용 · 개념02 섹션4)

문자열 하나 또는 문자열 배열을 받아, 언제나 배열로 돌려주는 함수를 만드세요.

기대 출력

— '라떼'

— '라떼', '아메리카노'

기대 검사 결과: 조용함

```
{
```

toList 를 만들고 두 번 부르세요

```
}
```

문제 9 (응용 · 개념02 섹션1)

기존 JS 프로젝트를 옮길 때 파일 순서를 고르세요. 아래 넷을 옮기는 순서대로 " → " 로 이어 출력하세요.

화면 / 계산 함수 / 상수·유틸 / 데이터 다루는 곳

기대 출력 상수·유틸 → 계산 함수 → 데이터 다루는 곳 → 화면

기대 검사 결과: 조용함

```
{
```

순서대로 출력하세요

```
}
```

문제 10 ([도전] · 개념01~03 종합)

옆 폴더의 옮기기_예제/이전.js 를 참고해서, 아래 JS 스타일 코드를 옮기세요. ① CartItem 타입을 만든다
② 세 함수에 매개변수·반환 타입을 적는다 ③ 그러면 걸리는 줄이 하나 생긴다. 값이 맞게 나오도록 고친다
④ 총액을 "총액: 15500원" 형태로 출력한다

힌트: shippingText 는 입력창에서 온 문자열입니다.

기대 출력 총액: 15500원

기대 검사 결과: 조용함

```
{
```

위 네 가지를 하세요

```
const cart = [  
  { name: "아메리카노", price: 4000, count: 2 },  
  { name: "라떼", price: 4500, count: 1 },  
];  
const shippingText = "3000";  
}
```

문제 11 (개념04 섹션1)

4500원짜리 메뉴 하나를 살 때의 값을 순서대로 출력하세요. ① 기본할인(DEFAULT_PERCENT)을 적용한 값 ② 그 값에 붙는 배송비 ③ 둘을 더한 총액

[이번에 쓸 것] 위에서 import 한 셋을 그대로 씁니다.

```
getDiscounted(가격, 할인율) · getShipping(금액) · DEFAULT_PERCENT
```

```
기대 출력    4050  
              3000  
              7050
```

기대 검사 결과: 조용함

```
{
```

①②③ 을 각각 출력하세요

```
}
```


문제 12 (개념04 섹션4)

장바구니 합계를 구한 뒤 20% 할인과 배송비를 붙이세요. ① 상품 합계 ② 20% 할인한 값 ③ 배송비까지 더한 총액

합계는 JS자료 08단원의 reduce 로 구하세요.

기대 출력	12500
	10000
	13000

기대 검사 결과: 조용함

```
{
  const cart = [
    { price: 4000, count: 2 },
    { price: 4500, count: 1 },
  ];
```

①②③ 을 각각 출력하세요

```
}
```

문제 13 (응용 · 개념04 섹션7)

주문 금액 세 건에 각각 배송비를 붙인 총액을 배열째 출력하세요. 30000원 이상이면 배송비가 0 입니다.

기대 출력

— 15000, 35000, 11000

기대 검사 결과: 조용함

```
{
  const 주문금액 = [12000, 35000, 8000];
```

map 으로 각 금액에 배송비를 더해 배열째 출력하세요

```
}
```

문제 14 (응용 · 개념04 섹션5)

선언 파일은 '약속' 일 뿐이라 실제와 다를 수 있습니다. 정말 맞는지 실행해서 확인하세요. ①
getShipping(12500) 이 실제로 무슨 타입인지 `typeof` 로 출력 ② DEFAULT_PERCENT 가 실제로 무슨
타입인지 `typeof` 로 출력

둘 다 number 가 나오면 할인계산기.d.ts 가 .js 와 맞게 적혀 있다는 뜻입니다.

기대 출력 number
 number

기대 검사 결과: 조용함

```
{
```

`typeof` 로 두 줄을 출력하세요

```
}
```

문제 15 (에러 확인 · 개념03 섹션5)

아래 두 줄의 `//` 를 지우고,

① `npm run check` 로 검사해 보세요 → 무슨 에러가 나나요? ② 주석을 되돌린 뒤, 이 자료가 왜 `enum` 을
막아 두었는지 생각해 보세요.

확인했으면 다시 `//` 를 붙이세요.

```
{
```

```
enum Color { Red, Blue }  
console.log(Color.Red);
```

```
}
```

개념02 부속 — 옮기기 전의 코드 (그냥 자바스크립트)

★ 이 파일은 .js 입니다. 일부러 고치지 않은 채 두었습니다.

실행해 보세요. 에러 한 줄 없이 끝까지 돌아갑니다.

node 08_기존_프로젝트에_타입_입히기/옮기기_예제/이전.js

그런데 마지막 줄의 총액을 보세요. 맞나요?

상품 12,500원 + 배송비 3,000원 = 15,500원이어야 합니다.

```
function getTotal(items) {
  let sum = 0;
  for (const item of items) {
    sum += item.price * item.count;
  }
  return sum;
}

function withShipping(total, shipping) {
  return total + shipping;
}

function describe(item) {
  return item.name + " " + item.count + "잔";
}

const cart = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "라떼", price: 4500, count: 1 },
];
```

배송비는 화면 입력창에서 온 값이라 문자열입니다.

```
const shippingText = "3000";

console.log(describe(cart[0]));
console.log("상품 합계:", getTotal(cart));
console.log("총액:", withShipping(getTotal(cart), shippingText) + "원");
```

08단원 ·

개념02 부속 — 옮긴 뒤의 코드

```
RUN node 08_기존_프로젝트에_타입_입히기/옮기기_예제/이후.ts
```

검사: npm run typecheck

옆의 이전.js 와 같은 코드입니다. 달라진 것은 두 가지뿐입니다.

① 확장자가 .ts ② 매개변수에 타입을 적음

그랬더니 이전.js 가 조용히 틀리던 곳이 드러났습니다.

```
type CartItem = {
  name: string;
  price: number;
  count: number;
};

function getTotal(items: CartItem[]): number {
  let sum = 0;
  for (const item of items) {
    sum += item.price * item.count;
  }
  return sum;
}

function withShipping(total: number, shipping: number): number {
  return total + shipping;
}

function describe(item: CartItem): string {
  return item.name + " " + item.count + "잔";
}

const cart: CartItem[] = [
  { name: "아메리카노", price: 4000, count: 2 },
  { name: "라떼", price: 4500, count: 1 },
];

const shippingText = "3000";
```

여기가 이전.js 가 조용히 틀리던 자리입니다

이 코드는 검사에서 걸립니다

TS2345

```
console.log("총액:", withShipping(getTotal(cart), shippingText) + "원");
```

Argument of type 'string' is not assignable to parameter of type 'number'.

이전.js에서는 이 줄이 "125003000원" 을 조용히 내놓았습니다.

withShipping 의 shipping 에 : **number** 를 적었더니 그 자리에서 걸립니다.
고치는 방법은 넘기기 전에 숫자로 바꾸는 것입니다.

```
const first = cart[0];  
console.log(first === undefined ? "빈 장바구니" : describe(first));
```

출력 아메리카노 2잔

```
console.log("상품 합계:", getTotal(cart));
```

출력 상품 합계: 12500

```
console.log("총액:", withShipping(getTotal(cart), Number(shippingText)) + "원");
```

출력 총액: 15500원

15500원. 이제 맞습니다.

고친 것은 Number () 한 번뿐인데, 그것을 '어디서' 고쳐야 하는지를 타입이 정확히 짚어 줬다는 것이 핵심입니다. 이전.js에서는 화면에 125003000원이 찍히고 나서야 찾기 시작했을 것입니다.

개념04 부속 — 위 .js 에 손으로 붙인 타입 (선언 파일)

★ .d.ts 는 '선언 파일' 입니다. 모양만 적고 실행되는 코드는 한 줄도 없습니다.

이 파일이 있으면 할인계산기.js 를 타입스크립트에서 안전하게 쓸 수 있습니다.

없으면 TS7016 이 납니다. 개념04 섹션1에서 직접 확인합니다.

declare 는 “코드는 저쪽에 있고, 모양만 여기 적는다” 는 뜻입니다.

```
export declare function getDiscounted(price: number, percent: number): number;

export declare function getShipping(total: number): number;

export declare const DEFAULT_PERCENT: number;
```

개념04 부속 — 타입이 없는 모듈 (그냥 자바스크립트)

★ 이 파일은 .js 입니다. 일부러 고치지 않은 채 두었습니다.

‘남이 만든 패키지’ 라고 생각하세요.

우리 프로젝트를 타입스크립트로 옮기더라도, 남의 패키지는 못 고칩니다.

그럴 때 어떻게 하는지가 개념04의 내용입니다.

```
export function getDiscounted(price, percent) {  
  return Math.round(price * (1 - percent / 100));  
}  
  
export function getShipping(total) {  
  return total >= 30000 ? 0 : 3000;  
}  
  
export const DEFAULT_PERCENT = 10;
```


종합 실습

OPEN 09_종합_실습

```
const rawMenus = [  
const rawOrders = [  
{ menuId: 3, count: 1 },  
{ menuId: 99, count: 1 },
```

01 주문 관리

02 재고 보고서

03 서버 응답 다루기

주문 관리

RUN node 종합01_주문_관리.ts

채점: npm run grade

검사: npm run check

사용 단원: 03(함수) · 04(객체·type) · 05(유니온·좁히기)

카페 주문 관리를 만듭니다. 아래 데이터는 고치지 말고 TODO 부분만 채우세요.

★ 이 실습의 규칙 셋 — 자료 전체에서 정한 것과 같습니다.

`any` 를 쓰지 않습니다 · `!` 를 쓰지 않습니다 · `as` 를 쓰지 않습니다

앞 문제에서 만든 타입과 함수를 뒤에서 쓰니 순서대로 푸세요. 막히면 종합01_주문_관리_정답.ts 를 보세요. 다만 먼저 혼자 고민해 보세요.

주어진 데이터 (고치지 마세요)

```
const rawMenus = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "카페라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
];

const rawOrders = [
  { menuId: 1, count: 2, memo: "얼음 적게" },
  { menuId: 3, count: 1 },
  { menuId: 99, count: 1 },
];

console.log("===== 주문 관리 =====");
```

문제 1 타입 만들기 (04단원 개념02)

Menu 와 Order 타입을 만들고 위 데이터에 붙이세요.

```
Menu   : id(숫자) · name(글자) · price(숫자)
Order  : menuId(숫자) · count(숫자) · memo(글자, 없어도 됨)
```

불인 뒤 메뉴 개수와 주문 개수를 출력하세요.

기대 출력 메뉴 3개 / 주문 3건

`type Menu` · `type Order` 를 만들고, `menus` · `orders` 에 붙여 출력하세요

문제 2 메뉴 찾기 (03단원 · 05단원)

`id` 로 메뉴를 찾는 `findMenu` 를 만드세요. 없을 수도 있으니 반환 타입을 사실대로 적어야 합니다.

기대 출력 카페라떼
 없는 메뉴

`findMenu` 를 만들고 2번과 99번을 각각 찾아 이름을 출력하세요

(없으면 "없는 메뉴" 를 출력. ?? 를 쓰면 짱습니다)

문제 3 주문 한 줄 (05단원 줍히기)

주문 하나를 "이름 x수량 = 금액원" 으로 만드는 `orderLine` 을 만드세요. 메뉴를 못 찾으면 "알 수 없는 메뉴 (99번)" 형태로 돌려주세요.

기대 출력 아메리카노 x2 = 8000원
 케이크 x1 = 6000원
 알 수 없는 메뉴(99번)

`orderLine` 을 만들고 `orders` 를 하나씩 출력하세요

문제 4 메모 (04단원 선택 속성)

메모가 있는 주문만 골라 "메뉴이름: 메모" 로 출력하세요. memo 는 없을 수도 있는 속성입니다. 확인하고 쓰세요.

기대 출력	아메리카노: 얼음 적게
메모가 있는 주문만 출력하세요	

문제 5 주문 상태 (05단원 리터럴 유니온)

상태는 "대기" · "조리중" · "완료" 셋뿐입니다. 상태를 받아 문구를 돌려주는 `statusText` 를 `switch` 로 만드세요. 반환 타입 : `string` 을 꼭 적으세요(빠뜨린 `case` 를 잡아 줍니다).

"대기" → 곧 시작합니다 "조리중" → 만들고 있어요 "완료" → 나왔습니다

기대 출력	곧 시작합니다 나왔습니다
<code>type Status</code> 와 <code>statusText</code> 를 만들고 "대기" · "완료" 를 출력하세요	

문제 6 총액 (03단원 기본값)

주문 전체의 총액을 구하는 `total` 을 만드세요. · 못 찾는 메뉴는 0원으로 칩니다 · 할인율을 선택으로 받습니다. 안 넘기면 0

기대 출력	14000 12600
-------	----------------

total 을 만들고, 할인 없이 한 번 · 10% 할인으로 한 번 출력하세요

문제 7 [도전] 영수증

위에서 만든 것들을 모아 영수증을 출력하세요. ① “----- 영수증 -----” ② 주문 줄 세 개 (문제 3의 orderLine) ③ “합계: 14000원” ④ 상태 한 줄 (문제 5의 statusText 로 “조리중”)

```
기대 출력      ----- 영수증 -----
                아메리카노 x2 = 8000원
                케이크 x1 = 6000원
                알 수 없는 메뉴(99번)
                합계: 14000원
                만들고 있어요
```

영수증을 출력하세요

재고 보고서

RUN node 종합02_재고_보고서.ts

채점: npm run grade

검사: npm run check

사용 단원: 02(추론) · 05(판별 유니온) · 06(제네릭)

재고를 보고 “주문이 필요한가” 를 판단하는 보고서를 만듭니다.

★ 규칙 — any · ! · as 를 쓰지 않습니다.

앞 문제에서 만든 것을 뒤에서 쓰니 순서대로 푸세요.

주어진 데이터 (고치지 마세요) —————

```
const products = [
  { id: 1, name: "아메리카노", stock: 12 },
  { id: 2, name: "카페라떼", stock: 3 },
  { id: 3, name: "케이크", stock: 0 },
];
```

모양이 다른 두 번째 목록입니다. id 가 있다는 것만 같습니다.

```
const suppliers = [
  { id: 7, company: "봄날로스터리", phone: "010-1111-2222" },
  { id: 9, company: "여름제과", phone: "010-3333-4444" },
];

console.log("===== 재고 보고서 =====");
```

문제 1 제네릭 first · last (06단원 개념01)

배열의 첫 값과 마지막 값을 돌려주는 first · last 를 만드세요. 어떤 배열에도 쓸 수 있어야 합니다. 빈 배열이면 undefined 입니다.

기대 출력	아메리카노
	케이크
	비었음

first · last 를 만들고

① products 의 첫 이름 ② 마지막 이름 ③ 빈 문자열 배열의 첫 값(없으면 "비었음")

문제 2 유니온 정규화 (05단원 개념02)

문자열 하나 또는 문자열 배열을 받아, 언제나 배열로 돌려주는 toList 를 만드세요.

기대 출력

— ‘아메리카노’

— ‘아메리카노’, ‘케이크’

toList 를 만들고 두 가지를 각각 넘겨 출력하세요

문제 3 재고 상태 (05단원 개념04 판별 유니온)

재고 수를 세 상태 중 하나로 나눕니다.

0개 → { level: "품절" }
1~5개 → { level: "부족", left: 개수 }

6개 이상 → { level: "충분", left: 개수 }

① StockState 타입을 판별 유니온으로 만드세요 (표착지 이름은 level). ② 숫자를 받아 StockState 를 돌려주는 checkStock 을 만드세요. ③ StockState 를 받아 문구를 돌려주는 stockText 를 switch 로 만드세요.

품절 → "품절입니다"
부족 → "3개 남았습니다 (주문 필요)"
충분 → "12개 있습니다"

기대 출력 12개 있습니다
 3개 남았습니다 (주문 필요)
 품절입니다

`StockState · checkStock · stockText` 를 만들고 `12 · 3 · 0` 을 넣어 출력하세요

문제 4 제네릭 찾기 (06단원 개념02 extends)

`id` 를 가진 것이면 무엇이든 찾을 수 있는 `findById` 를 만드세요. 못 찾으면 `undefined` 입니다.

★ `products` 와 `suppliers` 는 모양이 다릅니다. 같은 것은 `id` 뿐입니다. 그런데도 함수 하나로 둘 다 찾을 수 있어야 합니다. 그게 제네릭을 쓰는 이유입니다. 찾아온 값에는 원래 모양이 그대로 붙어 있어야 합니다 (`products` 에서 찾으면 `.name` 이, `suppliers` 에서 찾으면 `.company` 가 됩니다).

기대 출력 카페라떼
 없음
 여름제과

`findById` 를 만들고

① `products` 에서 2번 ② `products` 에서 99번 ③ `suppliers` 에서 9번
을 각각 찾아 이름(회사명)을 출력하세요
(없으면 "없음". `?.` 와 `??` 를 쓰면 짧습니다)

문제 5 답는 상자 (06단원 개념02 섹션4)

{ ok: boolean; data: T } 모양의 ApiResponse 타입을 만드세요. 상품 목록을 담은 것과 문자열을 담은 것을 각각 만들어 출력하세요.

기대 출력 3
 4

type ApiResponse<T> 를 만들고

① 상품 목록을 담아 data 의 개수 ② "봄날카페" 를 담아 data 의 글자 수

문제 6 [도전] 보고서

위에서 만든 것을 모아 보고서를 출력하세요. ① "----- 재고 보고서 -----" ② 상품마다 "이름: 상태문구" ③ 주문이 필요한 건수 (품절 + 부족)

힌트: 주문 필요 여부는 level 이 "충분" 이 아닌 것으로 세면 됩니다.

기대 출력 ----- 재고 보고서 -----
아메리카노: 12개 있습니다
카페라떼: 3개 남았습니다 (주문 필요)
케이크: 품절입니다
주문 필요: 2건

보고서를 출력하세요

서버 응답 다루기

RUN node 종합03_서버_응답.ts

채점: npm run grade

검사: npm run check

사용 단위: 04(서버 데이터) · 05(null·판별 유니온) · 08(unknown 확인)

서버가 보낸 JSON 을 안전하게 다루는 흐름을 만듭니다. 이 자료에서 가장 실무에 가까운 실습입니다.

★ 규칙 — any · ! · as 를 쓰지 않습니다. 특히 as 를 쓰고 싶어지는 자리가 나올 텐데, 쓰지 말고 확인하세요.

주어진 데이터 (고치지 마세요)

서버가 보냈다고 치는 문자열 넷입니다. 하나만 제대로 왔습니다.

```
const okText = '{"name":"라떼","price":4500}';
const missingText = '{"name":"라떼"}'; // price 가 없습니다
const wrongText = '{"name":123,"price":4500}'; // name 이 숫자입니다
const brokenText = "<html>502 Bad Gateway</html>"; // JSON 이 아예
아닙니다 console.log("===== 서버 응답 =====");
```

문제 1 모양을 적으면 생기는 일 (04단원 개념04)

Menu 타입(name: string, price: number)을 만들고 okText 와 missingText 를 각각 JSON.parse 해서 : Menu 로 받으세요. 그다음 "이름 가격" 을 출력하세요.

★ missingText 쪽에서 이상한 값이 나올 것입니다. 그게 이 문제의 요점입니다. 검사는 조용한데 실행하면 틀린 값이 나옵니다.

기대 출력	라떼 4500
	라떼 undefined

type Menu 를 만들고 둘을 받아 출력하세요

문제 2 확인하고 쓰기 (08단원 개념01 섹션3)

문자열을 받아 '확인을 통과한 Menu' 또는 `null` 을 돌려주는 `parseMenu` 를 만드세요. `JSON.parse` 를 `try / catch` 로 감쌉니다. 실패하면 `null` (`brokenText` 가 이 경우입니다) `JSON.parse` 결과를 `unknown` 으로 받습니다 (any 로 받으면 확인이 무의미합니다) `객체인지` `null` 이 아닌지 `이름이 있는지` `종류가 맞는지` 확인합니다 `하나라도 어긋나면 null`

★ `try` 를 빠뜨리면 `brokenText` 에서 프로그램이 통째로 죽습니다. 타입 검사는 이걸 못 잡습니다. 직접 빼 보고 확인하세요.

```
기대 출력    { name: '라떼', price: 4500 }
              null
              null
              null
```

`parseMenu` 를 만들고 `okText` `missingText` `wrongText` `brokenText` 를 각각 넘겨 출력하세요

문제 3 없을 수 있는 값 (05단원 개념03)

아래 가게 정보를 다룹니다. 타입은 이렇게 적으세요.

```
Shop = { name: string; owner: { name: string; phone: string | null } | null }
```

① `shop1` 의 주인 전화번호를 출력하세요. 없으면 "번호 없음" ② `shop2` 의 주인 이름을 출력하세요. 없으면 "주인 없음"

★ ! 를 쓰면 안 됩니다. ?. 와 ?? 로 하세요.

```
기대 출력    번호 없음
              주인 없음
```

`type Shop` 을 만들고 아래 두 값에 붙인 뒤 출력하세요

```
const rawShop1 = { name: "봄날카페", owner: { name: "홍길동", phone: null } };
const rawShop2 = { name: "무인카페", owner: null };
```

문제 4 화면 상태 (05단원 개념04)

응답 처리 결과를 판별 유니온으로 표현하세요.

LoadState =

```
| { status: "로딩중" }
| { status: "성공"; menu: Menu }
| { status: "실패"; message: string }
```

상태를 받아 문구를 돌려주는 render 를 switch 로 만드세요. 로딩중 → "불러오는 중..."

성공 → "라떼 4500원" (이름 가격원)
실패 → "오류: 형식이 다릅니다"

기대 출력 불러오는 중...
 라떼 4500원
 오류: 형식이 다릅니다

LoadState 와 render 를 만들고 세 상태를 각각 넘겨 출력하세요

문제 5 조립 (문제 2 + 문제 4)

문자열 하나를 받아 LoadState 를 돌려주는 load 를 만드세요.

- parseMenu 가 null 을 주면 → { status: "실패", message: "형식이 다릅니다" }
- 아니면 → { status: "성공", menu: ... }

기대 출력 라떼 4500원
 오류: 형식이 다릅니다

load 를 만들고 okText · missingText 를 각각 넣어 render 결과를 출력하세요

문제 6 [도전] 처리 결과표

세 응답을 전부 처리해서 표로 출력하세요. ① "----- 처리 결과 -----" ② "N번: 문구" (성공이면 render 결과, 실패면 message 그대로) ③ "성공 1건 / 실패 2건"

힌트: 성공·실패는 load 결과의 status 로 셉니다.

★ render 결과에 replace 를 쓰지 마세요. 표딱지가 있으면 표딱지로 갈라야 합니다.

기대 출력 ----- 처리 결과 -----
1번: 라떼 4500원
2번: 형식이 다릅니다
3번: 형식이 다릅니다
성공 1건 / 실패 2건

세 응답을 처리해 표를 출력하세요

부 록 가

연습문제·종합연습 정답

먼저 스스로 풀어 본 다음에 보세요. 정답과 다르게 썼더라도 기대 출력이 같으면 맞은 것입니다.
다만 “왜 그렇게 되는지” 설명은 꼭 읽으세요.

01단원 연습문제 정답.ts 정답 타입스크립트 시작하기

RUN node 연습문제_정답.ts

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
console.log("=== 01단원 연습문제 ===");
```

출력 === 01단원 연습문제 ===

문제 1

```
{
  const seatCount: number = 24;
  console.log(seatCount);
}
```

출력 24

```
}
```

해설 ① 이름 뒤에 : 과 종류를 씁니다. 이게 타입 표기의 전부입니다. 해설 ② 사실 이 줄은 타입을 안 붙여도 됩니다. 24 를 보고 타입스크립트가

알아서 **number** 라고 압니다. 그것을 '추론' 이라 하고 02단원에서 배웁니다.
지금은 손에 익히려고 일부러 적는 것입니다.

해설 ③ 흔한 실수 — `const seatCount = number 24;` 처럼 쓰는 것.

타입은 = 왼쪽, 값은 = 오른쪽입니다.

문제 2

```
{
  const menuName: string = "아메리카노";
  const isHot: boolean = true;
  console.log(menuName, isHot);
}
```


출력 아메리카노 true

```
}

```

해설 ① 참/거짓의 타입 이름은 boolean 입니다. bool 이나 Boolean 이 아닙니다. 해설 ② 대문자로 시작하는 String / Number / Boolean 도 문법상 되기는 하지만

전혀 다른 것이고 쓰면 안 됩니다. 02단원에서 이유를 설명합니다.

해설 ③ 타입 이름은 전부 소문자로 시작한다고 외워 두면 편합니다.

문제 3

```
{
  const quantityText = "2";
  const shipping = 3000;
  console.log(typeof (quantityText + shipping));
}
```

출력 string

```
}

```

해설 ① 문자열에 + 를 쓰면 숫자가 섞여 있어도 결과는 문자열입니다.

실제 값은 "23000" 입니다. $2 + 3000 = 3002$ 가 아닙니다.

해설 ② 이 줄은 타입 검사도 통과합니다. 개념01 섹션1에서 본 그대로입니다.

이어붙이기는 정당한 연산이라 타입스크립트가 막을 이유가 없습니다.

해설 ③ typeof 를 쓸 때 괄호를 빠뜨려 typeof quantityText + shipping 이라고 쓰면

"string3000" 이 나옵니다. typeof 가 먼저 붙어 버리기 때문입니다.

문제 4

```
{
  const quantityText = "2";
  const shipping = 3000;
  const total: number = Number(quantityText) + shipping;
  console.log(total);
}
```

출력 3002

```
}

```

해설 ① : number 라고 적어 두었기 때문에 Number() 를 빼면 그 자리에서 걸립니다.

```
error TS2322: Type 'string' is not assignable to type 'number'.
```

재현:

```
const quantityText = "2";
const shipping = 3000;
const total: number = quantityText + shipping;
```

해설 ② 이게 개념01 섹션3의 핵심입니다. 타입을 적어 둔 자리에서만 걸립니다.

문제 3처럼 그냥 출력만 하면 아무도 안 막아 줍니다.

해설 ③ Number() 대신 parseInt() 도 됩니다. JS자료 01단원 개념05를 보세요.

문제 5

```
{
  console.log(2);

```

```
출력      2

```

```
}

```

해설 ① 2번만 잡힙니다. 숫자 자리에 문자열이 왔기 때문입니다. 해설 ② 1번(-5)은 타입이 number 라서 통과합니다. 나이가 음수인 것이

말이 안 된다는 것은 타입의 관심사가 아닙니다.

해설 ③ 3번도 통과합니다. 1000 도 2000 도 결과도 전부 number 입니다.

"더해야 하는데 뺐다" 는 사람만 알 수 있는 일입니다.
→ 타입 검사를 통과했다는 것과 코드가 맞다는 것은 다른 말입니다.

문제 6

```
{
  console.log("4000");

```

출력 4000

```
console.log("string");
```

출력 string

```
}
```

해설 ① node 는 타입을 검사하지 않고 지웁니다. 그래서 4000 이 그대로 찍힙니다. 해설 ② 화면에 찍힌 4000 은 문자열 "4000" 입니다. 눈으로는 구별이 안 됩니다.

`typeof` 로 찍어 봐야 `string` 인 것이 드러납니다.

해설 ③ 이걸 직접 본 것이 개념02 섹션3의 틀린예제입니다.

문제 7

```
{
  console.log("통과");
}
```

출력 통과

```
console.log("VS Code 빨간 밑줄");
```

출력 VS Code 빨간 밑줄

```
console.log("npx tsc --noEmit");
```

출력 npx tsc --noEmit

```
}
```

해설 ① 둘뿐입니다. node 로 실행해서는 절대 안 보입니다. 그게 이 단원 전체의 이야기입니다. 해설 ② tsc 는 문제가 없으면 아무 말도 안 합니다. '성공' 같은 글자는 안 나옵니다. 해설 ③ 처음에는 "명령이 안 먹었나?" 싶는데 그게 정상입니다. 해설 ④ 반대로 뭔가 나왔다면 전부 고쳐야 할 것입니다. 경고와 에러의 구분이 없습니다.

문제 8

```
{
  const seatCount: number = 24;
  console.log(seatCount + 1);
}
```

출력 25

```
}
```

해설 ① 고치기 전 에러는 이렇습니다.

```
연습문제.ts(줄,9): error TS2322: Type 'string' is not assignable to type 'number'.
재현:
const seatCount: number = "24";
console.log(seatCount + 1);
괄호 안에서 앞이 줄, 뒤가 열(그 줄의 몇 번째 글자)입니다.
```

해설 ② 고치기 전에 node 로 돌리면 "241" 이 나옵니다. 에러가 아닙니다.

"24" + 1 이 이어붙이기가 되기 때문입니다(01단원 개념01 섹션1).
→ 검사는 걸리고 실행은 조용히 틀린, 이 단원의 전형적인 자리입니다.

해설 ③ 열 번호는 한 줄에 값이 여러 개일 때 쓸모가 있습니다.

배열 [90, 85, "칠십"] 에서 어느 값이 문제인지 알려 줍니다.

문제 9

```
{
  console.log("내가 넣은 것:", "boolean");
```

출력 내가 넣은 것: boolean

```
console.log("들어가야 하는 것:", "string");
```

출력 들어가야 하는 것: string

```
}
```

해설 ① Type 'A' is not assignable to type 'B' 에서

A 가 내가 넣은 것, B 가 들어가야 하는 것입니다.

해설 ② 순서를 거꾸로 읽으면 엉뚱한 곳을 고치게 됩니다. 가장 흔한 오독입니다. 해설 ③ 외우는 법 — 앞에 있는 것이 '지금 상태', 뒤에 있는 것이 '되어야 할 상태'.

문제 10

```
{
  const stockText = "12";
  const sold = 5;
  const left: number = Number(stockText) - sold;
  console.log(left);
}
```

출력 7

```
}
```

해설 ① 걸리는 줄은 `const left = stockText - sold;` 입니다.

```
error TS2362: The left-hand side of an arithmetic operation must be ...
```

재현:

```
const stockText = "12";
const sold = 5;
const left = stockText - sold;
```

해설 ② 문제 4와 달리 이번에는 `: number` 를 안 적어도 걸립니다.

- 는 숫자 전용 연산이라 그 자체로 안 되기 때문입니다.
+ 만 특별하다는 것을 다시 확인하세요.

해설 ③ JS에서는 이 줄이 조용히 7을 내놓았습니다. 우연히 맞았던 것입니다.

stockText 가 "12개" 였다면 NaN 이 됐을 것입니다.

문제 11

```
{
  console.log("TS2339");
}
```

출력 TS2339

```
}
```

해설 ① Property 'toUpperCase' does not exist on type 'number'.

숫자에는 toUpperCase 가 없습니다. 문자열에만 있습니다.

해설 ② TS2551 이 아닌 이유는 비슷한 이름 후보를 못 찾았기 때문입니다.

`number` 에 toUpperCase 와 닮은 이름이 아예 없습니다.

해설 ③ JS 에서는 실행하다가 TypeError 로 터지던 자리입니다.

타입스크립트는 실행 전에 잡습니다.

문제 12

```
{
  const unitPrice: number = 4000;
  const quantityText: string = "2";
  const shipping: number = 3000;

  const total: number = unitPrice * Number(quantityText) + shipping;
  console.log("총액: " + total + "원");
}
```

출력 총액: 11000원

```
}
```

해설 ① unitPrice * quantityText 를 그대로 쓰면 TS2363 으로 걸립니다.

곱하기의 오른쪽도 숫자여야 하기 때문입니다.
(왼쪽이면 TS2362, 오른쪽이면 TS2363 입니다)

해설 ② 마지막 줄의 + 는 일부러 쓴 이어붙이기입니다.

결과를 string 자리에 담을 것이므로 이걸 올바른 사용입니다.
console.log("총액:", total + "원") 처럼 써도 됩니다.

해설 ③ 흔한 실수 — Number() 를 total 전체에 씌우는 것.

Number(unitPrice * quantityText) 로는 안 걸러집니다.
문자열인 quantityText 하나에만 씌워야 합니다.

문제 13

주석을 풀면 이렇게 됩니다.

① node 연습문제.ts

여는 시간: 아홉시
 → 아무 일도 안 일어납니다. 그냥 돌아갑니다.
 : number 라고 적어 놓았는데 한글이 그대로 찍힙니다.

② npx tsc --noEmit

연습문제.ts(줄,열): error TS2322:
 Type 'string' is not assignable to type 'number'.

재현:

```
const openHour: number = "아홉시";
```

→ 여기서 걸립니다.

해설 ① 이 단원에서 제일 중요한 것이 이 차이입니다.

node 는 실행만, tsc 는 검사만 합니다.

해설 ② 그래서 "돌아가니까 맞겠지" 가 통하지 않습니다.

저장할 때마다 VS Code 의 빨간 밑줄을 보는 습관을 들이세요.

해설 ③ 07단원에서 Vite 를 쓸 때도 똑같습니다. Vite 도 타입을 지웁니다.

도구가 바뀌어도 이 구조는 그대로입니다.

02단원 연습문제 정답.ts 정답 기본 타입과 추론

RUN node 연습문제_정답.ts

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
console.log("=== 02단원 연습문제 ===");
```

출력 === 02단원 연습문제 ===

문제 1

```
{
  const menuName: string = "라떼";
  const price: number = 4500;
  const isIced: boolean = true;
  console.log(menuName, price, isIced);
}
```

출력 라떼 4500 true

```
}
```

해설 ① 타입 이름은 전부 소문자입니다. String/Number/Boolean 은 다른 것입니다. 해설 ② 참/거짓은 bool 이 아니라 boolean 입니다. 다른 언어를 해 본 분들이 자주 틀립니다. 해설 ③ 사실 이 세 줄은 전부 안 적어도 됩니다(문제 4를 보세요).

지금은 손에 익히려고 적는 것입니다.

문제 2

```
{
  const sizes: string[] = ["S", "M", "L"];
  const stocks: number[] = [12, 5];
  console.log(sizes.length, stocks.length);
}
```

출력 3 2


```
}

```

해설 ① `string[]` 은 “문자열이 들어 있는 배열” 입니다. `[]` 가 “여러 개” 라는 뜻입니다. 해설 ② 흔한 실수 — `[]string` 이라고 쓰는 것. 순서가 반대입니다. 해설 ③ 또 흔한 실수 — `array<string>` 이나 `Array` 같은 것을 찾는 것.

`Array<string>` 이라는 표기도 있기는 하지만 06단원에서 다룹니다.
지금은 `string[]` 하나만 쓰세요.

문제 3

```
{
  const prices: number[] = [4000, 4500, 5000];
  console.log(prices[0] + 100);
}

```

출력 4100

```
}

```

해설 ① `number[]` 에서 꺼낸 값에는 `number` 가 붙습니다. 그래서 그냥 더해집니다. 해설 ② 만약 `prices` 가 `string[]` 이었다면 “4000100” 이 나왔을 것입니다.

문자열에 `+` 는 이어붙이기니까요(01단원 개념01 섹션1).

해설 ③ 이게 배열에 한 종류만 넣어야 하는 이유입니다.

섞여 있으면 꺼낼 때마다 “이게 뭐지” 를 확인해야 합니다.

문제 4

```
{
  const shopName = "봄날카페";
  const seats = 24;
  console.log(shopName, seats);
}

```

출력 봄날카페 24

```
}

```

해설 ① 둘 다 지워도 됩니다. 값이 바로 옆에 있으니 추론됩니다. 해설 ② 지워도 검사는 똑같이 합니다.
`shopName.toFixed(2)` 를 쓰면 여전히 걸립니다.

"타입을 안 적으면 검사가 느슨해진다" 는 오해입니다.

해설 ③ 실무 코드에 타입이 생각보다 적게 적혀 있는 이유가 이것입니다.

적어야 할 곳(함수 매개변수 등)에만 적습니다.

문제 5

```
{  
  console.log("number[]");  
}
```

출력 number[]

```
}
```

해설 ① 마우스를 올리면 `const counts: number[]` 라고 뜹니다. 해설 ② 편집기가 없다면 `const peek: null = counts;` 를 써 보세요.

error TS2322: Type 'number[]' is not assignable to type 'null'.

재현:

```
const counts = [1, 2, 3];  
const peek: null = counts;
```

메시지 왼쪽에 진짜 타입이 그대로 나옵니다. 확인 후 지웁니다.

해설 ③ `const` 인데 왜 리터럴 타입이 아니냐면, 배열 '안' 의 값은 바뀔 수 있기

때문입니다. `const` 가 막는 건 `counts` 자체를 다른 배열로 바꾸는 것뿐입니다.

문제 6

```
{  
  console.log('커피');  
}
```

출력 "커피"

```
console.log("string");
```

출력 string

```
}

```

해설 ① `const` 는 다시 대입할 수 없으니 영원히 "커피" 입니다.

그래서 값 자체가 타입이 됩니다. 이것을 리터럴 타입이라고 합니다.

해설 ② `let` 은 나중에 바뀔 수 있으니 넉넉하게 `string` 으로 잡습니다.

단, 종류를 넘어가는 것은 막습니다. `b = 3` 은 걸립니다.

해설 ③ 이 차이가 왜 쓸모 있는지는 05단원에서 나옵니다.

"up" 이나 "down" 만 허용하는 값을 만들 때 씁니다.

문제 7

```
{
  const shop = {
    name: "봄날카페",
    menus: [
      { name: "아메리카노", price: 4000 },
      { name: "라떼", price: 4500 },
    ],
  };
  console.log(shop.menus[1].price);
}
```

출력 4500

```
}

```

해설 ① 아무것도 안 적었는데 `shop.menus[1].price` 까지 자동완성이 됩니다.

타입스크립트가 객체 안의 객체, 배열 안의 객체까지 전부 알아낸 것입니다.

해설 ② 이게 추론의 진짜 위력입니다. 손으로 적으면

```
{ name: string; menus: { name: string; price: number }[] } 이 됩니다.
읽기도 힘들고 유지하기도 힘듭니다.
```

해설 ③ 그럼 04단원에서 왜 모양을 적는 법을 배우냐면,

'값이 없는 자리'(함수 매개변수, 서버 데이터)에는 추론할 근거가 없기 때문입니다.

문제 8

```
{  
  console.log(2);
```

출력 2

```
}
```

해설 ① 1번은 4000 이 옆에 있으니 추론됩니다. 안 적어도 됩니다. 해설 ② 2번은 초기값이 없습니다. 볼 것이 없으니 알아낼 수가 없습니다.

`let selected: string;` 이라고 적어 줘야 합니다.

해설 ③ 규칙 한 줄 — 초기값이 옆에 있으면 안 적고, 없으면 적는다.

함수 매개변수가 '없는' 대표적인 자리라 무조건 적습니다(03단원).

문제 9

```
{  
  console.log("any");
```

출력 any

```
}
```

해설 ① any 는 타입을 적는 것이 아니라 타입 검사를 끄는 스위치입니다.

그래서 문자열을 `number` 자리에 넣어도 아무 말이 없습니다.

해설 ② : `number` 라고 적어 놓은 것이 완전히 무의미해집니다.

적어 둔 사람은 숫자가 들어올 거라 믿고 뒤에서 계산을 할 텐데,
실제로는 문자열이 들어와서 실행할 때 이상한 값이 나옵니다.

해설 ③ 이 자료의 규칙 — any 를 쓰지 않습니다.

문제 10

```
{
  console.log(2);
```

출력 2

```
}
```

해설 ① 1번은 점(.)으로 꺼낸 것이라 any 가 그대로 번집니다. 해설 ② 2번은 * 를 거쳤습니다. 곱하기의 결과는 무조건 숫자라

타입스크립트가 확신할 수 있어서 **number** 가 됩니다.
문자열에 + 한 결과도 마찬가지로 **string** 이 됩니다.

해설 ③ 그래서 “any 는 무조건 끝까지 번진다” 는 틀린 말입니다.

점을 따라갈 때 번지고, 계산을 거치면 끊깁니다.
다만 끊긴다고 안심할 일은 아닙니다. 값이 문자열 “14” 였다면
“14” * 2 = 28 로 우연히 맞고, “가” * 2 였다면 NaN 이 됩니다.

문제 11

```
{
  const parsed: { seats: number } = JSON.parse('{\"seats\":24}');
  console.log(parsed.seats + 10);
```

출력 34

```
}
```

해설 ① 모양을 적기 전에는 검사가 조용합니다. parsed 가 any 니까요.

그런데 실행하면 NaN 이 나옵니다. seat 은 없는 이름이라 undefined 이고,
undefined + 10 이 NaN 이기 때문입니다.
→ “검사도 통과하고 에러도 안 나는데 화면에 NaN” 이 any 의 전형적인 피해입니다.

해설 ② : { seats: number } 를 적는 순간 오타가 드러납니다.

error TS2551: Property 'seat' does not exist on type '{ seats: number; }'. Did you mean 'seats'?

재현:

```
const parsed: { seats: number } = JSON.parse('{ "seats": 24 }');
console.log(parsed.seat + 10);
```

한 줄 적었을 뿐인데 숨어 있던 버그가 튀어나옵니다.

해설 ③ 이게 04단원(모양 적기)이 필요한 이유입니다.

07단원에서 fetch 로 서버 데이터를 받을 때 똑같은 일이 생깁니다.

문제 12

```
{
  const mystery: unknown = "봄날카페";
  if (typeof mystery === "string") {
    console.log(mystery.length);
  }
}
```

출력 4

```
}
}
```

해설 ① unknown 은 확인하기 전까지 아무것도 못 하게 막습니다.

그냥 mystery.length 를 쓰면 TS18046 으로 걸립니다.

해설 ② typeof 로 확인한 if 안쪽에서는 타입스크립트가 string 인 것을 압니다.

그래서 .length 가 통과합니다. 이것을 '타입 좁히기' 라고 합니다.

해설 ③ any 와 unknown 의 차이 —

any 는 "검사하지 마세요", unknown 은 "확인하고 쓰겠습니다" 입니다.
모르는 값에는 unknown 을 쓰세요.

문제 13

```
{
  const shop = { revenue: 120000, seatsText: "24" };
  const perSeat: number = Math.floor(shop.revenue / Number(shop.seatsText));
  console.log("좌석당 매출: " + perSeat);
}
```

출력 좌석당 매출: 5000

```
}
```

해설 ① : any 를 지우면 { revenue: number; seatsText: string } 으로 추론됩니다.

그러면 shop.revenue / shop.seatsText 가 TS2363 으로 걸립니다.
(나누기의 오른쪽이 문자열이라서)

해설 ② any 가 있었을 때는 이 줄이 조용히 통과했습니다.

그리고 실행하면 "24" 가 우연히 숫자로 바뀌어 5000 이 나왔을 것입니다.
seatsText 가 "24석" 이었다면 NaN 이 됐을 것이고, 그때도 아무도 안 막습니다.

해설 ③ any 하나를 지웠더니 숨어 있던 진짜 문제가 드러난 것입니다.

이게 any 를 안 쓰는 이유 전부입니다.

문제 14

주석을 풀면 이렇게 됩니다.

① npx tsc --noEmit

조용합니다. 안 걸립니다.
value 가 any 라서 toFixed 가 있는지 없는지 아무도 안 봅니다.

② node 연습문제.ts

```
TypeError: value.toFixed is not a function
→ 여기서 프로그램이 멈춥니다.
"이 줄은 실행될까요?" 는 출력되지 않습니다.
```

해설 ① 01단원에서 본 것과 정반대입니다.

그때는 "돌아가는데 검사가 걸린다" 였고,
지금은 "검사는 통과하는데 돌리면 터진다" 입니다.

해설 ② 후자가 훨씬 나쁩니다. 검사를 믿을 수 없게 되기 때문입니다.

any 가 하나라도 있으면 "tsc 가 조용하다" 는 말이 의미를 잃습니다.

해설 ③ 그래서 규칙이 "any 를 쓰지 않는다" 입니다.

정말 모를 때는 **unknown** 을 쓰고, 확인한 뒤에 씁니다(문제 12).

03단원 연습문제 정답.ts 정답 함수 타입

RUN node 연습문제_정답.ts

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
console.log("=== 03단원 연습문제 ===");
```

출력 === 03단원 연습문제 ===

문제 1

```
{
  function callMenu(menu: string) {
    return menu + " 나왔습니다";
  }
  console.log(callMenu("라떼"));
```

출력 라떼 나왔습니다

```
}
```

해설 ① 안 적으면 TS7006 (implicitly has an 'any' type) 이 납니다. 해설 ② implicitly = "은근슬쩍". 적지도 않았는데 any 가 됐다는 뜻입니다.

02단원에서 any 를 안 쓰기로 했는데, 매개변수를 비우면 자동으로 any 가 됩니다.

해설 ③ 반환 타입은 안 적었습니다. 몸통을 보면 문자열인 게 뻔하니 추론됩니다.

문제 2

```
{
  function bigger(a: number, b: number) {
    return a > b ? a : b;
  }
  console.log(bigger(4, 9));
```

출력 9

```
}
```

해설 ① `Math.max(a, b)` 를 써도 정답입니다. 해설 ② 반환 타입을 안 적었지만 (`a: number, b: number`) ⇒ `number` 로 추론됩니다.

마우스를 올려 확인해 보세요.

해설 ③ 흔한 실수 — `function bigger(a, b: number)` 처럼 하나만 적는 것.

안 적은 `a` 만 `TS7006` 으로 걸립니다. 매개변수는 하나씩 따로 봅니다.

문제 3

```
{
  function getTotal(count: number): number {
    return count * 4000;
  }
  console.log(getTotal(2));
}
```

출력 8000

```
}
```

해설 ① : `number` 를 붙이는 순간 `return String(...)` 줄에서 걸립니다.

error TS2322: Type 'string' is not assignable to type 'number'.

재현:

```
function getTotal(count: number): number { return String(count * 4000); }
```

해설 ② 핵심은 '어디서' 걸리느냐입니다. 함수를 쓰는 쪽이 아니라 함수 '안' 입니다.

실수한 자리에서 바로 잡히니 원인을 찾을 필요가 없습니다.

해설 ③ 반환 타입을 안 적었다면 이 함수는 문자열을 돌려주는 함수가 되고,

문제가 이 값을 쓰는 먼 곳에서 드러났을 것입니다.
중요한 함수에 반환 타입을 적는 이유입니다.

문제 4

```
{
  function announce(message: string): void {
    console.log("[공지] " + message);
  }
  announce("문 달습니다");
}
```

출력 [공지] 문 달습니다

```
}
```

해설 ① void 는 “돌려주는 것이 없다” 는 뜻입니다. 해설 ② return 이 없으면 안 적어도 void 로 추론됩니다. 적는 것은 선택입니다.

다만 적어 두면 “이 함수는 값을 안 준다” 가 읽는 사람에게 분명해집니다.

해설 ③ announce(...).length 처럼 결과를 쓰려고 하면

TS2339 Property 'length' does not exist on type 'void'. 로 막힙니다.
JS 에서는 undefined 가 되어 실행 중에 터지던 자리입니다.

문제 5

```
{
  const half = (n: number): number => n / 2;
  console.log(half(42));
}
```

출력 21

```
}
```

해설 ① 적는 자리는 같습니다. 매개변수는 괄호 안, 반환값은 괄호 뒤입니다. 해설 ② const half = (n: number) => n / 2; 처럼 반환 타입을 빼도 정답입니다.

실무에서는 이쪽이 더 흔합니다.

해설 ③ 매개변수가 없어도 괄호는 필요합니다. const f = (): string => "x";

문제 6

```
{
  function order(menu: string, memo?: string) {
    console.log(menu, memo);
  }
  order("아메리카노");
```

출력 아메리카노 undefined

```
order("아메리카노", "얼음 적게");
```

출력 아메리카노 얼음 적게

```
}
```

해설 ① ? 를 붙이면 개수 검사(TS2554)를 통과합니다. 해설 ② 안 넘긴 값은 `undefined` 입니다. `null` 이 아닙니다. 해설 ③ 대가가 있습니다. `memo` 의 타입이 `string | undefined` 가 되어

`.length` 같은 것을 바로 못 씁니다. 문제 9가 그 이야기입니다.

문제 7

```
{
  function total(unit: number, count: number = 1) {
    return unit * count;
  }
  console.log(total(4000));
```

출력 4000

```
console.log(total(4000, 3));
```

출력 12000

```
}
```

해설 ① 기본값을 주면 ? 없이도 생략할 수 있습니다. 해설 ② 그리고 `count` 의 타입이 `number` 그대로입니다. `undefined` 가 안 섞입니다.

그래서 확인 없이 바로 곱셈에 쓸 수 있습니다. ? 보다 나은 점이 이것입니다.

해설 ③ count = 1 처럼 타입을 빼도 됩니다. 1 을 보고 number 로 추론합니다.

```
function total(unit: number, count = 1) 이 실무에서 가장 흔한 모양입니다.
```

문제 8

```
{
  const nums: number[] = [1, 2, 3];
  console.log(nums.map((n) => n * 10));
}
```

출력 [10, 20, 30]

```
}
```

해설 ① n 에 타입을 안 적었는데 TS7006 이 안 납니다.

nums 가 number[] 이므로 map 이 꺼내 주는 값은 number 일 수밖에 없어서,
타입스크립트가 문맥에서 알아낸 것입니다.

해설 ② 그래도 검사는 그대로 합니다. n.toUpperCase() 를 쓰면 TS2339 로 걸립니다. 해설 ③ 규칙 —
내가 만드는 함수의 매개변수는 적고, 남의 함수에 넘기는 콜백은 말긴다.

문제 9

```
{
  function showMemo(memo?: string) {
    if (memo) {
      console.log(memo.length);
    } else {
      console.log("메모 없음");
    }
  }
  showMemo("얼음 적게");
}
```

출력 5

```
showMemo();
```

출력 메모 없음

```
}
```

해설 ① 고치기 전에는 TS18048 'memo' is possibly 'undefined'. 가 납니다. 해설 ② if (memo) 안쪽에서는 memo 가 확실히 문자열입니다.

타입스크립트가 그것을 알고 .length 를 허락합니다. 이것이 '타입 좁히기' 입니다. 05단원의 주제이니 지금은 "if 로 감싸면 된다" 만 익히세요.

해설 ③ 혼한 실수 — memo!.length 처럼 느낌표를 붙여 넘어가는 것.

"내가 책임질 테니 그냥 해라" 라는 뜻이라 any 와 비슷하게 위험합니다. showMemo() 를 부르는 순간 실행 중에 터집니다.

문제 10

```
{  
  const format: (n: number) => string = (n) => n + "원";  
  console.log(format(4000));  
}
```

출력 4000원

```
}
```

해설 ① 함수 타입은 (매개변수) ⇒ 반환값 으로 적습니다.

함수를 '정의' 할 때 쓰는 콜론 (:)과 헷갈리지 마세요.
정의: (n: number): string => ...
타입: (n: number) => string

해설 ② 왼쪽에 타입을 적어 두면 오른쪽 (n) 에는 타입을 안 적어도 됩니다.

문제 8과 같은 문맥 추론입니다.

해설 ③ 이 표기가 길어지면 이름을 붙일 수 있습니다. 04단원에서 배웁니다.

문제 11

```
{
  function sumWith(values: number[], fn: (n: number) => number): number {
    let sum = 0;
    for (const v of values) {
      sum += fn(v);
    }
    return sum;
  }
  console.log(sumWith([1, 2, 3], (n) => n * 10));
}
```

출력 60

```
}
```

해설 ① `values.map(fn).reduce((a, b) => a + b, 0)` 으로 써도 정답입니다. 해설 ② `fn` 의 타입을 적어 두었기 때문에 넘기는 쪽의 `(n)` 이 자유로워집니다.

콜백을 받는 함수를 만들 때는 그 자리에 함수 타입을 꼭 적어야 합니다.
안 적으면 `TS7006` 으로 걸립니다.

해설 ③ 흔한 실수 — `sumWith([1,2,3], (n) => "값" + n)` 처럼 문자열을 돌려주는 것.

`TS2322` 로 걸립니다. 화살표 안쪽까지 검사해 줍니다.

문제 12

```
{
  function tag(message: string, prefix: string = "[알림]") {
    return prefix + " " + message;
  }
  console.log(tag("문 달습니다"));
}
```

출력 [알림] 문 달습니다

```
}
```

해설 ① 원래 코드는 `TS1016 A required parameter cannot follow an optional parameter.`

선택 매개변수 뒤에 필수 매개변수가 올 수 없습니다.

해설 ② 생각해 보면 당연합니다. tag("문 닫습니다") 라고 부르면

그 값이 prefix 인지 message 인지 알 방법이 없습니다.

해설 ③ 규칙 — 반드시 필요한 것을 먼저, 선택인 것을 뒤에.

? 대신 기본값을 쓴 것에도 주목하세요. 쓸 만한 기본값이 있으면 그쪽이 낫습니다.

문제 13

```
{
  const orders = [
    { menu: "아메리카노", price: 4000, isHot: true },
    { menu: "아이스티", price: 5000, isHot: false },
    { menu: "라떼", price: 4500, isHot: true },
  ];

  function sumHot(list: { menu: string; price: number; isHot: boolean }[],
    pick: (o: { menu: string; price: number; isHot: boolean }) => boolean,
  ): number {
    let sum = 0;
    for (const o of list.filter(pick)) {
      sum += o.price;
    }
    return sum;
  }

  console.log("뜨거운 음료 합계: " + sumHot(orders, (o) => o.isHot) + "원");
}
```

출력 뜨거운 음료 합계: 8500원

```
}
```

해설 ① $4000 + 4500 = 8500$ 입니다. 아이스티는 빠집니다. 해설 ② 매개변수 타입이 꼼꼼하게 겁니다.

{ menu: string; price: number; isHot: boolean } 가 두 번이나 나옵니다.
고칠 일이 생기면 두 군데를 다 고쳐야 합니다.
→ 이게 04만원(타입에 이름 붙이기)이 필요한 이유입니다.
04만원을 배우고 나면 이렇게 됩니다.

```
type Order = { menu: string; price: number; isHot: boolean };
function sumHot(list: Order[], pick: (o: Order) => boolean): number
```

해설 ③ 넘기는 쪽 (o) ⇒ o.isHot 에는 타입을 안 적었습니다.

pick 의 모양을 적어 두었으니 문맥에서 알아냅니다. 문제 8·11과 같습니다.

문제 14

주석을 풀면 이렇게 됩니다.

```
npm run check
```

03_함수_타입/연습문제.ts(줄,열): error TS7006: Parameter 'n' implicitly has an 'any' type. 재현:

```
function double(n) { return n * 2; }
```

해설 ① implicitly = “은근슬쩍”, “적지도 않았는데 그렇게 됐다” 는 뜻입니다.

여기서는 "n 이 자동으로 any 가 됐다" 입니다.

해설 ② node 로 돌리면 42 가 잘 나옵니다. 실행에는 아무 문제가 없습니다.

타입 검사에서만 걸립니다. 01단원 개념02의 그 구조 그대로입니다.

해설 ③ 이 에러는 이 자료에서 가장 자주 보게 될 것입니다.

implicitly 라는 단어가 보이면 "매개변수에 타입을 안 적었구나" 라고 바로 떠올리면 됩니다.

04단원 연습문제 정답.ts 정답 객체와 타입 별칭

RUN node 연습문제_정답.ts

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
console.log("=== 04단원 연습문제 ===");
```

출력 === 04단원 연습문제 ===

문제 1

```
{
  function printShop(shop: { name: string; seats: number }) {
    console.log(shop.name + " " + shop.seats + "석");
  }
  printShop({ name: "봄날카페", seats: 24 });
}
```

출력 봄날카페 24석

```
}
```

해설 ① 안 적으면 TS7006 입니다. 매개변수는 예외 없이 적습니다(03단원). 해설 ② 타입 안에서는 세미콜론(;), 값 안에서는 쉼표(,)로 통일하면 안 헛갈립니다. 해설 ③ 이제 매개변수 자리가 설명서입니다. 이 함수를 쓰는 사람은

몸통을 안 열어 봐도 무엇을 넘겨야 하는지 압니다.

문제 2

```
{
  type Shop = { name: string; seats: number };
  const shop: Shop = { name: "별빛카페", seats: 12 };
  console.log(shop.name, shop.seats);
}
```

출력 별빛카페 12

```
}
```

해설 ① 타입 이름은 대문자로 시작합니다. 변수(소문자)와 한눈에 구별됩니다. 해설 ② type 뒤에는 = 가 있습니다. interface 에는 없습니다. 짝지어 외우세요. 해설 ③ 이름을 붙이면 에러 메시지에도 그 이름이 나옵니다.

긴 모양이 통째로 찍히는 것보다 훨씬 읽기 쉽습니다.

문제 3

```
{
  type Menu = { name: string; price: number };
  const menus: Menu[] = [
    { name: "아메리카노", price: 4000 },
    { name: "라떼", price: 4500 },
  ];
  console.log(menus.map((m) => m.name));
}
```

출력 ['아메리카노', '라떼']

```
}
```

해설 ① 객체 배열은 Menu[] 입니다. [] 는 뒤에 붙습니다. 해설 ② m 에 타입을 안 적어도 됩니다. menus 가 Menu[] 라 문맥에서 알아냅니다. 해설 ③ 사실 이 문제는 타입을 안 적어도 돌아갑니다(추론이 되니까요).

적는 연습을 위한 문제이지만, 실무에서도 적어 둡니다.
나중에 빈 배열로 시작하거나 서버에서 받아올 때 근거가 없어지기 때문입니다.

문제 4

```
{
  type Order = { menu: string; memo?: string };
  const o: Order = { menu: "라떼" };
  console.log(o.menu, o.memo);
}
```

출력 라떼 undefined

```
}
```

해설 ① ? 를 붙이면 빠도 됩니다. 안 붙이면 TS2741(missing)이 납니다. 해설 ② 빠면 `undefined` 입니다. `null` 이 아닙니다. 해설 ③ 대가로 memo 의 타입이 `string` | `undefined` 가 됩니다.

`o.memo.length` 를 쓰면 TS18048 로 걸립니다. 문제 110이 그 이야기입니다.

문제 5

```
{
  type Receipt = { readonly id: number; menu: string };
  const r: Receipt = { id: 1001, menu: "라떼" };
  r.menu = "아메리카노";
  console.log(r.id, r.menu);
}
```

출력 1001 아메리카노

```
}
```

해설 ① `readonly` 를 붙이면 `r.id = 2002;` 가

TS2540 Cannot assign to 'id' because it is a read-only property. 로 걸립니다.

해설 ② `menu` 는 `readonly` 가 아니니 바뀌도 됩니다. 속성마다 따로 붙입니다. 해설 ③ `const` 와 헷갈리지 마세요.

`const r` 은 `r` 자체를 다른 객체로 바꾸는 것을 막고,
`readonly id` 는 객체 안의 `id` 를 바꾸는 것을 막습니다.
 JS자료 06단원에서 "`const` 객체도 속성은 바뀐다" 던 그 구멍을 막는 것입니다.

문제 6

```
{
  type Calc = (a: number, b: number) => number;
  const add: Calc = (a, b) => a + b;
  console.log(add(10, 20));
}
```

출력 30

```
}
```

해설 ① 함수 타입은 (매개변수) $\Rightarrow$ 반환값 입니다. 콜론이 아니라 화살표입니다. 해설 ② `(a, b)` 에 타입을 안 적은 것에 주목하세요.

왼쪽에 : Calc 라고 적어 두었으니 문맥에서 알아냅니다.

해설 ③ 같은 함수 타입을 두 번 이상 쓰게 되면 이렇게 이름을 붙입니다.

03단원 개념03 섹션4에서 예고한 것입니다.

문제 7

```
{
  type Menu = { name: string; price: number };
  const parsed: Menu = JSON.parse('{"name": "라떼", "price": 4500}');
  console.log(parsed.name, parsed.price);
}
```

출력 라떼 4500

```
}
```

해설 ① 안 적으면 any 입니다. parsed.아무거나 를 써도 안 걸립니다. 해설 ② : Menu 한 줄로 자동완성과 검사가 살아납니다. 비용 대비 효과가 가장 큰 한 줄입니다. 해설 ③ 다만 이것은 '주장' 이지 '확인' 이 아닙니다.

실제 응답에 price 가 없어도 조용히 통과합니다. 문제 13·15가 그 이야기입니다.

문제 8

```
{
  type Menu = { name: string; price: number };
  const text = '[{"name": "아메리카노", "price": 4000}, {"name": "라떼", "price": 4500}]';
  const list: Menu[] = JSON.parse(text);
  console.log(list.reduce((sum, m) => sum + m.price, 0));
}
```

출력 8500

```
}
```

해설 ① 목록이면 뒤에 [] 를 붙입니다. Menu 라고만 적으면

list.reduce 가 TS2339 로 걸립니다.

해설 ② reduce 의 두 번째 인자 0 을 빼면 안 됩니다.

빈 배열일 때 터지고, 타입도 이상해집니다.

해설 ㉓ for (const m of list) 로 더해도 정답입니다.

문제 9

{ console.log("TS2345");	
출력	TS2345
코드	▶ 출력
console.log("TS2741");	▶ TS2741
console.log("TS2322");	▶ TS2322
}	

해설 ㉑ 객체를 넘길 때 나는 에러는 세 갈래입니다.

종류가 통째로 다름 → TS2345
속성이 빠짐 → TS2741
속성 종류가 다름 → TS2322

해설 ㉒ 02단원의 “담을 때 TS2322, 넘길 때 TS2345” 규칙에서

'속성이 빠진 것' 만 예외입니다. 양쪽 다 TS2741 입니다. 빠진 속성 이름을 꼭 집어 주는 쪽이 쓸모 있기 때문입니다.

해설 ㉓ TS2322 는 객체 전체가 아니라 “비쌘” 자리를 가리킵니다.

열 번호를 보면 정확히 그 값입니다.

문제 10

{ console.log(1);	
출력	1
}	

해설 ㉑ 초과 속성 검사는 ‘그 자리에 직접 쓴 객체’에만 적용됩니다. 해설 ㉒ 2번처럼 변수를 거치면 통과합니다.

직접 쓴 것은 오타일 가능성이 높지만,
이미 만들어져 있는 값은 "필요한 게 다 있으니 됐다" 고 보기 때문입니다.
이런 방식을 구조적 타이핑이라고 합니다. 이름표가 아니라 모양만 봅니다.

해설 ③ 단, 느슨해지는 것은 초과 속성 검사뿐입니다.

속성이 '빠진' 것은 변수를 거쳐도 TS2741 로 걸립니다.

문제 11

```
{
  type Order = { menu: string; memo?: string };
  function showMemo(o: Order) {
    if (o.memo) {
      console.log(o.memo + " (" + o.memo.length + "글자)");
    } else {
      console.log("(메모 없음)");
    }
  }
}
showMemo({ menu: "라떼", memo: "얼음 적게" });
```

출력 얼음 적게 (5글자)

```
showMemo({ menu: "라떼" });
```

출력 (메모 없음)

```
}
```

해설 ① 고치기 전에는 TS18048 'o.memo' is possibly 'undefined'. 가 납니다. 해설 ② if (o.memo) 안쪽에서는 o.memo 가 확실히 문자열입니다.

한 번 확인하면 그 안에서는 몇 번을 써도 됩니다.

해설 ③ 흔한 실수 — o.memo!.length 처럼 느낌표로 넘어가는 것.

"내가 책임진다" 는 뜻이라 any 만큼 위험합니다.
메모 없는 주문이 들어오는 순간 실행 중에 터집니다.

문제 12

```
{
  type Shop = { name: string; seats: number };
  type Menu = { name: string; price: number };
  type Store = { shop: Shop; menus: Menu[] };

  const store: Store = {
    shop: { name: "봄날카페", seats: 24 },
    menus: [
      { name: "아메리카노", price: 4000 },
      { name: "라떼", price: 4500 },
    ],
  };
  console.log(store.menus[1].price);
}
```

출력 4500

```
}
```

해설 ① 만든 타입을 다른 타입 안에서 그대로 쓸 수 있습니다. 해설 ② store.menus[1]. 까지 치면 name / price 가 자동완성으로 뜹니다.

중첩이 깊어져도 끝까지 따라옵니다.

해설 ③ 안쪽이 틀리면 안쪽을 콧 집어 줍니다.

price: "4500원" 이라고 쓰면 Store 전체가 아니라 그 값을 가리킵니다.

문제 13

```
{
  console.log(undefined);
}
```

출력 undefined

```
}
```

해설 ① 검사는 통과합니다. JSON.parse 는 any 를 주고, any 는 어디에나 들어갑니다. 해설 ② : Menu 라고 적는 것은 "이런 모양일 것이다" 라는 내 주장입니다.

타입스크립트는 그 주장을 검사하지 않고 믿습니다.

node 는 타입 표기를 지우고 실행하니, 확인해 주는 코드가 어디에도 없습니다.

해설 ③ 정리하면 —

내가 쓴 코드끼리 안 맞는 것 → 잡아 준다

밖에서 들어온 값이 다른 것 → 못 잡는다

그래서 밖에서 온 값은 쓰기 전에 확인해야 합니다(05단원).

문제 14

```
{
  type Menu = { name: string; price: number; stock?: number };
  const text =
    '[{"name":"아메리카노","price":4000,"stock":3},' +
    '{"name":"아이스티","price":5000,"stock":0},' +
    '{"name":"라떼","price":4500,"stock":1},' +
    '{"name":"품절메뉴","price":9000}]';

  const menus: Menu[] = JSON.parse(text);
  const inStock = menus.filter((m) => m.stock !== undefined && m.stock > 0);
  const sum = inStock.reduce((acc, m) => acc + m.price, 0);
  console.log("재고 있는 메뉴 합계: " + sum + "원");
}
```

출력 재고 있는 메뉴 합계: 8500원

```
}
```

해설 ① 4000(아메리카노) + 4500(라떼) = 8500 입니다.

아이스티는 stock 이 0 이라 빠지고, 품절메뉴는 stock 이 아예 없어 빠집니다.

해설 ② m.stock > 0 만 쓰면 TS18048 로 걸립니다. stock 이 선택 속성이라

undefined 일 수 있기 때문입니다. 먼저 확인해야 합니다.

m.stock != null && m.stock > 0 으로 써도 되고,

05단원을 배우면 더 깔끔한 방법이 나옵니다.

해설 ③ 혼한 실수 — filter((m) => m.stock) 로 끝내는 것.

이러면 stock 이 0 인 아이스티도 falsy 라 빠지긴 하지만,

"0 은 있는 값인데 없는 것처럼 취급" 하는 것이라 의도가 흐려집니다.

JS자료 01단원의 falsy 목록에 0 이 있던 것을 떠올리세요.

문제 15

주석을 풀면 이렇게 됩니다.

① npm run check

조용합니다. 안 걸립니다.

② node 연습문제.ts

```
TypeError: Cannot read properties of undefined (reading 'toFixed')  
→ 여기서 프로그램이 멈춥니다.
```

해설 ① 02단원 개념03의 any 와 똑같은 구조입니다.

"검사는 통과하는데 돌리면 터진다" 입니다.

해설 ② 다른 점은, 이번엔 any 를 쓴 적이 없다는 것입니다.

모양을 성실하게 적었는데도 이렇게 됩니다.
적은 모양이 사실인지는 아무도 확인해 주지 않기 때문입니다.

해설 ③ 그래서 서버·파일·사용자 입력처럼 '밖에서 온 값' 은

쓰기 전에 확인하는 것이 원칙입니다.
확인하는 문법이 05단원의 주제입니다.

05단원 · 연습문제 정답

05단원 연습문제 정답.ts 정답 유니온과 타입 좁히기

RUN node 연습문제_정답.ts

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
console.log("=== 05단원 연습문제 ===");
```

출력 === 05단원 연습문제 ===

문제 1

```
{
  type Id = string | number;
  function printId(id: Id) {
    console.log(id);
  }
  printId(3);
}
```

출력 3

```
printId("A-3");
```

출력 A-3

```
}
```

해설 ① | 는 "둘 중 하나" 입니다. 개수 제한은 없습니다. 해설 ② `console.log` 는 아무거나 받으니까 통과합니다.

`id.toUpperCase()` 를 썼다면 걸렸을 것입니다. 숫자에는 없으니까요.

해설 ③ 유니온 값으로는 '양쪽에 공통으로 있는 것' 만 쓸 수 있습니다.

이게 05단원 전체를 관통하는 규칙입니다.

문제 2

```
{
  type Size = "S" | "M" | "L";
  function printSize(s: Size) {
    console.log("사이즈:", s);
  }
  printSize("M");
}
```

출력 사이즈: M

```
}
```

해설 ① 값을 그대로 적어 | 로 이으면 “이 값들 중 하나만” 이 됩니다. 해설 ② printSize(까지 치면 세 개가 자동완성으로 뜹니다. 외울 필요가 없습니다. 해설 ③ “XL” 을 넘기면 TS2345 로 걸립니다. 오타도 이걸로 잡힙니다.

JS에서는 오타가 조용히 통과해 if 비교가 영원히 false 가 됐습니다.

문제 3

```
{
  function describe(value: string | number) {
    if (typeof value === "string") {
      console.log(value.length);
    } else {
      console.log(value.toFixed(1));
    }
  }
  describe("아메리카노");
}
```

출력 5

```
describe(4500);
```

출력 4500.0

```
}
```

해설 ① if 안에서는 string, else에서는 number로 좁혀집니다. 해설 ② else 쪽이 재미있습니다. “문자열이 아니다”만 알려줬는데

`string` | `number` 에서 `string` 을 빼면 `number` 밖에 안 남으니 알아서 압니다.

해설 ③ 유니온이 셋 이상이면 `else` 에 둘이 남습니다.

그때는 `else if` 로 하나씩 더 걸러야 합니다.

문제 4

```
{
  type Status = "대기" | "조리중" | "완료";
  function message(status: Status): string {
    switch (status) {
      case "대기":
        return "곧 시작합니다";
      case "조리중":
        return "만들고 있어요";
      case "완료":
        return "나왔습니다";
    }
  }
  console.log(message("조리중"));
}
```

출력 만들고 있어요

```
}
```

해설 ① 세 경우를 다 적었으니 "return 이 없는 길" 이 없습니다. 그래서 통과합니다. 해설 ② case 를 하나 빠뜨리면 TS2366 으로 걸립니다.

나중에 `Status` 에 "취소" 를 추가하면 이 함수가 그 자리에서 걸립니다.
→ 고쳐야 할 곳을 타입이 전부 찾아 줍니다. 리터럴 유니온의 가장 큰 이득입니다.

해설 ③ `default` 로 뭉뚱그리면 편하지만 이 알림을 못 받습니다.

상태가 늘어날 수 있는 코드라면 `default` 없이 다 적는 편이 안전합니다.

문제 5

```
{
  function sumAll(v: number | number[]) {
    if (Array.isArray(v)) {
      console.log(v.reduce((a, b) => a + b, 0));
    } else {
      console.log(v);
    }
  }
}
sumAll(5);
```

출력 5

```
sumAll([1, 2, 3]);
```

출력 6

```
}
```

해설 ① `typeof` 로는 배열을 못 가립니다. `typeof []` 는 "object" 입니다.

JS자료 06단원에서 배운 그 함정입니다.

해설 ② `Array.isArray` 는 타입스크립트가 좁히기로 인정해 주는 함수입니다. 해설 ③ `v.reduce` 의 두 번째 인자 0 을 빼면 안 됩니다. 빈 배열일 때 터집니다.

문제 6

```
{
  type Shop = { name: string; owner: { name: string } | null };
  const shop1: Shop = { name: "봄날카페", owner: { name: "홍길동" } };
  const shop2: Shop = { name: "무인카페", owner: null };

  console.log(shop1.owner?.name);
```

출력 홍길동

```
console.log(shop2.owner?.name);
```

출력 undefined

```
}

```

해설 ① ?. 는 "앞엇것이 없으면 거기서 멈추고 `undefined`" 입니다. 해설 ② ?. 없이 `shop2.owner.name` 을 쓰면 TS18047 로 걸립니다.

JS 에서는 검사가 없으니 실행 중에 터지던 자리입니다.

해설 ③ ?. 의 결과에는 `undefined` 가 붙습니다. 문제가 사라진 게 아니라

다음으로 옮겨 간 것입니다. 그래서 문제 7처럼 ?? 와 짝지어 씁니다.

문제 7

```
{
  type Shop = { name: string; owner: { name: string } | null };
  const shop1: Shop = { name: "봄날카페", owner: { name: "홍길동" } };
  const shop2: Shop = { name: "무인카페", owner: null };

  console.log(shop1.owner?.name ?? "주인 없음");

```

출력 홍길동

```
console.log(shop2.owner?.name ?? "주인 없음");

```

출력 주인 없음

```
}

```

해설 ① ?? 는 "왼쪽이 `null` 이나 `undefined` 면 오른쪽" 입니다. 해설 ② 이제 결과에 `undefined` 가 안 붙으니 `const s: string = ...` 에 바로 담깁니다. 해설 ③ ?. 와 ?? 는 거의 항상 짝으로 나옵니다. 한 쌍으로 외우세요.

문제 8

```
{
  type Dog = { kind: "dog"; bark: string };
  type Cat = { kind: "cat"; meow: string };
  type Animal = Dog | Cat;

  function speak(a: Animal): string {
    switch (a.kind) {
      case "dog":
        return a.bark;
    }
  }

```

06단원 연습문제 정답.ts 정답 제네릭

RUN node 연습문제_정답.ts

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
console.log("=== 06단원 연습문제 ===");
```

출력 === 06단원 연습문제 ===

문제 1

```
{
  function last<T>(arr: T[]): T | undefined {
    return arr[arr.length - 1];
  }
  console.log(last(["가", "나", "다"]));
```

출력 다

```
console.log(last([10, 20, 30]));
```

출력 30

```
}
```

해설 ① <T> 는 “T 라는 이름의 타입을 하나 받겠다” 는 뜻입니다.

무엇인지는 부르는 쪽에서 정해집니다.

해설 ② 넘긴 값을 보고 알아내므로 last<string>(…) 처럼 적을 필요가 없습니다. 해설 ③ 반환 타입에 | undefined 를 꼭 적으세요.

빈 배열의 [length-1] 은 undefined 이기 때문입니다.
: T 라고만 적으면 쓰는 쪽이 확인 없이 쓰다가 실행 중에 터집니다.

문제 2

```
{
  function last<T>(arr: T[]): T | undefined {
    return arr[arr.length - 1];
  }
  const empty: string[] = [];
  console.log(last(empty) ?? "없음");
}
```

출력 없음

```
}
```

해설 ① last(empty) 는 string | undefined 입니다. ?? 로 마무리하면 string 이 됩니다. 해설 ② const empty = []; 처럼 타입 없이 쓰면 암시적 any[] 가 되어

TS7034 · TS7005 로 걸립니다. 검사를 꺾을 때와 같은 상태가 됩니다.
빈 배열로 시작할 때는 타입을 적어야 합니다(02단원 개념02 섹션5).

해설 ③ ?? 대신 if (v !== undefined) 로 확인해도 됩니다. 05단원 개념03입니다.

문제 3

```
{
  function pair<A, B>(a: A, b: B): [A, B] {
    return [a, b];
  }
  const [name, price] = pair("라떼", 4500);
  console.log(name, price);
}
```

출력 라떼 4500

```
}
```

해설 ① 타입 매개변수는 쉼표로 여러 개 받습니다. <A, B> 든 <T, U> 든 이름은 자유입니다. 해설 ② [A, B] 는 튜플입니다. 개수와 순서가 정해져 있고 자리마다 타입이 다릅니다.

name 은 string, price 는 number 로 정확히 나뉩니다.

해설 ③ 07단원의 useState 가 정확히 이 모양을 돌려줍니다.

const [count, setCount] = useState(0) 의 그 대괄호가 튜플 구조분해입니다.

문제 4

```
{
  function makeEmpty<T>(): T[] {
    return [];
  }
  const nums = makeEmpty<number>();
  nums.push(3);
  console.log(nums[0] + 1);
}
```

출력 4

```
}
```

해설 ① 넘기는 값이 없으니 T 를 알아낼 근거가 없습니다. 그럴 때만 직접 적습니다. 해설 ② 안 적으면 T 가 unknown 이 되고, nums[0] + 1 이

TS2571 Object is of type 'unknown'. 으로 걸립니다.
(이름 붙은 변수였다면 TS18046 입니다. 개념01 섹션5의 TS2532/TS18048 과 같은 구분입니다)
push(3) 까지는 통과해서 더 헛갈립니다. 꺼내 쓸 때 드러납니다.

해설 ③ 규칙 — 알아낼 수 있으면 말기고, 없으면 적습니다.

굳이 적으면 코드만 길어집니다.

문제 5

```
{
  function getLength<T extends { length: number }>(value: T): number {
    return value.length;
  }
  console.log(getLength("아메리카노"));
}
```

출력 5

```
console.log(getLength([1, 2, 3]));
```

출력 3

```
}
```

해설 ① 조건 없는 T 로는 아무것도 못 합니다. T 는 '모든 타입' 이라

숫자일 수도 있고, 숫자에는 length 가 없기 때문입니다.
05단원의 "공통으로 있는 것만" 규칙과 같은 이유입니다.

해설 ② extends 는 상속이 아니라 "적어도 이걸 갖고 있어야 한다" 로 읽으세요. 해설 ③ getLength(123) 은 부르는 쪽에서 TS2345 로 막힙니다.

함수 안이 아니라 부르는 자리에서 막아 주는 것이 핵심입니다.

문제 6

```
{
  type ApiResponse<T> = { ok: boolean; data: T };
  const result: ApiResponse<string> = { ok: true, data: "봄날카페" };
  console.log(result.data.length);
}
```

출력 4

```
}
```

해설 ① type 에도 <T> 를 붙일 수 있습니다. 쓸 때 <> 안에 답을 것을 적습니다. 해설 ② data 가 string 이라고 정해 줬으니 .length 가 통과합니다.

<number> 였다면 .toFixed 가 되고 .length 는 걸립니다.

해설 ③ 서버 응답이 언제나 { ok, data } 모양이라면

이렇게 한 번 만들어 두고 data 자리만 바꿔 쓰면 됩니다.

문제 7

```
{
  const menus: Array<string> = ["아메리카노", "라떼"];
  console.log(menus.length);
}
```

출력 2

```
}
```

해설 ① string[] 과 Array<string> 은 완전히 같습니다. 서로 오갈 수도 있습니다. 해설 ② 즉 02단원에서 string[] 을 배웠을 때 이미 제네릭을 쓰고 있었던 것입니다. 해설 ③ 이 자료는 짧은 string[] 쪽을 씁니다.

Array<...> 는 남의 코드에서 볼 때 알아보기만 하면 됩니다.

문제 8

```
{
  async function loadCount(): Promise<number> {
    return 42;
  }
  console.log(await loadCount());
}
```

출력 42

```
}
```

해설 ① async 함수의 반환 타입은 반드시 Promise<> 로 감싸야 합니다.

: number 라고만 쓰면 TS1064 로 걸리고, 메시지가 답까지 알려 줍니다.

해설 ② await 하면 Promise 가 벗겨지고 알맹이만 남습니다.

```
loadCount()      → Promise<number>
await loadCount() → number
```

해설 ③ await 를 빠뜨리면 Promise<number> 에 숫자 기능을 쓰려다 걸립니다.

JS 에서는 [object Promise] 가 조용히 찍히던 자리입니다(문제 14).

문제 9

```
{
  const stock = new Map<string, number>();
  stock.set("아메리카노", 12);
  console.log(stock.get("아메리카노") ?? 0);
}
```

출력 12

```
console.log(stock.get("없는메뉴") ?? 0);
```

출력 0

```
}
```

해설 ① Map<K, V> 는 열쇠와 값 두 개를 받습니다. K 는 Key, V 는 Value 입니다. 해설 ② get 의 반환 타입은 V | undefined 입니다.

없는 열쇠를 물어볼 수 있으니 사실대로 적힌 것입니다.
문제 1의 last 와 같은 이유입니다.

해설 ③ ?? 0 을 안 붙이고 + 1 을 하면 TS2532 Object is possibly 'undefined'. 입니다.

문제 10

```
{
  console.log("돌려받은 값의 타입이 살아 있어서");
}
```

출력 돌려받은 값의 타입이 살아 있어서

```
}
```

해설 ① any[] 로 만들면 함수는 하나로 줄지만 돌려받은 값이 any 가 됩니다.

그러면 무엇을 해도 안 막히고, 실행할 때 터집니다.

해설 ② 제네릭은 함수도 하나이고 검사도 살아 있습니다. 둘 다 얻습니다. 해설 ③ 제네릭은 “무엇이든 받는다” 지 “검사를 끈다” 가 아닙니다.

이 구분이 06단원 전체의 요점입니다.

문제 11

```
{
  console.log("string[]");
}
```

출력 string[]

```
console.log("number[]");
```

출력 number[]

```
}
```

해설 ① map 은 콜백이 돌려주는 것에 따라 결과 타입이 바뀝니다.

n + "원" 이 문자열이니 string[] 입니다.

해설 ② filter 는 걸러 내기만 하고 종류를 바꾸지 않으니 원래 타입 그대로입니다. 해설 ③ 이 둘이 다르게 동작하는 이유가 제네릭입니다.

map<U> 는 U 를 콜백에서 새로 받고, filter 는 T 를 그대로 씁니다.

문제 12

```
{
  type Menu = { id: number; name: string };
  type User = { id: number; email: string };
  const menus: Menu[] = [
    { id: 1, name: "아메리카노" },
    { id: 2, name: "라떼" },
  ];
  const users: User[] = [{ id: 7, email: "a@b.c" }];

  function findById<T extends { id: number }>(items: T[], id: number): T | undefined {
    return items.find((item) => item.id === id);
  }

  console.log(findById(menus, 2)?.name ?? "없음");
}
```

출력 라떼

코드

▶ 출력

console.log(findById(users, 7)?.email ?? "없음");

▶ a@b.c

console.log(findById(menus, 99)?.name ?? "없음");

▶ 없음

}

해설 ① 함수는 하나인데 Menu 도 User 도 찾습니다.

그런데 돌려받은 값에는 각각 제대로 된 타입이 붙어 있습니다.

해설 ② findById(menus, 2)?.email 을 쓰면

TS2339 Property 'email' does not exist on type 'Menu'. 로 걸립니다.
any 로 만들었다면 이게 조용히 통과했을 것입니다.

해설 ③ extends { id: number } 가 없으면 item.id 에서 걸립니다.

T 로는 id 가 있는지 알 수 없기 때문입니다.

문제 13

```
{
  type Menu = { id: number; name: string };
  type ApiResponse<T> = { ok: boolean; data: T };
  const menus: Menu[] = [
    { id: 1, name: "아메리카노" },
    { id: 2, name: "라떼" },
  ];

  async function load<T>(data: T): Promise<ApiResponse<T>> {
    return { ok: true, data };
  }

  const res = await load(menus);
  console.log(res.data[0]?.name ?? "비어 있음");
}
```

출력 아메리카노

```
}
```

해설 ① Promise<ApiResponse<T>> 처럼 제네릭 안에 제네릭이 들어갑니다.

읽는 법은 안에서부터입니다 - "T 를 담은 ApiResponse 를 담은 Promise".

해설 ② await 로 Promise 를 벗기면 ApiResponse<Menu[]> 가 남고,

res.data 는 Menu[] 입니다. 아무것도 안 적었는데 끝까지 따라옵니다.

해설 ③ res.data[0] 에 ?. 를 붙인 이유 — 배열의 [0] 은 빈 배열이면 undefined 입니다.

타입스크립트는 기본 설정에서 이걸 안 잡아 주므로(noUncheckedIndexedAccess 를 켜야 잡습니다) 습관으로 ?. 를 붙이는 편이 안전합니다.

문제 14

```
{
  type Cafe = { name: string; open: boolean; seats: number };
  const cafe: Cafe = { name: "봄날카페", open: true, seats: 24 };

  function getField<T, K extends keyof T>(item: T, key: K): T[K] {
    return item[key];
  }

  console.log(getField(cafe, "name"));
}
```

출력 봄날카페

코드	▶ 출력
console.log(getField(cafe, "seats"));	▶ 24
console.log(getField(cafe, "name").length);	▶ 4

```
}
```

해설 ① key: string 으로 받으면 TS7053 입니다.

Cafe 에 없는 이름이 올 수도 있어서 무엇을 돌려줄지 모르기 때문입니다.

해설 ② K extends keyof T 는 "T 의 속성 이름 중 하나" 라는 뜻입니다.

keyof Cafe 는 "name" | "open" | "seats" 입니다.

해설 ③ 돌려주는 타입이 T[K] 라 "name" 이면 string, "seats" 면 number 가 옵니다.

그래서 마지막 줄의 .length 가 그대로 통과합니다.
"seats" 에 .length 를 쓰면 걸립니다. 넘긴 이름에 따라 달라집니다.

문제 15

```
{
  type Book = { title: string; stock: number };
  const books: Book[] = [
    { title: "타입 입문", stock: 3 },
    { title: "제네릭 연습", stock: 0 },
    { title: "실전 TS", stock: 7 },
  ];
}
```


09단원 ·

07단원 종합 04 정답 주문 화면

보기: npm run dev → 왼쪽 목록에서 고르기

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
import { useState } from "react";
import type { ReactNode } from "react";
import Summary from "../_ui/Summary.tsx";
```

문제 1 props 에 타입 (07단원 개념01)

```
type Menu = { id: number; name: string; price: number };

type MenuItemProps = {
  menu: Menu;
  onAdd: (menu: Menu) => void;
};

function MenuItem({ menu, onAdd }: MenuItemProps) {
  return (
    <li>
      {menu.name} - {menu.price}원{" "}
      <button type="button" onClick={() => onAdd(menu)}>
        담기
      </button> </li>
    );
}
```

해설 ① props 는 객체 하나이므로 04단원의 객체 타입을 그대로 씁니다. 해설 ② 함수 props 는 (인자) ⇒ 반환값 입니다. 돌려줄 게 없으면 ⇒ void. 해설 ③ 안 적으면 TS7031(Binding element ... implicitly has an 'any' type)입니다.

문제 2 children 받는 상자 (07단원 개념01 섹션4)

```
type PanelProps = { title: string; children: ReactNode };

function Panel({ title, children }: PanelProps) {
  return (
    <div style={{ border: "1px solid #ddd", borderRadius: 8, padding: 12,
marginBottom: 12 }}> <h3 style={{ margin: "0 0 8px" }}>{title}</h3>
      {children}
    </div>
  );
}
```

해설 ① children 은 자동으로 안 생깁니다. 직접 적어야 합니다. 해설 ② ReactNode 는 “화면에 그릴 수 있는 것 전부” 입니다. 해설 ③ import type 을 쓰면 “타입만 가져온다” 가 분명해집니다.

문제 3 화면 상태 (판별 유니온)

```
type CartItem = { menu: Menu; count: number };
```

서버라고 치는 함수입니다. 400ms 뒤에 JSON 문자열을 돌려줍니다. 총액이 20000원 이상이면 서버가 이상한 것을 보내는 상황을 흉내 냅니다. (new Promise 는 안 배웠습니다. 그냥 쓰세요 — await 하면 문자열이 나옵니다)

```
function 주문전송(총액: number): Promise<string> {
  const 응답 =
    총액 >= 20000
      ? "<html>502 Bad Gateway</html>"
      : `{"ok":true,"orderId":${1000 + (총액 % 1000)}}`;
  return new Promise((resolve) => setTimeout(() => resolve(응답), 400));
}

type Status = "대기" | "조리중" | "완료";

type OrderState =
  | { status: "시작전" }
  | { status: "보내는중" }
  | { status: "성공"; total: number }
  | { status: "실패"; message: string };

const MENUS: Menu[] = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "카페라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
];

const STATUSES: Status[] = ["대기", "조리중", "완료"];

function statusText(s: Status): string {
  switch (s) {
    case "대기":
      return "곧 시작합니다";
    case "조리중":
      return "만들고 있어요";
    case "완료":
      return "나왔습니다";
  }
}

function OrderView({ state }: { state: OrderState }) {
```

```

switch (state.status) {
  case "시작전":
    return <p>담고 주문 버튼을 눌러 보세요.</p>;
  case "보내는중":
    return <p>보내는 중...</p>;
  case "성공":
    return <p style={{ color: "#2d6cdf" }}>주문 완료 - {state.total}원</p>;
  case "실패":
    return <p style={{ color: "#c00" }}>오류: {state.message}</p>;
}
}

```

해설 ① "성공"에만 total 이, "실패"에만 message 가 있습니다.

성공인데 오류 메시지가 있는 상태를 아예 못 만듭니다.

해설 ② case 안에서 state.total 을 확인 없이 씁니다. 타입에 적혀 있으니까요. 해설 ③ 상태를 하나 늘리면 이 switch 가 TS2366 으로 걸려서 고칠 곳을 알려 줍니다.

문제 4 useState 세 가지 (07단원 개념02)

정리 —————

- 1 props 는 객체 타입 그대로. 함수 props 는 (인자) ⇒ void.
- 2 children 은 직접 적는다 — children: ReactNode.
- 3 useState 에 타입을 적어야 하는 세 경우가 이 화면에 다 나온다.
- 4 빈 배열 useState([]) · null useState(null)
- 5 리터럴 유니온 useState("대기")
- 6 이벤트는 인라인이면 안 적고, 따로 빼면 적는다.
- 7 화면 상태는 판별 유니온 하나로 묶고 switch 로 그린다.
- 8 any · ! · as 를 한 번도 안 썼다.

```
export default function Page() {
  const [cart, setCart] = useState<CartItem[]>([]); // 빈 배열로 시작 const
  [selected, setSelected] = useState<Menu | null>(null); // null 로 시작 const
  [status, setStatus] = useState<Status>("대기"); // 리터럴 유니온 const [order,
  setOrder] = useState<OrderState>({ status: "시작전" });
  const [memo, setMemo] = useState("");

  const total = cart.reduce((sum, item) => sum + item.menu.price * item.count, 0);
```

문제 5 장바구니 담기 (불변 갱신)

```
function addToCart(menu: Menu) {
  setSelected(menu);
  const found = cart.find((c) => c.menu.id === menu.id);
  if (found === undefined) {
    setCart([...cart, { menu, count: 1 }]);
    return;
  }
  setCart(cart.map((c) => (c.menu.id === menu.id ? { ...c, count: c.count + 1 } :
  c)));
}
```

문제 6 품과 입력 (07단원 개념03)

```
async function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
  e.preventDefault();
  if (cart.length === 0) {
    setOrder({ status: "실패", message: "장바구니가 비었습니다" });
    return;
  }

  setOrder({ status: "보내는중" });
  const text = await 주문전송(total);
```

여기서부터는 종합03 문제2와 똑같습니다. 받은 것은 그냥 문자열입니다.

```

let data: unknown;
try {
  data = JSON.parse(text);
} catch {
  setOrder({ status: "실패", message: "서버 응답을 알아볼 수 없습니다" });
  return;
}

if (typeof data === "object" && data !== null && "ok" in data && data.ok ===
true) {
  setOrder({ status: "성공", total });
} else {
  setOrder({ status: "실패", message: "주문이 접수되지 않았습니다" });
}
}

function handleMemo(e: React.ChangeEvent<HTMLInputElement>) {
  setMemo(e.target.value);
}

return (
  <div>
    <h2>종합 04 정답 - 주문 화면</h2> <Panel title="메뉴"> <ul>
      {MENUS.map((m) => (
        <MenuItem key={m.id} menu={m} onAdd={addToCart} />
      ))}
    </ul> <p>마지막으로 고른 것: {selected?.name ?? "없음"}
  </p> </Panel> <Panel title="장바구니">
    {cart.length === 0 ? (
      <p>비어 있습니다.</p>
    ) : (

```

```

        <ul>
          {cart.map((c) => (
            <li key={c.menu.id}>
              {c.menu.name} x{c.count} = {c.menu.price * c.count}원
            </li>
          ))}
        </ul>
      )}
      <p>합계: {total}원</p>
      <button type="button" onClick={() => setCart([])}>
        비우기
      </button>
    </Panel>

    <Panel title="주문"> <form onSubmit={handleSubmit}> <input value=
    {memo} onChange={handleMemo} placeholder="메모 (선택)" />{ " "}
      <button type="submit">주문하기</button> </form> <p>메모: {memo.trim() ===
    "" ? "(없음)" : memo}</p> <OrderView state={order} /> </Panel> <Panel title="상태">
    {STATUSES.map((s) => (
      <button key={s} type="button" onClick={() => setStatus(s)}
        style={{ marginRight: 6 }}
      >
        {s}
      </button>
    ))}
    <p>{statusText(status)}</p> </Panel>

  </div>
);
}

```

해설 ① useState 에 타입을 적어야 하는 세 경우가 이 화면에 전부 나옵니다.

useState([]) 로 두면 **never[]** 가 되어 setCart 가 통째로 막힙니다.
 useState(null) 로 두면 null 전용 상자가 되어 setSelected(menu) 가 걸립니다.
 useState("대기") 로 두면 **string** 이 되어 statusText(status) 가 TS2345 로 걸립니다.

해설 ② e.preventDefault() 를 빠뜨리면 화면이 새로고침되면서 장바구니가 사라집니다.

타입은 이걸 안 막아 줍니다. 01단원의 "타입이 못 잡는 것" 입니다.

해설 ③ OrderView 의 switch 는 네 상태를 다 덮습니다.

상태를 하나 늘리면 그 자리에서 걸려서, 고칠 곳을 타입이 찾아 줍니다.
 반면 statusText 는 Status 를 늘릴 때 걸립니다. 서로 독립적입니다.

07단원 연습문제 정답.tsx 정답 React와 타입스크립트

보기: npm run dev → 왼쪽 목록에서 "연습문제 정답" 고르기

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
import { useState } from "react";
import type { ReactNode } from "react";
import Summary from "../_ui/Summary.tsx";
```

문제 1

```
type MenuItemProps = { name: string; price: number };

function MenuItem({ name, price }: MenuItemProps) {
  return (
    <li>
      {name} - {price}원
    </li>
  );
}
```

해설 ① 안 붙이면 TS7031 Binding element 'name' implicitly has an 'any' type.

03단원의 TS7006 과 형제입니다. 구조분해한 자리라 이름만 다릅니다.

해설 ② props 는 객체 하나이므로 04단원의 객체 타입을 그대로 씁니다. 해설 ③ 이제 <MenuItem name="라떼" /> 처럼 빠뜨리면 그 자리에서 걸립니다.

JS 였다면 화면에 "undefined원" 이 찍히고 나서야 알았습니다.

문제 2

```
type BadgeProps = { text: string; color?: string };

function Badge({ text, color = "#555" }: BadgeProps) {
  return (
    <span style={{ background: color, color: "#fff", padding: "2px 8px",
borderRadius: 4 }}>
      {text}
    </span>
  );
}
```

해설 ① ? 를 붙이면 안 넘겨도 됩니다. 04단원 개념03 섹션1 그대로입니다. 해설 ② 기본값(= "#555")을 주면 함수 안에서 color 가 string 입니다.

undefined 가 안 섞이니 style 에 바로 넣을 수 있습니다.
? 만 붙이고 기본값을 안 주면 string | undefined 라 배경색이 사라집니다.

해설 ③ 03단원 개념02의 "쓸 만한 기본값이 있으면 ? 보다 기본값" 이 여기서도 통합니다.

문제 3

```
type BoxProps = { title: string; children: ReactNode };

function Box({ title, children }: BoxProps) {
  return (
    <div style={{ border: "1px solid #ddd", borderRadius: 8, padding: 12,
marginBottom: 12 }}> <h3 style={{ margin: "0 0 8px" }}>{title}</h3>
      {children}
    </div>
  );
}
```

해설 ① children 은 자동으로 생기지 않습니다. props 에 직접 적어야 합니다.

(예전 React.FC 를 쓰던 시절에는 자동이었지만 지금은 아닙니다)

해설 ② ReactNode 는 "화면에 그릴 수 있는 것 전부" 입니다.

글자 · 숫자 · 태그 · 배열 · null 이 다 들어갑니다.

해설 ③ import type { ReactNode } 처럼 type 을 붙이면

"이건 타입만 가져오는 것" 이 분명해집니다.

문제 4

```
type Menu = { id: number; name: string };

function Cart() {
  const [items, setItems] = useState<Menu[]>([]);

  return (
    <div> <button type="button" onClick={() => setItems([...items, { id: items.length
+ 1, name: "라떼" }])}>
      >
      담기 ({items.length})
    </button> <ul>
      {items.map((m) => (
        <li key={m.id}>{m.name}</li>
      ))}
    </ul> </div>
  );
}
```

해설 ① `useState([])` 는 `never[]` 로 잡힙니다.

`never` 는 "값이 있을 수 없는 타입" 이라 아무것도 못 담습니다.

해설 ② 에러가 두 군데에서 납니다 — 담을 때(TS2322 ... not assignable to type 'never')와

꺼낼 때(TS2339 Property 'id' does not exist on type 'never').
원인은 하나인데 증상이 여러 개인 전형적인 경우입니다.
01단원 개념03 섹션3의 "맨 위부터 하나씩" 이 여기서 통합니다.

해설 ③ 에러에 `never` 가 보이면 `useState([])` 를 의심하세요. 이게 07단원 최다 질문입니다.

문제 5

```
function Selected() {
  const [selected, setSelected] = useState<Menu | null>(null);

  return (
    <div> <button type="button" onClick={() => setSelected({ id: 1, name: "라떼" })}>
      라떼 고르기
    </button> <p>고른 것: {selected?.name ?? "없음"}</p> </div>
  );
}
```

해설 ① `useState(null)` 만 쓰면 "null 만 담는 상자" 로 정해집니다.

나중에 값을 넣으려 하면 걸립니다.

해설 ② `<Menu | null>` 을 적으면 둘 다 담깁니다.

대신 `selected` 를 쓸 때 확인이 필요합니다. 05단원 개념03 그대로입니다.

해설 ③ `selected?.name ?? "없음"` 이 그 확인입니다.

`selected!.name` 으로 넘어가면 안 됩니다. 아직 아무것도 안 골랐을 때 터집니다.

문제 6

```
function TextInput() {
  const [text, setText] = useState("");

  function handleChange(e: React.ChangeEvent<HTMLInputElement>) {
    setText(e.target.value);
  }

  return (
    <div> <input value={text} onChange={handleChange} placeholder="메뉴 이름" /> <p>
      {text.length}자</p> </div>
  );
}
```

해설 ① 함수를 밖으로 빼면 문맥이 없어서 TS7006 이 납니다.

태그 안에 인라인으로 썼다면 안 적어도 됐습니다.

해설 ② <HTMLInputElement> 를 꼭 적어야 e.target.value 가 통과합니다.

어느 태그에서 난 이벤트인지 알려 줘야 target 에 무엇이 있는지도 압니다.

해설 ③ 타입 이름을 외우지 마세요.

onChange={e => ...} 로 인라인으로 써 보고 e 에 마우스를 올리면
이름이 그대로 뜹니다. 그걸 베끼면 됩니다.

문제 7

```
type Status = "대기" | "조리중" | "완료";
const STATUSES: Status[] = ["대기", "조리중", "완료"];

function statusLabel(s: Status): string {
  return s === "완료" ? "다 됐습니다" : s;
}

function StatusPicker() {
  const [status, setStatus] = useState<Status>("대기");

  return (
    <div>
      {STATUSES.map((s) => (
        <button key={s} type="button" onClick={() => setStatus(s)} style={{
marginRight: 6 }}>
          {s}
        </button>
      ))}
      <p>현재: {statusLabel(status)}</p> </div>
    );
  }
}
```

해설 ① useState("대기") 는 string 으로 넓혀 잡습니다.

02단원 개념02 섹션3에서 let 이 그랬던 것과 같은 이유입니다.
상태는 바뀌는 값이니 넉넉하게 잡는 것입니다.

해설 ② 그러면 statusLabel(status) 가 TS2345 로 걸립니다.

string 에는 "취소" 같은 것도 들어갈 수 있으니까요.

해설 ③ <Status> 를 적어 두면 setStatus("취소") 같은 오타도 그 자리에서 막힙니다.

useState 에 타입을 적어야 하는 세 경우 중 하나입니다.

문제 8

```
function OrderForm() {
  const [menu, setMenu] = useState("");
  const [orders, setOrders] = useState<string[]>([]);

  function handleSubmit(e: React.FormEvent<HTMLFormElement>) {
    e.preventDefault();
    if (menu.trim() === "") return;
    setOrders([...orders, menu]);
    setMenu("");
  }

  return (
    <form onSubmit={handleSubmit}> <input value={menu} onChange={(e) =>
setMenu(e.target.value)} placeholder="주문할 메뉴" />{" "}
    <button type="submit">주문</button> <ul>
      {orders.map((o, i) => (
        <li key={i}>{o}</li>
      ))}
    </ul> </form>
  );
}
```

해설 ① 품은 `React.FormEvent<HTMLFormElement>` 입니다. 해설 ② `e.preventDefault()` 를 빠뜨리면 화면이 새로고침되면서

지금까지 쌓은 목록이 전부 사라집니다. `React`자료 06단원에서 겪은 그것입니다. 타입은 이걸 안 막아 줍니다. 01단원의 "타입이 못 잡는 것" 그대로입니다.

해설 ③ `onChange` 쪽의 `(e)` 는 인라인이라 안 적었습니다.

같은 파일 안에서도 적는 곳과 안 적는 곳이 갈립니다. 기준은 '문맥이 있느냐' 입니다.

문제 9

```
type StepperProps = {
  label: string;
  onStep: (next: number) => void;
};
```

```
function Stepper({ label, onStep }: StepperProps) {
  const [n, setN] = useState(0);

  function handleClick() {
    const next = n + 1;
    setN(next);
    onStep(next);
  }

  return (
    <button type="button" onClick={handleClick}>
      {label}: {n}
    </button>
  );
}
```

해설 ① 함수 props 는 03단원의 함수 타입 그대로입니다. (인자) ⇒ 반환값. 해설 ② 돌려줄 것이 없으면 ⇒ void 입니다. 해설 ③ 부모가 onStep={setLast} 를 넘깁니다.

setLast 의 타입은 Dispatch<SetStateAction<number>> 인데
(next: number) => void 자리에 들어갑니다.

받는 쪽이 넉넉하면 통과하는 것 - 04단원 개념02의 "개수가 모자란 것은 봐준다" 와
같은 원리(구조적 타이핑)입니다.

문제 10

```

type LoadState =
  | { status: "시작전" }
  | { status: "로딩중" }
  | { status: "성공"; data: string[] }
  | { status: "실패"; message: string };

function Loader() {
  const [state, setState] = useState<LoadState>({ status: "시작전" });

  function load(shouldFail: boolean) {
    setState({ status: "로딩중" });
    setTimeout(() => {
      if (shouldFail) setState({ status: "실패", message: "서버가 응답하지 않습니다"
    });
      else setState({ status: "성공", data: ["아메리카노", "라떼"] });
    }, 400);
  }

  return (
    <div> <button type="button" onClick={() => load(false)}>
      성공
    </button>{" "}
    <button type="button" onClick={() => load(true)}>
      실패
    </button> <LoaderView state={state} /> </div>
  );
}

function LoaderView({ state }: { state: LoadState }) {
  switch (state.status) {
    case "시작전":
      return <p>버튼을 눌러 보세요.</p>;
    case "로딩중":
      return <p>불러오는 중...</p>;
    case "실패":
      return <p style={{ color: "#c00" }}>오류: {state.message}</p>;
    case "성공":
      return <p>{state.data.length}개: {state.data.join(", ")}</p>;
  }
}

```

해설 ① 상태 세 개가 하나로 줄었습니다.

`setError(null)` 을 빠뜨려서 옛 오류가 남는 사고가 구조적으로 안 생깁니다.

해설 ② 화면 조건이 `!loading && error === null` 같은 '아닌 것' 의 나열에서

`switch` 하나로 바뀌었습니다. 읽기도 쉽고 빠뜨릴 자리도 없습니다.

해설 ③ 여기에 `{ status: "빈결과" }` 를 추가해 보세요.

`LoaderView` 의 `switch` 가 TS2366 으로 걸립니다.

"고쳐야 할 곳을 타입이 전부 찾아 준다" - 이게 이 패턴을 쓰는 진짜 이유입니다.

화면

정리

- 1 props 타입을 안 붙이면 TS7031(Binding element). TS7006 과 형제다.
- 2 children 은 자동으로 안 생긴다. children: ReactNode 를 직접 적는다.
- 3 `useState([])` 는 `never[]` 가 된다. 에러에 never 가 보이면 여기를 의심한다.
- 4 `useState(null)` 은 `null` 전용 상자가 된다. 을 적는다.
- 5 `useState("대기")` 는 string 이 된다. 리터럴 유니온은 를 적는다.
- 6 이벤트 타입은 인라인으로 써 보고 마우스를 올려 베킨다.
- 7 화면 상태는 판별 유니온 하나로 묶는다. switch 하나로 그려진다.

```
export default function Page() {
  const [last, setLast] = useState(0);

  return (
    <div>
      <h2>07단원 연습문제 정답</h2> <p>문제
1</p> <ul> <MenuItem name="아메리카노" price={4000} /> </ul> <p>문제
2</p> <Badge text="기본색" /> <Badge text="지정색" color="#2d6cdf" /> <p>문제
3</p> <Box title="상자 제목">여기가 children 자리입니다.</Box> <p>문제 4</p> <Cart
/> <p>문제 5</p> <Selected /> <p>문제 6</p> <TextInput /> <p>문제 7</p> <StatusPicker
/> <p>문제 8</p> <OrderForm /> <p>문제 9</p> <Stepper label="더하기" onStep={setLast}
/> <p>마지막 값: {last}</p>
```



```
    <p>문제 10</p>
    <Loader />

  </div>
);
}
```

08단원 연습문제 정답.ts 정답 기존 프로젝트에 타입 입히기

```
RUN node 연습문제_정답.ts
```

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

문제 11~14에서 쓰는 모듈입니다(개념04).

```
import { getDiscounted, getShipping, DEFAULT_PERCENT } from "../타입없는_모듈/
할인계산기.js";

console.log("=== 08단원 연습문제 ===");
```

출력 === 08단원 연습문제 ===

문제 1

```
{
  console.log(undefined);
```

출력 undefined

```
}
```

해설 ① as 는 값을 바꾸지 않습니다. 타입스크립트의 생각만 바꿉니다.

JSON 에 price 가 없으니 실제로는 undefined 입니다.

해설 ② 검사는 조용합니다. m.price.toFixed(0) 을 쓰면 실행할 때 터집니다.

02단원의 any, 05단원의 ! 와 완전히 같은 구조입니다.

해설 ③ 04단원 개념04의 "타입은 약속이지 검사가 아니다" 를

as 로 한 번 더 확인한 것입니다.

문제 2

```
{
  type Menu = { name: string; price: number };
  const menu = { name: "아메리카노", price: 4000 } satisfies Menu;
  console.log(menu.name);
}
```

출력 아메리카노

```
}
```

해설 ① as 는 "Menu 라고 쳐 줘"(검사 안 함),

`satisfies` 는 "Menu 가 맞는지 봐 줘"(검사 함) 입니다.

해설 ② `satisfies` 는 확인만 하고 변수의 타입은 안 바꿉니다.

여기 `Menu` 는 `name` 이 `string` 이라 : `Menu` 와 결과가 같습니다.
차이는 목표 타입이 할당할 때 드러납니다(개념01 섹션4의 설정 예).

해설 ③ 값이 눈앞에 있을 때는 `as` 대신 `satisfies` 나 : 타입 을 쓰세요.

문제 3

```
{
  type Menu = { name: string; price: number };
  const menu = { name: "라떼", price: 4500 } satisfies Menu;
  console.log(menu.price);
}
```

출력 4500

```
}
```

해설 ① `satisfies` 로 바꾸면 `size` 자리에서 걸립니다.

```
error TS2353: Object literal may only specify known properties,
and 'size' does not exist in type 'Menu'.
```

재현:

```
type Menu = { name: string; price: number };
const menu = { name: "라떼", price: 4500, size: "L" } satisfies Menu;
void menu;
```

해설 ② as Menu 었을 때는 조용히 통과했습니다.

오타로 sizee 라고 썼어도 아무도 안 알려 줬을 것입니다.

해설 ③ 04단원 개념01 섹션3의 초과 속성 검사가 satisfies 에서도 그대로 동작합니다.

문제 4

```
{  
  const legacy: unknown = " 글자입니다 ";  
  if (typeof legacy === "string") {  
    console.log(legacy.trim());  
  }  
}
```

출력 글자입니다

```
}  
}
```

해설 ① any 었을 때는 legacy.trim() 이 그냥 통과했습니다.

문자열이 아니었다면 실행할 때 터졌을 것입니다.

해설 ② unknown 으로 바꾸면 TS18046 'legacy' is of type 'unknown'. 이 납니다.

확인하고 쓰라는 뜻입니다.

해설 ③ 옮기는 중에 막히면 any 말고 unknown 을 쓰라는 이유가 이것입니다.

any 는 빛이 번지고, unknown 은 그 자리에 갑니다.

문제 5

```
{  
  const o: { name?: string } = {};  
  o.name = "라떼";  
  console.log(o.name);  
}
```

출력 라떼

```
}
```

해설 ① const o = {} 는 "속성이 하나도 없는 객체" 로 추론됩니다.

그래서 `o.name` 이 TS2339 로 걸립니다.

해설 ② JS 에서 아주 흔하던 패턴이라 옮길 때 반드시 만납니다. 해설 ③ 더 나은 방법은 한 번에 다 적는 것입니다.

```
const o = { name: "라떼" };
나중에 붙여야만 한다면 위처럼 타입을 미리 정해 둡니다.
```

문제 6

```
{
  type Status = "대기" | "조리중" | "완료";
  const current: Status = "조리중";
  console.log(current);
}
```

출력 조리중

```
}
```

해설 ① `enum` 이 하려던 일을 리터럴 유니온이 거의 다 합니다. 해설 ② 게다가 실행할 때 아무것도 안 남고 사라집니다.

`enum` 은 진짜 객체를 만들어서 번들에 코드가 남습니다.

해설 ③ 이 자료는 `erasableSyntaxOnly` 로 `enum` 을 아예 막아 두었습니다.

`node` 가 `.ts` 를 실행할 때 `enum` 은 '지우기' 만으로 처리가 안 되기 때문입니다.

문제 7

```
{
  console.log("string");
}
```

출력 string

```
}
```

해설 ① `as unknown as number` 는 “그래도 하겠다” 고 뚫은 것입니다.

타입스크립트는 `number` 라고 믿지만 실제 값은 문자열 그대로입니다.

해설 ② “4500” `as number` 만 쓰면 TS2352 로 말립니다.

그런데 메시지가 "unknown 을 거쳐 가세요" 라고 방법까지 알려 줍니다.
말리면서 뚫는 법을 알려 주는 셈이라 오히려 위험합니다.

해설 ③ 남의 코드에서 as unknown as 를 보면 그 근처를 의심하세요.

거의 항상 설계가 잘못된 자리입니다.

문제 8

```
{  
  function toList(value: string | string[]): string[] {  
    return Array.isArray(value) ? value : [value];  
  }  
  console.log(toList("라떼"));
```

출력 ['라떼']

```
console.log(toList(["라떼", "아메리카노"]));
```

출력 ['라떼', '아메리카노']

```
}
```

해설 ① JS 에서 "문자열도 받고 배열도 받던" 함수를 옮길 때 쓰는 방법입니다. 해설 ② `typeof` 로는 배열을 못 가립니다. `typeof []` 는 "object" 입니다.

`Array.isArray` 를 씁니다(05단원 개념02 섹션3).

해설 ③ 반환 타입 : `string[]` 을 적어 두면

실수로 문자열을 그냥 돌려주는 것을 함수 안에서 잡아 줍니다.

문제 9

```
{  
  console.log("상수·유틸 → 계산 함수 → 데이터 다루는 곳 → 화면");
```

출력 상수·유틸 → 계산 함수 → 데이터 다루는 곳 → 화면

```
}
```

해설 ① 아래에서 위로 올라가는 순서입니다. 해설 ② A 가 B 를 쓰는데 B 에 타입이 없으면, A 를 먼저 옮겨도 B 쪽이 any 라

소득이 적습니다. 반대로 B 를 먼저 옮기면 A 가 아직 .js 여도 덕을 봅니다.

해설 ③ 화면(컴포넌트)이 가장 많이 얹혀 있어 제일 어렵습니다. 마지막에 하세요.

문제 10

```
{
  type CartItem = { name: string; price: number; count: number };

  function getTotal(items: CartItem[]): number {
    let sum = 0;
    for (const item of items) {
      sum += item.price * item.count;
    }
    return sum;
  }

  function withShipping(total: number, shipping: number): number {
    return total + shipping;
  }

  const cart: CartItem[] = [
    { name: "아메리카노", price: 4000, count: 2 },
    { name: "라떼", price: 4500, count: 1 },
  ];
  const shippingText = "3000";

  console.log("총액: " + withShipping(getTotal(cart), Number(shippingText)) + "원");
}
```

출력 총액: 15500원

```
}
```

해설 ① 걸리는 줄은 withShipping(getTotal(cart), shippingText) 입니다.

error TS2345: Argument of type 'string' is not assignable to parameter of type 'number'.

재현:

```
function withShipping(total: number, shipping: number): number { return total + shipping; }
const shippingText = "3000";
console.log(withShipping(12500, shippingText));
```

해설 ② 타입을 안 적었다면(= 옮기기 전 .js 상태) 이 줄이 조용히 통과하고

"125003000원" 이 나왔을 것입니다. 옮기기_예제/이전.js 로 직접 확인해 보세요.

해설 ③ 고친 것은 Number() 하나뿐입니다.

중요한 것은 '어디를' 고쳐야 하는지를 타입이 정확히 짚어 줬다는 점입니다.
JS 였다면 화면에 이상한 숫자가 찍히고 나서야 찾기 시작했을 것입니다.

문제 11

```
{
  const 할인가 = getDiscounted(4500, DEFAULT_PERCENT);
  console.log(할인가);
```

출력 4050

코드

console.log(getShipping(할인가));

console.log(할인가 + getShipping(할인가));

▶ 출력

▶ 3000

▶ 7050

```
}
```

해설 ① 남이 만든 .js 인데도 계산이 전부 숫자로 이어집니다.

짜이 되는 할인계산기.d.ts 가 있어서입니다.

해설 ② .d.ts 를 지우면 TS7016 이 나고, 이 세 줄이 전부 any 가 됩니다. 해설 ③ getDiscounted(4500, "10") 처럼 문자열을 넘기면 TS2345 로 걸립니다.

타입을 적어 두면 남의 패키지를 쓸 때도 이렇게 막아 줍니다.

문제 12

```
{
  const cart = [
    { price: 4000, count: 2 },
    { price: 4500, count: 1 },
  ];

  const 합계 = cart.reduce((total, item) => total + item.price * item.count, 0);
  console.log(합계);
```

출력 12500

```
const 할인후 = getDiscounted(합계, 20);
console.log(할인후);
```

출력 10000

```
console.log(할인후 + getShipping(할인후));
```

출력 13000

```
}
```

해설 ① reduce 의 시작값 0 이 숫자라 total 도 숫자로 정해집니다(JS자료 08단원). 해설 ② 합계가 number 니까 getDiscounted 에 그대로 넘어갑니다.

중간에 문자열이 섞였다면 여기서 걸렸을 것입니다.

해설 ③ 할인 뒤 금액이 30000 미만이라 배송비 3000 이 붙습니다.

20% 를 DEFAULT_PERCENT 로 바꾸면 $11250 + 3000 = 14250$ 이 됩니다.

문제 13

```
{
  const 주문금액 = [12000, 35000, 8000];

  console.log(주문금액.map((금액) => 금액 + getShipping(금액)));
```

출력 [15000, 35000, 11000]

```
}
```

해설 ① 35000 은 30000 이상이라 배송비가 0 입니다. 그래서 그대로 35000 입니다. 해설 ② map 이 number[] 를 돌려줍니다. getShipping 의 반환이 number 라고

.d.ts 에 적혀 있기 때문입니다.

해설 ③ 만약 .d.ts 에 string 이라고 잘못 적혀 있었어도 출력은 그대로입니다.

.d.ts 는 검사할 때만 읽고 실행에는 아무 영향이 없기 때문입니다.
실제 .js 가 숫자를 돌려주니 + 는 그대로 숫자 덧셈입니다.
위험한 것은 출력이 바뀌는 게 아니라, 검사가 조용한 채로
getShipping(12500).toUpperCase() 같은 줄을 통과시킨다는 점입니다.
그 줄은 실행해야 TypeError 로 터집니다. 개념04 섹션5가 말하는 위험입니다.

문제 14

```
{  
  console.log(typeof getShipping(12500));  
}
```

출력 number

```
console.log(typeof DEFAULT_PERCENT);
```

출력 number

```
}
```

해설 ① typeof 는 실행할 때 진짜 타입을 봅니다. 타입 검사와는 다른 것입니다. 해설 ② .d.ts 에 적은 것과 실체가 같으면 이렇게 number 가 나옵니다.

다르게 적혀 있어도 검사는 조용하니, 이렇게 한 번 찍어 보는 것이
손으로 적은 선언 파일을 확인하는 가장 쉬운 방법입니다.

해설 ③ 04단원 개념04 섹션3의 “타입은 약속이지 검사가 아니다” 가

선언 파일에서는 특히 위험합니다. 약속을 내가 적기 때문입니다.

문제 15

주석을 풀면 이렇게 됩니다.

```
npm run check
```

08_기존_프로젝트에_타입_입히기/연습문제.ts(줄,열): error TS1294: This syntax is not allowed when 'erasableSyntaxOnly' is enabled. 재현:

```
enum Color { Red, Blue }
void Color;
```

해설 ① node 가 .ts 를 실행하는 방식은 '타입을 지우는 것' 뿐입니다.

`enum` 은 지운다고 되는 문법이 아닙니다. 실제로 객체를 만들어야 합니다.

해설 ② 그래서 그냥 두면 실행할 때 이렇게 터집니다.

```
SyntaxError [ERR_UNSUPPORTED_TYPESCRIPT_SYNTAX]:
TypeScript enum is not supported in strip-only mode
검사 단계에서 미리 막아 두면 이 사고를 안 겪습니다.
```

해설 ③ 같은 이유로 클래스의 '매개변수 속성'(constructor(private name: string))도

막혀 있습니다. 둘 다 안 써도 되는 문법입니다.
`enum` 은 05단원의 리터럴 유니온으로 대신합니다(문제 6).

09단원 종합 01 정답 주문 관리

RUN node 종합01_주문_관리_정답.ts

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
const rawMenus = [
  { id: 1, name: "아메리카노", price: 4000 },
  { id: 2, name: "카페라떼", price: 4500 },
  { id: 3, name: "케이크", price: 6000 },
];

const rawOrders = [
  { menuId: 1, count: 2, memo: "얼음 적게" },
  { menuId: 3, count: 1 },
  { menuId: 99, count: 1 },
];

console.log("===== 주문 관리 =====");
```

출력 ===== 주문 관리 =====

문제 1

```
type Menu = { id: number; name: string; price: number };
type Order = { menuId: number; count: number; memo?: string };

const menus: Menu[] = rawMenus;
const orders: Order[] = rawOrders;

console.log("메뉴 " + menus.length + "개 / 주문 " + orders.length + "건");
```

출력 메뉴 3개 / 주문 3건

해설 ① memo 에 ? 를 붙여야 합니다. 세 주문 중 하나만 memo 가 있으니까요.

안 붙이면 rawOrders 를 Order[] 에 담는 순간 TS2741 로 걸립니다.

해설 ② rawMenus 를 그대로 담을 수 있는 이유는 모양이 맞기 때문입니다.

이름표가 아니라 모양만 봅니다(04단원 개념01 섹션4, 구조적 타이핑).

해설 ③ 흔한 실수 — rawMenus 자체에 : Menu[] 를 붙이려 하는 것.

그것도 되지만 문제가 "고치지 마세요" 라고 했으니 새 이름에 답습니다.

문제 2

```
function findMenu(id: number): Menu | undefined {
  return menus.find((m) => m.id === id);
}

console.log(findMenu(2)?.name ?? "없는 메뉴");
```

출력 카페라떼

```
console.log(findMenu(99)?.name ?? "없는 메뉴");
```

출력 없는 메뉴

해설 ① find 는 못 찾으면 undefined 를 줍니다. 반환 타입에 사실대로 적어야 합니다.

: Menu 라고만 적으면 그 자리에서 TS2322 로 막힙니다.
find 가 Menu | undefined 를 주는데 Menu 에 담을 수 없기 때문입니다.
★ 여기는 타입스크립트가 잡아 주는 자리입니다. 거짓말을 아예 못 적습니다.
반대로 종합02의 arr[0] 은 안 잡아 줍니다(그쪽 문제1 해설 ①을 보세요).
같은 "사실대로 적어라" 인데 한쪽만 강제되는 이유 - 05단원 개념03입니다.

해설 ② ?. 와 ?? 를 짝지어 쓰면 한 줄로 끝납니다(05단원 개념03).

if (menu !== undefined) 로 풀어 써도 정답입니다.

해설 ③ 흔한 실수 — findMenu(99)!.name 처럼 ! 로 넘어가는 것.

99번은 실제로 없으니 그 자리에서 터집니다.

문제 3

```
function orderLine(order: Order): string {
  const menu = findMenu(order.menuId);
  if (menu === undefined) {
    return "알 수 없는 메뉴(" + order.menuId + "번)";
  }
  return menu.name + " x" + order.count + " = " + menu.price * order.count + "원";
}

for (const o of orders) {
  console.log(orderLine(o));
}
```

출력 아메리카노 x2 = 8000원
 케이크 x1 = 6000원
 알 수 없는 메뉴(99번)

해설 ① 못 찾은 경우를 먼저 걸러 내고 돌려보냅니다.

그 아래부터는 menu 가 확실히 Menu 라 menu.name 을 그냥 씁니다.
05단원 개념02 섹션5의 '이른 반환' 입니다.

해설 ② if 로 안 감싸고 menu.name 을 쓰면 TS18048 로 걸립니다.

"없을 수도 있는데 그냥 쓰시려고요?" 입니다.

해설 ③ 반환 타입 : string 을 적어 두면 어느 갈래에서든 문자열을 돌려주는지

함수 안에서 확인해 줍니다.

문제 4

```
for (const o of orders) {
  if (o.memo !== undefined) {
    console.log(findMenu(o.menuId)?.name + ": " + o.memo);
  }
}
```

출력 아메리카노: 얼음 적게

해설 ① 이 if 는 타입이 강제하는 게 아니라 '메모 있는 주문만 찍으려고' 두는 것입니다.

o.memo 를 가드 없이 " : " + o.memo 로 이어도 에러가 안 납니다.
문자열 결합 자리에서는 undefined 도 그냥 허용되기 때문입니다.
* 타입이 안 잡아 주는 자리입니다. 빼고 돌려 보면 "undefined" 가 찍힙니다.

해설 ② if (o.memo) 로 써도 지금 데이터에서는 같은 결과입니다.

다만 메모가 "" 인 주문이 들어오면 '없는 것' 으로 취급됩니다.
"빈 메모" 와 "메모 없음" 을 구별해야 한다면 !== undefined 를 쓰세요
(05단원 개념02 섹션4).

해설 ③ orders.filter((o) => o.memo !== undefined) 로 걸러도 정답입니다.

문제 5

```
type Status = "대기" | "조리중" | "완료";

function statusText(status: Status): string {
  switch (status) {
    case "대기":
      return "곧 시작합니다";
    case "조리중":
      return "만들고 있어요";
    case "완료":
      return "나왔습니다";
  }
}

console.log(statusText("대기"));
```

출력 곧 시작합니다

```
console.log(statusText("완료"));
```

출력 나왔습니다

해설 ① 세 경우를 다 적었으니 "return 이 없는 길" 이 없습니다.

하나라도 빠뜨리면 TS2366 으로 걸립니다. 그게 반환 타입을 적는 이유입니다.

해설 ② statusText("취소") 는 TS2345 로 막힙니다. 오타도 이걸로 잡힙니다. 해설 ③ 나중에 Status 에 "취소" 를 추가하면 이 함수가 그 자리에서 걸립니다.

고쳐야 할 곳을 타입이 찾아 주는 것 - 리터럴 유니온의 가장 큰 이득입니다.

문제 6

```
function total(list: Order[], discount: number = 0): number {
  let sum = 0;
  for (const o of list) {
    const menu = findMenu(o.menuId);
    if (menu === undefined) continue;
    sum += menu.price * o.count;
  }
  return Math.round(sum * (1 - discount));
}

console.log(total(orders));
```

출력 14000

```
console.log(total(orders, 0.1));
```

출력 12600

해설 ① $8000 + 6000 = 14000$. 99번은 못 찾으니까 0원으로 건너뛰니다. 해설 ② 할인율에 기본값 0 을 주면 discount 의 타입이 number 그대로입니다.

? 를 붙였다면 `number | undefined` 가 되어 곱하기 전에 확인해야 했습니다.
쓸 만한 기본값이 있으면 ? 보다 기본값입니다(03단원 개념02 섹션3).

해설 ③ `Math.round` 를 쓴 이유 — 할인율에 따라 소수가 나올 수 있어서입니다.

$14000 * 0.9$ 는 마침 정수지만, 0.07 같은 값이면 소수가 됩니다.

문제 7

```
console.log("----- 영수증 -----");
```

출력 ----- 영수증 -----

```
for (const o of orders) {
  console.log(orderLine(o));
}
```


출력	아메리카노 x2 = 8000원 케이크 x1 = 6000원 알 수 없는 메뉴(99번)
----	--

코드	▶ 출력
<code>console.log("합계: " + total(orders) + "원");</code>	▶ 합계: 14000원
<code>console.log(statusText("조리중"));</code>	▶ 만들고 있어요

해설 ① 앞에서 만든 `orderLine` · `total` · `statusText` 를 그대로 가져다 씁니다.

새로 쓴 코드가 거의 없습니다. 그게 함수로 쪼개 둔 이유입니다.

해설 ② 타입을 적어 둔 덕분에 이 조립이 안전합니다.

`total` 에 문자열을 넘기거나 `statusText` 에 "취소" 를 넘기면 그 자리에서 걸립니다.

해설 ③ 이 실습에서 `any` · `!` · `as` 를 한 번도 안 썼습니다.

"없을 수도 있는 값" 은 전부 `?.` · `??` · `if` 로 처리했습니다.
실무에서도 이 정도면 충분합니다.

09단원 종합 02 정답 재고 보고서

```
RUN node 종합02_재고_보고서_정답.ts
```

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
const products = [
  { id: 1, name: "아메리카노", stock: 12 },
  { id: 2, name: "카페라떼", stock: 3 },
  { id: 3, name: "케이크", stock: 0 },
];
```

모양이 다른 두 번째 목록입니다. id 가 있다는 것만 같습니다.

```
const suppliers = [
  { id: 7, company: "봄날로스터리", phone: "010-1111-2222" },
  { id: 9, company: "여름제과", phone: "010-3333-4444" },
];

console.log("===== 재고 보고서 =====");
```

```
출력      ===== 재고 보고서 =====
```

문제 1

```
function first<T>(arr: T[]): T | undefined {
  return arr[0];
}
function last<T>(arr: T[]): T | undefined {
  return arr[arr.length - 1];
}

console.log(first(products)?.name ?? "비었음");
```

```
출력      아메리카노
```

```
console.log(last(products)?.name ?? "비었음");
```

```
출력      케이크
```

```
const empty: string[] = [];
console.log(first(empty) ?? "비었음");
```

출력 비었음

해설 ① 반환 타입에 `undefined` 를 꼭 적어야 합니다. 빈 배열이면 없으니까요.

`: T` 라고만 적으면 쓰는 쪽이 확인 없이 쓰다가 실행 중에 터집니다.

해설 ② `const empty = []`; 처럼 타입 없이 쓰면 안 됩니다.

빈 배열로 시작할 때는 `: string[]` 을 적어야 합니다(02단원 개념02 섹션5).

해설 ③ `products` 를 넘기면 `T` 가 `{ id, name, stock }` 으로 정해집니다.

그래서 `?.name` 이 통과합니다. `any[]` 였다면 오타도 안 걸렸을 것입니다.

문제 2

```
function toList(value: string | string[]): string[] {
  return Array.isArray(value) ? value : [value];
}
```

```
console.log(toList("아메리카노"));
```

출력 ['아메리카노']

```
console.log(toList(["아메리카노", "케이크"]));
```

출력 ['아메리카노', '케이크']

해설 ① `typeof` 로는 배열을 못 가립니다. `typeof []` 는 "object" 입니다.

`Array.isArray` 를 씁니다(05단원 개념02 섹션3).

해설 ② 반환 타입 `: string[]` 을 적어 두면

실수로 문자열을 그냥 돌려주는 것을 함수 안에서 잡아 줍니다.

해설 ③ `if / else` 로 풀어 써도 정답입니다. 삼항이 짧을 뿐입니다.

문제 3

```
type StockState =
  | { level: "품절" }
  | { level: "부족"; left: number }
  | { level: "충분"; left: number };

function checkStock(stock: number): StockState {
  if (stock === 0) return { level: "품절" };
  if (stock <= 5) return { level: "부족", left: stock };
  return { level: "충분", left: stock };
}

function stockText(s: StockState): string {
  switch (s.level) {
    case "품절":
      return "품절입니다";
    case "부족":
      return s.left + "개 남았습니다 (주문 필요)";
    case "충분":
      return s.left + "개 있습니다";
  }
}

console.log(stockText(checkStock(12)));
```

출력 12개 있습니다

```
console.log(stockText(checkStock(3)));
```

출력 3개 남았습니다 (주문 필요)

```
console.log(stockText(checkStock(0)));
```

출력 품절입니다

해설 ① “품절” 에는 left 가 아예 없습니다. 그게 판별 유니온의 요점입니다.

품절인데 남은 개수가 있는 상태를 아예 못 만듭니다.

해설 ② case “품절” 안에서 s.left 를 쓰면 TS2339 로 걸립니다.

그 갈래에는 없는 속성이니까요. 타입이 갈래마다 다르게 좁혀집니다.

해설 ③ level 을 string 으로 적으면 좁히기가 통째로 안 됩니다.

"품절" 처럼 값을 그대로 적어야 합니다(05단원 개념04 섹션5).
 판별 유니온에서 가장 흔하고 가장 찾기 어려운 실수입니다.

문제 4

```
function findById<T extends { id: number }>(items: T[], id: number): T | undefined {
    return items.find((item) => item.id === id);
}
```

```
console.log(findById(products, 2)?.name ?? "없음");
```

출력 카페라떼

```
console.log(findById(products, 99)?.name ?? "없음");
```

출력 없음

```
console.log(findById(suppliers, 9)?.company ?? "없음");
```

출력 여름제과

해설 ① extends { id: number } 가 없으면 item.id 에서 걸립니다.

조건 없는 T 는 '모든 타입' 이라 id 가 있는지 알 수 없기 때문입니다.

해설 ② extends 는 상속이 아니라 "적어도 이걸 갖고 있어야 한다" 로 읽으세요. 해설 ③ ★ 돌려받은 값에는 넘긴 것의 모양이 그대로 붙습니다.

products 에서 찾으면 .name 이 되고, suppliers 에서 찾으면 .company 가 됩니다.
 함수는 하나인데 나오는 타입이 다릅니다. 이게 제네릭을 쓰는 이유입니다.
 findById(products, 2)?.company 는 TS2339 로 걸립니다.
 any[] 로 만들었다면 셋 다 조용히 통과하고 실행할 때 undefined 가 나왔을 것입니다.

문제 5

```
type ApiResult<T> = { ok: boolean; data: T };

const listResult: ApiResult<{ id: number; name: string; stock: number }[]> = {
    ok: true,
    data: products,
};

console.log(listResult.data.length);
```

출력 3

```
const textResult: ApiResult<string> = { ok: true, data: "봄날카페" };  
console.log(textResult.data.length);
```

출력 4

해설 ① data 자리만 바뀌고 ok 는 그대로입니다.

서버 응답이 늘 같은 모양이면 이렇게 한 번 만들어 두고 씁니다.

해설 ② <string> 이면 .length 가 글자 수, <배열> 이면 개수입니다.

같은 .length 인데 담은 것에 따라 뜻이 달라지는데, 타입이 알아서 맞춰 줍니다.

해설 ③ 상품 타입에 이름을 붙여 두었다면 ApiResult<Product[]> 로 짧아집니다.

같은 모양을 두 번 이상 쓰면 이름을 붙이세요(04단원 개념02 섹션5).

문제 6

```
console.log("----- 재고 보고서 -----");
```

출력 ----- 재고 보고서 -----

```
let needOrder = 0;  
for (const p of products) {  
  const state = checkStock(p.stock);  
  console.log(p.name + ": " + stockText(state));  
  if (state.level !== "충분") needOrder++;  
}
```

출력 아메리카노: 12개 있습니다
카페라떼: 3개 남았습니다 (주문 필요)
케이크: 품절입니다

```
console.log("주문 필요: " + needOrder + "건");
```

출력 주문 필요: 2건

해설 ① 앞에서 만든 checkStock · stockText 를 그대로 씁니다.

새로 쓴 것은 세는 줄 하나뿐입니다.

해설 ② state.level !== "충분" 으로 썼습니다.

나중에 "예약중" 같은 상태가 늘어나도 이 줄은 그대로 맞습니다.
반대로 === "품절" || === "부족" 으로 썼다면 그때 고쳐야 합니다.

해설 ③ 그런데 stockText 는 상태가 늘면 TS2366 으로 걸립니다.

"고쳐야 할 곳만 정확히 걸리고, 안 고쳐도 되는 곳은 안 걸리는" 것이
판별 유니온을 쓰는 이유입니다.

09단원 종합 03 정답 서버 응답 다루기

RUN node 종합03_서버_응답_정답.ts

검사: npm run typecheck

코드보다 해설이 본체입니다. 맞았어도 해설은 읽고 넘어가세요.

```
const okText = '{"name":"라떼","price":4500}';
const missingText = '{"name":"라떼"}';
const wrongText = '{"name":123,"price":4500}';
const brokenText = "<html>502 Bad Gateway</html>"; // JSON 이 아예 아닌
응답 console.log("===== 서버 응답 =====");
```

출력 ===== 서버 응답 =====

문제 1

```
type Menu = { name: string; price: number };

const a: Menu = JSON.parse(okText);
console.log(a.name, a.price);
```

출력 라떼 4500

```
const b: Menu = JSON.parse(missingText);
console.log(b.name, b.price);
```

출력 라떼 undefined

해설 ① 두 줄 다 검사를 통과합니다. `JSON.parse` 는 `any` 를 주고,

`any` 는 어디에나 들어가기 때문입니다.

해설 ② : `Menu` 라고 적는 것은 “이런 모양일 것이다” 라는 내 주장입니다.

타입스크립트는 그 주장을 검사하지 않고 믿습니다.
node 는 타입 표기를 지우고 실행하니 확인해 주는 코드가 어디에도 없습니다.

해설 ③ `b.price.toFixed(0)` 이었다면 여기서 터졌을 것입니다.

undefined 가 화면에 찍히고 나서야 알게 되는 것 -
이게 04단원 개념04의 "타입은 약속이지 검사가 아니다" 입니다.

문제 2

```
function parseMenu(text: string): Menu | null {
  let data: unknown;
  try {
    data = JSON.parse(text);
  } catch {
    return null; // JSON 이 아니면 여기서 끝. 이 줄이 없으면 프로그램이 죽습니다
  }
  if (
    typeof data === "object" &&
    data !== null &&
    "name" in data &&
    "price" in data &&
    typeof data.name === "string" &&
    typeof data.price === "number"
  ) {
    return { name: data.name, price: data.price };
  }
  return null;
}

console.log(parseMenu(okText));
```

출력 { name: '라떼', price: 4500 }

코드	▶ 출력
console.log(parseMenu(missingText));	▶ null
console.log(parseMenu(wrongText));	▶ null
console.log(parseMenu(brokenText));	▶ null

해설 ① : unknown 으로 받는 것이 핵심입니다.

any 로 받으면 아래 확인을 안 해도 통과해 버려서 확인이 무의미해집니다.
unknown 은 확인을 통과하기 전에는 아무것도 못 하게 막습니다.

해설 ② 조건이 여섯 줄인 이유 — 앞에서부터 하나씩 좁혀 나가기 때문입니다.

`typeof null` 이 "object" 라서 ②(null 이 아닌가)를 따로 봐야 합니다.
"name" in data 로 이름이 있는지 본 뒤에야 data.name 을 읽을 수 있습니다.

해설 ③ wrongText 는 name 이 숫자 123 이라 ⑤ 에서 걸러집니다.

as Menu 로 우겼다면 셋 다 통과하고 나중에 터졌을 것입니다.
실무에서는 이 일을 zod 에 맡깁니다(08단원 개념03 섹션4).

해설 ④ ★ try / catch 가 왜 필요한가 — unknown 확인은 '값의 모양' 만 봅니다.

JSON.parse 는 그 확인에 도달하기도 전에 예외를 던집니다.
서버가 죽으면 JSON 이 아니라 502 HTML 페이지나 빈 문자열이 옵니다.
그러면 try 가 없는 코드는 화면이 통째로 멈춥니다.
타입 검사로는 절대 안 잡히는 자리입니다. JSON.parse 는 어디서나 이렇게 감쌉니다.

문제 3

```
type Shop = { name: string; owner: { name: string; phone: string | null } | null };

const rawShop1 = { name: "봄날카페", owner: { name: "홍길동", phone: null } };
const rawShop2 = { name: "무인카페", owner: null };

const shop1: Shop = rawShop1;
const shop2: Shop = rawShop2;

console.log(shop1.owner?.phone ?? "번호 없음");
```

출력 번호 없음

```
console.log(shop2.owner?.name ?? "주인 없음");
```

출력 주인 없음

해설 ① ?. 는 '없을 수 있는 자리마다' 필요합니다.

shop1 은 owner 가 있지만 phone 이 null 이라, ?? 가 그걸 받아 줍니다.
shop2 는 owner 부터 없어서 ?. 가 거기서 멈춥니다.

해설 ② shop2.owner!.name 으로 썼다면 그 줄에서 터집니다.

shop2 는 owner 가 null 인데 ! 로 "없을 리 없다" 고 우긴 것이라
 Cannot read properties of null 이 납니다.
 (shop1.owner!.phone 은 owner 가 있어서 안 터집니다. 그래서 더 위험합니다 -
 ! 는 터질 때까지 아무 말도 안 해 줍니다)
 ! 는 문제를 숨기고 ?. ?? 는 문제를 처리합니다.

해설 ③ rawShop1 을 그대로 담을 수 있는 이유는 모양이 맞기 때문입니다.

phone: null 은 **string** | null 자리에 들어갑니다.

문제 4

```
type LoadState =
  | { status: "로딩중" }
  | { status: "성공"; menu: Menu }
  | { status: "실패"; message: string };

function render(state: LoadState): string {
  switch (state.status) {
    case "로딩중":
      return "불러오는 중...";
    case "성공":
      return state.menu.name + " " + state.menu.price + "원";
    case "실패":
      return "오류: " + state.message;
  }
}

console.log(render({ status: "로딩중" }));
```

출력 불러오는 중...

```
console.log(render({ status: "성공", menu: { name: "라떼", price: 4500 } }));
```

출력 라떼 4500원

```
console.log(render({ status: "실패", message: "형식이 다릅니다" }));
```

출력 오류: 형식이 다릅니다

해설 ① case "성공" 안에서 state.menu 를 확인 없이 씁니다.

"성공이면 menu 가 반드시 있다" 가 타입에 적혀 있기 때문입니다.

해설 ② { status: "성공" } 만 쓰면 TS2322 로 걸립니다. menu 가 빠졌으니까요.

성공인데 데이터가 없는 상태를 아예 못 만듭니다.

해설 ③ 상태를 하나 늘리면 이 switch 가 TS2366 으로 걸립니다.

고쳐야 할 곳을 타입이 찾아 줍니다.

문제 5

```
function load(text: string): LoadState {
  const menu = parseMenu(text);
  if (menu === null) {
    return { status: "실패", message: "형식이 다릅니다" };
  }
  return { status: "성공", menu };
}
```

```
console.log(render(load(okText)));
```

출력 라떼 4500원

```
console.log(render(load(missingText)));
```

출력 오류: 형식이 다릅니다

해설 ① parseMenu 가 Menu | null 을 주니, null 을 먼저 걸러 내고 돌려보냅니다.

그 아래부터 menu 는 확실히 Menu 입니다(이른 반환).

해설 ② { status: "성공", menu } 는 { status: "성공", menu: menu } 의 줄임입니다.

JS자료 09단원의 속성 축약입니다.

해설 ③ 문제 1과 비교해 보세요. 같은 missingText 인데

문제 1은 화면에 undefined 를 찍었고, 여기는 "형식이 다릅니다" 로 처리합니다.
확인 한 번을 넣었을 뿐인데 결과가 완전히 달라집니다.

문제 6

```
console.log("----- 처리 결과 -----");
```

출력 ----- 처리 결과 -----

```
const responses = [okText, missingText, wrongText];
let ok = 0;
let fail = 0;

for (let i = 0; i < responses.length; i++) {
  const text = responses[i];
  const state = load(text);
  if (state.status === "성공") ok++;
  else fail++;
  const 문구 = state.status === "실패" ? state.message : render(state);
  console.log(i + 1 + "번: " + 문구);
}
```

출력 1번: 라떼 4500원
 2번: 형식이 다릅니다
 3번: 형식이 다릅니다

```
console.log("성공 " + ok + "건 / 실패 " + fail + "건");
```

출력 성공 1건 / 실패 2건

해설 ① state.status === "성공" 으로 세면 됩니다. 문자열 비교가 아니라

타입에 적힌 세 값 중 하나라, 오타를 내면 TS2367 로 걸립니다.

해설 ② responses[i] 가 undefined 일 수 있다고 걱정할 수 있는데,

기본 설정에서는 그냥 string 으로 나옵니다.
noUncheckedIndexedAccess 를 켜면 확인이 필요해집니다(08단원 개념03 섹션3).

해설 ③ ★ "오류: " 를 떼려고 render 결과에 replace 를 쓰면 안 됩니다.

판별 유니온으로 갈라 놓고 다시 문자열을 뜯어보는 것은 되돌아가는 것입니다.
게다가 replace 는 앞머리만 보는 게 아니라 첫 일치치를 아무 데서나 지웁니다.
메뉴 이름이 "오류: 라떼" 라면 성공 결과까지 깎입니다.
표딱지가 있으면 표딱지로 갈라야 합니다 - 05단원 개념04가 이 이야기입니다.

해설 ④ 이 실습 전체에서 any · ! · as 를 한 번도 안 썼습니다.

JSON 이 아닌 응답까지 try / catch 로 받아 내므로
서버가 무엇을 보내든 화면이 안 터집니다. 그게 목표였습니다.

부 록 나

심화문제 정답

심화문제는 선택 과제입니다. 풀지 않고 넘어가도 다음 단원에 지장이 없습니다.